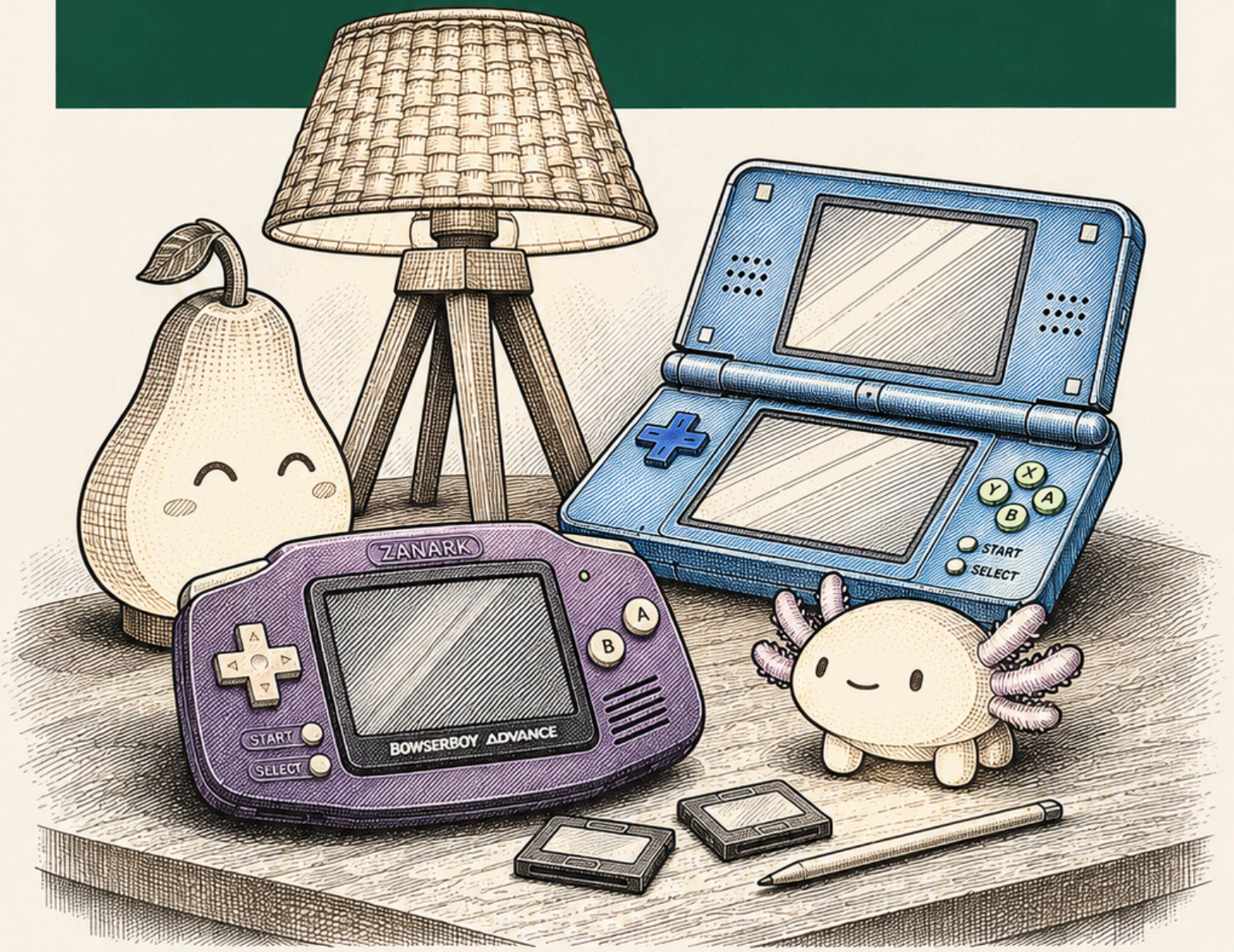
THE BOWSERBOY ENGINEERING BOOK

SECOND EDITION / NOVEMBER 2026

From Shapes to a Living Desk

The Complete BowserBOY Procedural Modeling Handbook



State. Shape. Light. Evidence.

Zanark

Contents

Thirty-eight source-pinned chapters. Detailed mechanisms, construction recipes, worked examples and explained exercises.

How to use this reconstruction manual 4

Part 1 - Foundations and the Game Boy Advance 9

MD01 - Evidence, libraries and the actual toolchain 10
MD02 - The modeling contract and visual references 18
MD03 - Coordinate systems and transform ownership 26
MD04 - The procedural geometry toolkit 34
MD05 - GBA shell, silhouette and layered construction 42
MD06 - GBA LCD, curved glass and frame 51
MD07 - GBA controls, grips and tactile details 59
MD08 - Cartridges, small details and markings 67
MD09 - Procedural textures and asset boundaries 76

Part 2 - The articulated Nintendo DS 84

MD10 - DS assembly and the parameter map 85
MD11 - Hinges, lids and articulation 93
MD12 - DS shells, recesses and hardware 100
MD13 - DS controls and readable markings 108
MD14 - Dual LCDs and curved glass 116
MD15 - Stylus construction and touch geometry 124
MD16 - Cartridge insertion and loading choreography 132
MD17 - Inspection, camera and manipulation 140
MD18 - The DS model regression atlas 149

Part 3 - Characters, softness and movement 157

MD19 - Pear Boy: the parametric body 158
MD20 - Projecting a face onto a curved shell 166
MD21 - Expressions, gaze and six visible freckles 174
MD22 - Limbs, rope and soft motion 182
MD23 - Axolotl: the silicone body 190
MD24 - Gills, tail and cute features 199
MD25 - Companion light emission and material tuning 207
MD26 - Dragging, gravity and dropping 215
MD27 - Clearance, collision envelopes and home poses 223

Part 4 - A complete, verifiable 3D world 232

MD28 - The desk, room and tripod lamp **233**
MD29 - Light units, exposure and color **242**
MD30 - Shadows, reflections and shaders **249**
MD31 - Screen materials and pixel preservation **256**
MD32 - Picking, occlusion and input ownership **263**
MD33 - Render loops, GPU budgets and lifecycle **271**
MD34 - Algorithms and data structures **278**
MD35 - Design principles and failure boundaries **286**
MD36 - Validation and bounded recovery **295**
MD37 - The causal modeling history **303**
MD38 - Agent reconstruction and extension **311**
The standalone agent handoff **319**
Sources, model coverage and research limits **326**

How to use this reconstruction manual

This book explains how BowserBOY's first-party three-dimensional objects are constructed, assembled, animated and verified. Its central subject is not the emulator instruction set. It is the deceptively large amount of geometry and engineering required to make a small handheld console, a second articulated console, two expressive companions and a believable desk behave as one coherent interactive world.

The purpose is transfer. A future agent should be able to inspect the implementation, identify the mechanism behind a visible detail, reconstruct that mechanism in a small experiment, and extend it without relying on this conversation. The book therefore keeps exact source references, parameter meanings, worked calculations, changed-condition retries and explained exercises beside the construction discussion. It does not claim to copy a model's hidden reasoning, transfer every previous memory or guarantee that a different agent will make identical artistic judgments.

What belongs to this edition

The implementation cutoff is `1e4ef518d6943ab1af39cef3b4d558f209a17891` in the public `Zanark/BowserBoy` repository. A cutoff makes the teaching inspectable: a later improvement must not silently become evidence for an earlier explanation. The manuscript, diagnostic atlas and publishing tools were created after that implementation revision. Their exact input hashes belong to the edition manifests, rather than being misrepresented as files already present at the cutoff.

There are four parts. The first establishes libraries, references, coordinates, reusable geometry, the Game Boy Advance shell and its small physical details. The second follows the Nintendo DS from lower deck to articulated lid, dual display, stylus, controls and cartridge loading. The third constructs Pear Boy and the axolotl, including curved face projection, expressions, soft-looking surfaces, movement and clearance. The fourth explains the supporting desk, room, tripod lamp, rendering, input ownership, performance, algorithms, design principles, regression evidence and historical changes.

The desk and wall are not incidental background. A convincing tabletop controls apparent contact, scale, shadow direction and whether a moved object seems to float. A lamp is not only a mesh: the visible shade, world-space emitter, exposure and shadow policy have to agree. A face is not only a set of eyes: its features have to remain outside the actual body surface as it follows the pointer. Those connections are part of modeling, not unrelated implementation detail.

This is a third, independent book. The existing GBA engineering book and the separate NDS development-journey book remain useful references and are not overwritten. This handbook deliberately includes both console families and their common systems because the subject is the complete modeling workflow. It does not import the GBA chapters into the NDS-only book.

Read the evidence labels literally

Four classes of statements occur throughout the chapters.

Implemented behavior describes source at the pinned revision. The strongest references identify the file, the relevant symbol or section, and actual line ranges. A source reference can establish what a constructor creates or what an update function calculates. It cannot by itself prove what a particular browser visibly displayed.

Historical evidence describes a change supported by a commit, tracked workstream or existing regression record. A development history reconstructed from those records is not a verbatim transcript of an author's thoughts. A later test name is not proof that the test existed when an earlier shape was first modeled.

Researched guidance comes from an identified external reference, such as Three.js or a browser-platform reference. Documentation consulted later can explain a general mechanism or suggest a better experiment. It does not retroactively prove the project's original intent, and a current documentation page need not describe every detail of the installed library version.

Proposed extensions are recommendations. A proposed optimization, alternate modeling tool or broader test matrix is not already implemented because it appears in a textbook. Carry that distinction into future changes and into their completion reports.

Hash equality is also a narrow claim. It can establish the identity of a file, an illustration or an artifact. It does not establish semantic correctness, aesthetic approval, reproducibility on an untested machine or the rights to redistribute a third-party work. The native PDF and browser checks make specific assertions about content, dimensions, links and layout; they are not an all-purpose certificate.

Follow a model through five layers

For each object, begin with its silhouette. If a reader cannot recognize the console or companion from an uncluttered three-quarter view, tiny labels will not rescue the design. Reconstruct the main masses first, establish an origin and local axes, then add recesses, controls and markings.

Next, inspect the surface construction. A pale silicone body, painted shell, rubber pad, LCD and glass overlay should not be treated as interchangeable material presets. Ask which geometry owns the visible contour, which material receives light, which surface is unlit by design and which layer exists only to provide a reflection.

Then identify the transform owner. A DS lid rotates around a hinge-related local origin; a whole console moves through another group; a companion's gaze can move its facial features without dragging its entire body. An attractive pose in one hierarchy is not evidence that the hierarchy remains correct when a parent rotates.

The fourth layer is interaction. A button can be visually present but geometrically occluded. A face can report a visible flag yet be buried beneath another surface. A lamp can remain in the correct pose while its effective light position fails to follow. Inspect the world-space result and the foremost rendered surface, not only a convenient state field.

Finally, examine lifecycle and regression evidence. A model factory allocates buffers, materials and textures. A render loop decides when motion is sampled and when shadows or reflections are refreshed. A change to a limb can enlarge a collision envelope even if its resting pose looks unchanged. Modeling precision therefore includes resource ownership, animation timing and clearance.

The atlas is a controlled experiment

The illustrations in `assets/atlas` are fresh renders of the actual procedural factory geometry. There are front, back and three-quarter orthographic views of the GBA, DS, Pear Boy, axolotl, tripod lamp and cartridge. The original factory code was not edited to make the images.

The capture environment intentionally differs from the application. It uses a neutral studio background, separate studio lights, ACES tone mapping at exposure one and an orthographic camera fitted to each complete model. Factory light objects are hidden and material emission is disabled so that base surfaces remain inspectable. The DS's returned lid group is rotated fifteen degrees around local X for a readable diagnostic open pose. The application sets its own poses, lighting and exposure.

Those conditions matter. A pale companion in the atlas is not a demonstration of its powered lamp appearance. The captured GBA's white display is its factory material, not proof of game execution. A visible model edge is useful construction evidence; it does not demonstrate a shadow produced by the table lamp in the real room. Captions and the capture manifest retain these limits.

The generator loads the shared brand images before constructing the GBA. Its canvas textures use decoded image data immediately; simply importing a constructor is insufficient. The loopback server serves only approved module and image resources, and the browser blocks external requests. No ROM, firmware, native emulator startup or private recording is needed to reproduce these illustrations.

The source identity check distinguishes Windows line endings from substantive edits. It compares pinned and working source after explicit CRLF-to-LF normalization, while recording the raw capture-input hashes separately. Do not report those two identities as byte equality. The manifest also records actual served resource hashes, browser version, model bounds and mesh counts.



Figure O.1. The original six-model cover plate is composed from actual three-quarter captures. It compares shapes, not physical scale between objects or application-room lighting.

Use the learning structure

Every chapter has a stable `MDxx` identity. A `W` section is a worked example: follow the concrete setup and explain why the result follows. An `R` section changes a condition and requires a new decision rather than copying the previous answer. A `Q` section presents an exercise; its paired `A` section explains the reasoning and the failure modes, not just a final number.

When adapting a technique, write down the invariant before changing the code. For a face, an invariant might concern surface clearance and visibility across all expressions. For a console, it might concern the relation between the bezel opening, active screen and glass. For a movable companion, it includes the complete component envelope and a collision-free settled pose.

Use the exercises to find a misunderstanding before making a broad patch. If the proposed change cannot be explained in local coordinates, isolate a small model and draw the parent-child relationship. If a lighting adjustment cannot distinguish material emission from external illumination, test those contributions separately. A short controlled experiment is more useful than repeatedly tuning several unrelated constants.

Reconstruct before you generalize

The shortest route through the book for a new agent is foundations, the model family being changed, algorithms, design principles and reconstruction. Read the neighboring interaction and lifecycle chapters before changing a shape that is animated or draggable. The book's source inventory and standalone handoff help locate those boundaries, but neither substitutes for reading the relevant implementation.

Reproduce the existing behavior first. Establish the baseline silhouette, dimensions, material roles, transform hierarchy and a few representative views. Save the exact source revision and capture conditions. Only then change one construction layer and evaluate the result. Otherwise a new lighting setup can disguise a bad normal, a different camera can hide an intersection and a broad refactor can make the cause of an improvement impossible to identify.

Do not install an alternative authoring tool merely because it appears in a researched recommendation. The established application constructs its models procedurally with Three.js and browser canvas textures. An external mesh editor can be a reasonable future choice, but it introduces a different source-of-truth, export, scale, normal, texture and licensing contract. It is a design decision, not a missing step that this book quietly assumes.

For reproducible commands, start in the repository root. The publishing and atlas recipes use the existing Node dependencies, Python image/PDF tools and an installed supported Chromium browser. Restore a missing dependency only when the chosen validation reports it missing, or when changing its manifest. The documented local qualification does not claim a fresh-machine setup or cross-platform test run.

Know when the work is actually done

A successful modeling change has several different outcomes. The intended geometry exists. The intended effect is visible from the relevant views. Interaction still selects the correct surface. Motion and clearance remain valid. Relevant regressions pass or an explicitly recorded limitation remains. Documentation explains the change without exporting private inputs.

A successful book has its own separate outcomes. The assembly contains the intended chapters and illustrations; real chapter prose is not replaced by code dumps; the printed PDF contains at least two hundred pages and at least two hundred unique chapter-content pages; links and learning markers survive native export; and the standalone reader opens without scripts or network dependencies. Cover, contents, part openers, chapter openers and appendices do not satisfy the content-page minimum.

Mechanical results, bounded visual inspection, user acceptance and publication are distinct. Read the exact edition's review note. Do not turn a generated contact sheet into a claim that every figure was inspected at full size, a deployment badge into a gameplay test, or a previous edition's review into approval of new bytes.

The transferable method is a sequence of evidence-backed decisions, not a list of magical parameter values. Preserve that sequence when improving the project, and the next agent will inherit a usable engineering manual instead of another long document that still requires the original conversation.

PART 1

Foundations and the Game Boy Advance

1. Evidence, libraries and the actual toolchain
2. The modeling contract and visual references
3. Coordinate systems and transform ownership
4. The procedural geometry toolkit
5. GBA shell, silhouette and layered construction
6. GBA LCD, curved glass and frame
7. GBA controls, grips and tactile details
8. Cartridges, small details and markings
9. Procedural textures and asset boundaries

PART 1 / FOUNDATIONS AND THE GAME BOY ADVANCE

MD01

Evidence, libraries and the actual toolchain

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Evidence, libraries and the actual toolchain

What this reconstruction can establish

A useful modeling book must explain how to rebuild an object, not merely identify the library that draws it. This part reconstructs the Game Boy Advance family from executable geometry, image-processing operations, materials, tests, and recorded changes. Its implementation baseline is commit `1e4ef518d6943ab1af39cef3b4d558f209a17891`. Numbers described as implementation values belong to that baseline. Exercises and proposed diagnostics are teaching additions, not claims that the original author used those procedures.

The central evidence is unusually direct: `createHandheld()` creates a group and populates it with named meshes; `createCartridge()` creates a second group; `createWood()` paints a texture. These are construction programs rather than imported finished models. Reading them reveals shapes, coordinate relationships, material sharing, and initial visibility. It does not reveal the author's unrecorded reasoning, physical measurements, or subjective approval. See the factory boundaries (`src/handheld.js:392`) and the cartridge and wood factories (`src/handheld.js:645`).

Use three evidence categories throughout this part. **Implementation** means behavior visible in the pinned source. **Historical record** means a commit diff or public workstream says something happened; that statement retains its date and limitations. **Recommendation** means a reproducible technique proposed here. These categories prevent a common documentation error: converting a sensible present-day workflow into an invented account of how a model was originally made.

The small runtime toolchain

The root manifest declares one runtime npm dependency, Three.js `0.186.1`, and the development dependency `@playwright/test` `1.64.1`. It declares ES modules and Node `>=22`. The lock entries and installed package manifests inspected for this reconstruction agree with those two versions. Agreement verifies this workspace's declarations and installation; it is not evidence that a clean machine was installed or tested during this writing. Manifest (`package.json:1`); locked packages (`package-lock.json:20`).

Three.js provides the scene graph, shapes, triangulation, extrusions, lathe geometry, materials, and textures used by these factories. Browser Canvas 2D supplies text rasterization and image processing. Neither a Blender project nor a GLTF loader participates in the construction path examined here. This is a bounded claim about these sources, not a claim that no external creative tool ever existed. In particular, supplied brand artwork is an input asset whose earlier creation cannot be reconstructed from the geometry factory.

The browser resolves the bare module name `three` through an import map to `./vendor/three.module.js`. This is not a bundler rewriting application imports. The build copies allowlisted public files, including the Three module, core, light helpers, and license. Its separate demo-generation calls do not turn cartridge props into ROM-derived meshes. Browser entry (`web/index.html:8`); delivery map (`scripts/site-files.mjs:4`).

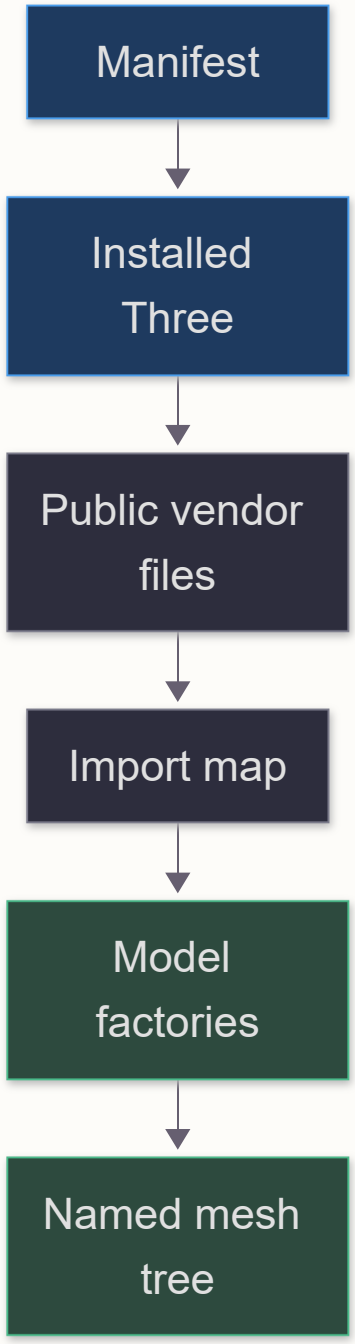


Figure MD01.1. The application resolves a packaged library into procedural factories; the delivery step does not author geometry.

Separate construction, presentation, and publishing

The book-production directory has its own manifest. Its Markdown parser, Mermaid, Paged.js, and browser automation dependencies produce documentation, not the handheld. Sharing a package family or version between production and tests does not merge their responsibilities. When recording a toolchain, assign each dependency to the operation it actually performs; otherwise a future agent might look for a CAD export that never existed. Production manifest (`docs/book/production/package.json:1`).

Similarly, a diagnostic render is a presentation of factory output. It can expose topology, layering, silhouette, or missing assets under controlled light. It cannot prove the room's final lighting behavior, emulator upload cadence, or accessible interaction. Those require integration evidence. The browser test that raycasts the LCD, speaker slot, and D-pad well is stronger evidence of those particular surfaces than a flattering front image, but it does not establish an accurate physical replica.

The historical record supports this distinction. The initial tabletop workstream records an actual-mesh requirement and later records rejection of the first shape despite passing tests. The reference correction explicitly separates independent geometry from extraction of a supplied reference model. Those records teach why visual and technical acceptance are different, without granting either one today. Recorded correction (`TODO/07_REFERENCE_SHAPE.todo:18`).

MD01-W01 Trace one visible object to its dependencies

Consider the A button. Begin at the named `A button` mesh, not a screenshot. The factory uses a lathed cap, selects a cloned rubber material, and attaches recessed lettering. The letter outline is recovered from browser-rendered Arial alpha; contour extraction turns that raster boundary into geometry. Thus the button depends on Three.js and browser text rasterization even though its final letter is not a painted decal. The distinction matters when comparing machines: the system font can affect a geometric cut.

Next identify dependencies that do **not** belong to that object. The loaded cartridge's title canvas does not determine the A glyph. The book renderer does not determine the lathe profile. The native emulator shader does not manufacture the cap. A rigorous receipt names these negative boundaries only after following the positive construction chain. Cap and glyph construction (`src/handheld.js:116`); glyph recovery (`src/handheld.js:232`).

The following original diagnostic is runnable in the browser developer console of an already-served project page. It does not load a ROM, call the complete handheld factory, or change the application. It creates a separate cartridge and reports its structural receipt. Its assumptions are a functioning page import map and browser Canvas 2D. Expected outcomes are five named grip meshes and two text planes, not a pixel-identical image across all operating systems.

```
// Pedagogical diagnostic; not application implementation.
const { createCartridge } =
  await import('./src/handheld.js');
const prop = createCartridge('#666574', 'TEST', 'LOCAL');
const names = [];
prop.traverse(node => {
  if (node.isMesh) names.push(node.name || '(text plane)');
});
console.table(names);
console.assert(
  names.filter(n => n === 'Cartridge grip').length === 5
);
```

Why inspect the object without rendering it? It isolates factory assembly from camera, lighting, and the live scene's state. A missing grip count is a construction problem. A present grip that cannot be seen in a render may instead be behind another surface or below image resolution. This staged distinction reduces speculative parameter editing.

MD01-R01 Retry when the library appears correct but imports fail

Suppose the installed package version is correct, yet the browser reports that `three` cannot be resolved. Reinstalling packages is not the first justified repair. Check whether the served HTML contains the import map, whether its relative vendor URL resolves under the current base path, and whether both module and core files were delivered. The build maps source locations to public destinations; a filesystem import that works in Node does not demonstrate browser delivery.

Now change the condition: the import succeeds, but `createHandheld()` fails while drawing the wordmark. This is a different boundary. `brandImage` is a live export populated by asynchronous image loading. The factory draws it directly. The relevant repair investigation is initialization order and asset decoding, not Three.js geometry segmentation. Brand loader (`src/brand.js:1`).

Record each retry with its changed condition and observable result. “Dependencies fixed” is too broad. “Vendor module now resolves; brand image still undecoded” preserves useful causal information. It also avoids claiming clean-machine reproduction merely because a warm workspace recovered. A future agent should be able to repeat the narrow observation without inheriting an unexplained success label.

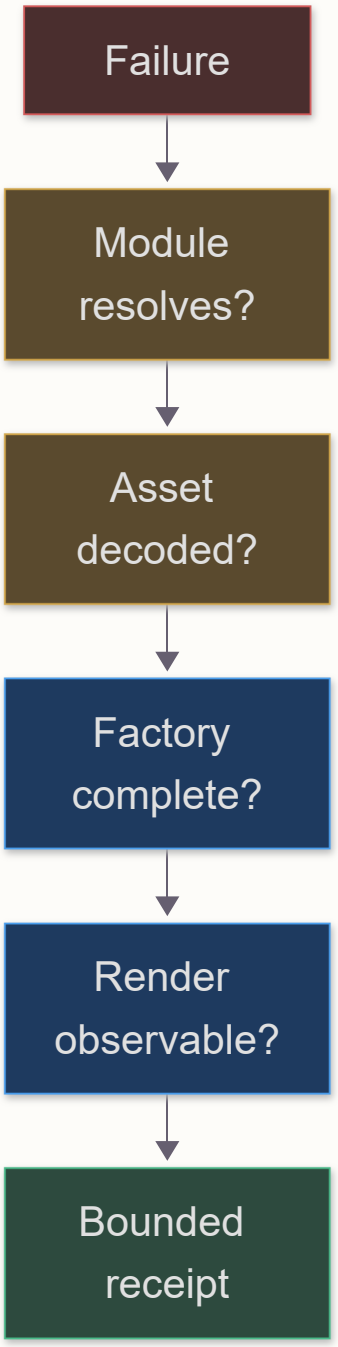


Figure MD01.2. Diagnose the earliest failed boundary before changing a later modeling parameter.

MD01-Q01 Decide what a successful test proves

Explained answer

An agent reports: “The real-3D browser case passed, so the GBA is dimensionally accurate, the source model was imported legally, every control remains illuminated while pressed, and the book is ready to publish.” Identify which parts follow from the inspected evidence. Design a stronger replacement report without running a publishing operation.

MD01-A01 Evidence has a measurable scope

Back to question

The report overreaches in four different directions. The real-3D case checks a triangle threshold, movement into play, front orientation, particular raycast surfaces, and further interaction behavior. Its surface assertions establish an unobstructed center LCD, a recessed speaker comparison, and a bounded pivot well. They do not compare a caliper measurement or CAD reference. Replace “dimensionally accurate” with those explicit observations. Surface assertions (`tests/browser/scene.spec.mjs:1351`).

The reference workstream says the geometry was independently authored and the external model was unavailable for extraction. That is a historical provenance statement, not a license grant discovered by a geometry test. The complete-press-response test is explicitly parked in the pinned suite, while separate aperture-lighting cases contain active assertions. Their existence cannot be summarized as a fresh pass of all controls under every light. Publication readiness requires its own authorized integration process; neither a mesh test nor this chapter performs it.

A defensible replacement report identifies the pinned revision, exact inspected or executed cases, active versus skipped status, and unmeasured properties. This writing inspected tests but did not execute them. That limitation is useful rather than embarrassing: it tells the next agent precisely where new evidence is required.

Research that informs recommendations, not history

Build a receipt that another agent can challenge

A useful reconstruction receipt has an input identity, an operation, an observation, and a limitation. For example, identify the pinned handheld factory, describe creating an independent cartridge, report five grip meshes, and state that no rendering or runtime insertion was exercised. A later agent can challenge any one of these elements without discarding the whole account. “Looks correct” has no comparable structure.

Line citations serve a related purpose. Cite the helper that defines a bevel convention as well as the call site that chooses a particular bevel. Citing only the call site leaves readers guessing whether the geometry is centered or how thickness expands. Citing only the helper cannot prove that the shell actually uses it. The source pair creates a short, auditable construction chain.

For version-sensitive recommendations, record the gap between what documentation describes and what the implementation invokes. A current API page can contain features added after the project was pinned. It is not safe to retrofit those features into the historical account simply because they would simplify the explanation. Prefer the subset confirmed by the source, then label newer alternatives explicitly.

Finally, preserve failures and skipped checks as first-class evidence. An unresolved retained-frame reload issue belongs beside the otherwise useful CRT assertions, not in an unrelated footnote. An unavailable font is an input limitation, not an excuse to claim exact contours anyway. This

approach makes a book useful as an engineering handoff: the next agent receives both a construction method and the edges beyond which it must gather new evidence.

The current Three.js Object3D documentation, BufferGeometry documentation, and CanvasTexture documentation were consulted on 2026-11-07. They support teaching about local/world transforms, indexed attributes and normals, and canvas-backed textures. They are latest documentation, not a frozen description of the project's original development environment. The implementation and lockfile remain authoritative for what this project actually calls and installs.

PART 1 / FOUNDATIONS AND THE GAME BOY ADVANCE

MD02

The modeling contract and visual references

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

The modeling contract and visual references

A reference is a question, not a mesh specification

The hardest modeling decision often precedes the first vertex: deciding what the object must communicate. Here the historical reference correction names a stepped upper crown, straighter sides, outward lower bow, lower-profile controls, and an angled recessed grille. These are actionable shape relationships. “Make it look more like a GBA” is not. The record also says the first model was rejected despite technical test success. Treat that as a warning against letting a passing runtime suite define aesthetic correctness. Reference contract (`TODO/07_REFERENCE_SHAPE.todo:1`).

This chapter reconstructs a contract from public artifacts; it does not recover the author's thoughts. The external reference's recorded download and license availability are historical observations, not a current audit of that service. The implemented model remains a set of independently specified curves and primitives. No dimensional drawing, calibrated photograph, or measurement survey is present in the evidence read for this part. Consequently, source units must remain scene units, not retroactively invented millimeters.

Begin with observable relationships: the crown is narrower than the widest side region; the screen sits centrally; the D-pad and action buttons occupy opposite sides; the speaker slots descend on the right; the rear has a separate battery cover. Each relationship can be traced to source coordinates. None requires reproducing copyrighted reference artwork. This distinction makes the workflow reusable: analyze organization and silhouette, then author original geometry whose parameters and limitations are explicit.

Translate appearance into layers of obligation

A useful contract has four layers. **Identity** asks whether the outline and landmark arrangement read as the intended device. **Construction** asks whether holes, floors, relief, and depth ordering actually exist. **Behavior** asks whether the assembled hierarchy can move and be inspected without separating parts. **Evidence** asks how each previous claim will be observed. Keeping these layers separate prevents an easy screenshot from becoming the only acceptance test.

The implementation provides a concrete identity scaffold. The outline reaches the central crown at `y=2.42`, uses side control points around `x=±4.28`, and bows downward through a control point at `y=-2.61`. These are path controls, not guaranteed final extrema. Extrusion bevels also expand the eventual surface. A bounding-box test must measure the generated geometry rather than substitute the largest literal found in the path. Outline construction (`src/handheld.js:410`).

Construction obligations are more specific. The thin front face receives control and speaker holes; recessed dark surfaces lie behind those openings. A mark intended to be molded uses geometry, while the bezel wordmark uses a processed texture. Behavior obligations preserve cap-child lettering and the loaded cartridge's parent relationship. These distinctions are not cosmetic bookkeeping: they decide which transformations and materials should change together.

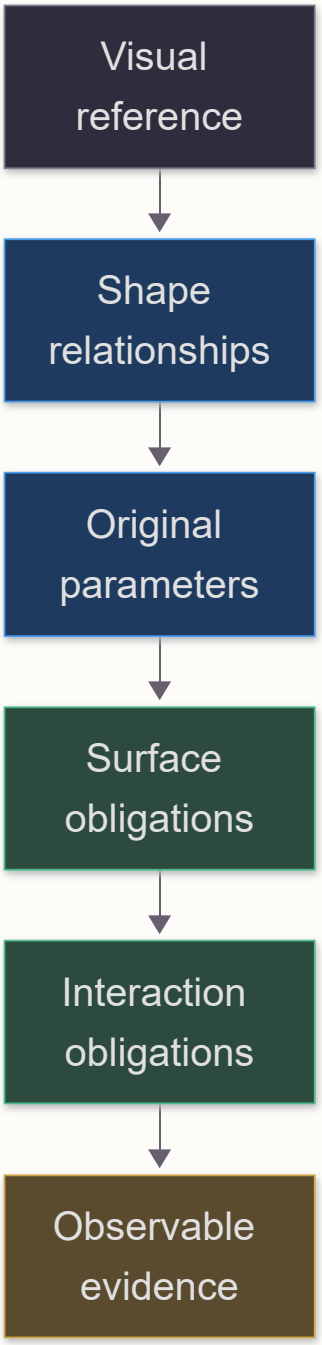


Figure MD02.1. A reference becomes useful when translated into independent parameters and bounded observations.

Choose landmarks before choosing detail density

Landmarks make comparisons interpretable. For this device, use the crown center, shoulder transitions, widest sides, bottom bow, screen center, control centers, and battery cover. Compare these before screw slots or chrome tint. A beautiful screw cannot repair an oval silhouette that violates the reference contract. This is a recommended analysis order, not a claim that the original implementation was written in this sequence.

The source's control table is especially valuable because it separates functional locations from visual commentary. A is at `(3.55,.50)`, B at `(2.76,-.02)`, Start at `(-2.64,-.98)`, and Select at `(-2.70,-1.5)`. Their socket radii differ from cap radii. That gives two independent checks: landmark position and local clearance. The D-pad opening is constructed from a scaled cross rather than a circular placeholder. Control landmarks (`src/handheld.js:434`).

Assets need their own contract. The brand loader decodes four supplied PNGs, whereas the factory derives a wordmark from `brandImage`. An independently modeled shell does not imply independently authored input artwork. Conversely, supplied artwork does not mean the shell came from an imported model. Document these ownership boundaries separately so a future agent does not confuse provenance of pixels with provenance of triangles. Asset inputs (`src/brand.js:1`).

MD02-W01 Derive a measurable speaker requirement

Suppose a reference review says “the grille should be recessed, not raised rails.” Translate that sentence into a surface experiment. First select a point near a slot center, `(3.02,-.79)`. Then select the neighboring solid face at `(3.02,-.87)`. Shoot both rays from the front along the device's negative local Z direction. Convert both hit positions back into root-local coordinates and subtract their depths. A positive face-minus-slot result shows a recession at these sample locations.

The existing browser case does exactly this kind of comparison and requires a difference above `.02` scene units. It is not a measurement of every point in all six slots. It does, however, reject a surface arrangement where both probes land on a raised decorative strip. Existing sample and threshold (`tests/browser/scene.spec.mjs:1370`).

An original pseudocode recipe extends the reasoning without pretending to be an existing test:

```
Pedagogical recipe; use a factory root with fresh matrices.
For each of six speaker slots:
  Raycast the center from local positive Z.
  Raycast a neighboring solid-face point.
  Convert both hits into root-local coordinates.
  Record faceZ minus slotZ and both mesh names.
Expect every chosen center to hit a lower receiving surface.
Reject missing hits; do not convert them into zero depth.
```

This recipe assumes the neighboring point really lies between slots, not inside a rotated aperture. The slots are angled `.23` radians, so a naive vertical offset can become ambiguous near their ends. Begin at the shared X center where the original comparison is understood, then derive offset points in the slot's local frame if broadening coverage. The experiment is geometric; it does not prove the opening looks good under every material or light.

Now connect the requirement to construction. The face receives a rounded hole path; the dark grille is a separate shape behind it. If the ray hits an underlying shell instead of the named grille, the depth result may still be positive. Recording hit names prevents silently accepting the wrong ownership. This is why a good modeling contract specifies both shape and observable component identity.

MD02-R01 Retry when a closer view changes the judgment

Imagine the front diagnostic looks convincing at thumbnail scale, but an oblique close-up makes the controls appear pasted on. Do not immediately increase bevel segments. First ask which obligation failed: missing cap curvature, missing socket clearance, floating marks, or wrong material response? The source uses a lathed cap and genuine A/B cutouts, so a current failure might instead arise from view angle, an altered transform, or loss of the intended floor depth.

Change one condition at a time. Hold the model pose and replace decorative lighting with neutral studio light for shape diagnosis. Then hold light and rotate to inspect depth. Finally restore the room to check presentation. The parent-provided atlas images in this book are diagnostic factory renders under neutral lighting; they are not historical screenshots or evidence of live-room behavior. That limitation is the reason for using them, not an apology for them.

The old geometry chapter records successive changes from raised markings through recessed shapes to recovered Arial contours. It demonstrates that “softer” can refer to color, outline, or shadow depth. A retry should therefore name the changed property explicitly. “Reduced groove depth while preserving contour and cap material” is reproducible; “made letters nicer” is not. Historical distinctions (`docs/book/chapters/04-handheld-geometry/README.md:39`).

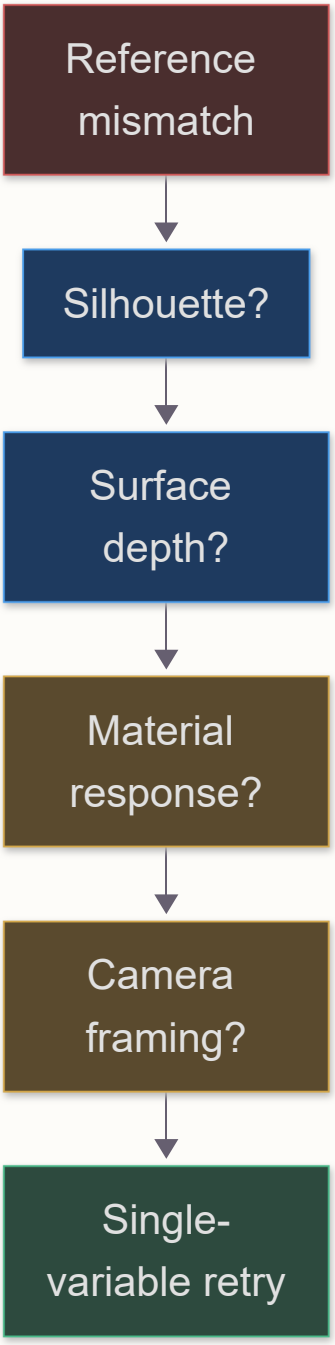


Figure MD02.2. Separate shape, material, and presentation hypotheses before deciding which parameter to change.

Precision without false metrology

Precision means controlling a defined quantity. It does not require pretending scene units are physical dimensions. A raycast depth comparison is precise in the model's coordinate system. A shortest-distance screen-edge fit is precise in sampled framebuffer pixels. A claim of hardware accuracy needs additional calibrated evidence that these sources do not supply. Keep the unit beside every threshold and explain the sample domain.

The LCD workstream is a useful precedent: it requires sampled image edges inside the geometry and a thin keyline, but explicitly calls the outline a fit rather than a guarantee for arbitrary shaders. Similarly, the player test changes native presets and checks that only CRT Geom uses curved geometry. That contract is conditional on the selected presentation, not an absolute requirement that every screen corner always be curved. LCD acceptance (`TODO/13_CURVED_LCD_BORDER.todo:17`); preset distinction (`tests/browser/player.spec.mjs:265`).

Choose comparisons that survive camera changes

A front silhouette comparison should use the device frame rather than the screen's current pixel rectangle. Perspective changes apparent spacing: a nearer shoulder can look wider without any geometry change. For a proposed parameter study, hold the camera fixed or compare orthographic diagnostic views, then use an oblique view to expose depth. This is a recommended experimental design, not a statement that the historical reference was calibrated.

Likewise, normalize a landmark comparison deliberately. Dividing every horizontal distance by shell width is useful for proportion, but it can hide an absolute clearance loss when the shell changes size. Record both the raw scene-unit gap and its normalized ratio. If the screen stays fixed while the case widens, these two quantities answer different questions: available space versus visual balance.

Do not treat every reference feature as mandatory. Some details may be occluded, licensed only for viewing, or ambiguous in perspective. Place those in an uncertainty list and author a bounded approximation instead of inventing unseen structure. That discipline is especially important for the simplified cartridge rear, whose absence of connector geometry is known from the factory rather than solved by imaginative reconstruction.

MD02-Q01 Define a contract for a wider handheld

Explained answer

A future agent proposes widening the shell by ten percent while leaving the screen unchanged. What observations should be required before accepting the change? Distinguish a visual redesign from faithful reconstruction, and explain why simply scaling the root is not the same experiment.

MD02-A01 Preserve the distinction between redesign and reconstruction

Back to question

A wider shell is a proposed extension unless new evidence establishes it belongs to the pinned design. Record it as such. Keeping the LCD unchanged means root scaling is unsuitable: root scaling also widens the screen, changes control spacing, stretches text, and alters all child geometry. Instead define which shell path coordinates change and which landmarks remain fixed. Then regenerate or reposition dependent openings deliberately rather than let them drift as an accidental side effect.

Acceptance should compare front and oblique silhouettes, distances from controls to shell edges, speaker clearance, shoulder alignment, and actual LCD visibility. It should also inspect rear cover placement and screw locations, because a wider front with an unchanged rear can look assembled from unrelated parts. Interaction targets need fresh projection checks in the host, although those runtime changes lie beyond this part's factory analysis.

The answer is not “all tests pass.” Existing tests sample the old arrangement and may miss a new collision. Derive new probe points from the changed geometry, preserve the original baseline as a separate specimen, and report which observations are source-grounded versus proposed. That is how a modeling contract supports future agents without falsely freezing every aesthetic decision into universal truth.

PART 1 / FOUNDATIONS AND THE GAME BOY ADVANCE

MD03

Coordinate systems and transform ownership

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Coordinate systems and transform ownership

The device is modeled in its own frame

Before editing any dimension, establish the local axes. The handheld's major outlines lie in XY. Positive X runs toward the action buttons, positive Y toward the cartridge edge, and positive Z toward the display side. The rear cover sits at negative Z. These conventions follow the factory positions, not a global law imposed by Three.js. Once the device is placed on a desk, its root may rotate; the device's front is no longer necessarily world positive Z. Front assembly (`src/handheld.js:462`); rear assembly (`src/handheld.js:631`).

Scene units are consistent within that frame but are not documented physical units. A value of `.075` can be a meaningful cap travel relative to a `.29` cap radius without being millimeters. Preserve this distinction in calculations. Ratios and clearances remain useful; undocumented conversions create false precision and can corrupt camera and light reasoning elsewhere in the book.

Geometry coordinates and object coordinates are separate layers. `solid()` extrudes a shape then translates its vertices by negative half the nominal depth. The returned mesh can therefore be positioned around a convenient center. Bevel thickness still extends the resulting geometry beyond the nominal extrusion interval. A mesh's `position.z` alone is not its front surface. Centered extrusion (`src/handheld.js:19`).

Ownership is the reason to use a hierarchy

The root owns placement of the entire device. A button owns its cap-relative lettering. The underside power group owns a track, slider, grip ridges, and printed label in a rotated frame. The rear engraving group owns the replacement cap, colored floor, and walls after turning them to face backward. These are not merely organizational containers: they encode which pieces inherit motion and orientation.

In mathematical terms, a local point reaches world space through the product of its ancestors' transforms and its own transform. Moving a parent changes every descendant consistently. Moving only a child changes that child's relationship to the assembly. The current Object3D documentation, consulted 2026-11-07, describes local position, quaternion, scale, and world matrices. That is present-day explanatory guidance; the pinned factory remains evidence for which hierarchy this project actually constructs.

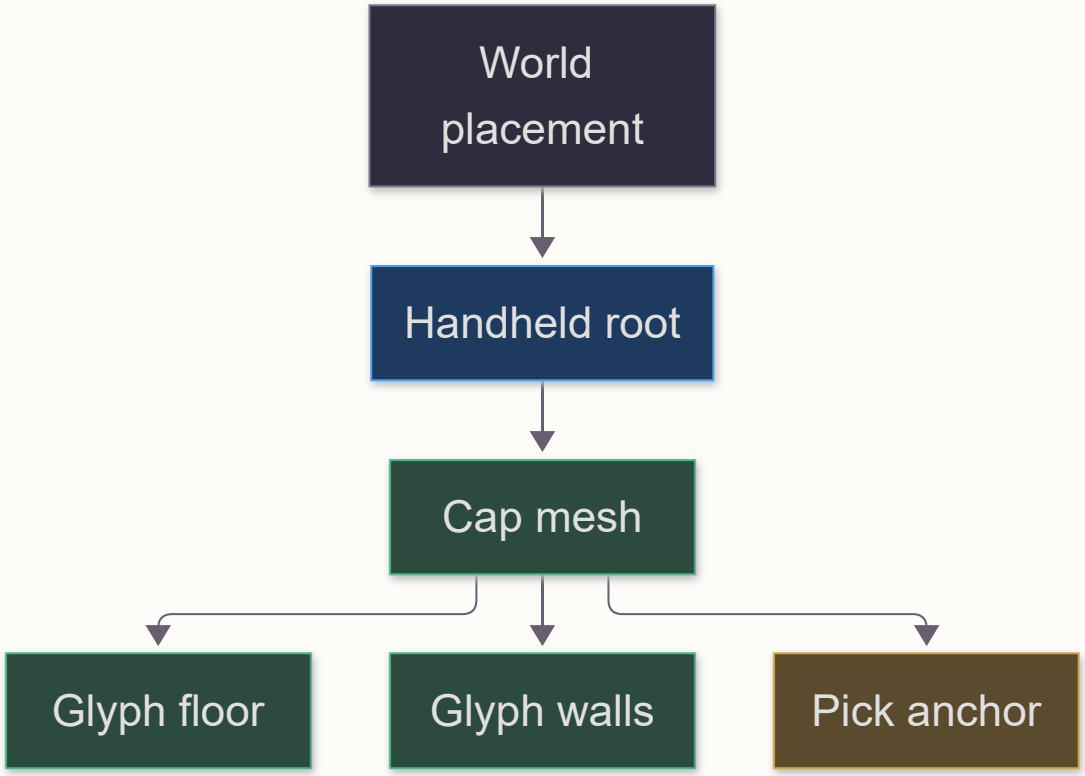


Figure MD03.1. Cap-relative marks inherit cap motion; an independently positioned root-level label would not.

The `control()` helper records both a moving mesh and an anchor. All four D-pad directions share the same mesh, but each gets a different anchor. This separates the object that rocks from the point used to locate a directional target. It also stores rest position and rest quaternion only when motion metadata is absent, avoiding four independent rest definitions on one D-pad. Control metadata (`src/handheld.js:553`).

Rotated geometry versus rotated objects

The cylinder helper rotates its geometry by ninety degrees around X so the cylinder's axis aligns with the device depth direction. That operation changes vertex coordinates before a mesh owns them. By contrast, the headphone mesh receives an additional object rotation later. Do not cancel or duplicate these transforms by intuition; inspect the helper and the call site together. A primitive's default axis is not necessarily the axis of the helper's output. Cylinder helper (`src/handheld.js:110`).

The power assembly makes the distinction concrete. Its group sits at `(1.55,-2.385,.12)` and has Euler rotation `(π/2,.15,0)`. Its children are authored as if looking straight at the local track face. The group turns that face toward the curved lower edge. This permits small parts to use simple local coordinates instead of repeating complicated world-space values. The code comment identifies local X as following the shell tangent; do not extend that into an unsupported exact surface-normal derivation. Underside assembly (`src/handheld.js:611`).

MD03-W01 Derive a transform-invariant depth probe

Suppose the console has been turned around for inspection and a test still needs the LCD's device-local depth. A world-space ray fixed along negative Z is now wrong. Instead define the ray origin and direction in the handheld frame, transform them into world space, intersect the world-space hierarchy, then transform the hit back. The root-local result remains meaningful despite the device pose.

The existing browser case follows this pattern: it updates world matrices, transforms a front origin with `localToWorld`, transforms the direction through `matrixWorld`, and calls `worldToLocal` on the hit. It then compares named surfaces and local depths. Existing transform-safe probe (`tests/browser/scene.spec.mjs:1370`).

The following original recipe assumes an already-created, finite, nonsingular factory root stored as `root`, and a browser import map. It is a reusable diagnostic rather than copied implementation. The expected center hit on an untouched handheld is `Live LCD`; its root-local Z is approximately `.782`, subject to float precision.

```
// Pedagogical diagnostic; root is an existing factory root.
const { Raycaster, Vector3 } = await import('three');
root.updateWorldMatrix(true, true);
const origin = root.localToWorld(new Vector3(0, .22, 4));
const direction = new Vector3(0, 0, -1)
  .transformDirection(root.matrixWorld);
const hit = new Raycaster(origin, direction)
  .intersectObject(root, true)[0];
if (!hit) throw new Error('No front surface');
console.log(hit.object.name);
console.log(root.worldToLocal(hit.point.clone()).z);
```

Why clone the point? Coordinate conversion mutates the vector passed to it. Keeping the original world-space point preserves evidence for later projection or distance analysis. Why update ancestors as well as children? A root can itself be attached to a moving parent. Stale matrices make an otherwise correct coordinate derivation produce inconsistent observations.

Under nonuniform scale, the ray still follows the transformed local depth direction, but Euclidean world distances no longer equal local distances. Compare local Z to local Z, not a ray distance in world units. Surface normals require their own transformation rule; a position transform is not a general normal transform. Avoid introducing nonuniform root scale merely to fit a viewport, because it changes the physical interpretation of every ratio.

MD03-R01 Retry when lettering stays behind during a press

A modified A button moves inward but its letter appears to float in place. Inspect parentage before changing Z offsets. The source adds cap face, floor, and walls as children of the button, so all inherit its motion. If an extension attached a new label to the handheld root using a copied current position, that copied position describes only one pose. It does not encode ownership. Reparenting with an explicit local transform is the conceptual repair.

Now suppose the parentage is correct but the letter slides sideways during compression. Check whether motion was applied in world space while the cap's rest position was recorded locally. The factory stores `restPosition` from `mesh.position.clone()` and a local travel vector. Runtime consumption lies beyond this chapter, but the factory contract is unambiguous: these values belong to the parent frame of that mesh, not the camera frame.

The original book describes markings as manufactured relief rather than detached decoration and distinguishes actual mesh motion from visual effects. That historical explanation supports the ownership requirement but does not replace reading current hierarchy construction. Earlier explanation (`docs/book/chapters/04-handheld-geometry/README.md:26`).

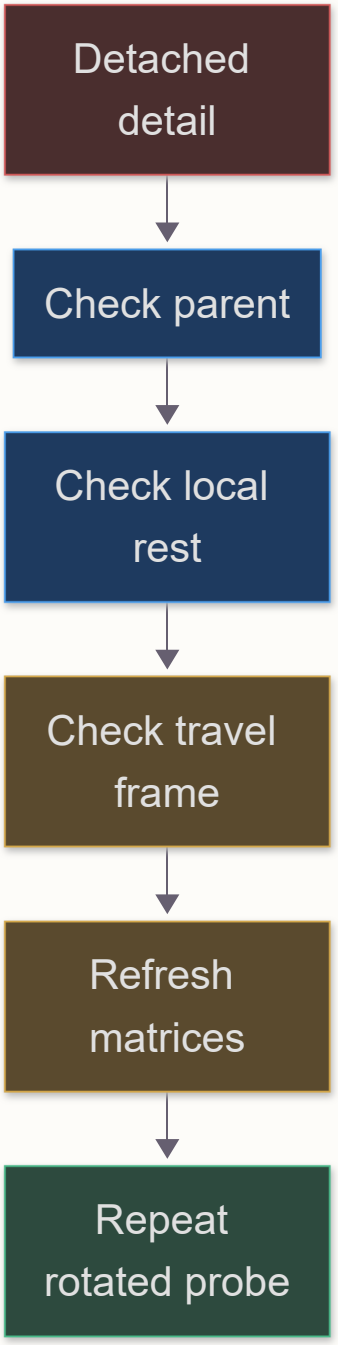


Figure MD03.2. A transform failure is investigated through ownership and coordinate frames before appearance parameters.

Pivots, mirrored views, and labels

The rear engraving group rotates around Y by π and starts at the rear surface of the battery cover. Its negative local groove depth therefore points inward relative to the backward-facing local artwork. Reading only the sign of a child Z value would give the wrong world-space interpretation. Follow the parent rotation before declaring a floor outside or inside. Rear transform (`src/handheld.js:285`).

Text has a second coordinate conversion: Canvas Y increases downward, whereas the glyph's geometric Y increases upward. The alpha projection explicitly uses `.5-py/height`. That sign change is not a rear-view mirror operation. Confusing the two can reverse counters or vertically invert a label while leaving its bounding rectangle correct. Brand decoding belongs to a separate input stage and cannot correct an erroneous projection sign. Glyph projection (`src/handheld.js:232`); decoded inputs (`src/brand.js:6`).

The reference contract's front/back inspection requirement makes these distinctions operational. A model that looks correct only from its initial camera pose fails that contract even if every mesh has plausible coordinates. Check at least one turned pose whenever adding a detail to a rotated subgroup. Inspection obligation (`TODO/07_REFERENCE_SHAPE.todo:18`).

Avoid compensating transforms that erase useful origins

A tempting repair for a misplaced detail is to bake its current world transform into its vertices and reset its object position. That can preserve one visible pose while destroying a useful local origin. For a cap, the origin is part of the motion contract: rest position, travel, lettering offsets, and anchors all assume a coherent parent frame. Baking only one component turns future edits into compensating offsets scattered across several objects.

Prefer a named subgroup when a set of details needs a new pivot. Place the subgroup at the desired mechanism center, express children relative to it, and verify that the rest pose is unchanged before animating. This is a proposed extension technique; the pinned factory already uses groups for the underside mount and rear engraving but does not expose a generalized pivot utility.

There is also a numerical reason to keep measurements local. Subtracting two large world coordinates can magnify floating-point sensitivity when the useful gap is tiny. Here the relevant clearances are small fractions of the device width. Converting both points back into the device frame makes the quantity being compared explicit and avoids confusing a global placement change with a surface-depth change. It does not create physical accuracy, but it improves the clarity and repeatability of model-space diagnostics.

MD03-Q01 Move the underside switch without moving its label

Explained answer

A redesign moves only the power slider along its track. Which children should follow the slider, which should remain on the mount, and how would you detect an accidental double transform? Explain the answer using the existing ownership structure rather than proposing a new global-coordinate table.

MD03-A01 Separate the moving part from its mounting frame

Back to question

The five grip ridges belong to the slider and must follow it. The track and POWER label belong to the mount and should remain fixed while the slider moves. Moving `powerMount` would move all of them and relocate the entire assembly; moving `power` changes only the slider and its descendants. This division is visible directly in the factory's `add()` calls, so it is stronger evidence than interpreting a screenshot.

An accidental double transform occurs when a child is assigned already-transformed world coordinates and then inherits the same mount transform again. Detect it by comparing the child's local position to the simple authored offsets, then comparing world positions before and after a known mount rotation. A ridge should remain at the same slider-relative location through that rotation. A label should remain mount-relative but not slider-relative.

Use the transform-invariant probe from the worked example to test the physical surface, and separately inspect the host's projected target if the change reaches integration. The browser suite includes underside-orientation and target checks, but this chapter did not execute them. The proper conclusion is a derived ownership rule and an available verification path, not a new claim of working interaction. Underside verification context (`tests/browser/scene.spec.mjs:846`).

MD04

The procedural geometry toolkit

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD04 / THE PROCEDURAL GEOMETRY TOOLKIT

Contents

The procedural geometry toolkit

Match the representation to the geometric problem

Procedural modeling becomes manageable when a small toolkit expresses recurring construction patterns. This project uses planar paths for outlines, extrusions for thickness and bevels, lathes for rotationally symmetric caps, and explicit triangle buffers for groove walls. It does not solve every object with the same primitive. Choosing the representation is consequential: a texture can describe color but cannot create an inward raycast hit; a dense sphere cannot conveniently preserve a flat manufactured face with exact holes.

The smallest reusable unit is `roundedShape(width,height,radius)`. It starts at the lower edge of a centered XY rectangle, connects straight segments with quadratic corner curves, and returns a shape. `solid()` extrudes that shape with one depth step, three bevel segments, and eighteen curve segments, then translates it by half its depth. `box()` composes these helpers. These operations are the actual toolkit, not an inferred CAD workflow. Core helpers (`src/handheld.js:4`).

The rounded corners are quadratic curves, not mathematically exact circular fillets. The radius argument controls where the straight segment ends and the corner control lies. Its practical meaning is a visual corner size. The helper does not clamp unreasonable radii; a proposed reusable API should validate positive dimensions and a radius compatible with the smaller dimension. That validation is a recommendation, not behavior already present.

Extrusion produces more than nominal depth

For nominal depth `d`, the helper recenters the extrusion's main interval around zero. Bevel thickness adds material beyond that interval. Therefore a box placed at Z `.3` with depth `.36` and bevel `.1` is not bounded simply by `.12` and `.48`. Its generated bounding box is the authoritative envelope. This matters when stacking the front face above the shell or locating a groove at the rear surface.

Bevel size and bevel thickness share the same argument here, but they describe different effects. Size changes the planar edge treatment; thickness changes depth. Keeping them equal is a compact stylistic convention, not a physical manufacturing law. Larger bevels can round a narrow feature out of existence or make a recess cover the surface it was meant to frame. More segments smooth tessellation; they do not repair an inappropriate bevel envelope.

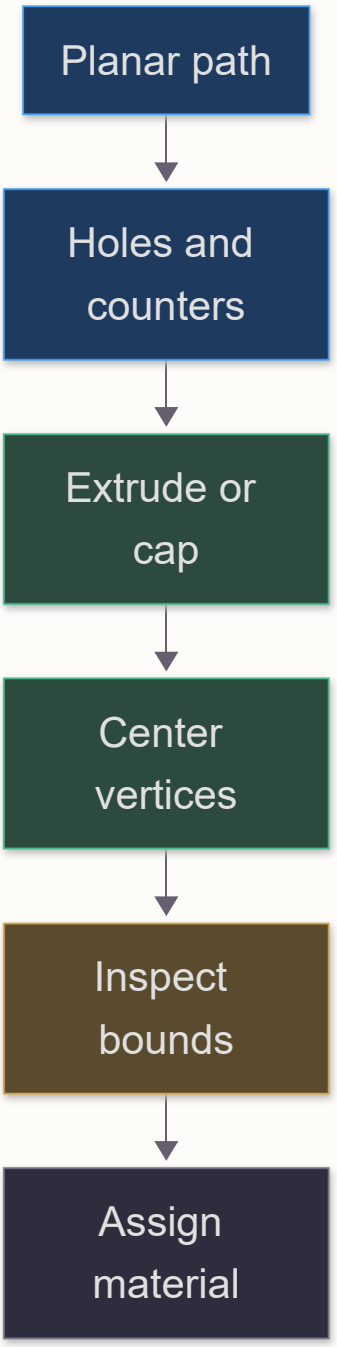


Figure MD04.1. Outline, topology, envelope, and material are separate decisions in the reusable geometry pipeline.

Holes need topology, not dark paint

A Three.js shape can contain hole paths. The front face uses them for controls and speaker apertures. Molded glyphs use counter paths for enclosed letter spaces. The rear engraving requires more elaborate nesting because supplied artwork can contain islands inside holes inside larger regions. Preserving this hierarchy is essential: a letter's counter is solid cap material in a recessed inscription, not another dark cavity.

The rear algorithm sorts contours by absolute area, identifies a containing parent using bounds plus point-in-polygon testing, assigns a nesting level, and adjusts winding according to parity. It then uses even and odd levels for different surface roles. Winding affects which side of a triangle or contour is interpreted as front-facing; nesting affects which regions are material versus void. These related concepts must not be collapsed into “reverse every path until it looks right.” Nested regions (`src/handheld.js:255`).

The old geometry chapter documents why counter islands and inward hits mattered in the lettering revisions. It is historical explanation, not a replacement specification for the pinned algorithm. When extending the toolkit, follow the current contours and surface construction, then use older records to understand which visible failure modes motivated additional checks. Earlier relief explanation (`docs/book/chapters/04-handheld-geometry/README.md:39`).

Lathes preserve cap profiles efficiently

The button cap is a lathe generated from seven profile points and forty radial segments. The profile closes toward the center near its bottom and dome. Rotating the resulting geometry around X aligns its depth with device Z. This is economical because the same profile governs every azimuth; editing one radial section changes the entire cap consistently.

Action buttons need an open center region for their reconstructed cap faces. The factory removes only triangles touching the upper center pole, retaining the original rim normals. It does not flatten the whole lathe into a cylinder. The later face uses a shallow radial dome and custom normals to continue the cap's appearance. That preserves the rim's highlight behavior while allowing genuine holes. Lathe and center removal (`src/handheld.js:116`).

MD04-W01 Build and measure a centered rounded solid

The following original browser-console recipe uses the exported helper, not a copied implementation. It assumes the project's import map is active and does not require the brand assets because it never calls the complete handheld factory. It is safe to inspect without attaching anything to the running scene.

```
// Pedagogical geometry experiment.
const { roundedBox } = await import('./src/handheld.js');
const geometry = roundedBox(2, 1, .2, .12, .03);
geometry.computeBoundingBox();
const b = geometry.boundingBox;
console.table({
  minZ: b.min.z,
  maxZ: b.max.z,
  centered: Math.abs(b.min.z + b.max.z) < 1e-6
});
console.assert(b.max.z > .1);
console.assert(b.min.z < -.1);
geometry.dispose();
```

The nominal un-beveled half-depth is `.1`. The expected bounds extend beyond it because bevel thickness is `.03`; symmetry remains approximately centered. Inspecting the actual values is better than asserting a long decimal literal because float-buffer storage and tessellation are implementation details. The meaningful invariant is symmetric extra extent around the nominal interval.

Now derive a stacking rule. If a second surface must sit entirely in front of this object, its nearest depth must exceed the first object's maximum depth in their common frame. Comparing only the two mesh centers is insufficient. If either object is rotated, transform its geometry bounds or use an appropriate object-space measurement before comparing. For overlapping manufactured layers, intentional penetration may be valid, but record that choice rather than calling every intersection a bug.

Finally vary bevel from `.03` to zero and repeat. The helper disables bevels when the argument is not positive, so the nominal depth becomes the principal Z extent. A wider-looking silhouette after enabling bevel is not a camera error. This derivation teaches an immediately reusable distinction between an authored dimension and the generated envelope.

Explicit triangles require explicit normal reasoning

Groove walls use two triangles per contour edge. For edge endpoints A and B, the vertices connect the surface edge to its deeper floor. The order of the cross product of triangle edges determines the face normal. Reverse triangle order and the front face reverses; merely changing material color cannot fix that orientation. The factory computes vertex normals after filling the position buffer. A/B wall triangles (`src/handheld.js:347`).

The latest BufferGeometry documentation, consulted 2026-11-07, distinguishes indexed triangles from non-indexed groups of three positions and explains normal averaging. For these explicit non-indexed walls, duplicated positions do not automatically share smoothed normals. That is

useful for crisp cavity walls. For a smooth shell, blindly duplicating every triangle would create faceting unless normals were assigned deliberately. This is recommendation-level reasoning grounded in the representation, not proof of an undocumented optimization.

MD04-R01 Retry when a new glyph loses its enclosed center

Suppose a proposed letter P renders as a filled disk-shaped patch rather than a stem with an enclosed counter. First determine which alphabet is involved. `moldedShapes()` only knows the explicitly listed polygon glyphs; it is not a general font engine. `actionButtonGlyph()` explicitly accepts only A and B. Passing P is outside both current contracts unless its polygons are added to the first or the second is redesigned.

If a new contour importer is being developed, inspect closed loops before triangulation. Record contour count, signed area, containment, and expected counter count. If the boundary graph is not closed, the extractor throws rather than silently returning a partial inscription. If the graph closes but nesting is wrong, a counter can become a cavity floor. If topology is right but the wall vanishes from one side, investigate winding and material sidedness.

The reference workstream's insistence on actual apertures is relevant here: a dark texture that visually suggests a counter cannot substitute for correct geometry when raycasts and oblique views must observe it. Geometric obligation (`TODO/07_REFERENCE_SHAPE.todo:18`).

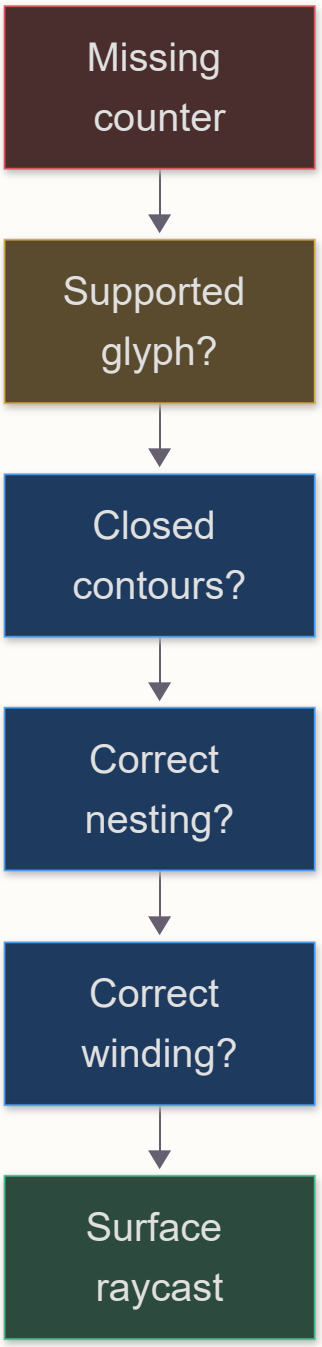


Figure MD04.2. A missing counter can originate in the input contract, contour graph, region classification, or triangle orientation.

Sampling changes topology as well as smoothness

The alpha contour extractor does more than outline occupied pixels. It examines four alpha samples per cell, finds threshold crossings on cell edges, assigns shared edge keys, and stitches a graph whose vertices must each have two neighbors. Crossings interpolate using the threshold and endpoint alpha values. This gives subpixel boundary positions rather than a staircase following whole pixel corners. Extractor (`src/handheld.js:174`).

Cells with four crossings need a connectivity decision. The implementation chooses paired edges according to the first corner's threshold state. That is a specific saddle-case rule, not a general proof that arbitrary noisy artwork will produce the intended topology. Very thin bridges or diagonal contacts can therefore change which regions connect when resolution or threshold changes.

For a new importer, save contour statistics before building triangles. Closed-loop count, signed area, minimum edge length, and nesting depth can reveal a problem earlier than a shaded render. A smooth-looking image is insufficient if a single bridge merges two cavities or removes a counter. Conversely, more sampling is not automatically safer: it can recover tiny noise islands and greatly increase wall triangles. Choose resolution against an explicit detail budget and preserve the original source image for comparison.

MD04-Q01 Choose between a decal and a cut

Explained answer

A future agent must add a tiny serial number to the cartridge and a deep logo to a replacement battery cover. Which representation should each use, and what evidence would justify changing that choice? Include performance and ownership tradeoffs rather than treating “more polygons” as automatically better.

MD04-A01 Spend geometry where shape must be observable

Back to question

A small serial number can reasonably begin as a transparent canvas-backed label if its purpose is readable color on an otherwise unchanged surface. It is cheap to author, supports arbitrary browser text, and does not require a complete polygon alphabet. The existing cartridge title uses this approach. Its limits include font dependence, transparent sorting, finite texture resolution, and no physical relief. Attach it to the cartridge so it follows the prop rather than the scene.

A deep battery logo needs geometry if the requirement includes inward ray hits, edge shadows, or visible walls under oblique inspection. Reuse a contour-and-floor construction with explicit nesting rather than placing a dark picture below an uncut cap. Preserve the surrounding bevel and material relationship; a deep logo must not accidentally replace the entire cover with a different surface.

Evidence can reverse either initial choice. If the serial number must be felt visually through grazing highlights, shallow relief may become justified. If the logo is only a distant printed label and never examined obliquely, geometry may be needless cost. The decision follows the observation contract. Existing browser surface probes demonstrate how specific geometry properties can be tested, but they do not automatically cover newly added details. Current surface probes (`tests/browser/scene.spec.mjs:1370`); browser asset decoding boundary (`src/brand.js:6`).

PART 1 / FOUNDATIONS AND THE GAME BOY ADVANCE

MD05

GBA shell, silhouette and layered construction

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD05 / GBA SHELL, SILHOUETTE AND LAYERED CONSTRUCTION

Contents

GBA shell, silhouette and layered construction

Start with the silhouette that carries identity

The shell is not a scaled ellipse or a rounded rectangle. Its path explicitly describes a raised central crown, descending shoulders, broad sides, sloping lower corners, and a bowed bottom. This matters because the historical reference correction rejected an earlier oval impression. The pinned source's curves are the reconstruction target; the reference record explains the type of mismatch being corrected without proving physical accuracy. Historical shape correction (`TODO/07_REFERENCE_SHAPE.todo:18`).

Begin at the upper-left side point `(-4.1,1.45)`. A short quadratic transition reaches `(-3.78,1.8)`, then line and curve segments climb toward the central crown. The top spans from approximately `x=-2.08` to `x=2.08` at `y=2.42`. The right half descends through the shoulder before bowing toward the lower-right corner. The bottom curve connects `(2.12,-2.17)` to `(-2.12,-2.17)` using `(0,-2.61)` as its control. The left side returns to the start. Complete shell path (`src/handheld.js:410`).

Those coordinates are scene units and Bézier controls, not measured hardware dimensions. In particular, the curve does not necessarily pass through its control point. Editing the bottom control changes curvature without placing the bottom at `-2.61`. This is a practical reason to distinguish path parameters from sampled silhouette bounds whenever documenting exact numbers.



Figure MD05.1. Diagnostic factory render under neutral studio lighting, used to inspect casing layers and silhouette; not a historical design screenshot or live-room proof.

Understand the crown as transitions, not a point list

The top's shape is communicated by changes in slope. Straight central segments read as a crown; the small curved joins soften transitions; lower shoulder slopes lead toward the side lobes. If an extension simply raises every top vertex, it may preserve width while destroying the stepping relationship. If it increases curve sampling, it makes the same incorrect outline smoother.

A recommended reconstruction sequence is to plot the major endpoints first, add transition controls second, and only then extrude. This is a teaching workflow derived from the source structure, not a recovered authoring timeline. Compare the outline as a flat silhouette before tuning materials. Otherwise a specular highlight can hide a kink in one view and exaggerate it in another.

The path is largely mirrored in X, which supplies a useful diagnostic. Sample the outline and compare left and right envelopes at corresponding heights. Do not require the whole assembled console to be symmetric: the control layout, speaker, ports, and switch are intentionally not mirrored. Symmetry is a property of the base shell construction, not a universal model requirement.

The casing is a stack of related solids

Four named pieces establish the primary depth structure. The seam uses nominal depth `.08`, bevel `.08`, and center Z `.05`. Rear and front shells each use depth `.36` and bevel `.1`, positioned at `-.26` and `.3`. A thin perforated face uses depth `.04`, bevel `.008`, and center `.592`. These are overlapping visual layers, not a watertight manufacturing assembly with documented wall thickness. Layer construction (`src/handheld.js:450`).

Why keep the face separate? It allows the visible front surface to carry apertures while the broader shell establishes body thickness and rounded edges. Dark receiving surfaces behind holes create visible recesses without boolean subtraction through every underlying shell. This is an efficient visual model, but it has a limitation: an aperture is not automatically a continuous hollow tunnel through the casing. Backside inspection of the internal stack is not equivalent to a solid-model manufacturing check.

The material ownership supports the stack. Front shell and face share the purple material, while the rear and battery cover share rear plastic. The seam has a darker rough material. Changing the shared purple changes multiple surfaces together; changing one mesh's material reference can deliberately break that relationship. Record whether a finish experiment edits a shared material or assigns a clone. Material definitions (`src/handheld.js:392`).

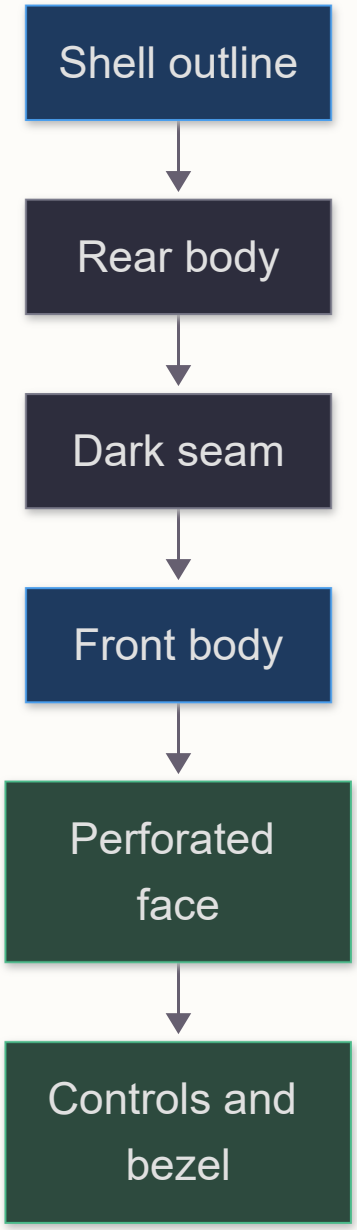


Figure MD05.2. The visual shell is built as a depth stack; the diagram is assembly logic, not a claim of disjoint physical solids.

MD05-W01 Calculate why the front face clears the body

Use the centered-extrusion convention to derive nominal envelopes. The front body at `.3` with depth `.36` occupies `.12` to `.48` before bevel thickness. With bevel `.1`, the generated extreme reaches approximately `.58`. The face at `.592` with depth `.04` spans `.572` to `.612` before its `.008` bevel, placing its outer face near `.620`. Thus the thin face intentionally overlaps the main body while presenting a higher visible surface.

That overlap is not an error by itself. It prevents a crack between two independently tessellated surfaces and lets holes reveal lower surfaces. The important requirements are stable depth ordering and the absence of an unwanted cap covering a feature. Exact generated bounds should still be measured rather than assumed; the helper's geometry translation and bevel options establish the calculation. Extrusion convention (`src/handheld.js:19`).

The following original console recipe assumes the project's page is served and the model factory may be instantiated independently. It loads the public brand images because the complete factory requires them. It does not attach the new model to the application or mutate the live model.

```
// Pedagogical envelope diagnostic.
const { loadBrand } = await import('./src/brand.js');
const { createHandheld } = await import('./src/handheld.js');
await loadBrand();
const specimen = createHandheld();
for (const name of ['Front shell',
  'Front face with speaker apertures']) {
  const mesh = specimen.root.getObjectByName(name);
  mesh.geometry.computeBoundingBox();
  const b = mesh.geometry.boundingBox;
  console.log(name,
    mesh.position.z + b.min.z,
    mesh.position.z + b.max.z);
}
```

Expected output places the face's outer bound ahead of the body's outer bound. The recipe assumes these two meshes are unrotated direct children, as in the pinned factory. Reusing the arithmetic on a rotated subgroup without transforming bounds would be incorrect. The diagnostic allocates an isolated model; a repeated production tool should explicitly manage disposal of its unique geometries, materials, and textures.

Now lower the face center hypothetically by `.05`. Its front bound approaches the body's front bound, and holes may become visually ambiguous or disappear behind the body. The correct diagnosis is a layer-order change, not loss of hole paths. This example shows how an invisible aperture can exist topologically while failing the visible-surface contract.

Apertures are manufactured cues with local rules

Six speaker slots share width `1.02`, height `.065`, and angle `.23`, with vertical centers descending by `.16`. Their hole paths are rotated around their own origin before translation into the face. The dark slot surfaces repeat the same angle and dimensions behind the face. Using the same parameter table avoids a common mismatch where the hole rotates but its backing remains horizontal. Speaker definitions and holes (`src/handheld.js:431`).

Control openings use a cross enlarged by `1.09` and circular paths wider than their caps. These openings are not interchangeable with raised socket lids. Historical commit `706563fe0355d6455a3c1ba1ff8fd9bafd3fa2a4` replaces such lids with face holes and receiving floors. That diff supports a concrete construction change; it does not license a claim that every possible lighting condition is now correct. Scoped historical change.

MD05-R01 Retry when a wider bevel hides a slot

Suppose an extension increases the thin face bevel to soften its appearance, and the narrow speaker openings nearly close. Investigate planar bevel expansion relative to slot height. A `.065`-high slot has only `.0325` half-height before accounting for curvature and bevel behavior. Increasing a bevel by a value comparable to that scale is not a small edit simply because it looks numerically small beside an eight-unit-wide shell.

Retry by holding path dimensions fixed and restoring the original bevel. If visibility returns, the hypothesis is supported. Then decide whether to reduce bevel, widen the aperture, or separate edge rounding from thickness. Widening is a redesign of the grille proportions; reducing bevel preserves the existing silhouette more closely. Do not silently change both and claim to have identified the cause.

The browser surface test offers an objective partial check: neighboring solid face depth must exceed the selected slot depth by more than `.02`. A neutral diagnostic render adds oblique readability, while the real room checks lighting response. These observations answer different questions. Depth assertion (`tests/browser/scene.spec.mjs:1382`).

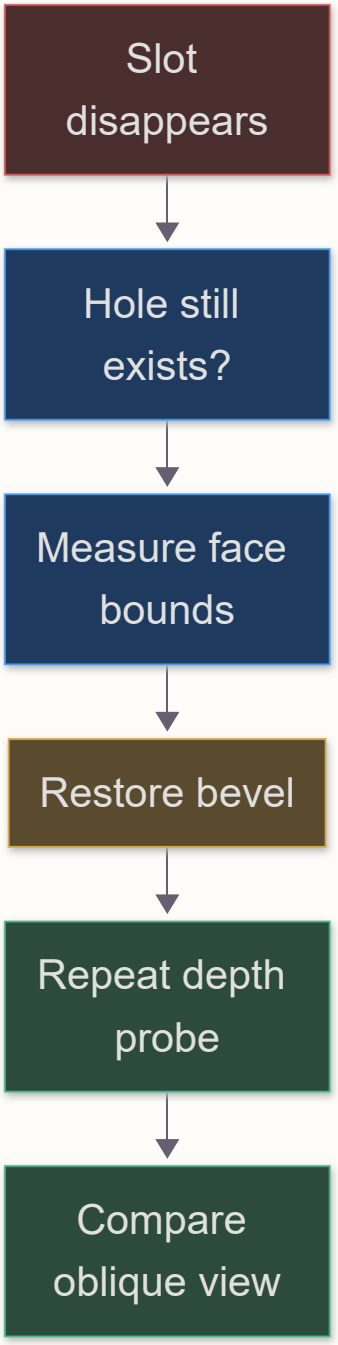


Figure MD05.3. A narrow opening can fail through bevel envelope or layer occlusion even when its path remains valid.

Derive the bottom bow instead of guessing its depth

The lower central segment is a quadratic Bézier with equal endpoint heights. At its midpoint, the weights are one quarter, one half, and one quarter. Applying them to endpoint Y values of negative two point one seven and control Y of negative two point six one gives a midpoint near negative two point three nine. This is already enough to show why reading the control point as the shell's lowest point would overstate the bow.

Because the endpoint X values are equal and opposite and the control X is zero, the midpoint also lies on the centerline. For this symmetric quadratic, the vertical derivative vanishes there. That makes it a useful local landmark for comparing the lower outline before extrusion. Bevel expansion then changes the rendered envelope, so keep the planar landmark separate from the final mesh bound.

This derivation suggests a disciplined parameter experiment. Move only the bottom control downward, leave endpoints fixed, and predict that the center deepens while the lower corner positions remain unchanged. Then sample the generated path and verify the prediction. If the whole bottom translates instead, the edit changed endpoints or a parent transform as well. This isolates curvature adjustment from overall scaling and makes “more bowed” a reproducible instruction rather than an aesthetic guess.

MD05-Q01 Is a watertight shell automatically a better model?

Explained answer

An agent proposes replacing the four casing layers with one boolean-subtracted watertight mesh. Explain what this could improve, what it could break, and which claims cannot be resolved from the inspected sources. Include the effect on material ownership and the existing diagnostic names.

MD05-A01 Representation quality depends on the contract

Back to question

A unified solid could improve internal consistency, remove hidden intersecting surfaces, and support physical cross-sections or manufacturing export. Those benefits matter if the new contract requires them. This project instead exposes an inspectable visual device whose named pieces support material assignment and targeted probes. A replacement must preserve front/back appearance, aperture behavior, movement ownership, and accessible integration rather than merely reduce the number of objects.

Boolean output can introduce sliver triangles, shading discontinuities, or difficult material groups. A single mesh also removes convenient names used by current tests unless equivalent component metadata is retained. The rear cover's separate geometry is modified by its engraving algorithm; merging it into the shell would require redesigning that operation. None of these risks proves booleans are wrong, but they establish work that a “cleaner mesh” claim must account for.

The sources do not provide physical wall thickness, tolerances, or watertightness acceptance criteria. Therefore one cannot certify the replacement as a more accurate manufactured replica from these records alone. Preserve the layered model as the pinned baseline, describe a unified solid as an extension, and derive fresh observations before accepting it. The brand loader and public delivery map further remind us that a shell rewrite must not inadvertently sever asset inputs or static module delivery. Brand initialization (`src/brand.js:6`); public source delivery (`scripts/site-files.mjs:18`).

MD06

GBA LCD, curved glass and frame

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD06 / GBA LCD, CURVED GLASS AND FRAME

Contents

GBA LCD, curved glass and frame

Name the actual modeled surface

The chapter title includes glass because a screen assembly must be discussed as a visual surface, but the inspected GBA factory does not create a separately named transmissive glass slab. It creates a bezel surround, charcoal bezel, LCD recess, and live LCD mesh. Do not infer thickness, refraction, or a transparent material merely from the word glass. The visible display uses `MeshBasicMaterial` with tone mapping disabled and both shadow flags false. Actual assembly (`src/handheld.js:467`).

That distinction protects the image pipeline. A live picture should not become brighter or darker merely because a room lamp illuminates its material. The factory's basic material provides an unlit display surface; the runtime later supplies the picture. This chapter analyzes the geometry and initial material, not the complete runtime texture lifecycle. The player test separately checks that native video directly supplies the material map rather than a host-side replacement CRT effect. Direct-video assertion (`tests/browser/player.spec.mjs:265`).

A curved outline is not a curved framebuffer mapping

`createLcdGeometries(curved)` fixes width at `4.05` and height at two thirds of width, giving `2.7` scene units. Flat mode uses a plane. Curved mode triangulates a specially authored shape with thirty-two curve segments. Both represent an image-bearing face in XY; the curved mode changes its boundary, not the Z curvature of a thick lens. LCD geometry factory (`src/handheld.js:49`).

The outline is a series of quadratic curves. Its bottom begins near `(-.451w,-.483h)` and bows through a control at `y=-.521h`; sides use controls near `x=±.503w`. These coefficients describe a fitted perimeter around the native CRT picture. They are not a physical lens equation or a universally correct CRT model. Like the shell's controls, curve control points are not guaranteed sampled extrema.

The decisive operation follows triangulation: every curved-screen UV becomes `u=x/w+.5`, `v=y/h+.5`. This preserves rectangular framebuffer coordinates across the clipped perimeter. It avoids applying another image warp after the native CRT shader has already shaped the picture. Geometry chooses where the image can be seen; UVs choose which image pixel appears there. Those are different responsibilities. Boundary and UV construction (`src/handheld.js:34`).

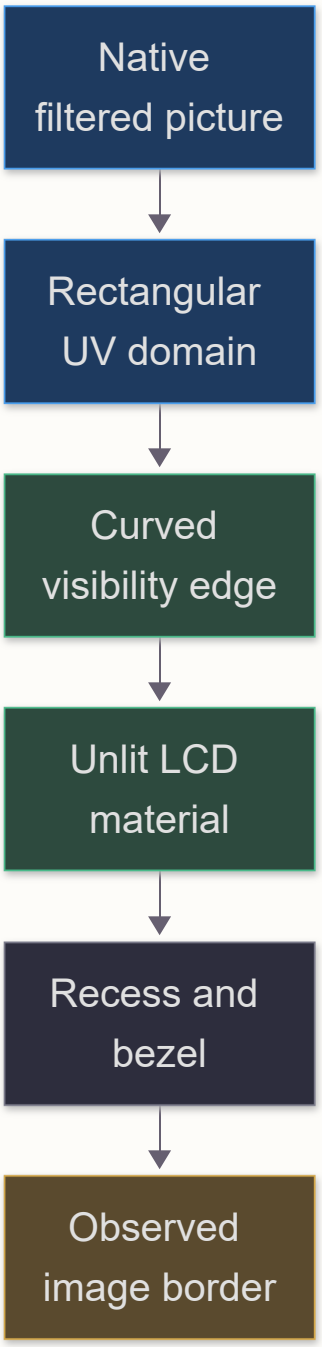


Figure MD06.1. The fitted boundary clips visibility while preserving the original image-coordinate mapping.

Stack the frame without burying the display

The surround is `4.47×3.39` with nominal depth `.045`; the charcoal bezel is `4.35×3.27` with nominal depth `.055`. Their centers share Y `.1`, while recess and screen share Y `.22`. Z centers progress from `.65` and `.685` to `.748` and `.782`. These values are not interchangeable: changing the screen center without the recess changes alignment; changing the bevel can alter occlusion without changing any center.

The recess outline is enlarged by `.12` in both width and height, then extruded with depth `.015` and bevel `.009`. In flat mode it uses a rounded rectangle with corner argument `.025`; in curved mode it uses the matching CRT outline. Thus the surrounding frame follows the effective screen mode. An overly thick recess bevel could cover the image plane, a problem the reference record explicitly notes was corrected during earlier shape work. Recorded occlusion correction (`TODO/07_REFERENCE_SHAPE.todo:43`).

The source revision introducing curved geometry had smaller dimensions than the pinned implementation. Historical commit `72da407f03ddf872bdd7b3c19e4a9bba92445998` is evidence for the change from rectangular border to fitted boundary and explicit UV restoration. It is not the source of today's `4.05` width. Keep historical values beside their commit rather than copying them into current parameter tables. Scoped historical diff.

MD06-W01 Derive the no-second-warp invariant

At the screen center, `x=0,y=0`, so the mapping gives `u=.5,v=.5`. At an arbitrary point `x=.25w,y=-.2h`, it gives `u=.75,v=.3`. These coordinates are independent of whether the outer edge is rectangular or bowed. A native pixel at three quarters of framebuffer width is not moved again simply because the border is curved.

The following original browser-console recipe is runnable without brand loading or a ROM. It checks all vertices of both modes against the same formula. The expected maximum error is near float precision, below `1e-6`; the recipe does not assert pixel content or rendered border quality.

```
// Pedagogical UV invariant check.
const { createLcdGeometries } =
  await import('./src/handheld.js');
for (const curved of [false, true]) {
  const { screen, recess } = createLcdGeometries(curved);
  const p = screen.getAttribute('position');
  const uv = screen.getAttribute('uv');
  const { width: w, height: h } = screen.userData.viewport;
  let error = 0;
  for (let i = 0; i < p.count; i++) {
    error = Math.max(error,
      Math.abs(uv.getX(i) - (p.getX(i) / w + .5)),
      Math.abs(uv.getY(i) - (p.getY(i) / h + .5)));
  }
  console.log({ curved, error });
  console.assert(error < 1e-6);
  screen.dispose(); recess.dispose();
}
```

Why check all vertices if the formula is already visible? The check catches future geometry replacements that inherit default shape UVs or change dimensions without updating viewport metadata. It is cheap and independent of screenshots. It still cannot detect clipping of native lit pixels: a perfectly linear UV mapping can sit on an outline that is too small.

For clipping, the existing browser case samples lit edges from the actual native output, raycasts corresponding positions onto the screen, and measures distance to a sampled perimeter. It requires more than two hundred samples, zero sampled clipping, maximum keyline distance below nine native pixels, and UV error below `.00001`. Those are separate invariants, not four names for the same visual judgment. Edge-fit assertions (`tests/browser/scene.spec.mjs:1475`).

Why shortest distance matters near a curved tangent

A horizontal gap from an image edge to a curved border can grow large near a nearly horizontal tangent even if the border is close in the normal direction. The test computes closest distance to each sampled line segment of the outline and takes the minimum. That quantity better matches “thin keyline” than a fixed-axis distance.

This remains a sampled approximation. The perimeter is discretized, rows are sampled at intervals, and lit pixels are selected by a brightness threshold. An all-black frame provides no useful image-edge sample. The historical LCD workstream explicitly limits its claim to a fitted outline for the pinned native shader. Do not generalize it into proof for every cartridge, shader parameter, output resolution, or browser. Recorded fit limits (`TODO/13_CURVED_LCD_BORDER.todo:17`).

MD06-R01 Retry when unfiltered corners disappear

Suppose CRT Geom looks correct, but switching to unfiltered output cuts off rectangular corners. The first hypothesis is stale curved geometry, not bad texture filtering. The player case checks that `screenCurved` is true only for `crt-geom.glslp`, while retaining the same native runtime identity across preset changes. That condition prevents a curved-only screen from becoming an unconditional model feature.

Retry with geometry inspection before image tuning. Record viewport dimensions, geometry type, curved-state flag, and the first raycast object at a corner-adjacent point. Then switch back and verify the corresponding border changes. If the mode updates but corners still vanish, inspect the recess envelope and material map; if the geometry does not update, changing anisotropy cannot solve the error.

The long CRT case is parked under CI in the pinned source because retained reload readiness remains unresolved. Its assertions are valuable specifications but not evidence of a current CI pass. A separate preset-loop case remains a different source of coverage. This distinction must survive future summaries, particularly when a chapter cites a test's thresholds. Skip condition (`tests/browser/scene.spec.mjs:1438`).

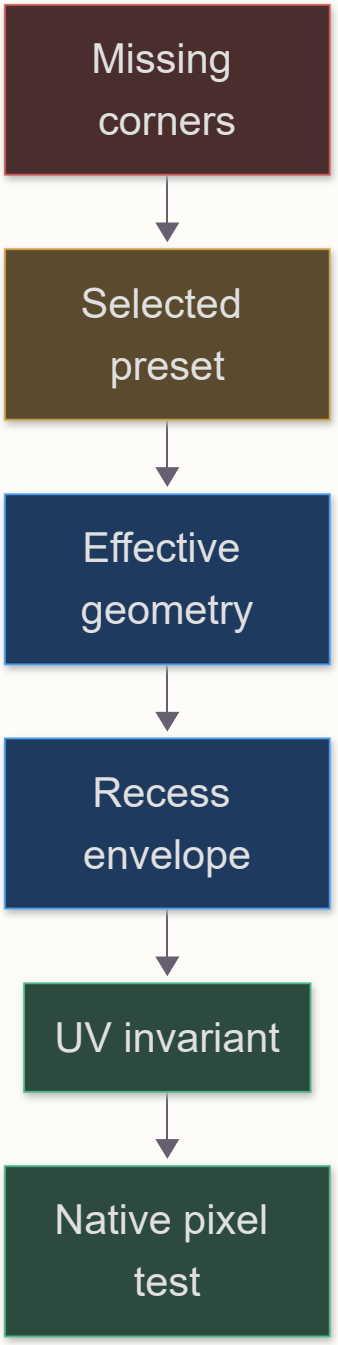


Figure MD06.2. Missing corners are investigated through mode, geometry, occlusion, mapping, and only then actual pixel output.

Separate image sharpness from perimeter density

Increasing the number of boundary segments adds triangles around the curved perimeter. It does not add native game pixels. If the image looks blurred throughout its interior, inspect texture filtering, source resolution, display scaling, and the renderer rather than increasing shape tessellation. If the perimeter looks faceted while interior pixels remain crisp, boundary sampling is a more plausible cause. These symptoms point to different representations.

The reverse mistake is to sharpen the image while ignoring geometric clipping. A nearest-sampled texture can still lose its corners if the outline is too tight. A high-resolution source can still sit behind an overly thick recess. The worked UV check, center raycast, and native edge fit form a useful diagnostic ladder precisely because each isolates a different property.

When comparing screen modes, hold the native output and camera as stable as the experiment permits. A moving game scene can change which edge pixels are lit, so a difference in sampled keyline distance may reflect content rather than geometry. The existing original-demo case uses a specific running environment; this chapter does not generalize its threshold into an arbitrary-content theorem.

Treat viewport metadata as a geometric promise

The screen geometry stores width and height in `userData.viewport`. The edge test uses that metadata to map framebuffer pixels into local coordinates. If an extension changes the geometry's scale by modifying vertex positions but leaves metadata unchanged, the test's rays can sample the wrong physical location even though its arithmetic still looks correct.

A safe extension updates the outline parameters, UV mapping, and viewport metadata as one coherent operation. If only the mesh object scale changes, make clear whether viewport dimensions remain local or describe the scaled result. Mixing those meanings produces subtle errors in picking and pixel measurements. The pinned factory's metadata describes its unscaled local screen.

Finally, do not interpret a dark border as missing alpha without inspecting material and source content. Native CRT output may include black pixels outside its visible curved picture, while the mesh itself remains fully opaque. Transparent glass, opaque black picture pixels, and absent geometry create different raycast and compositing behavior. A debugging receipt should name which of those quantities was actually measured.

MD06-Q01 Add a genuinely transparent glass layer

Explained answer

A future agent proposes a reflective transparent sheet above the live LCD. Explain why this is an extension, what new coordinate and material decisions it introduces, and why the existing “LCD center hit is Live LCD” assertion may need a carefully justified replacement rather than deletion.

MD06-A01 Preserve display ownership while adding optics

Back to question

It is an extension because the inspected factory has no separate glass slab. The new layer needs its own Z envelope, thickness policy, outline, material, shadow behavior, and ownership. If placed above the display, it may become the first raycast hit at the center even when the image is still correctly visible. A transparent surface is not automatically ignored by picking.

The existing assertion protects against an opaque bezel burying the image. Replace its literal first-object expectation only with an equally strong observation: for example, require an approved glass object first, then an unobstructed LCD intersection behind it, and verify actual image visibility. Otherwise a test “fix” could accept a new occluding panel.

Reflection should not be confused with image emission. Keep the image on its direct unlit material; let the optical layer contribute bounded reflection rather than multiply room lighting into game pixels. Current Three.js material guidance can inform a proposed design, but it cannot prove the original project used that design. The old pixels chapter explains the native-picture boundary, while the asset loader clarifies that supplied brand images are not the live framebuffer. Earlier pipeline account (`docs/book/chapters/05-pixels-and-light/README.md:5`); brand inputs (`src/brand.js:6`).

MD07

GBA controls, grips and tactile details

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

GBA controls, grips and tactile details

Tactility begins with clearance

A button cannot convincingly move into a case if the case is a solid lid beneath it. The factory therefore separates openings, receiving floors, moving caps, and cap-relative markings. This is a geometric contract before it is an animation effect. The historical aperture correction replaces intersecting raised socket surfaces with actual holes in the front face and lower dark floors. That change is visible in commit `706563fe0355d6455a3c1ba1ff8fd9bafd3fa2a4`, not inferred from a current screenshot. Historical change.

A and B use opening radius `.315` and cap radius `.29`, leaving a nominal radial gap `.025` before detailed bevel and pose effects. Start and Select use `.18` openings around `.155` caps, the same nominal gap. The source also stores target size separately: `.65` for action buttons and `.38` for Start/Select. Target dimensions are interaction metadata, not measurements of cap diameter. Control parameter table (`src/handheld.js:444`).

These are scene-unit clearances. They are not molding tolerances or guarantees against all animated intersections. A cap's curved profile, face bevel, and rocking motion can change the visible gap. The correct question is whether the complete surface remains coherent at rest and through its supported motion envelope, not whether one radius subtraction is positive.

The D-pad is one body with four directional anchors

The cross outline uses arm extent `.61` and half-arm width `.205`. Its opening is the same cross scaled by `1.09` and translated to `(-3.2,.26)`. The visible pad face contains a circular center hole of radius `.135`, with a lower circular pivot-well surface. Its nominal depth `.07` and bevel `.015` create a short manufactured profile rather than a flat sticker. D-pad construction (`src/handheld.js:563`).

Four directional controls share that one mesh. Their anchors sit at offsets of `.40` along the cardinal axes and local Z `.055`. Four small extruded arrows use rotated copies of a triangular path. This arrangement lets the pad rock as a body while preserving separate directional target locations. Creating four independent pads would change the physical model and could open cracks at their joins.

Material ownership adds a second layer of tactility. Rubber is cloned from a physical material initially used for silver-colored parts, then roughness rises to `.68` and clearcoat drops to `.06`. Arrow marks use another clone with a different color. Shared mesh motion does not require every marking to share exactly the same material object; it requires deliberate parentage and bounded finish differences. Material clones (`src/handheld.js:399`).

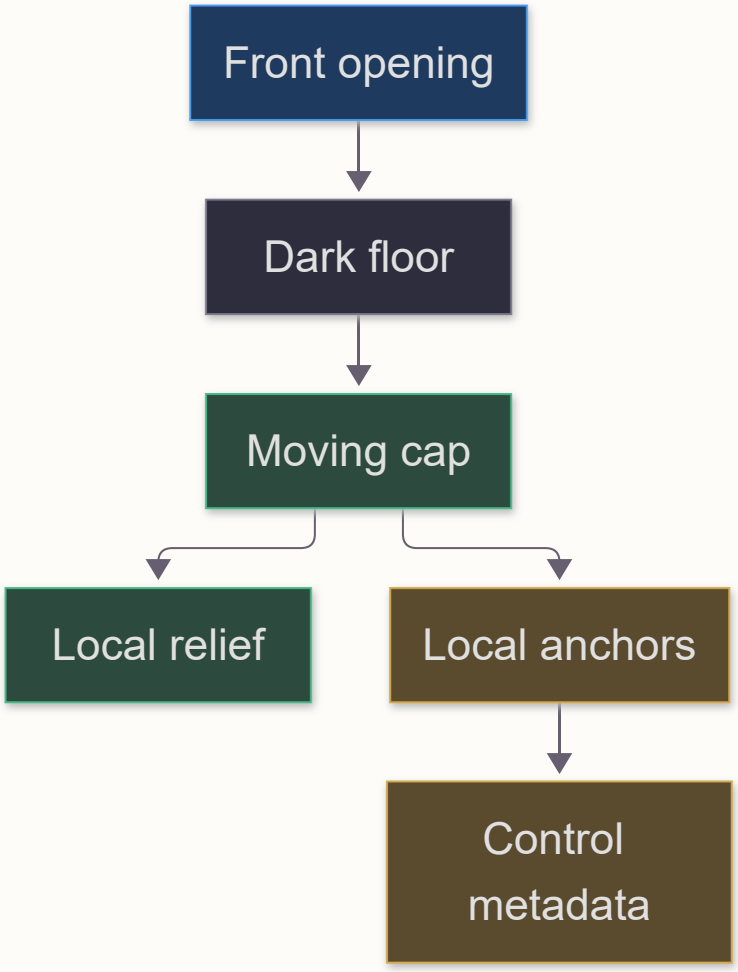


Figure MD07.1. Openings and floors provide physical clearance; moving caps own relief and target anchors.

Recessed A/B letters are surface reconstruction

The action cap begins as a forty-segment lathe. For action buttons, center-closing triangles are removed while rim normals remain. The letter is then recovered from browser-rendered `bold 95px Arial`, projected into a `.25×.28` footprint with Y offset `-.005`. A must produce two contours and B three, including their enclosed spaces. The extractor throws if that expectation fails. Glyph recovery contract (`src/handheld.js:232`).

The reconstructed top is a circular face with the glyph cut out and counter islands added back at cap height. Its dome follows `z=.055-.003*r/(radius*.6)`. The glyph floor lies at `.047`; walls connect floor and dome along all outer and counter contours. This is why the result remains a recessed letter from an oblique angle. Lowering a dark letter plane under an intact cap would hide it rather than engrave it. Recess construction (`src/handheld.js:324`).

Normals deserve attention because the new center meets an existing lathe rim. The implementation assigns top normals using a normalized profile-related vector rather than recomputing a flat face normal everywhere. Walls receive computed normals from their explicit

triangles. This separates smoothly continued cap shading from the cavity's edge behavior. A visible circular seam after an edit suggests a normal-continuity problem before it suggests a material-color problem.

The old geometry chapter records why replacing the original font with hand-drawn rounded contours was rejected. The later contour recovery preserves browser-font dependence; it does not bundle or license Arial. A future reproducibility effort should record browser and font availability, or explicitly propose a licensed deterministic outline asset as an extension. Historical typography distinction (`docs/book/chapters/04-handheld-geometry/README.md:55`).

MD07-W01 Derive resting and pressed action-cap heights

An action cap's mesh center is `.653`. Its uncut center dome reaches approximately local `.055`, so its resting center height is about `.708`. Factory motion metadata gives ordinary action controls travel `(0,0,-.075)`. At full nominal translation, the corresponding cap center surface is near `.633`. The visible face's outer envelope is near `.620`, so the cap approaches the opening while remaining slightly proud at that central location.

This is a local geometric derivation, not an assertion that the runtime always reaches exactly full travel or never intersects at every radial point. The groove floor is lower than the cap dome and moves with it. Its full-translation height would be `.653+.047-.075=.625`, which explains why tiny depth changes can matter to perceived occlusion. Motion and cap placement (`src/handheld.js:553`); button creation (`src/handheld.js:584`).

The following original browser-console recipe assumes a freshly created independent `model` from `createHandheld()`, after `loadBrand()`. It checks factory metadata without driving the live application. The observable outcomes are one shared D-pad mesh and `.075` nominal action travel.

```
// Pedagogical metadata check on an independent model.
const controls = model.controls;
console.assert(
  controls.get('up').mesh === controls.get('right').mesh
);
const a = controls.get('a').mesh.userData.motion;
console.log(a.restPosition.z, a.travel.toArray());
console.assert(Math.abs(a.travel.z + .075) < 1e-9);
console.assert(controls.get('up').anchor !==
  controls.get('right').anchor);
```

Next calculate why Start/Select should not inherit the same travel blindly. Their metadata uses `.04` inward motion and their caps are smaller. Shoulder controls move along negative local Y by `.12`, not along Z. Uniformly changing every control's Z coordinate would violate these different mechanisms. Geometry and interaction share a parameter map, but the mechanism remains control-specific.

Shoulders and grips complete the tactile vocabulary

L/R are rounded boxes `1.3×.39×.7` at `x=±3.45`, Y `1.81`, Z `.15`, with opposite small Z rotations. Their extruded L/R marks belong to the button. An anchor above the local surface locates the target independently of the button's center. Start/Select instead receive molded shell labels attached to the casing, so those words do not compress with the round caps.

The underside slider has five grip ridges; the battery cover has four thumb grips. Neither set is a generic “texture”: both are separate meshes whose depth can catch light. Grip spacing and ownership must follow the moving part they describe. A power ridge belongs to the slider; a battery grip belongs to the cover. The source's parent arguments encode that distinction directly.

MD07-R01 Retry when deeper motion makes a cap dark

An extension increases action travel and finds the pressed cap unexpectedly black. Do not repair this with emission or an unlit cap material. First verify that the opening still clears the cap and that the front face or another socket surface is not intercepting light. Then compare rest and pressed samples under the same light and camera. A geometry-induced shadow failure should be solved geometrically before changing finish.

The active aperture-lighting cases require resting probes above a brightness threshold and pressed probes above sixty percent of rest. A separate complete compression/release/unlit-response case is explicitly skipped in the pinned source. These are not interchangeable receipts. This writing inspected the assertions but did not run them. Aperture lighting assertions (`tests/browser/scene.spec.mjs:946`); parked response case (`tests/browser/scene.spec.mjs:986`).

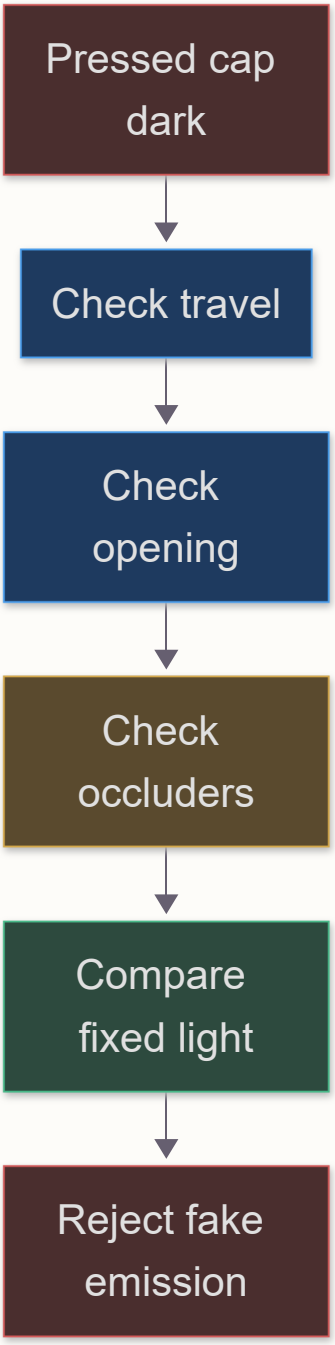


Figure MD07.2. Increased travel can recreate occlusion; artificial cap emission would conceal rather than solve that construction error.

Distinguish travel, rocking, and visual compression

The factory metadata records travel vectors and rest quaternions, but it does not itself execute a press. That boundary matters when writing reusable instructions. A correct factory can be driven incorrectly by a host animation, and a forgiving animation can conceal a geometric collision. Diagnose the rest assembly first, then inspect the host's supported pose envelope as a separate task.

For the shared D-pad, four anchors do not imply four independent travel accumulators. If simultaneous directions are represented by repeatedly translating the same mesh once per control, the result can exceed the intended movement. The factory's shared motion object is evidence that the pad is one body. A future host should combine directional intent into one bounded pose rather than sum unrelated mesh displacements.

Visual compression should also not be implemented by scaling the cap unless a redesign explicitly calls for squash. Scaling changes the glyph width, rim profile, and socket clearance, whereas the source metadata describes translation and orientation around a preserved rest shape. The old book's stronger-feedback discussion distinguishes deeper motion from altered cap shape; current factory values establish the baseline being explained. Historical feedback account (`docs/book/chapters/04-handheld-geometry/README.md:129`).

Diagnose finish without confusing it with shape

A rougher cap spreads highlights and may make a shallow groove less legible under broad light. That does not mean the groove disappeared. Compare raycast depth and grazing illumination before changing its contour. Conversely, a high-contrast dark floor can make a tiny cavity seem deeper than it is. Neither appearance alone measures geometry.

A practical proposed study holds the cap mesh fixed and varies only light direction, then holds light fixed and varies only floor depth. The first reveals shading sensitivity; the second reveals geometric depth sensitivity. Keep screenshots labeled with the changed condition and retain the same camera. This simple experiment avoids treating material roughness, letter color, and cut depth as interchangeable controls named “softness.”

MD07-Q01 Make only the action-letter grooves deeper

Explained answer

A request asks for more pronounced A/B engraving while preserving the original cap silhouette, lettering footprint, and all other controls. Which parameters and surfaces belong to the change? What must not change, and why is darkening the material alone insufficient?

MD07-A01 Change the cavity, not the alphabet or cap

Back to question

The change belongs to the A/B floor depth and walls connecting that floor to the unchanged dome. Preserve recovered contours, counter islands, cap radius, rim normals, and mesh position. Lowering the floor requires rebuilding the wall vertices consistently; otherwise the cavity may have a gap or walls ending at the old depth. Do not edit the shared molded alphabet, because it also supplies shell labels and shoulder marks.

Darkening the groove material changes contrast but not depth, ray intersections, or wall shadows. It may make letters look heavier without satisfying the geometric request. Conversely, deeper walls can make an unchanged font look sharper through occlusion. The historical record

documents that distinction, so a good retry includes front and grazing-angle comparisons while holding light fixed.

Recheck pressed clearance because a lower floor approaches the shell's visible depth. Verify actual floor hits and cap-height counters, not mesh names alone. The broader input suite confirms that physical controls reach the native core, but those integration assertions do not measure newly changed engraving depth. Input integration boundary (`tests/browser/player.spec.mjs:693`); asset initialization prerequisite (`src/brand.js:6`).

MD08

Cartridges, small details and markings

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD08 / CARTRIDGES, SMALL DETAILS AND MARKINGS

Contents

Cartridges, small details and markings

Small parts explain what the large shape is

The shell carries silhouette, but a cartridge slot, battery cover, screw sockets, and edge controls explain how the object is used. These details should be modeled according to their observable role. A screw slot needs a thin dark geometric cue; a deep rear logo needs an actual cut surface; a cartridge title can be a canvas-backed label. Treating every detail as an image loses depth, while extruding every pixel adds complexity without a corresponding observation requirement.

The pinned factory returns a cartridge reference alongside root, screen, recess, controls, LED, and bezel texture. That return contract makes the cartridge a manipulable component rather than an anonymous decoration. Its initial state is invisible; host behavior later makes it meaningful during loading. This chapter explains the factory structure, not the full insertion choreography. Returned model and loaded cartridge (`src/handheld.js:622`).



Figure MD08.1. Diagnostic cartridge factory render under neutral studio lighting, showing prop thickness, label layering, and grip cues; not a historical loading screenshot.

The reusable cartridge is deliberately simple

`createCartridge(color,title,subtitle)` returns a group. Its body is a rounded extruded box `1.48×1.13×.22` with corner argument `.1`. A cream label backing `1.17×.65×.025` sits at Y `-.02`, Z `.15`. Two transparent text planes sit at Z `.21`, one centered at Y `.11`, the other at `-.12`. Five box grips span the lower region at X `-.53+i*.26`, Y `-.43`, Z `.14`. Complete prop factory (`src/handheld.js:645`).

The body uses roughness `.72`; the label backing uses `.85`. These choices describe a matte prop, not a measured material sample. The text helper creates a standard lit material with transparency and disabled depth writing. That lets antialiased letter edges reveal the backing while avoiding writing the full rectangular plane into the depth buffer. It does not remove all transparent-ordering limitations.

No separate contacts, connector cavity, rear screw, or detailed molded seam is created in this factory. A future chapter must not describe a cartridge back render as evidence for geometry that the factory lacks. Its value is precisely that it reveals the simplified rear boundary. Adding contacts would be a proposed extension with new dimensions, materials, and tests, not an explanation of existing implementation.

MD08-W01 Fit the loaded cartridge to the top slot

The handheld creates a default prop with body color `#666574`, title `BBA`, and subtitle `LOCAL CARTRIDGE`. It scales X and Y by `1.9` while leaving Z at one, then positions the group at `(0,1.52,-.3)`. Nominal body width becomes `1.48*1.9=2.812`; nominal body height becomes `1.13*1.9=2.147`. Thickness remains `.22` before bevel effects. These calculations are dimensions of the base body, not the complete label-and-grip envelope.

The named slot is `3.05` wide, so comparing nominal widths leaves `.238` total horizontal difference before bevel expansion and other geometry. This is a first check, not a collision guarantee. The factory's nonuniform scaling also changes corner proportions in the XY plane relative to depth; that is an authored prop-fitting choice, not a physical cartridge standard.

The following original recipe assumes a browser page with the project import map. It creates only the standalone prop, so it needs no brand decoding. It measures the full visible group after the same scale used by the handheld and compares that envelope with the nominal slot width.

```
// Pedagogical fitting diagnostic.
const { Box3, Vector3 } = await import('three');
const { createCartridge } =
  await import('./src/handheld.js');
const prop = createCartridge('#666574', 'TEST', 'LOCAL');
prop.scale.set(1.9, 1.9, 1);
prop.updateWorldMatrix(true, true);
const size = new Box3().setFromObject(prop)
  .getSize(new Vector3());
console.log(size.toArray());
console.log('Width remaining:', 3.05 - size.x);
console.assert(size.x < 3.05);
```

The expected width is larger than the un-beveled `2.812` because the body helper adds bevel extent. Measuring all children also catches a future grip or label that protrudes sideways. The check assumes no rotation and does not prove the cartridge clears the tabletop throughout insertion. The scene tests sample actual motion and table clearance; a factory envelope is only one part of that integration story. Insertion sampling (`tests/browser/scene.spec.mjs:229`); motion assertions (`tests/browser/scene.spec.mjs:283`).

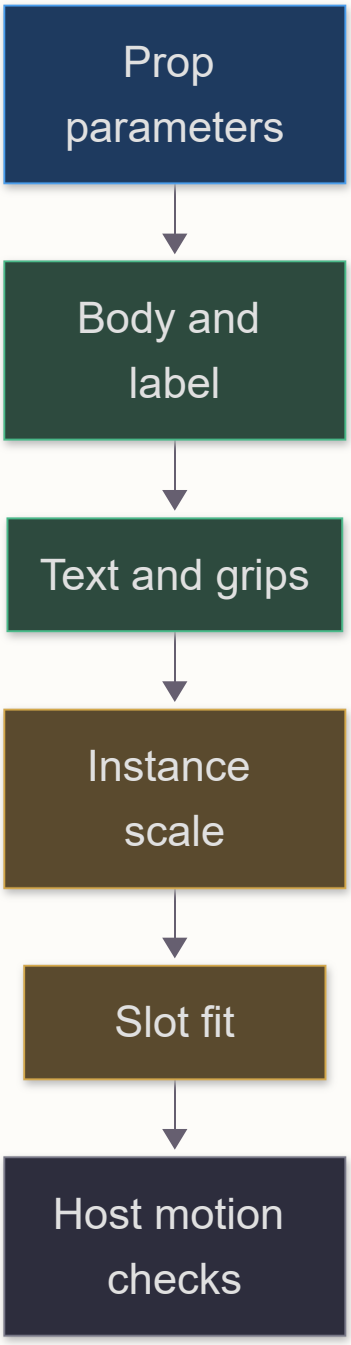


Figure MD08.2. Reusable cartridge geometry is distinct from instance fitting and runtime loading motion.

Rear cover engraving preserves the original rim

The rear cover is a rounded `3.65×2.9×.06` box at `(0,-.2,-.57)`. Four small thumb grips are children of the cover. Its logo is not simply a plane pasted above the plastic. `engraveRearLogo()` reads the supplied image alpha through a `512×171` mask, projects contours into a `3.45`-wide local region centered at Y `-.35`, and cuts a `.024`-deep engraving.

The algorithm computes the original cover's minimum geometry Z, removes triangles lying wholly on that rear cap, and retains side and bevel geometry. A subgroup at that surface rotates around Y by π so its front-facing shapes point backward. A replacement face contains the cut regions, colored floors sit inward, and explicit wall triangles connect the boundaries. Keeping the original outer bevel avoids replacing the entire cover with a visibly different object. Rear-cap replacement (`src/handheld.js:285`).

Nested contours preserve plastic islands inside transparent counters. Bounds accelerate containment selection, but an actual point-in-polygon test determines the relationship. Winding is adjusted by nesting parity. The floor UVs map back into the original artwork, so the colored logo survives as recessed color rather than becoming a uniformly dark silhouette. The floor's basic material is transparent, not tone-mapped, and does not cast or receive shadows; the plastic walls carry geometric shading.

Historical commit `cf53ecc05ea27a9c26e60a87d0306de1bce78bc5` changes width from `2.95` to `3.45` and depth from `.006` to `.024`. This supports the current enlargement and deeper cavity as an observed source change. It does not establish a measured real-world engraving depth or a new human approval. Scoped enlargement diff.



Figure MD08.3. Diagnostic rear factory render under neutral studio lighting, intended to reveal cover relief, grips, and screw cues rather than prove live-room illumination.

Molded marks and edge hardware use different ownership

The maker mark is a shell-owned group at `(0,2.10,.625)`. It contains extruded polygon lettering and a chamfered outline. Start/Select labels use rounded outline rings and molded text, rotated together on the shell. These letters come from an explicit small polygon alphabet, unlike the recovered Arial action glyphs and canvas cartridge text. Reusing the wrong alphabet would change appearance and could fail for unsupported characters. Molded labels and maker mark (`src/handheld.js:362`).

Five screw positions each receive a dark socket, a smaller silver cylinder, and a thin dark slot box. These are visual hardware cues, not threaded fasteners. The link port is a box; the headphone socket uses an additionally rotated cylinder. Their simplified representations are adequate for the inspected exterior contract but do not describe internal electronics. Documenting that limit makes extension safer: a future agent knows which details are abstractions rather than missing pieces of an unseen CAD model.

The brand loader is responsible for decoding the PNGs before the rear mask is generated. A blank engraving can therefore originate at input loading, alpha extraction, region classification, or surface replacement. These boundaries should be investigated independently. Brand decoding (`src/brand.js:6`).

MD08-R01 Retry when a new logo loses thin features

Suppose a licensed replacement logo has fine translucent lines that disappear from the cavity while the colored floor still contains them. The mask is sampled at `512×171` and uses alpha threshold `16`, while the color texture remains `1024×342`. Geometry and color therefore have different sampling responsibilities. Inspect the mask before increasing depth or brightening the floor.

A line narrower than the mask's effective sampling can merge, vanish, or create unexpected contour connections. Increasing mask resolution is a proposed change with higher contour and wall cost. Lowering the threshold may recover faint pixels but also admit noise and detached regions. Neither is automatically correct. Compare contour count, nesting, and wall complexity against the intended licensed artwork, then inspect an oblique render for artifacts.

Do not silently edit the original PNG to suit a cavity algorithm. Preserve the source asset and derive a documented mask. The historical reference record explicitly distinguishes independently authored geometry from extracted assets, and the same provenance discipline applies to replacement marks. Provenance boundary (`TODO/07_REFERENCE_SHAPE.todo:19`).

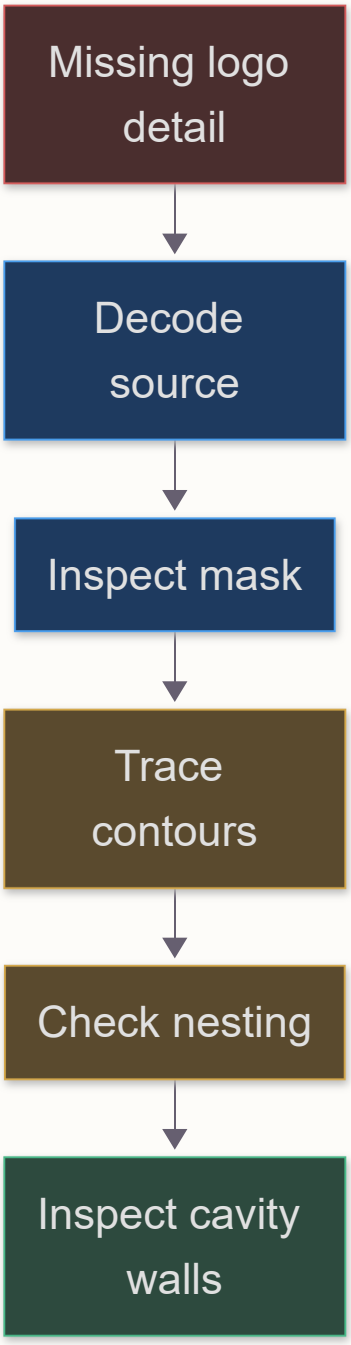


Figure MD08.4. Rear-logo failures are traced from source alpha to the final cavity rather than treated as generic material problems.

Keep tiny detail readable without falsifying construction

Small parts face a sampling limit: a real geometric slot may project to less than one pixel in a distant view. Increasing its contrast can improve recognition, but enlarging the slot changes the model. Decide whether the contract is close inspection or distant readability before choosing a remedy. The same source can reasonably support a detailed inspection view and a simpler distant presentation, but this factory does not implement a separate level-of-detail system.

Screw slots illustrate the distinction. The thin dark box communicates orientation and recess-like contrast, yet it is not a modeled helical thread or a fully subtracted driver cavity. Describe the actual cue without promoting it into absent engineering detail. A future physically detailed fastener would require a new profile and an observation budget that makes those triangles useful.

For the rear engraving, detail scale is coupled to mask sampling rather than only camera distance. Enlarging the final mesh makes existing contour segments more visible but does not recover alpha detail discarded during extraction. Rebuilding the mask at a different resolution changes geometry; scaling the subgroup does not. This is another reason to record source resolution, projection width, and groove depth as separate parameters.

The cartridge title has a similar separation. A wider plane can stretch existing text without changing its raster input, while rerasterizing at a different aspect changes layout. Preserve readable naming by editing the intended stage rather than compensating with a transform somewhere else.

MD08-Q01 Add rear contacts to the cartridge

Explained answer

Design a bounded extension for visible cartridge contacts. What must be established before choosing a gold material and placing strips? Explain how to avoid claiming that a diagnostic back image already proves those contacts exist.

MD08-A01 Establish the missing geometry contract first

Back to question

The current factory has no contacts, so the extension must define a connector region, strip count, spacing, recess, thickness, and rear-facing orientation from an authorized reference or an explicitly fictional design. A metallic color alone does not establish any of these. Keep dimensions in scene units unless calibrated evidence supports a physical conversion.

Attach contact geometry to the cartridge root or a named connector subgroup. Its rear-facing transform must be derived from the prop's frame, not copied from the handheld's battery-cover engraving. Preserve the existing label and grips, then measure the complete envelope again: a new rear protrusion may affect slot fitting or table clearance even if width is unchanged.

A back diagnostic of the old factory documents absence and simplification, not hidden detail. Produce separate baseline and extension observations and label them accordingly. Existing loading tests check motion and seating relationships; they would need additional contact visibility and clearance observations before covering the new geometry. The original book's relief-versus-color explanation provides a useful conceptual test, but it does not supply missing contact dimensions. Earlier geometry principle (`docs/book/chapters/04-handheld-geometry/README.md:26`).

MD09

Procedural textures and asset boundaries

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Procedural textures and asset boundaries

Procedural does not mean asset-free

The project creates many textures at runtime, but their inputs differ. Wood grain is generated from seeded arithmetic. Cartridge text is rasterized by the browser. The bezel wordmark is derived from supplied artwork. Rear engraving turns supplied alpha into geometry while retaining image color on the cavity floor. Calling all of these “generated textures” hides important reproducibility and provenance boundaries.

The shared `canvasTexture(width,height,paint)` helper creates a canvas, obtains a 2D context, invokes a painter, and wraps the result in `THREE.CanvasTexture`. It throws an explicit error if drawing is unavailable. The texture is marked sRGB and assigned anisotropy `4`. These are actual implementation settings; they do not prove every consuming material or renderer uses the same sampling behavior. Texture helper (`src/handheld.js:68`).

The current CanvasTexture documentation, consulted 2026-11-07, describes canvas-backed texture construction and inherited sampling defaults. Use it as explanatory guidance, not a version-pinned historical receipt. This project explicitly overrides color space and anisotropy, so copying generic defaults into a parameter table would be wrong.

Color data and coverage data have different jobs

sRGB marks how encoded color should be interpreted. Alpha describes coverage, not brightness. The bezel derivative uses source RGB to decide which pixels belong to neutral lettering, but ultimately preserves a reconstructed alpha silhouette and fills that silhouette with stylized color. The rear engraving uses alpha to create contours, while its floor samples the color image. A bright RGB pixel with zero alpha contributes no visible mark; a dim but opaque pixel may still be important to geometry.

This distinction explains why debugging should inspect channels separately. If a logo is missing, first determine whether the color is dark, alpha was removed, UVs point elsewhere, or another surface occludes it. Increasing light only addresses one of those possibilities. Disabling depth writing on a transparent label addresses a different problem again: whether the plane's rectangle prevents later geometry from drawing.

Text labels use a canvas height of `128` and width `ceil(128*aspect)`, centered text alignment, middle baseline, and a maximum drawing width eighteen pixels smaller than the canvas. The world plane's width-to-height ratio supplies the aspect. Thus text size, canvas resolution, and world footprint are related but not identical. A font size in canvas pixels is not a world-space letter height. Text and label helpers (`src/handheld.js:81`).

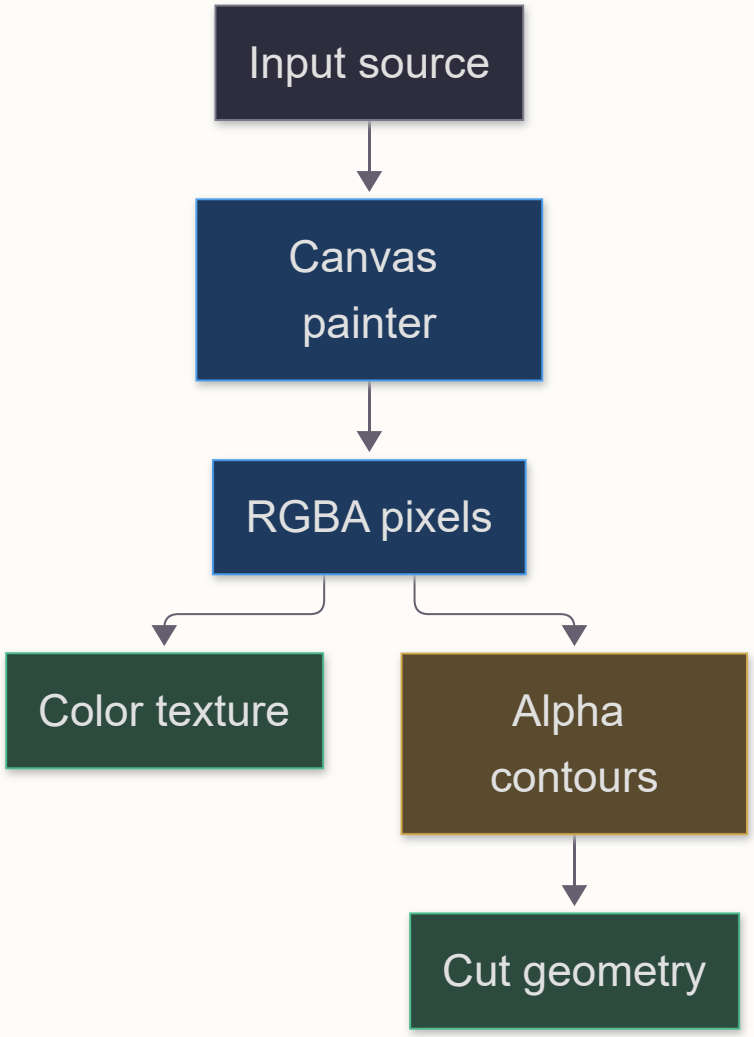


Figure MD09.1. The same RGBA input can supply visible color or geometric coverage, but those outputs have different contracts.

The bezel wordmark is a derived asset

`loadBrand()` decodes `logo.png`, `star.png`, `orbit.png`, and `glow.png`. The handheld module imports `brandImage`; it does not use all four images directly. The full wordmark texture is drawn at `1024×342` for rear artwork. A separate bezel canvas is `1536×174`, cropped from the supplied image and placed into a centered region occupying sixty percent of each canvas dimension. Preserve that distinction: the rear uses full artwork, while the front derives letter-only branding. Input loader (`src/brand.js:1`); initial derivative (`src/handheld.js:475`).

The derivative first computes the minimum RGB channel and uses it to attenuate alpha, rejecting saturated orbital accents while retaining neutral lettering and highlights. It then finds solid alpha cores using a seven-by-seven neighborhood. Those seeds flood into connected nonzero-alpha pixels. The seed test selects components; the flood reconstructs their original edge pixels rather than simply eroding the whole mark.

This algorithm has a concrete failure boundary: if no solid cores remain, it throws. A replacement artwork with thin lettering may therefore be unsupported even when it looks valid as a PNG. Broadening the threshold is not automatically safe because detached exterior contours may return. Inspect the retained components before changing the test. Core selection and flood (`src/handheld.js:487`).

Finally the canvas applies vertical chrome color bands with `source-in`, adds a horizontal iridescent tint with `source-atop`, and restores `source-over`. The resulting material uses roughness `.4` and metalness `.1`. The colored bands are baked stylization, not physically traced reflections or a thin-film simulation. Their appearance can coexist with room lighting, but the gradient itself does not respond to camera movement. Compositing and material (`src/handheld.js:529`).

MD09-W01 Reproduce the wood generator's controlled variation

`createWood()` paints a `1024×1024` canvas. It starts from seed `41` and advances a 32-bit linear congruential generator using multiplier `1664525` and increment `1013904223`. Eight bands of `128` pixels establish planks. Each receives low-alpha tonal variation, then `110` grain strokes whose paths combine sampled Y positions with sine displacement. Thin dark and light seams separate planks. The texture repeats twice along both axes. Complete wood painter (`src/handheld.js:657`).

The seed makes the arithmetic sequence repeatable. It does not promise identical raster pixels across every browser implementation, GPU, and operating system. Canvas line rasterization and color handling can differ. The useful invariant is stronger within one environment: two calls follow the same painter and seeded sequence, so their pixel results should agree there.

The following original browser-console recipe assumes the project's import map and a usable 2D canvas. It hashes canvas pixels, not GPU readback, and creates no scene. Expected results are equal hashes in the same execution environment and repeat values `[2,2]`. It is a diagnostic proposal, not a historical test.

```
// Pedagogical deterministic-painter check.
const { createWood } = await import('./src/handheld.js');
function hash(texture) {
  const c = texture.image;
  const bytes = c.getContext('2d')
    .getImageData(0, 0, c.width, c.height).data;
  let value = 2166136261;
  for (const byte of bytes) {
    value = Math.imul(value ^ byte, 16777619);
  }
  return value >>> 0;
}
const a = createWood(), b = createWood();
console.assert(hash(a) === hash(b));
console.log(a.repeat.toArray());
a.dispose(); b.dispose();
```

Derive what changing seed versus repeat means. A seed change alters the generated grain arrangement while preserving canvas size and the eight-band structure. A repeat change alters how often the same image tiles over the receiving geometry; it does not repaint new planks. If a desk looks too repetitive, changing only seed cannot remove the fact that the same tile repeats twice. If grain appears stretched, inspect mesh UVs and aspect before repainting.

The function returns a texture, not a desk mesh or material. Its placement and physical scale belong to the consumer outside this part's scope. Recording that boundary prevents “wood model” from becoming an unsupported claim about desk dimensions or UV orientation.

MD09-R01 Retry when a replacement wordmark fails

Suppose a new licensed wordmark decodes correctly but the factory throws that it has no solid lettering. Begin with the derivative canvas, not the mesh. The crop constants refer to the current source artwork layout. A differently arranged image may place its letters outside that crop or make them too thin after resampling. Successful decode proves only that the image loaded.

Retry by inspecting crop coverage, alpha thresholds, and seed neighborhoods separately. First verify that the crop contains intended letters. Then inspect neutral-channel selection. Finally inspect whether any seven-by-seven solid regions survive. Changing all three at once can produce a plausible output while leaving the cause unknown. A replacement should carry its own explicit derivation parameters rather than silently inheriting assumptions about the old image.

The earlier geometry chapter records why threshold-only filtering damaged lettering and why component reconstruction was introduced. That historical explanation supports retaining component identity, but the exact current loop remains authoritative. It also distinguishes the

later colored finish from earlier neutral ink; do not describe both as simultaneous source behavior. Recorded derivative evolution (`docs/book/chapters/04-handheld-geometry/README.md:95`).

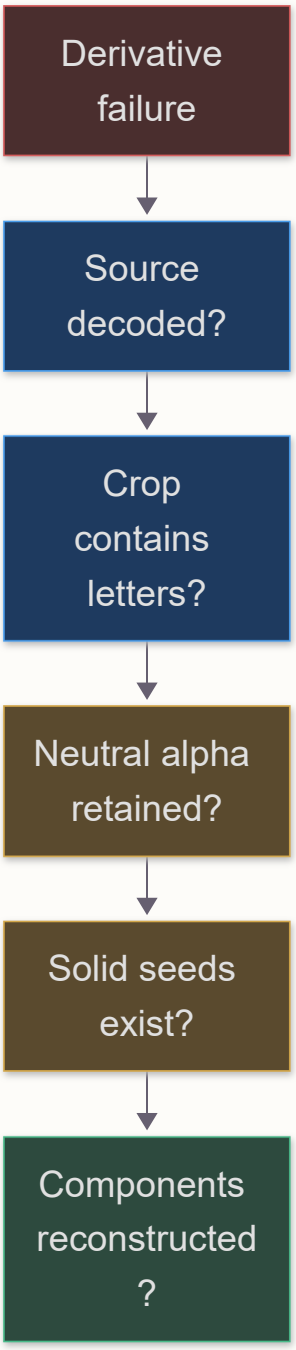


Figure MD09.2. Wordmark diagnosis follows each image-processing boundary instead of treating every missing pixel as an asset-loading error.

Asset delivery and legal provenance remain separate

The static allowlist copies top-level JS, SVG, and PNG files from `src` and `assets`, maps the inspection logo separately, and vendors selected Three files and license. A runtime-generated texture need not exist as an exported PNG to be reproducible. Conversely, a local asset outside the

allowlist is not automatically available at the same browser URL. Public delivery rules (`scripts/site-files.mjs:4`).

The root manifest's Three dependency establishes the geometry library; it does not establish rights to every supplied image or system font. The historical reference workstream explicitly avoids extracting unavailable model assets. Preserve that discipline when adding references: use authorized inputs, document derivation, and do not infer permission from technical accessibility. This chapter does not perform a new rights audit. Library declaration (`package.json:7`); historical provenance limit (`TODO/07_REFERENCE_SHAPE.todo:19`).

Own the lifetime of generated resources

Each helper call creates a new canvas and texture. The text contour routine disposes its temporary texture after reading pixels, and the rear-mask routine similarly disposes the mask. Those calls release texture resources; they do not imply that every scene object is automatically cleaned up when removed from a group. Temporary glyph resource (`src/handheld.js:232`); temporary rear mask (`src/handheld.js:255`).

An agent writing a repeated diagnostic should therefore distinguish temporary extraction inputs from textures retained by materials. Disposing a shared retained texture while another material still uses it can invalidate the display. Failing to release unique diagnostic resources can accumulate allocations across many specimens. Track ownership explicitly instead of assuming traversal and removal constitute disposal.

The same principle applies to caches. The brand loader returns early only when all four image exports are present; that is an input cache, not a cache of generated handheld geometries or wordmark textures. Calling the factory twice still constructs two model trees and derived canvases. A proposed factory cache would need rules for material mutation, texture reuse, and disposal, especially because cloned finishes and mutable control state belong to individual instances. Resource reuse is an extension design, not something established by the current loader's early return.

MD09-Q01 Make the book fully reproducible across fonts

Explained answer

An agent wants every A/B contour and cartridge title to match across machines. Is a fixed random seed enough? Propose a bounded extension and explain what existing behavior it would change. Include why matching the bezel bitmap would not solve the action glyph problem.

MD09-A01 Control the actual source of variability

Back to question

A seed controls wood's arithmetic choices, not browser font rasterization. Action glyph geometry begins with Arial text rendered to a canvas, and cartridge labels use browser text as well. If the selected font differs, contours and line breaks can differ even with identical Three versions. The

bezel is derived from a supplied image, so matching its bitmap does not determine the separately rendered A/B letters.

A deterministic extension could supply appropriately licensed font data and await its loading before rasterization, or provide authorized fixed vector contours for the required glyphs. Each changes the input contract and should be labeled as a new design. The first retains rasterization dependence but controls font selection; the second removes raster contour recovery for those glyphs but introduces vector-asset maintenance and provenance responsibilities.

Verification should compare contour topology, footprint, counter preservation, and actual rendered readability, not only a file hash. Keep the existing baseline available so the extension is not misrepresented as recovered historical tooling. The player test's direct-video assertion further reminds us that decorative texture reproducibility must not replace the live framebuffer with a baked image. The model's labels, brand derivations, and game display are three different asset boundaries. Live-video boundary (`tests/browser/player.spec.mjs:265`).

PART 2

The articulated Nintendo DS

1. DS assembly and the parameter map
2. Hinges, lids and articulation
3. DS shells, recesses and hardware
4. DS controls and readable markings
5. Dual LCDs and curved glass
6. Stylus construction and touch geometry
7. Cartridge insertion and loading choreography
8. Inspection, camera and manipulation
9. The DS model regression atlas

PART 2 / THE ARTICULATED NINTENDO DS

MD10

DS assembly and the parameter map

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

DS assembly and the parameter map

Read the assembly as a set of contracts

The DS is not one sculpted object. It is an assembly whose convincing appearance depends on separate transform spaces, layered solids, readable controls and a stable pixel surface. The reconstruction problem is therefore not “make a blue rectangle.” It is to preserve relationships while parts move: the upper display follows the lid, the lower display stays with the base, and the stylus remains available without stretching the camera envelope toward the pointer.

Implemented evidence. `createDsHandheld()` imports Three.js and three helpers from `handheld.js`; it constructs groups, extrusions, cylinders, lathed caps, planes and canvas textures. There is no model-file loader in this factory. The repository and installed package both identify Three.js **0.186.1**. This chapter describes JavaScript procedural authoring, not a Blender, CAD or imported-mesh workflow. Read the factory entry and materials (`src/ds-handheld.js:1`) and dependency declaration (`package.json:7`).

Numbers below are **model-space units**, not measured millimetres. A screen width of 4.5 establishes a useful internal scale, but it does not establish a manufactured part's dimensions. Surface names are authoring labels, not a promise of mechanical equivalence to every Nintendo DS revision. Begin a reconstruction by recording those limits; otherwise seemingly precise arithmetic becomes misleading hardware documentation.

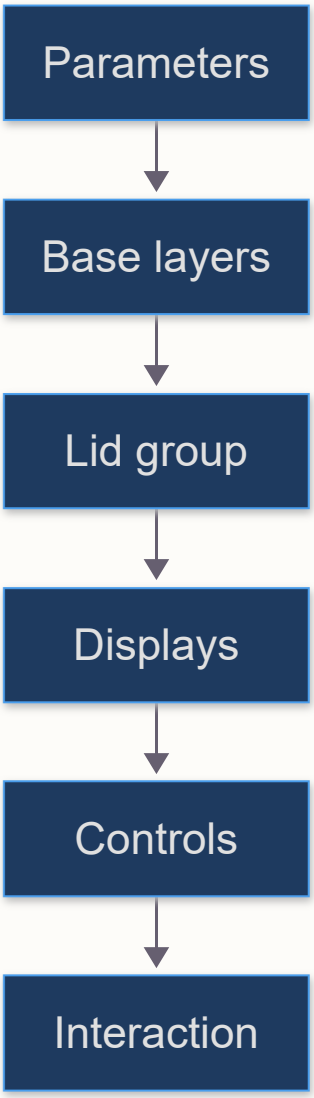


Figure MD10.1. Reconstruct in dependency order: silhouette and ownership precede decorative details and interactive targets.

Build a parameter ledger before changing geometry

The lower assembly contains a parting line, rear shell and control deck centered at local Y −2.43. Their common nominal width is 8.5 and height 4.75. The lid's nominal footprint is 8.5 by 4.65, centered at its own Y 2.43. The opposing signs place the joint near the root origin instead of at either shell's center. This origin choice makes articulation simpler: the lid rotates around its parent origin while its geometry is offset away from that origin.

Parameter	Implemented value
Lower footprint	8.5 × 4.75
Upper footprint	8.5 × 4.65

Lower center Y	−2.43
Upper center Y	2.43
LCD viewport	4.5 × 3.375
Hinge radius	.255
Default lid X rotation	70 degrees

These are nominal constructor inputs, not final beveled bounds. The DS wrapper supplies bevel `min(.02, depth/3, radius/3)` to the shared rounded-box helper. Extrusion bevels extend the outline and thickness; an 8.5-wide shape can therefore occupy more than 8.5 in its generated bounding box. The shell construction (`src/ds-handheld.js:68`) must be read with the centered extrusion helper (`src/handheld.js:19`). That combination explains why center-to-center spacing is insufficient for clearance work.

The lower rear shell uses depth .19 at Z −.135; the control deck uses depth .22 at Z .105. The parting line is much thinner, .025 at Z −.025. These layers establish visible seams without requiring a single watertight enclosure. Do not silently reinterpret them as a production injection-molding cross-section: intersecting layered meshes are acceptable here, but manufacturing wall thickness and internal fasteners are not being solved.

Assign transform ownership before assembly

The root owns the base, lower LCD, controls, card and stylus. The `lid` owns its shell layers, upper LCD, speaker details, bumpers and hinge leaves. The separate `hinge` group is root-owned; its barrel does not rotate merely because the lid rotates. This distinction prevents double articulation. Attaching the hinge under the lid and then rotating both would create a plausible-looking but structurally wrong joint.

The return object is the public assembly interface: `root`, `lid`, `hinge`, `screens`, `screen`, `controls`, `led`, `power`, `cartridge`, stylus references, branding and accent materials. `screen` aliases the second entry of `screens`, so downstream lower-touch logic can address the intended surface directly. The returned interface (`src/ds-handheld.js:273`) is more reliable than guessing targets from traversal order.

Research guidance, not pinned implementation evidence. Current Three.js Object3D documentation, consulted 2026-11-07, distinguishes local transforms, world matrices and radian rotations. Use that model to reason about parent-child composition. Do not adopt a newer documented property merely because it appears on the current page; this implementation uses group origins and offsets, not a newly introduced pivot API.

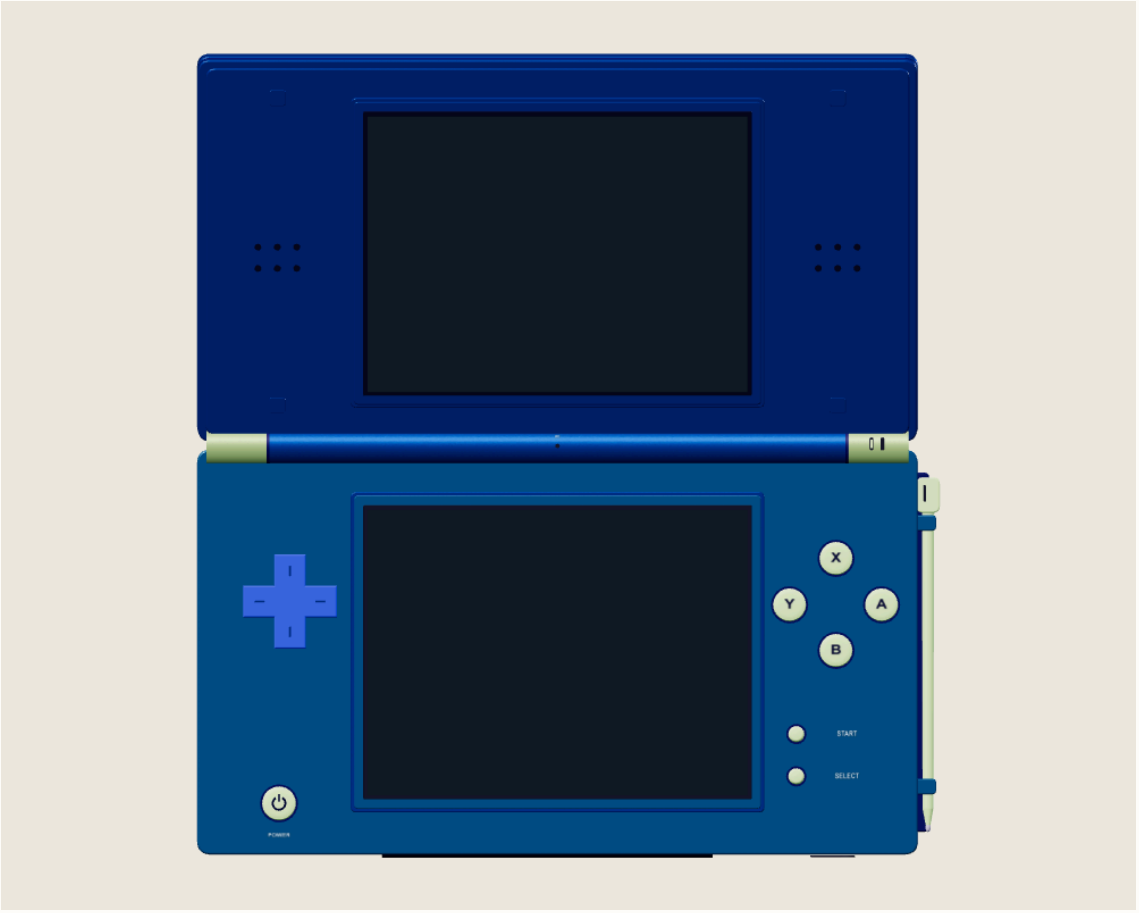


Figure MD10.2. Parent-supplied neutral-studio constructor diagnostic, intended for silhouette and layering inspection; it is not native gameplay, historical development evidence or a claim that every control is unobstructed in every pose.

Separate visual construction from application completion

A factory instance is not the finished application state. Its branding plane starts invisible and without the supplied logo map. `DsController.initialize()` loads the image, assigns an sRGB texture and reveals the plane. Its screens initially use unlit materials, but the controller constructor subsequently selects reflective powered-off glass. A bare-constructor diagnostic must therefore identify which initialization stages it includes.

The controller initialization (`src/ds-controller.js:186`) establishes this asset boundary. It is a supplied artwork texture on procedural geometry, not evidence that the whole console was imported. Likewise the scene puts the root into a desk or focused-play transform; a local positive Y coordinate is not automatically “up” in the room.

The scene pose constants (`src/scene.js:18`) use a rotated table quaternion and an identity default play quaternion. Keeping that distinction in the ledger prevents mistaken descriptions of the card's travel or the lower expansion slot. The modeled front/bottom expansion opening is separate from the rear DS game-card slot near the hinge; the factory names and places both.

MD10-W01 Audit a face-button clearance numerically

Consider the leftmost face button, Y, at X 2.75 with cap radius .195. Its recessed socket has radius `.195 + .025 = .220`, so the socket's leftmost nominal X is `2.75 − .220 = 2.530`. The outer touchscreen bezel starts with width 4.88, giving half-width 2.440, but its .035 depth limits the DS wrapper bevel to `.035/3 ≈ .011667`. The outer bound is therefore approximately 2.451667. The nominal horizontal gap is about **.078333**.

That result explains two decisions. First, the socket, not just the smaller cap, is the limiting feature. Second, ignoring the bezel's extrusion bevel would overstate the available gap. The browser test requires greater than .05 between every cap/socket bound and the actual bezel, and greater than .1 toward the deck edge. Read the bound-based assertions (`tests/browser/ds.spec.mjs:1309`).

Original diagnostic pseudocode follows. Assumptions: a browser has constructed the model, matrices are current, and `boundsInBase()` transforms all eight geometry-box corners into root space. The expected result is positive margins, not an assertion that this recipe was executed here.

```
bezel = boundsInBase("Touch fine cyan bezel edge")
deck = boundsInBase("Metallic blue control deck")
for label in X, A, B, Y:
  for part in button, recessed_socket:
    box = boundsInBase(label, part)
    require box.minX - bezel.maxX > .05
    require deck.maxX - box.maxX > .10
```

For a beveled corner, an axis-aligned box is conservative: it can reject space that is geometrically empty near the rounded edge. That is useful for a readable diagram and predictable safety margin, but it is not exact mesh-to-mesh distance. Record which measurement you use rather than switching between them when a test fails.

MD10-R01 Retry after widening the screen surround

Suppose a proposed surround change increases its width by .12 without changing controls. Its right edge advances .06. The previously calculated .078333 gap becomes about .018333 and violates the .05 criterion. Moving only the Y legend does nothing; moving only its cap leaves the socket overlapping. Shrinking the LCD would change a separate display contract.

The corrected proposal must move the entire diamond or redesign the surround, then recalculate both inner and outer margins. Moving all four controls outward by .04 might recover the inner threshold but consumes .04 of the A button's deck-edge margin. Inspect both limits before choosing a value. Historical commit 89d905f moved the complete cluster outward by .2; that recorded correction supports the method, not permission to move it indefinitely.

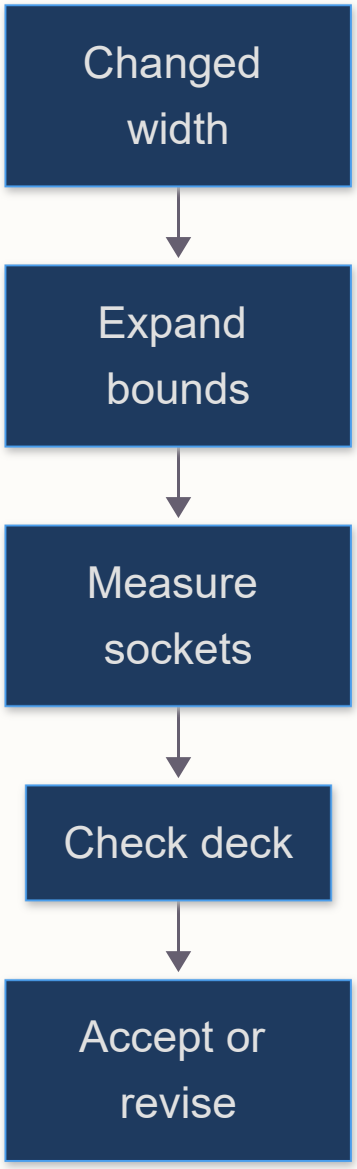


Figure MD10.3. A parameter change is accepted only after checking both adjacent constraints; passing one side does not preserve the assembly.

Make parameter changes transactional

Proposed reconstruction practice. Keep a small change ledger with four columns: parameter, dependent geometry, dependent behavior and evidence to regenerate. For example, changing LCD width affects the plane, all three surround layers, illumination viewport metadata, control clearance and camera bounds. Changing only shell roughness affects appearance but should not change those dimensions. This classification keeps an agent from treating every visual complaint as a material problem.

Work from a reversible baseline. Change one relationship, rebuild the affected geometry and capture numerical bounds before interpreting a new image. A plane becoming wider without a matching viewport metadata change is an internal inconsistency even if the image still looks acceptable. A moved cap with an unmoved shell label may be visually inconsistent even though its input anchor follows correctly.

Resource lifetime is another dependency. The existing controller disposes replaced display materials and maps; proposed geometry replacement should similarly account for the old geometry rather than retaining every revision indefinitely. Do not copy that disposal strategy blindly onto shared objects: first determine whether another mesh still references the resource. The factory intentionally shares several materials, including the shell/rim relationship, so mutation has a wider scope than one named mesh.

Finally, save both the intended effect and the invariant that must remain unchanged. “Move the diamond outward while keeping screen size” is testable; “improve the controls” is not. This makes later debugging a comparison of contracts rather than a reconstruction of undocumented taste.

MD10-Q01 Exercise: choose the correct editable object

Explained answer

A diagnostic operator wants the top display to fold while the lower touchscreen, battery cover and branding remain fixed. They propose rotating `root`, then counter-rotating every lower component. Explain the ownership error, choose the minimal correct transform, and identify one downstream calculation that must be refreshed.

MD10-A01 Explained answer

Back to question

Rotate `lid.rotation.x`, not the root. Counter-rotation replaces a clear hierarchy contract with many compensating transforms; every later base detail becomes another opportunity for a missed correction. The upper LCD already belongs to the lid and therefore follows automatically. The battery cover and underside branding remain root-owned. The hinge barrel also remains root-owned while the lid-owned leaves move with the shell.

Refresh articulated bounds and view matrices before trusting projected input. `dsPoseBounds()` evaluates separate root and lid transforms; a single cached flat rectangle cannot describe both poses. This is a geometric correctness requirement, not merely a camera convenience. The pose/bounds implementation (`src/scene.js:502`) supplies the actual application mechanism. This chapter read source and assertions; it did not rerun browser tests, render production images or establish owner approval.

PART 2 / THE ARTICULATED NINTENDO DS

MD11

Hinges, lids and articulation

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Hinges, lids and articulation

A hinge is first a coordinate-system decision

An articulated object fails when its pivot is chosen after its geometry. A lid centered on its own origin spins like a signboard; it does not fold around its lower edge. This factory instead places the lid group's origin at the joint and offsets its visible shell to local Y 2.43. The group receives an initial X rotation of 70 degrees and a Z offset of .07. The upper screen and surrounding details are children, so one transform carries the complete upper assembly.

Implemented evidence. The lid creation (`src/ds-handheld.js:71`) precedes the separately created barrel group. That ordering makes the visual joint independent of the mathematical articulation. No torque, detent spring, hinge friction or collision-constrained rigid-body joint is implemented by these groups. The physical mechanism is approximated through recognizable geometry and bounded pose editing.

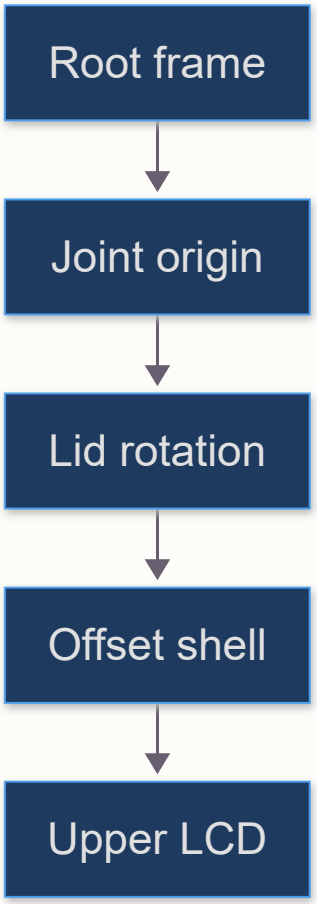


Figure MD11.1. Offset geometry creates the lever arm; rotation belongs to the joint frame rather than the shell center.

Construct the barrel without confusing its axis

Three.js cylinders are initially aligned along their local Y axis. The central barrel uses radius .255, width 6.90 and 48 radial segments, then rotates around Z by π/2 so its length lies along the hinge's X axis. A right lime cap at X 3.80 uses width .72. The left end is assembled from two half-cylinder sections at X −3.80 with angular starts −π/2 and π/2. Their names distinguish front and rear sections, but both use the same lime material at this cutoff.

Two collars sit at X ±3.43 with radius .258 and width .035. The slight radial increase creates a legible seam without making the joint look like disconnected cylinders. Lid-owned leaves at those X positions bridge the upper shell toward the barrel. The microphone, MIC legend and indicator recesses belong to the barrel frame, so they do not unexpectedly orbit the upper LCD. Inspect the complete joint details (`src/ds-handheld.js:111`).

The barrel is a visual approximation rather than a mechanically interlocking mesh. A continuous cylinder and narrow collars are inexpensive and readable at desk scale; modeling internal axles would add surfaces that the current interaction does not expose. If an extension requires an exploded mechanism, add that as a new documented model contract rather than claiming the existing continuous barrel already contains one.

Opening degrees and stored rotation are different quantities

The scene's getter reports `180 - radToDeg(lid.rotation.x)`. Its setter clamps an opening angle to 30–180 degrees and stores the complement in radians. Thus an opening of 180 corresponds to zero stored rotation, 145 corresponds to 35 degrees, and the factory's 70-degree rotation corresponds to a 110-degree opening. These conversions are not interchangeable descriptions of one raw number.

Human opening	Lid X rotation
180 degrees	0 radians
145 degrees	about .610865 radians
110 degrees	about 1.221730 radians
30 degrees	about 2.617994 radians

The actual setter and lock guards (`src/scene.js:467`) reject non-finite angles and edit only an unlocked DS in Inspect outside a transition. A correct-looking console command can do nothing if those preconditions are false. That is deliberate state ownership, not a conversion failure.

Research guidance. Current Object3D documentation, consulted 2026-11-07, describes rotations in radians and local positions relative to parents. This supports the mathematical explanation; it does not establish version-specific behavior beyond the pinned Three.js 0.186.1

implementation actually read. The factory uses ordinary groups and rotations, not a physics-joint library.

MD11-W01 Calculate a useful diagnostic pose

Use the lid center point `(0, 2.43, 0)` in the lid's local coordinates, ignoring shell thickness for this calculation. After an X rotation θ, the root-space point becomes approximately `(0, 2.43 cos θ, .07 + 2.43 sin θ)`. At a 145-degree opening, θ is 35 degrees, giving Y about 1.9905 and Z about 1.4638. The center rises away from the lower assembly in root space while retaining the joint origin.

This is not yet a room-space prediction. If the root has the tabletop quaternion, multiply the result by that root transform before calling a direction world-up or world-forward. The useful diagnostic outcome is relative: the upper screen changes orientation and the lower screen's local transform does not. A root rotation would move both, exposing a hierarchy error immediately.

The following original browser-module recipe assumes a constructor-capable DOM and imported Three.js. It creates a new diagnostic instance, not a live application mutation. Expected result: a 145-degree opening under the scene convention, with updated world matrices. No production render is requested.

```
import * as THREE from 'three';
import { createDsHandheld } from './src/ds-handheld.js';

const ds = createDsHandheld();
ds.lid.rotation.x = THREE.MathUtils.degToRad(35);
ds.root.updateWorldMatrix(true, true);
console.assert(
  Math.abs(ds.lid.rotation.x - Math.PI * 35 / 180) < 1e-12
);
```

There is **no hinge setter returned by the factory**. The returned `lid` is the exact factory API for this operation. In the application, use `view.setDsHinge(145)` only after entering unlocked DS Inspect and waiting for the transition to finish; that method accepts opening **degrees** and also reframes the camera. Do not call `setDsHinge(35)` expecting a 35-degree stored rotation.

Preserve three pose purposes

Desk pose, Inspect pose and optional gameplay lock serve different jobs. The desk has a remembered placement and resting hinge. Inspect allows independent root rotation and hinge edits. Locking records a clone of the root quaternion plus the current lid rotation; it does not rewrite the saved desk arrangement. Unlocked gameplay defaults to the flat, front-facing configuration.

Historical commit e885636 introduced independent hinge controls and locked gameplay poses. The associated construction task (`TODO/87_DS_HINGE_GAMEPLAY_POSE.todo:1`) explicitly separates the desk's 110-degree resting opening from gameplay. Those records ground the feature's purpose; they are not a fresh execution or a substitute for reading the current setter.

The pose lock is session state, not permanent storage. Cloning the quaternion matters because keeping a mutable reference would allow later root edits to mutate the supposed saved pose. Clearing drag and pinch state at the lock boundary similarly prevents an earlier gesture from continuing through a new ownership rule.

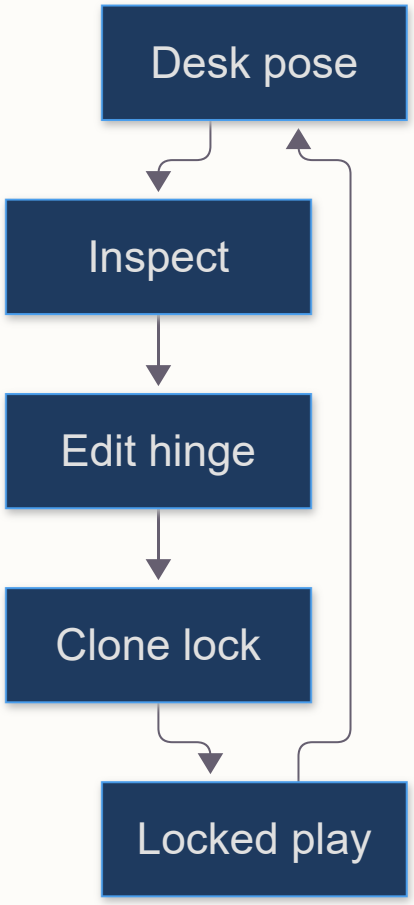


Figure MD11.2. A gameplay lock captures a chosen pose without converting that pose into the console's remembered desk placement.

MD11-R01 Retry with a nearly folded lid

Change the opening from 145 to 30 degrees. The stored rotation becomes 150 degrees, placing much of the upper assembly over the lower half. A previously visible control may now be covered even if the whole console remains inside the camera frustum. The correct retry does not increase exposure, disable depth testing or keep invisible controls clickable.

Recompute articulated bounds and update matrices, then let occlusion decide availability. `projectControls()` checks facing, clip-space limits and lid intersections before exposing projected HTML buttons. Its projection path (`src/ds-controller.js:684`) explains why a folded control can disappear correctly. For touch, the first relevant root intersection must be the lower LCD, not simply any intersection farther down the ray.

Also recheck materials at an oblique angle. Shared rounded solids obtain their bevels through ExtrudeGeometry construction (`src/handheld.js:19`). A highlight vanishing on a grazing face is not automatically a missing normal; first determine whether the face points away from the lamp or is physically hidden.

Precision checks for a convincing joint

Recommended diagnostic procedure. Inspect the joint from front, rear and a grazing side direction while holding the root transform fixed. A front view checks continuity across barrel sections; a rear view checks whether the leaves remain attached to the shell; the side view reveals an offset pivot that a frontal image can conceal. These are complementary questions rather than three decorative views of the same proof.

Measure in the joint's local frame before measuring in world space. A collar radius .258 versus barrel radius .255 creates a radial difference of .003. That is a deliberate small geometric distinction in the source, not a reason to round every number to two decimals. A rounded diagnostic export could erase the distinction even though the original geometry retains it. Conversely, do not report the .003 as a manufacturing clearance: both are visual surfaces and are not an axle-and-bearing tolerance pair.

If a collar appears detached, inspect its width and axial position independently of radius. A larger radius repairs a radial silhouette gap but cannot repair an axial separation. If a leaf penetrates the barrel, first decide whether the overlap is intentional cover geometry or a visible discontinuity. A mechanical mesh requires different checks from an economical visual joint, and the current source promises the latter.

For a proposed alternate shell size, preserve the pivot origin and change the child offset deliberately. Moving the lid center without revisiting its nearest edge can create an exposed gap at the joint. Calculate the edge position from half-height, center offset and bevel extension, then inspect the resulting seam through the allowed opening range. Checking only the 145-degree diagnostic pose can miss a collision or gap near 30 degrees.

Finally distinguish geometry precision from input precision. A correctly rotating lid can still leave stale hit targets if the scene matrix refresh is omitted. Record the opening angle, stored radians, root quaternion and matrix-update stage together. That record turns an intermittent “hinge bug” into a reproducible question about either geometry, state guards or projection.

MD11-Q01 Exercise: diagnose a lid that translates while folding

Explained answer

A reconstruction rotates a shell mesh about its own center and then updates the lid group's position on each frame to make the lower edge “approximately stay put.” The barrel is also parented under that group. Why is the result fragile, and what should a replacement validation measure?

MD11-A01 Explained answer

Back to question

The implementation has replaced one stable joint frame with two coupled approximations: rotation plus corrective translation. Small changes to shell dimensions invalidate the correction, and the barrel moves with the lid even though it should identify the fixed joint axis. Put the lid origin at the joint, offset shell geometry in the child frame, and keep the barrel root-owned. The replacement needs only one articulation transform.

Measure a joint-adjacent reference point over several openings and confirm it follows the intended hinge neighborhood rather than circling the shell center. Check that the lower LCD, battery cover and branding retain their root-local transforms. Then validate occlusion and actual input in the articulated pose. The browser's independent-hinge scenario (`tests/browser/ds.spec.mjs:927`) is read evidence for those application expectations, not a test rerun in this authoring task. A stationary hinge point alone cannot prove usable screens or correct projected input.

PART 2 / THE ARTICULATED NINTENDO DS

MD12

DS shells, recesses and hardware

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD12 / DS SHELLS, RECESSES AND HARDWARE

Contents

DS shells, recesses and hardware

Use depth where depth is part of the evidence

Small hardware details communicate how an object is assembled. A seam says two shells meet; a screw recess suggests fastening; an inset floor says a depression has depth. These meanings are not interchangeable. Painting a dark square can suggest a recess from one camera, but it does not create an aperture that a ray can pass through before hitting a lower surface.

Implemented evidence. The DS outer lid actually contains holes in its `Shape`. Two rounded-square paths are added before extrusion, while separate blue floors sit behind them. By contrast, the speaker openings are small dark cylinder meshes and the rear card slot is a dark rounded box. The construction deliberately mixes true outline topology with economical surface approximations. The lid aperture construction (`src/ds-handheld.js:77`) is the authority for which technique applies.

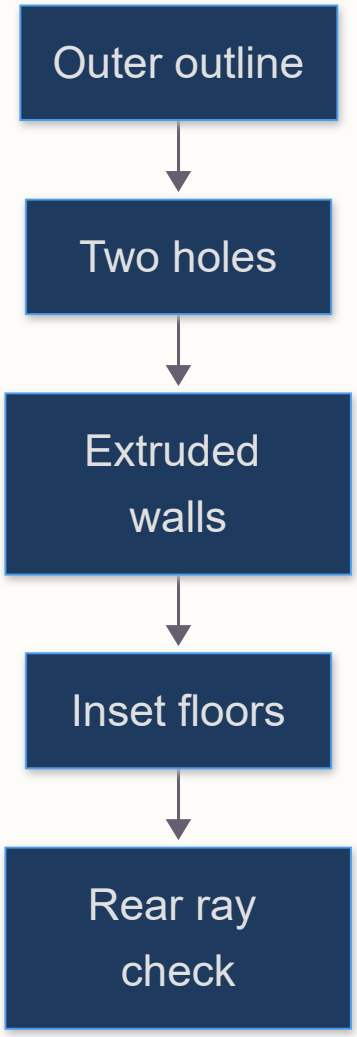


Figure MD12.1. A genuine modeled recess needs an open outline and a separately positioned floor, not merely a darker top-facing patch.

Reconstruct the outer lid cross-section

Start with a centered rounded outline 8.5 by 4.65 and corner radius .14. Generate each hole from a .54-by-.54 rounded shape with radius .055, sample its points and shift them by local Y ±.38. The hole X offset is zero. When the complete lid mesh is placed at Y 2.43, the hole centers become Y 2.05 and 2.81 in the lid group.

The outer geometry extrudes .12 with bevel size and thickness .02, three bevel segments and eighteen curve segments, then translates Z by −.06. Its mesh position adds another −.10. Ignoring curvature at the bevel, the main extrusion therefore spans approximately Z −.16 to −.04 before the bevel thickness expands that range to about −.18 through −.02. The inset-floor centers are at Z −.14, with .008 nominal thickness and the DS helper's small depth-limited bevel.

This arrangement is most usefully understood from the **rear**, looking along positive local Z. A ray through intact outer shell first encounters the rear skin around −.18. At a hole center that skin is absent, allowing the ray to continue to the floor around −.147. The floor is deeper into the lid from that rear viewpoint. Saying “behind” without specifying the viewing side can reverse the explanation.

The shared solid helper (`src/handheld.js:19`) explains centered extrusion and bevel expansion. **Research guidance:** the current ExtrudeGeometry documentation, consulted 2026-11-07, distinguishes depth, bevel size, bevel thickness and segment counts. It also notes that modifying the recorded constructor parameters does not rebuild geometry. Treat that documentation as current guidance; the pinned factory's actual values and geometry creation remain the implementation evidence.

MD12-W01 Verify two recesses and the land between them

The diagnostic needs three rear rays, not a screenshot of two dark squares. Cast along positive local Z through `(0, 2.05)`, `(0, 2.81)` and `(0, 2.43)`. The first two should hit the named inset-floor meshes; the midpoint should hit the metallic outer lid. Transform both origins and directions through the lid's current world matrix so the same test works after articulation.

Original pseudocode, assuming a constructed instance and current matrices:

```
for y in 2.05, 2.81, 2.43:
  origin = lid.localToWorld(0, y, -1)
  direction = transformDirection(lid.matrixWorld, 0, 0, 1)
  first = raycast(lid, origin, direction)
  record y, first.object.name
expect floor, floor, outer_lid
```

The expected names correspond to the actual browser surface test (`tests/browser/ds.spec.mjs:1324`). This chapter read that assertion; it did not execute it. If all three rays hit the outer lid, the hole paths are not cutting the intended outline. If the first two miss everything, inspect floor placement and face orientation before changing ray distances.

A rear-side ray is important because planes and mesh triangles have oriented faces. Blindly making every material double-sided can hide a reversed-normal defect. Diagnose winding, geometry transforms and viewing side first. Double-sided shading is a deliberate rendering choice, not a general repair for uncertain construction.

Lower-shell hardware is its own subsystem

The battery cover is root-owned on the underside: a 2.81-by-3.51 seam at X 2.35, Y −2.43, Z −.27, then a slightly smaller cover at Z −.278. Five screw recesses include one associated with that cover; their head slots sit farther toward negative Z. Four pale-green square pads occupy X ±3.65 and Y −.70/−4.30 at Z −.30.

The underside hardware construction (`src/ds-handheld.js:227`) uses overlapping shallow solids, not boolean cavities cut into every base layer. Screw slots are boxes, not full threaded fasteners. The pads clone button plastic and raise roughness to .9, separating a matte support surface from the metallic hinge trim. These approximations are appropriate to the existing desk model but should be disclosed in a reusable reconstruction manual.

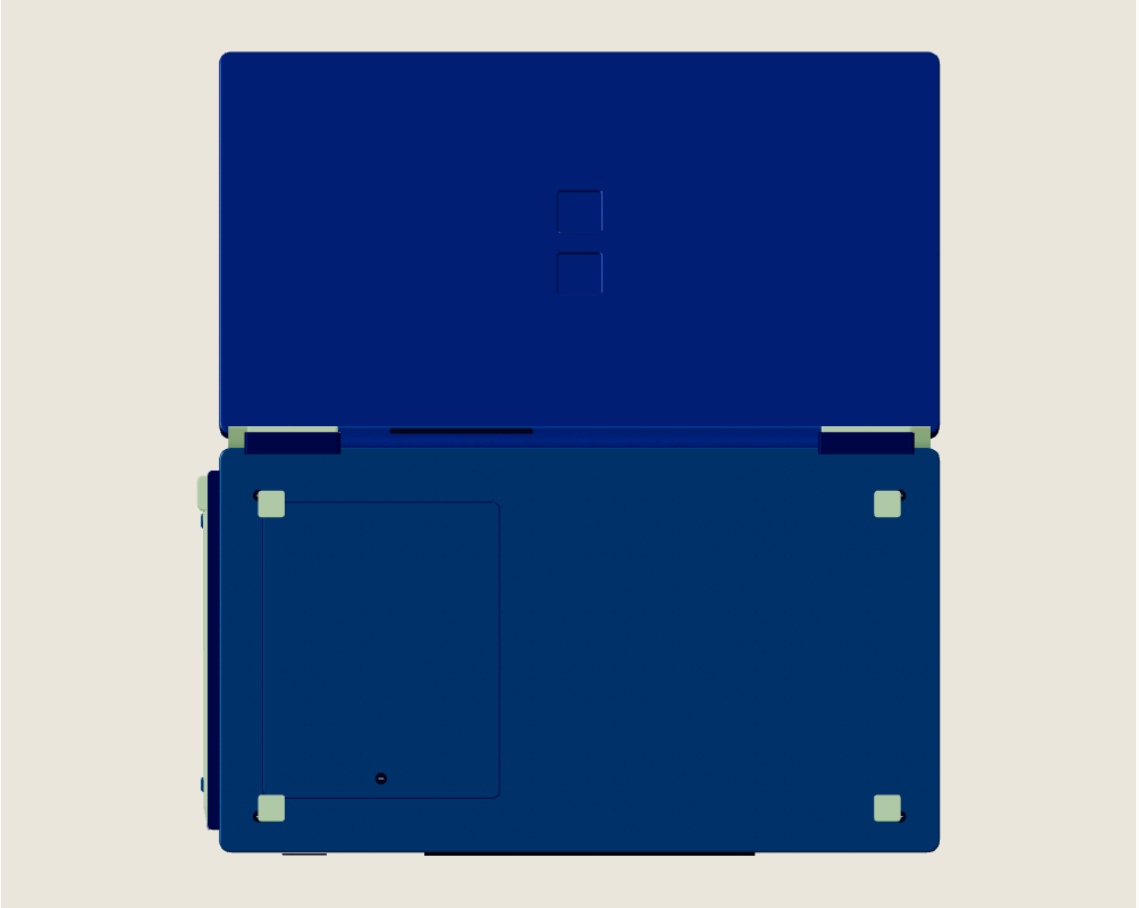


Figure MD12.2. Parent-supplied neutral-studio underside diagnostic for shell layers, battery-area layout and rear detail placement; constructor-only branding may remain hidden until its supplied texture is initialized.

The lower-base branding plane is centered at X −1.65, Y −2.43, Z −.265, faces outward after a Y rotation of π, and preserves a 2149:732 artwork aspect. It belongs to the lower root, clear of the right-side battery cover, not to the swinging lid. `initialize()` supplies its image later. See branding geometry (`src/ds-handheld.js:138`) and texture initialization (`src/ds-controller.js:196`).

Distinguish the two slots explicitly

The front navy expansion opening is placed at local Y −4.842 and rotated around X by π/2. It represents the lower/front expansion-slot feature, not the original hardware's rear DS-card slot. The separately named rear DS game-card slot is a 1.68-by-.18-by-.24 dark rounded box at X 1.4, Y .1, Z −.02. The insertable DS card is aligned to that rear-side region.

Do not infer insertion direction from the word “bottom” in a caption. Local Y becomes a different room direction after the root's table rotation. Nor should a dark slot-shaped box be described as a fully hollow connector with modeled contacts: the source supplies a visual opening approximation. The scene's root placement constants (`src/scene.js:18`) establish the transform boundary that makes direction language conditional.

Historical correction: topology and parenting changed together

Commit 6f5083a changed the hole-point offsets and inset-floor X positions from −2.8 to zero. The same diff moved the emblem from the lid to the base underside. This is stronger evidence than a generic statement that the rear “looked better”: both the topology and the floor placements were corrected, while branding changed ownership.

The rear-markings task (`TODO/95_DS_REAR_MARKINGS.todo:5`) records the requested centerline and battery-clearance requirements. Its historical verification is not a new verification of this manuscript. Preserve that distinction when turning a development ledger into teaching material.

MD12-R01 Retry after increasing the floor thickness

Suppose a proposed floor depth changes from .008 to .08 while its center stays at −.14. Its rear face moves much farther toward the outer skin and may become flush with or protrude beyond the aperture entrance once bevel thickness is included. The source name “inset floor” would no longer guarantee an inset appearance.

Compute the floor's actual geometry bounds after rebuilding, compare the rear-most floor Z against the aperture entrance, and repeat the three rays. If the goal is a deeper recess, moving the floor farther positive Z from the rear viewpoint may be appropriate, but check the inner shell and display layers too. Making a floor thinner or darker cannot compensate for incorrect world-space occlusion.

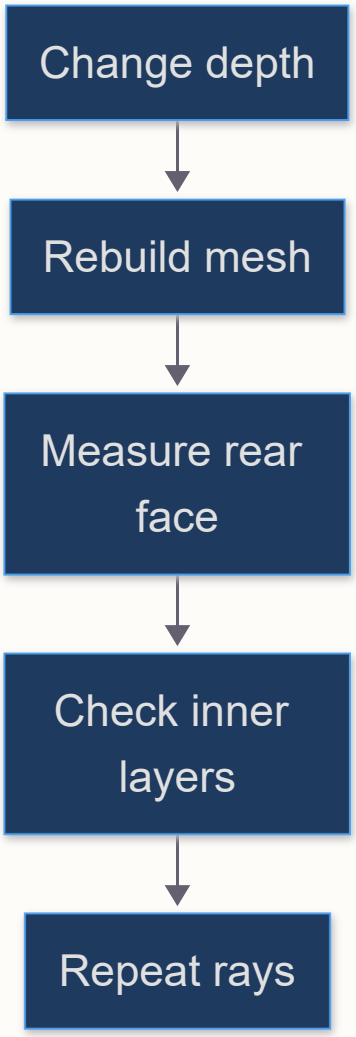


Figure MD12.3. A depth edit changes both exterior evidence and interior clearance; rebuilding and remeasuring are separate from changing a parameter label.

Debug topology without hiding it under materials

Recommended diagnostic procedure. Temporarily reason about the aperture with a single neutral material and explicit ray origins before comparing metallic highlights. This is a proposed inspection technique, not a change made to the shipped model. A blue floor can visually merge with a blue shell under diffuse illumination even when the recess topology is correct; a sharp reflection can make a shallow overlap look deeper than it is.

The floor is slightly wider than the nominal hole: .56 versus .54. That gives a nominal .01 overlap on each side before bevel details. The overlap helps avoid an uncovered edge when looking obliquely, but its sufficiency should be judged from generated geometry and the intended viewing range. It is not a universal anti-crack constant. If the hole radius or floor corner radius changes independently, the edge relationship changes even when both widths stay the same.

Inspect boundary continuity separately from shading continuity. A hole can exist while its side-wall normals produce an unexpected highlight, and a smoothly shaded decal can look recessed without any hole. The rear rays establish surface ordering; a side view establishes wall visibility; a normal-oriented lighting pass can help isolate shading. No single one of those observations replaces the others.

Also account for layered shell intersections. The rear skin, inset floors, perimeter lip and inner lid are separate meshes. A proposed deeper aperture might expose an unrelated layer rather than create empty interior space. Trace a ray through the entire lid hierarchy and record all relevant hit distances when diagnosing that condition, instead of stopping at a conveniently named first hit.

These practices preserve the distinction between an aesthetic recess and a physically bored opening. The existing paired recesses intentionally terminate at floors. Calling them through-holes into an empty enclosure would be as inaccurate as calling them flat dark decals.

MD12-Q01 Exercise: can a logo be moved onto the battery cover?

Explained answer

A proposed wider emblem still fits inside the total lower-shell rectangle but overlaps the battery-cover region. Its author argues that transparent pixels make the overlap harmless. Identify the missing geometric constraint and the ownership rule that must remain unchanged.

MD12-A01 Explained answer

Back to question

The constraint is not only the shell boundary. The emblem must remain in the clear area beside the battery cover, with positive measured separation from that cover. Transparent pixels do not change the plane's layout contract or preserve a readable service-panel boundary. Measure the actual plane bounds, not only its opaque artwork, and compare them with the battery cover in the same root space.

Keep the emblem root-owned and outward-facing. Moving it onto the lid to find more area would violate the documented lower-base placement even if the image became easier to see. The regression requires battery gap greater than .25 and meaningful shell margins; these are explicit model checks, not inferred hardware tolerances. The same discipline applies throughout this chapter: preserve named relationships, then improve appearance within them.

MD13

DS controls and readable markings

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD13 / DS CONTROLS AND READABLE MARKINGS

Contents

DS controls and readable markings

A control has more than a visible cap

For this model a control combines geometry, a rest transform, an input identifier and a projection anchor. Those components solve different problems. A cap communicates touchability; its socket communicates mounting; its anchor places an HTML input target; its identifier routes native button state. Editing only one can make a screenshot look corrected while the actual control still overlaps a bezel or receives input in the wrong place.

Implemented evidence. The factory stores controls in a `Map`. Face buttons use their mesh as anchor, while the D-pad uses four separate child anchors on one shared cross. The cap builder and face layout (`src/ds-handheld.js:145`) establish that construction. Socket radius is cap radius plus .025; clearing only cap centers cannot prove the whole assembly fits.

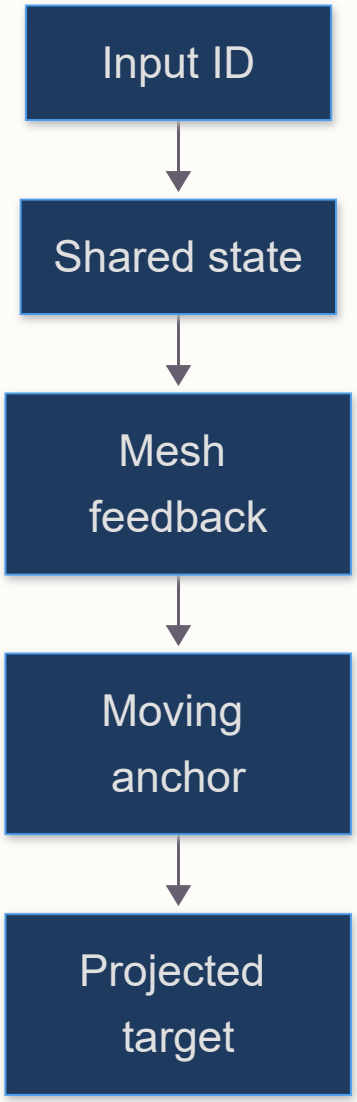


Figure MD13.1. Geometry and interaction meet at the anchor: a pressed visual should not leave its input target behind.

Lathe the caps rather than stacking flat discs

The cap profile starts on the axis, moves outward across the underside, climbs a rounded side and returns toward the axis over the top. For radius r, the radial samples include `r - .018`, `r`, `r - .015` and `.65r`; axial samples range from −.0425 to .047. `LatheGeometry` revolves this profile with 32 segments, then the geometry rotates around X by π/2 before placement at Z .29.

That rotation has a useful consequence: the lathe's original axial coordinate becomes local Z. The subsequent press animation can move `mesh.position.z` inward without changing the cap's orientation. The visible legend is a child plane at Z .052, so it follows the cap instead of floating above the deck. This is a geometric reason to attach a marking to its moving owner, not simply a convenient grouping habit.

The four .195-radius caps form a diamond at X/Y `(3.3, -1.32)`, `(3.84, -1.87)`, `(3.3, -2.41)` and `(2.75, -1.87)`. START and SELECT use smaller .095-radius blank caps with separate light labels on the shell. Power uses radius .20 plus actual torus-and-stem icon geometry. These differences prevent a one-size-fits-all label helper from dictating the hardware layout.

The text helper paints bold Arial into a canvas, then uses a transparent `MeshBasicMaterial` with depth writing disabled and tone mapping disabled. It chooses 96 pixels for a single character and 512 for longer labels. See the text and circle helpers (`src/ds-handheld.js:54`) and the shared texture helper (`src/handheld.js:68`). These are procedural raster markings on modeled parts, not engraved font outlines.

Build one molded D-pad with four semantic directions

The cross is a twelve-corner `Shape`, extruded .085 with .018 bevel size and thickness. It is placed in a group at `(-3.17, -1.83, .285)`. Four small rectangular direction marks occupy its arms, and four child anchors sit near the arm tips. All direction entries refer to the **same mesh group**; they differ in anchor and indicator.

The cross and anchors (`src/ds-handheld.js:165`) explain why the controller must compute one composite pose. Rotating the cross once per input entry would apply competing updates to one object. Instead, held inputs become a signed two-axis vector, normalized so a diagonal does not exceed the intended .16-radian combined tilt.

The response uses elapsed-time damping, with stronger release response than press response. A .012 compression accompanies any directional hold. Opposing directions cancel the corresponding rotational axis but still count as held and still highlight their individual marks. That combination distinguishes physical pose from semantic input state: zero tilt does not necessarily mean no buttons are held.

MD13-W01 Derive the Up-plus-Right pose

For Up and Right held, the controller computes `x = down - up = -1` and `y = right - left = 1`. Magnitude is √2, so target pitch is approximately −.113137 radians and target yaw approximately .113137 radians. Their Euclidean magnitude is .16, matching the cardinal target rather than exceeding it by √2.

Rest Z .285 becomes .273. For an approximate arm-surface point at local Y .5 and local Z .068, negative pitch lowers the upper arm in root-local Z. A small-angle estimate gives a top-versus-bottom edge difference around `sin(-.113137) ≈ -.1129` across unit separation, while the simultaneous yaw tilts the horizontal axis. Exact transformed heights should be measured with the mesh matrix, not inferred from Euler signs alone.

Original runnable arithmetic recipe, requiring only JavaScript:

```
const held = new Set(['up', 'right']);
const x = +held.has('down') - +held.has('up');
const y = +held.has('right') - +held.has('left');
const n = Math.max(1, Math.hypot(x, y));
const target = { pitch: x * .16 / n, yaw: y * .16 / n };
console.assert(Math.abs(Math.hypot(
  target.pitch, target.yaw
) - .16) < 1e-12);
console.log(target);
```

The expected output has negative pitch and positive yaw. It calculates targets, not damped intermediate frames. The actual feedback implementation (`src/ds-controller.js:634`) applies damping and exact near-target snapping. The unit assertions (`tests/unit/ds-controls.test.mjs:29`) check signed edge heights, diagonal bounds, neutral return and a surface remaining above .235.

Shoulder orientation turns local travel into edge travel

L/R housings sit at X ±3.4, Y .08, Z −.255 and rotate around X by −π/2. Their sockets and caps are children of those rotated housings. The existing generic depression moves cap local Z by −.065; under the housing rotation that becomes motion into the top-edge mounting direction rather than through the underside.

The shoulder construction (`src/ds-handheld.js:192`) is a reusable recipe: orient the control frame once, then retain a consistent local press axis. Do not scatter world-axis exceptions throughout the animator. However, the controller's projected facing normal is derived from the root, so do not claim every shoulder target is a full perspective-correct projection of its rotated cap surface.

Contrast is deliberately split between surfaces

The pale accent is `#C5F2B1`. Button lettering and the power symbol are dark navy; shell-mounted START, SELECT and POWER labels remain light. Metallic lime hinge trim uses physical paint, while matte pads use rough plastic. Read the material setup (`src/ds-handheld.js:35`). A single universal text color would lose contrast on either the pale caps or the darker shell.

Documented history. Commit c59de69 replaced repeated flat depression with bounded rocking and held-direction material changes. The D-pad task history (`TODO/97_DS_DIRECTIONAL_DPAD.todo:5`) records readability as the purpose. It does not justify adding extra lights or changing native input timing merely to improve a screenshot.

MD13-R01 Retry with Up and Down held together

Change the example to Up plus Down. Both signed axes become zero, but `held` remains true. The correct result is neutral rotation with .012 compression and both direction marks highlighted. An implementation that uses only vector magnitude to decide whether anything is pressed would incorrectly lift the cross and clear the cues.

Now add Right. The vertical axis still cancels; yaw reaches the cardinal .16 target. Release only Up and verify Down continues to contribute. Native state ownership matters here: the controller receives authoritative `bba:control` messages and uses them for visual feedback, rather than assuming a local keydown is the only source.

Projected controls also need current matrices and occlusion checks. The scene's immediate update path (`src/scene.js:494`) avoids waiting for another eligible GPU paint before refreshing target alignment. This is essential when the pose changes under sparse rendering.

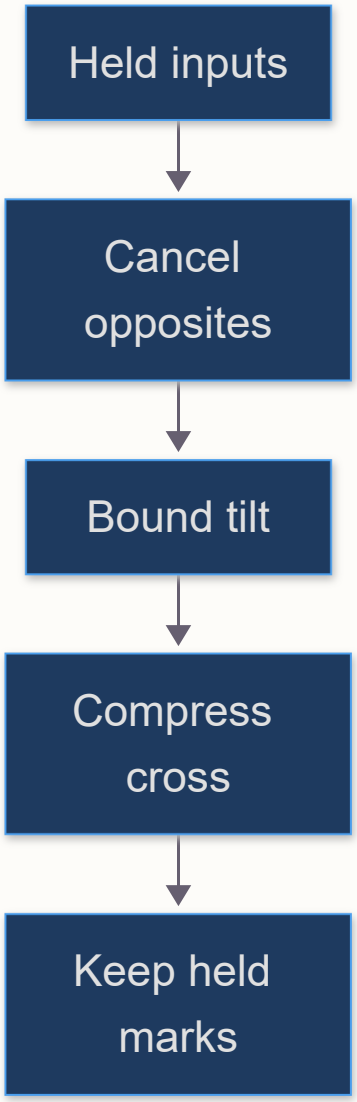


Figure MD13.2. Opposite directions cancel rotation, not their individual held state or the cross's shared compression.

Diagnose legends, normals and travel independently

Recommended reconstruction practice. Begin a marking check in the unpressed state, with a known camera and deterministic lighting. Confirm that the label is on the intended parent and lies in front of the visible cap surface. Then press the control and compare the relative transform. If the label remains at the old world height, its ownership is wrong; changing font size will not repair that defect.

The cap's lathed profile includes both a side wall and a gently raised top. Its appearance depends on the generated surface normals as well as color. A bright ring at the shoulder of the profile can be a legitimate specular highlight, while a discontinuity repeated at every segment can indicate

insufficient tessellation or a shading issue. The current 32-segment lathe is a concrete implementation choice, not proof that more segments always improve the result. Evaluate projected size before increasing geometry.

Travel must be checked in the correct local frame. Face caps start at Z .29 and depress by .065, leaving their group origins at .225. That origin can lie below the deck's front region while the cap's raised surface remains visible. Testing the origin alone would produce a false penetration diagnosis. The D-pad unit test correctly samples a surface point, not just the group center.

Contrast diagnosis likewise needs the right surface. Dark navy text on a pale cap and light text on a blue shell solve different contrast problems. A blanket rule such as “all legends should be white” discards the material context. If a label becomes unreadable only under one light, first separate a low-contrast legend from an overexposed cap, and compare the same geometry point across conditions.

For a proposed alternate theme, keep identifiers, anchor sizes and native mappings unchanged while evaluating new colors. That isolates visual design from behavior. Only after the theme passes geometry and contrast checks should a separate interaction change be considered. A reusable agent manual should make these change boundaries explicit so a typography edit cannot accidentally become an input remap.

MD13-Q01 Exercise: separate a geometry bug from a key-map bug

Explained answer

The Y cap is visually clear of the screen, but pressing A on the keyboard activates a shoulder when the emulator frame has focus. Should the modeler move the Y cap, enlarge its hit target or change the runtime mapping? Explain the evidence needed before editing.

MD13-A01 Explained answer

Back to question

This symptom depends on keyboard focus, not geometry. Inspect fixed DS mappings and the runtime owner before moving any mesh. `DS_CONTROLS` assigns KeyA to Y and KeyS to X, while shoulders use Q/W. The mapping declaration (`src/ds-cartridge.js:7`) establishes the intended routing, and the browser native-key scenario (`tests/browser/ds.spec.mjs:271`) checks scene and runtime focus separately.

A geometry repair cannot prevent an inherited keyboard preference from shadowing a DS binding. Conversely, correct native button IDs do not prove sockets clear the beveled screen. Keep both classes of evidence, name the failed boundary, and avoid using one successful test as a substitute for the other. The cited tests were read, not rerun here.

PART 2 / THE ARTICULATED NINTENDO DS

MD14

Dual LCDs and curved glass

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD14 / DUAL LCDS AND CURVED GLASS

Contents

Dual LCDs and curved glass

Correct the premise before modeling the glass

The chapter title names a useful topic boundary, not a claim that the DS constructor contains curved display glass. **At this cutoff both DS active surfaces are planar** `PlaneGeometry(4.5, 3.375)` meshes. The repository has a separate curved-outline LCD helper, but the scene's curve setter applies it to the GBA `this.model`, not to `this.ds`. Applying that helper to the DS by association would invent an authoring step and risk breaking lower-screen UV input.

The DS display loop (`src/ds-handheld.js:98`) is decisive. Read it alongside the shared curved-outline helper (`src/handheld.js:49`) and GBA curve setter (`src/scene.js:1122`). “Curved” there describes a shaped outline with explicitly remapped UVs, not evidence of a bulging DS surface.

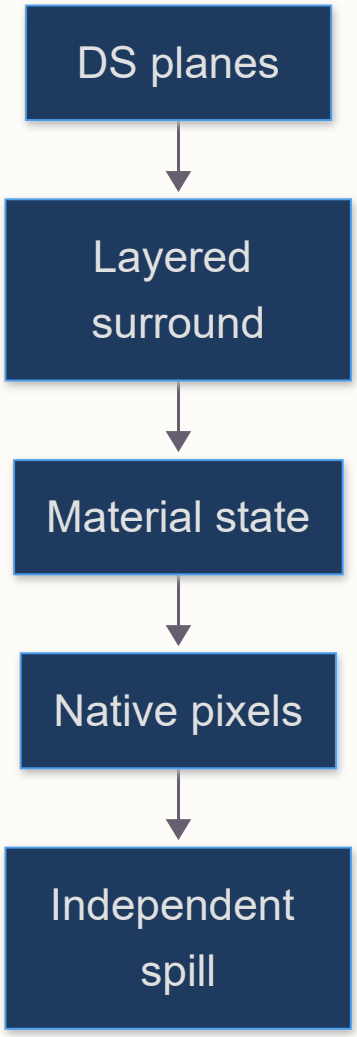


Figure MD14.1. The DS display illusion comes from geometry layers and state-specific materials, not an undocumented curved overlay.

Construct a display cross-section from outside inward

Each screen assembly starts with a fine cyan bezel edge, nominally 4.88 by 3.77 and .035 deep. A 4.80-by-3.69 metallic surround follows at Z offset .018, then a 4.60-by-3.475 black recess at offset .048. The active 4.5-by-3.375 plane sits at offset .085. The upper assembly's base Z is .18; the lower assembly's base Z is .265.

Thus the active upper plane is at lid-local Z .265 and the active lower plane at root-local Z .350. Each is forward of its recess. These offsets separate visible layers and reduce coplanar competition. They do not form a physically simulated stack of polarizers, liquid crystal, cover glass and air gaps. Using that manufacturing vocabulary without qualification would overstate what the meshes contain.

The common aspect is `4.5 / 3.375 = 4/3`, matching `256 / 192`. Both geometry objects expose `userData.viewport` width and height for illumination. The lower surface also carries a touchscreen flag. The upper parent is `lid`; the lower parent is `root`. Parenting is part of the display definition because the upper emission and picking surfaces must fold with their shell.

Short, nonzero layer offsets should be evaluated relative to the camera's depth precision. If flicker appears only at certain distances, first inspect nearly coplanar triangles and material depth settings. Do not solve every flicker by increasing all offsets: that can make the screen look detached from the bezel and alter occlusion near the controls.

Powered-off glass and active LCDs obey different invariants

The bare factory creates dark unlit screen materials. The controller subsequently uses `showPowerOff()` to replace them with `MeshPhysicalMaterial`: dark green-black color, roughness .24, clearcoat 1 and clearcoat roughness .22. This is reflective powered-off glass. It is expected to respond to lighting and view angle.

Status and active native screens instead use `MeshBasicMaterial` with `toneMapped: false`. Status screens use procedural 512-by-384 canvases; native outputs use 256-by-192 canvases with nearest filtering and mipmaps disabled. The controller disposes replaced materials and maps before assigning new ones. See the power-off and status states (`src/ds-controller.js:266`) and native material handoff (`src/ds-controller.js:426`).

The active invariant is narrow and important: scene-light changes should not relight or tone-map the active pixel surface. It does **not** say powered-off glass must remain unchanged, or that screen pixels are unaffected by texture sampling, browser color management or the selected framebuffer content. The upright-lighting regression (`tests/browser/ds.spec.mjs:1425`) deliberately selects static status LCDs before comparing pixels across lighting states.

Split the combined framebuffer without resampling twice

`dsScreenRegions(width, height)` accepts a top/bottom image whose width is an integer multiple of 256 up to 2048 and whose height is 1.5 times its width. It returns equal upper and lower halves. For native width 256 and height 384, the regions are `(0,0,256,192)` and `(0,192,256,192)`.

The region validator (`src/ds-cartridge.js:48`) rejects side-by-side layouts and gaps rather than trying to guess. `paintScreens()` copies each region into its own output canvas with smoothing disabled. The copy path (`src/ds-controller.js:665`) then marks the textures for update.

This separation matters for debugging: a wrong half-image can be a region-layout failure even when geometry, materials and UVs are correct. Conversely, correct output canvases can still appear wrong if the upper and lower texture references are swapped. Inspect the intermediate canvas before touching the model.

MD14-W01 Calculate pixel scale and a touch location

One horizontal pixel corresponds to `4.5/256 = .017578125` model units. One vertical pixel corresponds to `3.375/192`, exactly the same value. This equality is a useful construction check: the planar display has square source pixels in its own coordinate system, independent of perspective foreshortening.

At UV `(0.25, 0.75)`, the touch converter yields X `floor(.25 × 256) = 64` and Y `floor((1 − .75) × 192) = 48`. The vertical reversal maps the plane's upward-increasing UV coordinate to the native screen's downward-increasing row index. At UV 1, clamping prevents the inclusive edge from producing an out-of-range pixel.

Original arithmetic recipe, suitable for Node or a browser:

```
const width = 4.5, height = 3.375;
console.assert(width / 256 === height / 192);
const uv = { x: .25, y: .75 };
const pixel = {
  x: Math.min(255, Math.floor(uv.x * 256)),
  y: Math.min(191, Math.floor((1 - uv.y) * 192))
};
console.log(pixel); // { x: 64, y: 48 }
```

Assumptions are an accepted lower-LCD ray hit and UVs inside zero through one. This recipe does not perform the raycast or permit clicks through the lid. Preserving those preconditions is as important as preserving the arithmetic.

Emission is modeled separately from displayed color

Two `ScreenIllumination` instances attach area lights to their respective screens. Each uses the viewport metadata, a small sampled canvas, linear-channel averaging and damped light color. The additional control-spill emitter is explicitly labeled experimental in source; it approximates first-bounce visibility for nearby controls rather than solving global illumination.

The illumination implementation (`src/lighting.js:126`) samples live sources at intervals of 125 ms and derives intensity from the peak averaged channel. The scene update (`src/scene.js:1353`) clears and hides DS illumination when no powered source exists. This keeps lighting separate from the active LCD material.

Research guidance. Current Object3D documentation, consulted 2026-11-07, supports the explanation that child emitters inherit their screen transform. That current reference is not a claim that every newer rendering feature is used by Three.js 0.186.1 here. Inspect actual light classes and renderer paths before proposing a different illumination system.

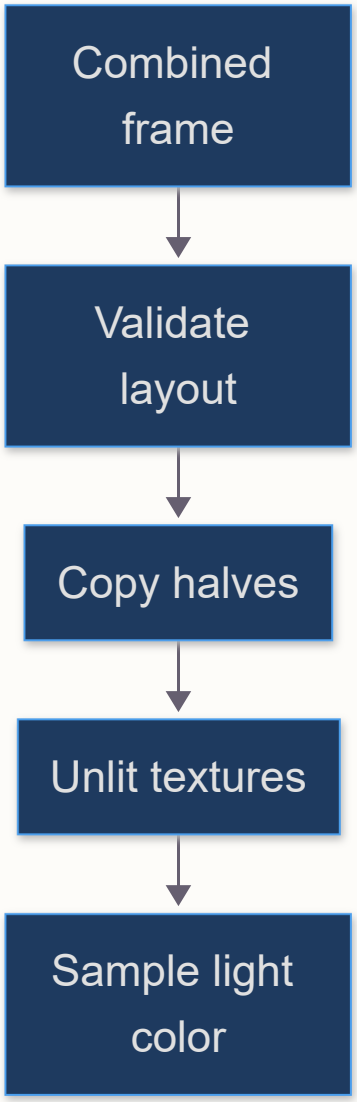


Figure MD14.2. Pixel presentation and illumination share a source image but use different materials and update responsibilities.

MD14-R01 Retry with a proposed curved DS surface

Change the assumption: an extension now requests an actually bulging DS cover. That is a **proposal**, not the existing workflow. A geometric bulge changes normals, silhouette and ray intersections; an outline-only shape changes the boundary but remains planar. Decide which effect is needed before choosing an algorithm.

For a true bulge, define a height function, preserve a well-understood UV map, recompute appropriate normals and test lower-screen ray-to-pixel correspondence over the full surface. Keep active pixel material behavior separate from any reflective overlay. A transparent overlay can become the first ray hit and block input unless picking ownership explicitly accounts for it. Never add it and then globally ignore every non-LCD hit; that would also allow touches through the folded lid.

A failure tree for apparently damaged pixels

Recommended diagnostic procedure. Start at the output canvas rather than the final camera image. If the canvas already contains the wrong half-frame, investigate layout validation and region copying. If it contains correct pixels but the model shows the other display, investigate screen-array ownership and map assignment. If both are correct but diagonal edges look soft, investigate filtering and projected scale before changing geometry.

Nearest filtering preserves discrete source samples; it cannot guarantee every source pixel occupies an integer number of final screen pixels under perspective. A tilted plane naturally foreshortens, and minification can produce uneven apparent cell widths. The honest invariant is that the application avoids an additional smoothing copy and uses nearest texture sampling for native outputs. “Perfect integer pixels at every angle” would be an unsupported stronger claim.

The status-texture path and native-output path also have different source resolutions. Status is drawn at 512 by 384, while native output canvases are 256 by 192. Both share the same physical viewport aspect, but a text edge in one is not an appropriate one-pixel reference for the other. Choose a deterministic fixture that matches the property being measured.

For a proposed reflective active overlay, write the visibility and input consequences before implementing it. A clearcoat-like sheen on the same material is not equivalent to a separate transparent mesh. A separate mesh can introduce additional depth ordering, reflections and first-hit ownership. If the overlay should not receive native input, that exception must remain narrow enough that shell and lid occlusion still work.

Finally inspect normals and front/back visibility in an oblique pose. The planar LCD has a defined front side; rotating the entire console behind the camera-facing direction does not make the display physically visible through its back. Making the material double-sided may be appropriate for a diagnostic, but it changes the presentation contract and must not silently become the production repair.

These recommendations extend the source-grounded workflow without claiming an unimplemented glass stack. They give a later modeler an ordered experiment: source image, texture, geometry, material, projection and occlusion. Changing all six at once prevents a useful causal diagnosis and makes any apparent improvement difficult to preserve.

MD14-Q01 Exercise: why did the lighting test fail with power off?

Explained answer

A test expects identical LCD RGB values with the lamp on and off, but it never powers the DS or installs status textures. It observes different pixels and proposes making all screens unlit permanently. Identify the mistaken fixture and the functionality that proposal would remove.

MD14-A01 Explained answer

Back to question

The fixture selected reflective powered-off glass, not an active LCD. Its lighting response is intentional. Install a deterministic active status texture before testing the unlit, non-tone-mapped invariant; qualify reflective glass separately with view-dependent expectations. Making every state permanently unlit would remove the powered-off material distinction rather than repair the test.

The historical upright-lighting task (`TODO/116_DS_UPRIGHT_LIGHTING.todo:40`) records exactly this fixture correction. Its reported outcomes are historical evidence, not tests rerun for this chapter. The lesson is to state the material state before stating the invariant.

MD15

Stylus construction and touch geometry

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD15 / STYLUS CONSTRUCTION AND TOUCH GEOMETRY

Contents

Stylus construction and touch geometry

Separate the prop, the pointer and the contact

A stylus appears simple until it must work over an articulated screen. Three distinct systems are involved: the docked pen is part of the console silhouette; the held pen is a camera-oriented visual; the native touch contact is a pixel coordinate plus a pressed state. Making the held mesh itself the input sensor couples decoration to gameplay and can cause the pen to intercept its own ray.

Implemented evidence. The constructor creates a docked group at `(4.39, -2.43, -.025)`, while the controller clones that group into a separate scene object, scales it to .8 and disables raycasting on every clone descendant. The original remains in the DS hierarchy and is hidden while held. See the docked construction (`src/ds-handheld.js:248`) and held-clone setup (`src/ds-controller.js:25`).

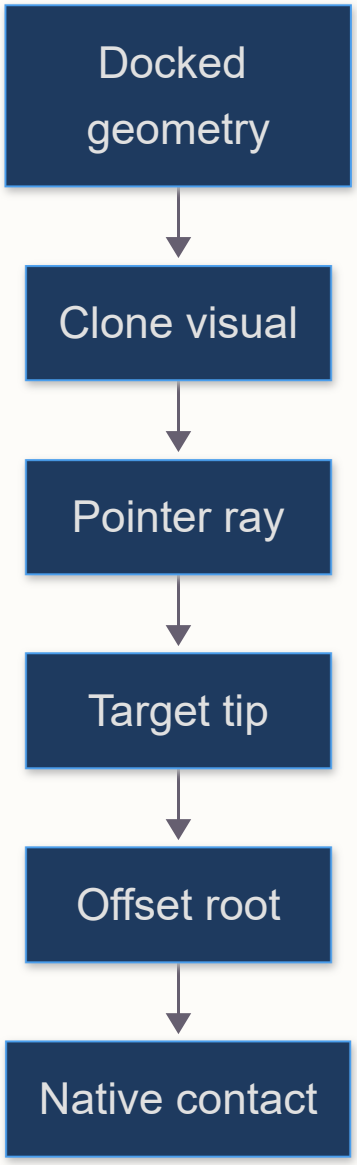


Figure MD15.1. The held mesh illustrates the contact; it does not replace the lower LCD's ray-to-pixel input contract.

Assemble the pen around a known contact point

The shaft is a 3.64-long cylinder with radii .073 and .065, positioned slightly off the group center. A flat pull tab, short retention hook and release groove establish the docking end. The other end uses a .20-long tapered cylinder, followed by a sphere of radius .032 scaled by `(1, 1.8, 1)` to make a short rounded nib.

The exact contact anchor lies at Y `-2.07 - .032 × 1.8 = -2.1276`, not at the sphere center. That distinction matters when solving a held pose. Aligning the group origin or nib center to the pointer would leave the visible tip systematically displaced. A named `Object3D` anchor records the intended extreme without requiring the controller to rediscover mesh topology.

The shaft's cylinder orientation already follows the pen's local Y direction, unlike the button cylinders rotated to face local Z. Keeping those conventions explicit avoids an incorrect held quaternion. The controller aligns local positive Y toward the camera-side “back” direction; the nib lies toward negative Y.

The docking channel is a separate dark rounded box at X 4.27; two shell-owned clips remain on the base. The hover halo is a back-sided transparent cylinder with depth writing disabled, hidden by default. Stylus plastic is cloned from button material so hover emissive changes do not recolor every console cap. These are small ownership choices with large debugging value.

Solve the held pose from the tip backward

`paintStylus()` converts the pointer into normalized device coordinates and casts through the camera. It takes the first non-stylus intersection with the DS root, falling back to a camera-facing plane through the lower display's world position. That fallback keeps the visual moving outside the screen; it does not create valid native touches outside the LCD.

The target tip is lifted along the **pointer ray**, by −.06 while hovering or −.008 during contact. A shift along a perspective ray changes depth without moving the projected point. A shift along the LCD normal generally changes screen position in an oblique pose. This is the geometrical reason for the implementation's direction choice.

The controller chooses a camera-relative back vector with X/Y signs adjusted near the top and right edges, rotates local positive Y to it with `setFromUnitVectors`, then computes the transformed tip offset. The group position becomes `targetTip - rotatedScaledLocalTip`. The complete placement algorithm (`src/ds-controller.js:611`) is a tip-constraint solver, not a cursor sprite approximation.

MD15-W01 Calculate the root-to-tip offset

With uniform scale .8, the local tip's Y magnitude becomes `2.1276 × .8 = 1.70208`. If a diagnostic orientation maps local positive Y to world positive Z, the nib offset is `(0,0,-1.70208)`. To put the nib at target `(1,2,3)`, place the group root at `(1,2,4.70208)`. The back extends toward the camera while the tip remains at the requested contact.

Original pseudocode, assuming a unit quaternion q, scale s and an already chosen world target:

```
localTip = (0, -2.1276, 0)
offset = rotate(q, componentMultiply(localTip, s))
rootPosition = targetTip - offset
reconstructedTip = rootPosition + offset
require distance(reconstructedTip, targetTip) < tolerance
```

The expected reconstruction error is numerical roundoff. This recipe isolates attachment math from raycasting, so it can diagnose a wrong anchor or scale without needing native gameplay. For nonuniform scale, component multiplication must happen before quaternion rotation, matching

the source's operation order.

Research guidance. Current Three.js Raycaster documentation, consulted 2026-11-07, describes NDC input, distance-sorted intersections and hit UVs. Its face-orientation warning is relevant when a rotated display unexpectedly stops receiving hits. The recommendation is to inspect facing and ownership first, not to assume the current documentation proves every behavior of the pinned 0.186.1 application.

Native touch belongs to the lower display only

`touchAt()` casts through the entire model, ignores stylus-tagged hits only while the pen is held, and accepts a contact only if the first remaining object is `screens[1]`. It does not search arbitrarily deeper for any LCD hit. A folded lid or shell in front of the screen therefore blocks input.

The touch hit gate (`src/ds-controller.js:490`) feeds UV conversion (`src/ds-cartridge.js:55`). Finite UVs within zero through one map to X 0–255 and Y 0–191, with vertical inversion. Input capture preserves a single active pointer, while cancellation and lost capture release the native contact.

Direct mouse or touch drawing remains valid without holding the modeled pen. This is important for accessibility and testing: the decorative pickup workflow should not become an unexplained prerequisite for the actual touchscreen. Conversely, moving the held visual off the LCD must not produce out-of-bounds contact coordinates.

Contact edges must survive asynchronous delivery

`DsTouchQueue` stores down, move and up samples. Consecutive move samples can replace one another, but contact edges remain separate. After each dispatched sample, the queue waits for `nextFrame`, preserving observable transitions across native polls. A generation counter invalidates stale work after cancellation; cancellation also releases a previously dispatched down contact immediately.

The queue implementation (`src/ds-touch.js:11`) and unit cases (`tests/unit/ds-touch.test.mjs:16`) distinguish edge preservation from throughput. Coalescing an entire quick tap into a final unpressed sample would be efficient but wrong: the native game might never observe contact.

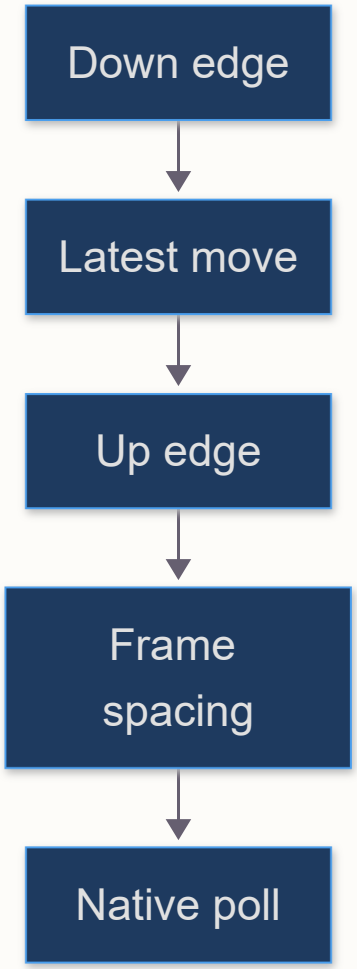


Figure MD15.2. Move compression is safe only when the beginning and end of contact remain independently observable.

MD15-R01 Retry under an oblique locked pose

Change the assumption from a flat front-facing display to a rotated body with a partially folded lid. A constant screen-space offset may appear correct at the center yet drift at the corners. The corrected approach keeps using a camera ray, obtains the actual hit point and solves the tip offset in world space. Do not repair drift with a hand-tuned X/Y cursor correction.

Refresh camera and root matrices immediately after pose or viewport changes. The scene update boundary (`src/scene.js:495`) explicitly avoids waiting for another GPU paint before projection and touch rays become current. When debugging, separate stale matrices from a wrong UV map: the first shifts intersections; the second shifts pixels after a correct intersection.

The browser orientation scenarios (`tests/browser/ds.spec.mjs:413`) cover locked orientations, zoom and high-DPI touch. They were read, not rerun here. Historical commit ofdd81b introduced the pickable pen; that history supports feature provenance, not a universal accuracy guarantee.

Debug the pen with three separate errors

Recommended diagnostic procedure. Record visual-tip error, hit-point error and native-pixel error independently. Visual-tip error is the distance between the projected nib anchor and pointer. Hit-point error asks whether the ray reached the intended visible surface. Native-pixel error compares the accepted UV conversion with the game-space location. A correct-looking pen can still draw at the wrong pixel, and correct drawing can coexist with a visibly misplaced pen.

To isolate the first error, hold a deterministic quaternion and scale and test the tip reconstruction formula without a raycaster. To isolate the second, remove only the held clone from consideration and record the first real DS hit. To isolate the third, log accepted UVs and converted coordinates without introducing a hand-tuned cursor offset. These are proposed diagnostic steps, not operations performed during this authoring task.

The pen's cloned material references also deserve care. The model separately clones stylus plastic from the button material, which isolates hover treatment from console buttons. Cloning the object hierarchy does not imply a fresh independent material for every descendant. A later extension that recolors only the held pen should inspect material sharing first rather than assuming scene ownership guarantees material ownership.

At pointer exit, the held visual may disappear while its held state remains available for return. That is distinct from a native pressed contact, which must be released when the input path no longer has a valid lower-LCD hit. Treat visibility, pickup state and contact state as separate booleans in a debugging log. Otherwise an invisible cursor can be misdiagnosed as a stuck native press.

For touch cancellation, verify the release edge as well as the final queue contents. Emptying pending samples alone does not release a contact already dispatched to the runtime. The generation mechanism and immediate release in `cancel()` address that distinction. This is a reusable principle for any modeled tool whose visual movement is more frequent than the native consumer's polling: preserve semantic edges even when intermediate motion is compressed.

MD15-Q01 Exercise: why keep an invisible docked pen?

Explained answer

A proposed optimization removes the docked pen from the DS hierarchy while held and reparents it to the scene. The camera then changes framing as the pointer moves. Explain the coupling and choose a safer visual strategy.

MD15-A01 Explained answer

Back to question

The pose-bounds calculation traverses geometry in the DS hierarchy, including geometry whose visibility may be false. The original implementation deliberately retains the docked geometry as a stable part of that envelope and moves only a scene-owned clone. Reparenting changes the

envelope; moving a still-parented held pen can make the envelope chase the pointer. Neither effect is desirable for stable gameplay framing.

Keep the original docked transform, hide its visual during pickup and move the non-pickable clone. Return paths should restore visibility, release contact and reset cursor state. This is not a general claim that invisible meshes never matter: here they remain relevant to the specific bounds traversal. Read ownership and consumers together before deleting apparently redundant geometry.

MD16

Cartridge insertion and loading choreography

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD16 / CARTRIDGE INSERTION AND LOADING CHOREOGRAPHY

Contents

Cartridge insertion and loading choreography

There are three different loading stories

Selecting a file begins application work, not merely a mesh animation. The console must become visible while identification is pending; a loading LCD must communicate progress or waiting; a card prop may move toward its seated position. Native startup is a fourth boundary whose completion is not determined by the prop's position.

Implemented evidence. `load()` closes DS tools, selects the DS and enters play before awaiting ROM identification. It calls `loadingStatus('READING LOCAL ROM')` before the expensive asynchronous work. Only after identification and retained-state lookup does it clear the old session, retain the new file and configure card insertion. The load ordering (`src/ds-controller.js:345`) is the authoritative workflow, not a generic “load then show console” sequence.

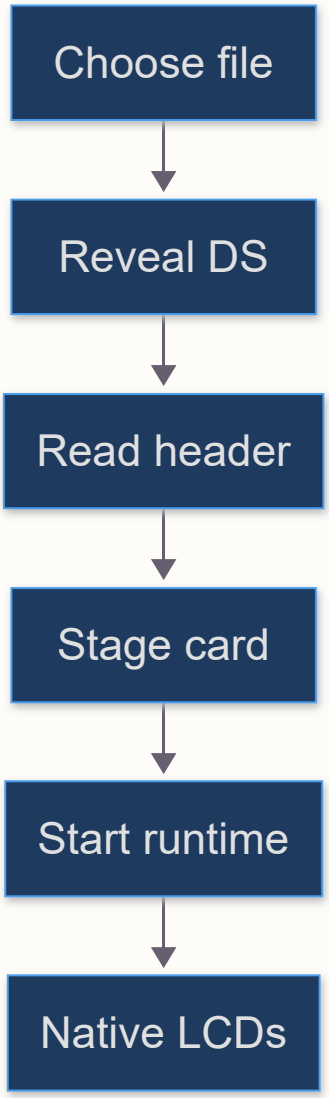


Figure MD16.1. Presentation begins before slow file work, while native readiness remains a separate success condition.

Model the card and identify the correct opening

The root-owned card group starts at `(1.4, .44, -.02)` and is hidden until needed. Its dark shell is 1.46 by 1.56 by .13, with a smaller raised label and two procedural text lines. It is not a detailed scan of a commercial cartridge. The card geometry (`src/ds-handheld.js:239`) uses the same rounded-box and text vocabulary as the console.

The nearby rear DS game-card slot sits at X 1.4, Y .1, Z –.02. The front navy expansion opening sits near Y –4.842 and has a different orientation. **The modeled bottom/front expansion slot is not the actual original hardware's rear DS-card slot.** Describing the animated DS card as entering the bottom opening would conflate two separately authored features.

The card translates along **root-local Y**, from 3.1 toward .44. Depending on the root quaternion, that is not necessarily world-down, room-back or screen-down. The scene pose constants (`src/scene.js:18`) show why a direction must name its coordinate space. A camera-facing image can be useful evidence of presentation, but not proof of a mechanically exact connector path.

The slot is a dark rounded box, not a fully modeled socket with contacts, retention spring and collision-tested internal cavity. The card visibly approaches a representative slot region. If an extension requires physical insertion tolerances, it needs new cavity topology and clearance tests rather than stronger adjectives for the existing animation.

DS insertion uses painted updates

When motion is allowed, `load()` places the card at Y 3.1 and creates `{elapsed: 0, lastPaint: null}`. Reduced motion seats it directly at .44 and creates no insertion motion. During controller `paint()`, elapsed time increases by the nonnegative difference from the previous paint; the first call adds zero. Fraction is clamped at `elapsed/1800`, eased with `f²(3−2f)` and used to interpolate Y.

The DS animation code (`src/ds-controller.js:624`) is called after the scene's eligible-paint gate. Read the scene scheduling boundary (`src/scene.js:1339`). It is inaccurate to call this an unconditional wall-clock timer independent of rendering.

The nuance is equally important: elapsed differences between painted calls can include a long real-time gap. The DS path does not reserve a midpoint boundary inside its 1800 ms slide. A sparse subsequent paint can therefore advance a long way. “Paint-driven” does not guarantee every intermediate pose is shown.

Do not borrow the GBA's timing contract

The shared `cartridge-motion.js` helper belongs to the scene's other cartridge path. It defines 850 ms framing and 1350 ms sliding, preserves elapsed time when paint is not due, and clamps advancement at a framing boundary before allowing seating. `transition-motion.js` similarly reserves a painted midpoint for desk transitions.

The cartridge helper (`src/cartridge-motion.js:4`) and transition helper (`src/transition-motion.js:1`) explain the repository's broader sparse-paint strategy. They do **not** make the DS card use those exact durations or boundaries. Reusable-agent documentation must identify which caller owns a helper before transferring its guarantees.

MD16-W01 Calculate the DS halfway position

Assume the first eligible DS paint records time 100 ms and elapsed remains zero. A later paint at 1000 ms adds 900 ms. Fraction is .5, and smoothstep easing is also .5. The card position is `3.1 + (.44−3.1) × .5 = 1.77`. Another 900 ms of painted-call interval reaches elapsed 1800, Y .44 and the seated dataset state.

Original arithmetic recipe:

```
function cardY(elapsed) {
  const f = Math.min(1, Math.max(0, elapsed) / 1800);
  const e = f * f * (3 - 2 * f);
  return 3.1 + (.44 - 3.1) * e;
}
console.assert(Math.abs(cardY(900) - 1.77) < 1e-12);
console.assert(Math.abs(cardY(1800) - .44) < 1e-12);
```

Assumptions are the current DS endpoints and duration, not a physically measured insertion speed. The zero derivative at both ends reduces abrupt starts and stops in the displayed motion. It does not simulate friction, a latch click or card mass. The expected result is a reproducible root-local position at a chosen interpolation fraction.

For comparison, a 450 ms elapsed value gives fraction .25 and ease .15625, producing Y 2.684375. Equal time steps do not produce equal travel distances. This is useful when checking captures: a slowly moving initial card can be correct easing rather than a stalled loader.

The loading LCD is an illustration, not a progress meter

`paintLoading()` chooses one of twelve frames with `floor(time/160) % 12`; reduced motion fixes frame zero. It draws an original card and slot illustration, text and three cycling dots onto the lower status texture. Its card Y is `52 + min(frame/8,1) × 148`, with clipping around the illustrated slot.

The loading painter (`src/ds-controller.js:310`) does not measure bytes downloaded or native initialization percentage. Calling its loop “90% complete” would invent information. It communicates that the application is working and names the current phase: reading a local file or starting DeSmuME.

Canvas generation uses the shared texture helper (`src/handheld.js:68`), while native success replaces status maps with separate 256-by-192 outputs. The visual card loop and 3D card can coexist without being synchronized mechanical representations of the same event.

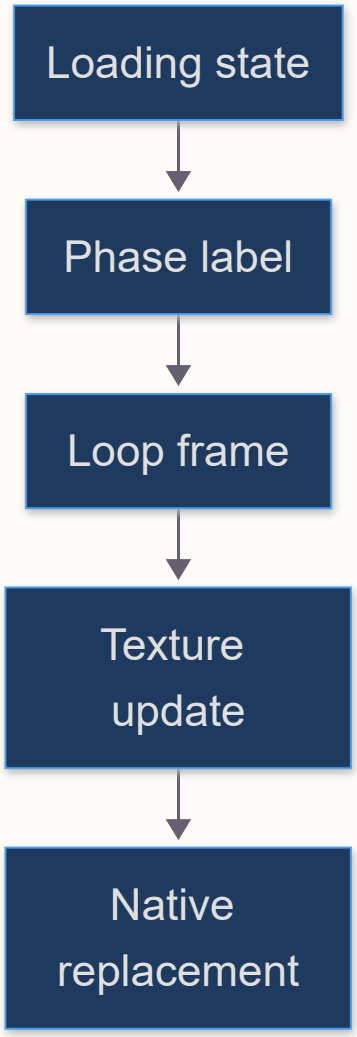


Figure MD16.2. The loading illustration signals a phase, while real native startup determines when live display textures replace it.

MD16-R01 Retry with a rejected replacement file

Change the assumption: an existing session is running and the replacement file is invalid. `loadingStatus()` avoids replacing active screen materials when `running` is true, although it still records loading state. Identification occurs before `clear()`. A rejection therefore need not destroy the previous source, frame or game identity.

The browser upload scenario (`tests/browser/ds.spec.mjs:116`) explicitly checks retained session objects and restored native pixels after an invalid replacement. It also checks reduced-motion loading and cancelled startup separately. These assertions were read, not rerun here.

A corrected retry preserves the prior session until a new file passes identification, clears loading state in the finalizer and surfaces the existing error UI. Do not equate a failed file read with successful replacement merely because the 3D prop reached its seat. Conversely, do not erase the old game at the first upload event just to simplify the visual state machine.

Documented history. Commit e1896ce moved visible DS feedback ahead of slow identification. The immediate-loading task (`TODO/115_DS_IMMEDIATE_LOADING.todo:5`) records the motivating sidebar delay. That history supports the ordering explanation, not a fresh startup-performance measurement.

Validate choreography without private content

Recommended diagnostic procedure. Use the repository's original homebrew integration fixture or synthetic phase control when validating presentation; do not introduce private cartridge images merely to make the loading path look authentic. The browser cases deliberately hold the upload read so the immediate UI transition and changing LCD can be observed before native startup. This isolates an ordering property that a fast local file might conceal.

For a purely geometric check, inspect the card shell and slot bounds in root space at staged fractions. Their construction uses the same centered extrusion helper (`src/handheld.js:19`), so nominal width and depth are not necessarily final outer bounds. A card fitting within the nominal slot width does not establish a physically valid cavity when the slot itself is represented by a dark solid.

For timing, record the sequence of eligible paint timestamps, not only the upload timestamp and a final screenshot. An 1800 ms nominal interpolation duration describes elapsed differences consumed by DS paint calls. A blocked or skipped interval can change which intermediate positions are actually visible. That observation should be reported as scheduling evidence rather than rounded into a guaranteed smooth frame rate.

The loading illustration's twelve-frame loop has another important limit: its phase can jump when time advances. It is an animation frame selector, not a queue that must display every preceding illustration. Reduced motion selects a static frame by policy. A static image under reduced motion is therefore expected behavior, while a static image under ordinary motion needs investigation of loading state, frame selection and texture upload.

Cancellation deserves its own evidence record. A failed runtime launch should remove the pending frame, clear loading and busy state, and surface a retryable explanation through the established operation path. A rejected replacement while an old session is running has a different preservation requirement. Combining those into a single “error case” hides whether the application retained or discarded the right state.

Keep proposed refinements separate from implemented guarantees. A future DS-specific midpoint reservation could make a sparse paint show a staged insertion pose before seating, but that is not present in the current controller's slide loop. It would require explicit new tests and should not be inferred from the neighboring shared GBA helper.

MD16-Q01 Exercise: can seated mean ready?

Explained answer

A proposed UI hides Cancel as soon as the card reaches Y .44. The native runtime is still fetching and has not sent its started message. Explain why this couples unrelated completion conditions and choose the proper readiness evidence.

MD16-A01 Explained answer

Back to question

Seating is visual interpolation completion; readiness is a validated native display handoff. The controller checks framebuffer layout and retained drawing-buffer support before installing output canvases and setting `running`. Cancel belongs to the pending launch callbacks, not to the mesh's Y coordinate. A seated prop can accompany a slow or failed native launch.

Keep the loading state and error recovery until native success or failure resolves the operation. Preserve the 90-second startup timeout and explicit cancellation rather than inventing readiness from animation duration. This separation makes the model choreography useful feedback without allowing it to misrepresent actual application state.

PART 2 / THE ARTICULATED NINTENDO DS

MD17

Inspection, camera and manipulation

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Inspection, camera and manipulation

A good model needs a truthful viewing instrument

A procedural assembly can be correct and still look broken if its camera crops the lid, places the shell outside the lamp's useful region or leaves projected controls aligned to an old pose. Inspection is therefore part of the modeling workflow. It must expose geometry without rewriting the object merely to flatter one view.

Implemented evidence. The DS camera uses articulated geometry bounds, not a hard-coded flat-screen rectangle. Body rotation and hinge edits are separate actions; an optional gameplay lock preserves both. The Inspect controls and pose setters (`src/scene.js:460`) define those ownership rules. The constructor supplies a root and a lid, so the camera must respect two transform spaces rather than pretending the whole object is rigid.

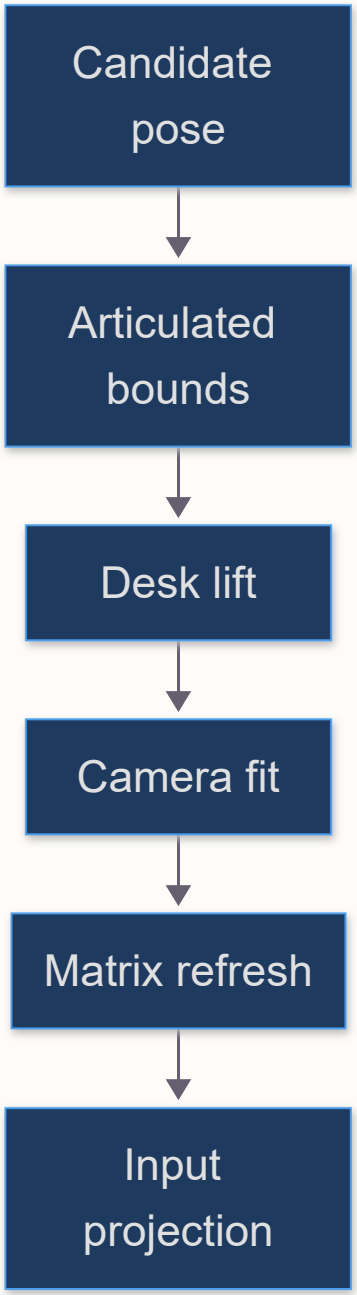


Figure MD17.1. Pose editing must update geometry envelopes and input matrices, not merely the rendered root rotation.

Evaluate a candidate pose without flattening its hierarchy

`dsPoseBounds(rotation, hinge)` first updates current world matrices. It composes a candidate root matrix at the shared play position and calculates a root delta relative to the current root matrix. It separately composes the candidate lid transform, then derives a lid delta relative to the current lid world matrix.

For each geometry object, the traversal determines whether its ancestry reaches the lid or root first. It computes a geometry bounding box when needed, applies the appropriate delta multiplied by the object's current world matrix, then unions the transformed box into an assembly envelope. The bounds implementation (`src/scene.js:502`) preserves nested local details while evaluating a different overall pose.

This is conservative box geometry, not an exact silhouette or a minimum-volume oriented hull. Rotated boxes can include empty corners. The advantage is predictable fitting and inexpensive computation across many separate meshes. The limitation is extra framing space around some oblique poses. Do not describe that slack as a mesh defect without checking the actual rendered silhouette.

Visibility is not filtered in that traversal. Hidden card or stylus geometry can still contribute to the envelope. This is especially relevant to the held stylus: its docked original remains in the DS hierarchy while only a clone moves in the scene. The controller clone rationale (`src/ds-controller.js:25`) deliberately preserves stable bounds.

Fit width, height and depth with interface reservations

The DS camera path gets the candidate envelope, then lifts the focused target so its minimum world Y is at least 2. It extracts size and computes a distance using field-of-view tangent, aspect ratio, width padding, height padding and half-depth. Narrow Inspect viewports reserve more top space for controls.

The fit formula (`src/scene.js:940`) uses top reservation 260 for phone Inspect and 90 otherwise; bottom reservation is 140 below width 600 and 120 otherwise. A minimum usable height prevents division by an arbitrarily tiny content area. A minimum width envelope of 12 also participates, so the formula is not merely the tightest possible fit to visible shell vertices.

The look target shifts vertically to account for asymmetric top and bottom reservations. The camera then sits along positive world Z from that target. Changing the object quaternion changes the evaluated bounds, not necessarily the camera's world viewing axis. This separation is useful when comparing front, back and three-quarter diagnostic poses.



Figure MD17.2. Parent-supplied neutral-studio constructor diagnostic for thickness and articulated relationships; the verified capture pose belongs to the parent render, not to an inferred native room or gameplay camera.

MD17-W01 Work a camera-fit term by hand

Assume a hypothetical candidate envelope of width 9, height 10 and depth 3, a 960-by-720 viewport, a 40-degree vertical field of view, Inspect mode and zoom 1. These are teaching assumptions, not a claim about a captured production camera. Aspect is 4/3, and tangent of half-FOV is about .36397.

The width term is `max(12, 9+1.2)/(4/3) = 9`. With top 90 and bottom 120, usable height is 510, so the height term is `(10+.8) × 720/510 ≈ 15.2471`. Height wins. Distance is approximately `15.2471/(2×.36397) + 1.5 ≈ 22.45`. This demonstrates why leaving room for UI can dominate a shell-only fit.

Original arithmetic recipe:

```
const w = 960, h = 720, fov = 40;
const size = { x: 9, y: 10, z: 3 };
const tangent = Math.tan(fov * Math.PI / 360);
const widthNeed = Math.max(12, size.x + 1.2) / (w / h);
const heightNeed = (size.y + .8) * h / (h - 90 - 120);
const distance = Math.max(widthNeed, heightNeed)
  / (2 * tangent) + size.z / 2;
console.log(distance);
```

Expected output is about 22.45 model units under those assumptions. The real path additionally applies zoom and the gameplay play-zoom factor. A fit calculation should report those multipliers rather than silently treating every mode as Inspect.

Input matrices cannot wait for an expensive paint

`reframeDsPose()` marks visuals and reflections dirty, updates camera targets, moves the root to the play target, copies the camera and look positions, then calls `updateViewMatrices()`. That function runs `lookAt`, refreshes camera and active-model matrices, and projects controls immediately.

The matrix-update contract (`src/scene.js:494`) explicitly decouples input correctness from the next eligible GPU paint. Under rendering backpressure, waiting for another painted frame could leave rays using a previous camera pose. The correct repair is not a permanent cursor offset; it is current transform data.

Research guidance. Current Object3D documentation distinguishes local and world matrices, and current Raycaster documentation specifies normalized-device coordinates and nearest-first hits. Both were consulted 2026-11-07. They support this diagnostic method, while the repository's pinned Three.js 0.186.1 code remains the version-specific implementation authority.

The controller's projected HTML buttons also have explicit limits. They use anchor world positions, a root-derived facing normal and a distance-based size estimate, with minimum sizes of 44 pixels for power and 14 for other controls. They test lid occlusion but are not exact projected cap polygons. Read the projection implementation (`src/ds-controller.js:684`) before calling it physically exact.

Lighting diagnosis begins with position, not paint

A dark metallic shell is not automatically a bad material. The model uses roughness, grain, clearcoat and environment reflections; its appearance depends on actual room geometry and emitters. Moving the focused console outside the lamp's emitting region can make the front unreadable even though the same paint looks good on the table.

The upright-lighting history (`TODO/116_DS_UPRIGHT_LIGHTING.todo:25`) records an initial lighting improvement at `(-2,5.1,-2)` that failed rotation clearance, followed by the final shared target `(-2,5.1,-8)`. Commit 9be851e grounds that correction. These are historical measurements, not readings repeated during chapter authoring.

The lesson is causal: inspect which surface a sample ray actually hits, then inspect light direction and object placement. Increasing emissive intensity or global exposure can hide the symptom while breaking dark-room behavior and active-screen comparisons. Material tuning belongs after spatial diagnosis.

MD17-R01 Retry on a phone-sized Inspect viewport

Change the example to a 390-by-844 viewport in Inspect. Aspect drops sharply and the top reservation becomes 260, while bottom becomes 140. Reusing the desktop distance can crop the stylus or force the hinge controls over the display. Recompute the same envelope under the new reservations rather than reducing renderer resolution or shrinking the physical model.

Then verify actual picking at the lower-screen corners. A fit that includes the whole object does not guarantee that the lid leaves the touchscreen exposed. Also check the projected power target after resizing; its minimum touch size and anchor alignment are separate from the shell's visibility. The browser phone and hinge scenarios (`tests/browser/ds.spec.mjs:1063`) encode those expectations. They were read, not rerun here.

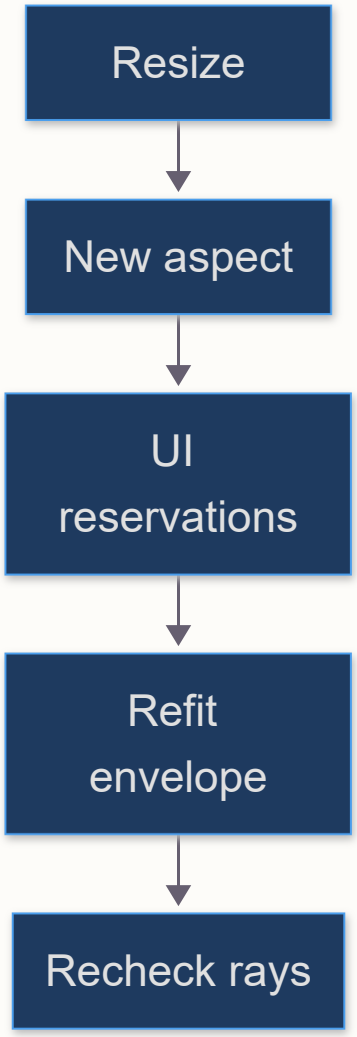


Figure MD17.3. Responsive fitting preserves geometry and pixel resolution while changing the viewing envelope and projected interaction layout.

Keep inspection instruments calibrated

The model's nominal dimensions depend on the shared centered rounded-solid helper (`src/handheld.js:19`). That helper's bevel expansion is one reason the camera measures generated geometry rather than rebuilding an envelope from a table of nominal dimensions. If a future toolkit change increases bevel thickness, an unchanged DS constructor can still need a different fitted distance.

Recommended diagnostic procedure. Record the candidate envelope before applying camera padding. This separates a larger geometry envelope from a larger UI reservation. If the model suddenly looks smaller on screen, compare those intermediate values: the shell may be unchanged while the viewport's reserved region grew, or an unexpectedly moved hidden child may have enlarged the measured bounds.

Also separate body rotation from camera orbit. The existing inspection interaction rotates the active model; a different tool might orbit the camera around a fixed model. Both can produce similar images, but their effects on world-space lighting and picking are not identical. A camera-relative highlight change is not interchangeable with rotating a metallic shell relative to a fixed lamp.

Avoid reusing local normals as world directions. A normal `(0,0,1)` on the control deck must be transformed by the root matrix before comparing it with the vector toward the camera. The controller performs that transformation for its broad facing check. A diagnostic that compares the untransformed vector can label a rear-facing console as front-facing after root rotation.

For a proposed alternative camera, preserve immediate matrix updates and rerun corner rays after resize, zoom and hinge edits. Then check scene clearance separately. A camera can fit an object that physically overlaps a companion, and collision-free placement can still be cropped. Combining those as one generic “framing pass” makes it difficult to identify which constraint failed.

Finally distinguish a stable mathematical fit from a pleasing composition. The source formula reserves interface space and fits conservative geometry, not an art-directed photographic composition for every angle. A proposed manual diagnostic camera may choose a tighter crop, provided its caption states that scope and does not present it as the native application's default view.

MD17-Q01 Exercise: why does fitting not prove accessibility?

Explained answer

A folded DS fits entirely inside the camera frustum. The author concludes that every control should remain enabled because no geometry is cropped. Explain the missing visibility test and identify the correct behavior for a hidden lower LCD.

MD17-A01 Explained answer

Back to question

Frustum inclusion only means the assembly envelope lies within the view. It says nothing about one component occluding another. The lid can cover the lower display while both remain inside that envelope. Projected controls must respect lid intersections, and touch must accept only a first relevant hit on the lower LCD.

Do not force hidden controls to remain clickable by searching through occluders. That would make the model visually truthful but behaviorally false. Preserve the lower-screen hit gate (`src/ds-controller.js:494`) and teach operators to open the hinge or change pose. Inspection is useful precisely because it exposes such constraints rather than concealing them.

PART 2 / THE ARTICULATED NINTENDO DS

MD18

The DS model regression atlas

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD18 / THE DS MODEL REGRESSION ATLAS

Contents

The DS model regression atlas

An atlas records questions, not just attractive images

A useful model atlas explains what each observation can establish. A front image can reveal a control overlap; an underside ray can establish that branding faces outward; a unit test can establish a bounded D-pad target. None alone proves the whole console is correct. The goal is to preserve independent evidence channels so a later agent can diagnose a changed model without confusing appearance, geometry and native behavior.

Evidence status for this part. Source, selected TODO histories, scoped commit diffs and test assertions were read at cutoff `1e4ef518d6943ab1af39cef3b4d558f209a17891`. This authoring task did not rerun browser or unit suites, produce production renders, inspect supplied images, publish or establish owner approval. Parent-supplied constructor PNGs are diagnostic illustrations with a separate verified capture pose; they are not native-runtime or historical-development proof.

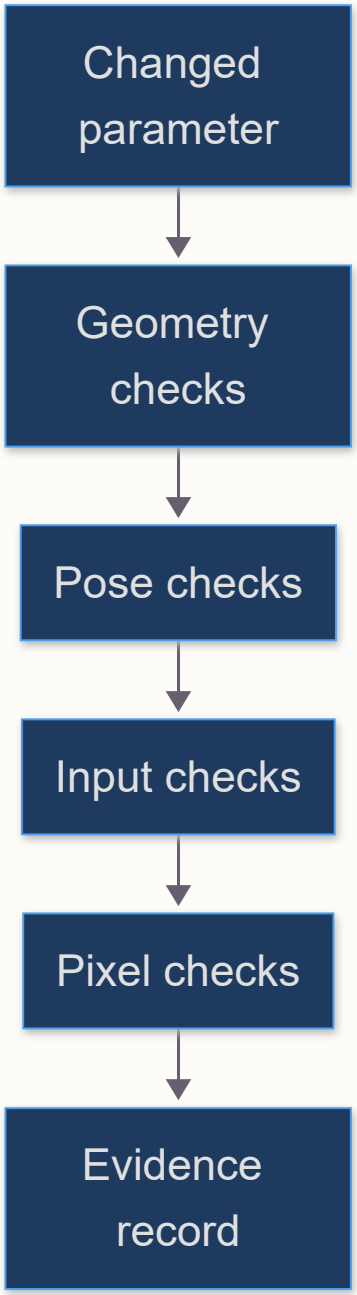


Figure MD18.1. Independent checks narrow the failure boundary; an attractive final image does not replace the earlier measurements.

Establish a minimal constructor baseline

The constructor baseline should identify the root quaternion, lid rotation, camera, lights, renderer and material initialization stage. Without those facts two images can disagree even when the factory source is identical. In particular, a bare factory has hidden branding and initial unlit screens; the controller later loads the supplied emblem and selects reflective powered-off glass.

The factory return interface (`src/ds-handheld.js:273`) identifies the stable diagnostic handles. The controller initialization (`src/ds-controller.js:196`) and power-off material state (`src/ds-controller.js:266`) explain why a constructor image is not the same fixture as a running application.

For an open diagnostic pose, set the returned `lid.rotation.x` to `degToRad(35)`, corresponding to a 145-degree opening under the scene convention. This is a safe value within the scene's supported 30–180-degree opening range, not a factory-enforced constraint. The factory exposes no setter; direct diagnostic assignment bypasses the application's guards and reframe logic.

The shared rounded-solid helper (`src/handheld.js:19`) must also be pinned. Changing its bevel behavior can change many apparent dimensions without editing the DS file. A regression atlas that records only `ds-handheld.js` can therefore miss a real geometry dependency.

Geometry atlas: surfaces, envelopes and ownership

The browser placement scenario combines several distinct checks. Rear rays hit the two inset floors and intact shell between them. The underside emblem is root-owned, faces outward and clears the battery cover. Face-cap and socket bounds clear the actual beveled touchscreen bezel and remain inside the deck. Four pads are square, matte and hittable from the rear; shoulders have the intended top-edge location and press normal.

The surface and bound assertions (`tests/browser/ds.spec.mjs:1347`) are a compact regression specification. Notice the measurement diversity: names from rays, parent identity, axis-aligned gaps, material properties and transformed normals. They are not redundant ways to prove the same fact.

Atlas view/check	What it can establish
Front diagnostic	Silhouette and obvious overlap
Rear surface rays	Apertures and outward surfaces
Root-space bounds	Numeric layout margins
Articulated envelope	Pose-dependent camera fit
Native contact sample	Actual pixel/input behavior

Conservative boxes are especially useful for repeatable clearance checks. They do not prove exact Euclidean distance between rounded surfaces. The atlas should say “root-space bounding gap” when that is what was measured, instead of implying a manufactured minimum air gap.

MD18-W01 Design a regression for a wider Y socket

Assume a proposed edit increases only face-socket radius from .220 to .250 while keeping the Y center at X 2.75. Its left bound moves from 2.530 to 2.500. The current beveled bezel right bound is approximately 2.451667, leaving .048333. That falls below the existing greater-than-.05

requirement even though the cap radius remains .195 and the cap itself still appears clear.

The correct regression selects both cap and socket, calculates their boxes in root coordinates and compares against the generated bezel, not the nominal width alone. It should report which named part failed and the measured gap. Then it should check the outer A-side deck margin before accepting any outward correction.

Original pseudocode, assuming a browser fixture with current world matrices:

```
for part in all_face_caps_and_sockets:
  b = rootSpaceBounds(part)
  innerGap = b.minX - bezel.maxX
  outerGap = deck.maxX - b.maxX
  report part.name, innerGap, outerGap
  require innerGap > .05
  require outerGap > .10
```

Expected result under the changed assumption: the enlarged Y socket fails the inner criterion. Do not weaken the threshold to accommodate the edit. The threshold expresses a preserved model contract, while the proposed radius is the variable under evaluation. An implementation change can be rejected without treating the existing test as an obstacle.

Pose atlas: fixed references and editable poses coexist

An atlas needs both reproducible reference poses and varied interaction poses. A fixed historical lighting pose supports color comparisons; an unlocked default play pose supports everyday usability; a 30-degree opening tests occlusion; a tilted lock tests preserved articulation. Substituting one for another can make an otherwise valid photometric expectation fail for the wrong reason.

The hinge setter and bounds path (`src/scene.js:467`) define allowed edits and candidate evaluation. The camera-fit formula (`src/scene.js:935`) includes viewport reservations and desk lift. Record those conditions before interpreting a crop or brightness difference as a model defect.

Current Object3D documentation, consulted 2026-11-07, supports explicit world-matrix reasoning. Current Raycaster documentation, consulted the same day, supports nearest-first hit and face-orientation diagnostics. These are researched recommendations, not evidence that a different current Three.js release was used to author the pinned model.

Input atlas: measure behavior independently of artwork

The D-pad unit test uses a lightweight fixture and checks signed physical edge heights, diagonal magnitude, opposite-direction cancellation, exact rest return and idle-settled behavior. It also checks that the cap surface remains above the deck. The unit cases (`tests/unit/ds-controls.test.mjs:49`) are valuable arithmetic evidence but do not instantiate the complete shell.

The touch queue tests separately preserve quick down/up edges, coalesce drags and cancel stale work. Their fixture and expectations (`tests/unit/ds-touch.test.mjs:29`) establish asynchronous sequencing, not native-game success. The implementation is DsTouchQueue (`src/ds-touch.js:28`).

Browser scenarios then connect the pieces: real dual-screen startup, scene/runtime keyboard focus, stylus placement, quick taps, rotated poses and resize under deferred painting. Read the scenario inventory and native case (`tests/browser/ds.spec.mjs:168`) as integration intent. Reading an assertion is not observing its current pass status.

Pixel atlas: choose the material state first

The active-LCD invariant applies to unlit, non-tone-mapped status or native textures. Reflective powered-off glass is expected to change with lighting. The upright-lighting browser case explicitly installs static status LCDs, verifies sampled shell hits and compares LCD RGB values across lamp, dark and room states. That fixture avoids confusing a legitimate reflection with a pixel-presentation regression.

The pixel and material assertions (`tests/browser/ds.spec.mjs:1455`) require actual surface identity as well as color. Sampling a nearby background pixel can produce a plausible brightness number with no evidence about the shell. Keep the ray-hit name in the diagnostic output.

MD18-R01 Retry after a slow integrated browser case

Change the failure condition: an overpacked browser scenario times out after several correct native direction checks. Do not immediately reverse D-pad signs, reduce rendering quality or add retries. First inspect which checks completed and whether the deadline expired before the remaining independent behaviors were exercised.

The D-pad historical task (`TODO/97_DS_DIRECTIONAL_DPAD.todo:40`) records splitting scene keys, runtime keys, diagonal/opposite holds and mixed-source cleanup into independently booted scenarios while retaining assertions and deadlines. That is historical failure isolation, not evidence that every timeout has the same cause.

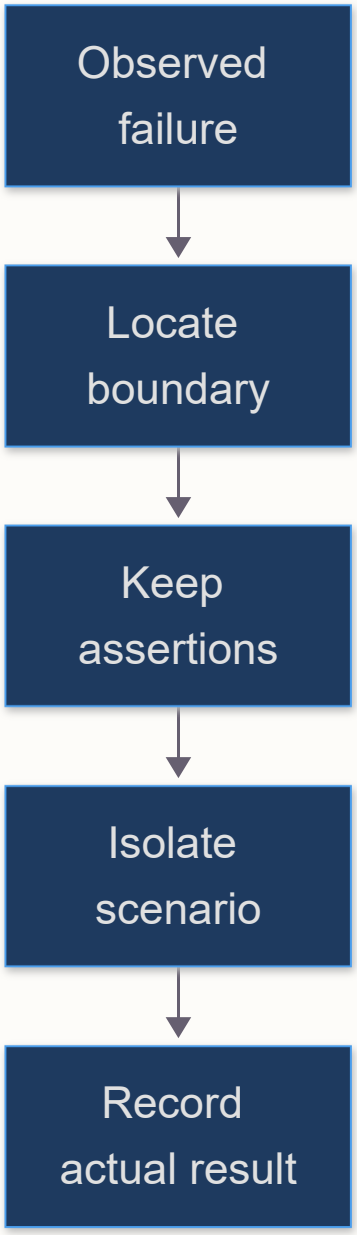


Figure MD18.2. Recovery changes the diagnostic arrangement only after evidence identifies the boundary; it must not silently relax the model contract.

Build a reusable evidence packet

Proposed agent procedure. For each model edit, retain a small packet containing the exact source cutoff, parameter delta, relevant constructor names, expected invariant, test selection and actual execution status. Add diagnostic images only when their fixture is known. A packet with fewer but well-scoped observations is more useful than many unlabeled screenshots.

For geometry, include generated bounds and the coordinate frame in which they were measured. For ray evidence, include origin, direction, first-hit name and relevant distance. For motion, include input state and timestamps or interpolation fraction. For pixel evidence, include the

material state, light state, sampled surface identity and viewport. These fields make a failure reproducible without requiring the next agent to infer hidden conditions.

Keep source-read status distinct from execution status in the packet. “Assertion exists at this line” is a useful fact. “Passed locally on a historical commit” is another useful fact. Neither means “passed now.” Likewise a successfully produced constructor image does not establish native framebuffer readiness. The atlas should preserve those categories rather than flatten them into one completion badge.

When updating documentation after a model change, recalculate arithmetic examples as well as replacing images. A moved Y button changes the worked socket gap; a new lid offset changes the pivot example; a new screen dimension changes pixel scale and illumination metadata. Leaving old arithmetic next to a fresh image can make a handbook internally inconsistent even when every individual source link still resolves.

The same principle applies to coverage. This part covers the DS shell, hinge, controls, screens, card, stylus, markings and interaction boundaries actually discussed. It does not claim exhaustive explanation of save formats, emulator internals, all renderer code or every source line. A precise boundary lets a future extension add missing topics honestly instead of accidentally inheriting an impossible completeness claim.

MD18-Q01 Exercise: write an honest atlas conclusion

Explained answer

You have read a geometry test, produced a neutral-studio front image and found a historical task reporting local passes. You have not run the current browser suite or inspected native drawing. What can the coverage record claim, and what must remain unverified?

MD18-A01 Explained answer

Back to question

It can claim source construction coverage, an identified assertion contract and a separately described diagnostic image. It can report the historical result explicitly as historical, with its source. It cannot claim fresh integration success, hosted qualification, native touch accuracy in the current environment or owner approval.

The coverage JSON for this part therefore names models, source symbols, components, chapters and tests **read**, then records limitations and unverified work. It is not a claim that every source line was explained or every hardware detail reproduced. This restraint makes the handbook reusable: a later agent can see exactly which evidence to regenerate rather than inheriting an inflated green status.

PART 3

Characters, softness and movement

1. Pear Boy: the parametric body
2. Projecting a face onto a curved shell
3. Expressions, gaze and six visible freckles
4. Limbs, rope and soft motion
5. Axolotl: the silicone body
6. Gills, tail and cute features
7. Companion light emission and material tuning
8. Dragging, gravity and dropping
9. Clearance, collision envelopes and home poses

PART 3 / CHARACTERS, SOFTNESS AND MOVEMENT

MD19

Pear Boy: the parametric body

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Pear Boy: the parametric body

Evidence boundary and construction objective

Implemented behavior. Pear Boy is authored by `createPearBoy`, not imported from a sculpting package. The factory assembles a lathed shell, projected facial geometry, simulated rope limbs, decorative stem and leaf, contact shadow, and interior lights. This chapter reconstructs the shell first because its sampled profile becomes the geometric authority for both facial depth and rope collision. The implementation baseline is commit `1e4ef518d6943ab1af39cef3b4d558f209a17891`; measurements below are model units, not inferred manufacturing dimensions. See factory and profile (src/pear-boy.js:5).

Design interpretation. A broad lower body and narrower upper neck can read as soft, stable, and toy-like. That is an interpretation of the silhouette, not proof that a user approved it or that a physical silicone prototype exists. The useful procedural objective is more concrete: preserve a continuous, rotationally symmetric shell whose front surface can be sampled consistently by independent features.

Build the profile before the surface

The profile starts as ten radius-height pairs in the XY plane. The endpoints are `(0,.16)` and `(0,6.85)`. Intermediate radii grow through `1.6`, `2.5`, and `2.85`, then taper through `2.8`, `2.5`, `1.95`, `1.6`, and `1`. The widest authored control point is therefore at height `1.3`, not at the middle of a generic sphere. The control-point list is interpolated with `CatmullRomCurve3`, sampled with `getPoints(96)`, converted to `Vector2`, and clamped to nonnegative radius. This produces 97 sampled profile points.

There are two distinct resolutions: 96 curve divisions describe the vertical silhouette; 64 lathe segments describe angular sampling. Increasing one cannot repair undersampling in the other direction. A coarsely sampled profile can produce a stepped shoulder despite a smooth circular cross-section. Conversely, a fine profile with too few radial segments can look polygonal from above. Do not silently replace `getPoints` with evenly spaced arc-length samples: that would change the correspondence used by the downstream height lookup. The actual choices appear together at shell construction (src/pear-boy.js:66).

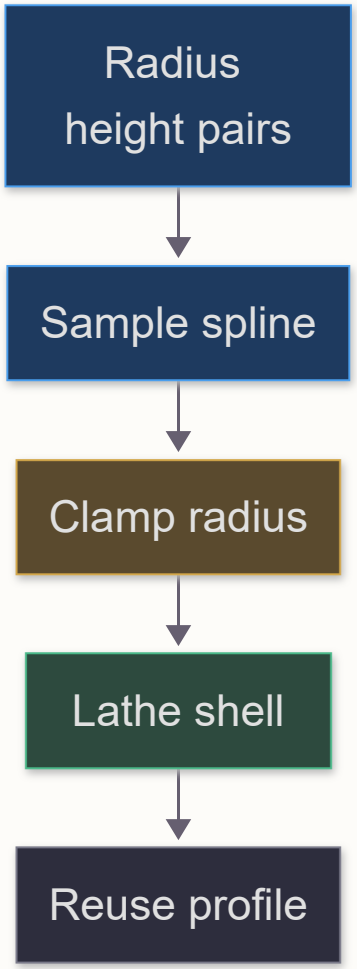


Figure MD19.1. One sampled profile supplies the lathe and the later shell queries; those consumers must not drift into independent approximations.

Derive the shell coordinates

For explanatory purposes, let `r(y)` denote the radius interpolated between neighboring sampled profile points. A rotational shell can be described by `x=r(y) sin(theta)` and `z=r(y) cos(theta)`. This is a derivation of the surface of revolution, not a transcription of Three.js's internal vertex ordering. Positive Z is the authored face direction. At each fixed height, the ideal cross-section satisfies `x²+z²=r(y)²`.

The code does not solve the original spline analytically for height. `radiusAt` finds the first sample after index eight with height at least the requested value, then linearly interpolates its radius against the preceding sample. The index guard avoids the beginning of the rounded base. Its validity depends on the authored profile behaving sensibly over the queried heights. A proposed profile with loops or decreasing height needs a new lookup contract; the existing function is not a general spline inversion algorithm.

This distinction explains why a visually plausible replacement sphere is insufficient. The face and collision code would continue asking the old profile for radii. A faithful rebuild must retain one authority, or deliberately update every dependent consumer and its tests.

Topology, normals, and ownership

The body uses `LatheGeometry(profile,64)` and a `MeshStandardMaterial`. The factory does not manually recalculate its normals after construction. Do not describe the body as a subdivision surface, voxel remesh, or normal-map sculpt: none of those stages appears here. The mesh helper marks the object with `userData.pearBoy`, sets `receiveShadow`, and adds it to the root. The root owns model placement; the body geometry remains in local coordinates.

Researched guidance. Current Three.js BufferGeometry documentation distinguishes indexed shared vertices from non-indexed triangles and explains that normal recomputation behaves differently for each. That reference was consulted on 2026-11-07; it is not an immutable description of this repository's dependency internals. If a proposed extension edits lathe positions directly, inspect its actual index and normal attributes before deciding how to rebuild shading. Updating positions alone is not a complete geometry-editing policy.

The material's cream base, roughness `.65`, warm emissive color, and intensity `77` are factory defaults. They suggest a luminous toy surface but do not implement subsurface scattering or elastic shell deformation.

Keep decoration separate from shell authority

The brown stem is a curved tube through heights `6.8`, `7.55`, and `8.02`, with radius `.22`. The leaf is a planar Bézier outline extruded `.08`, with `.04` bevel size and thickness. Its root translation is `(0,7.5,.2)`; the vein is another small tube. These parts extend the recognizable silhouette without changing `radiusAt`. See stem, leaf, and vein (src/pear-boy.js:285).

That separation matters in collision interpretation. A profile-radius query describes the pear shell, not the leaf or every decorative protrusion. Conversely, an object-wide box can include protrusions that are not intended as physical support. Neither representation replaces the other. The source uses several envelopes for different jobs; treating all of them as the literal surface would produce contradictory dimensions.



Figure MD19.2. Actual original-factory diagnostic under neutral studio lighting; useful for silhouette comparison, not proof of native-room illumination or animation.

MD19-W01 Reconstruct a cross-section consistently

Begin with the exact control points, create the 97-point profile, and select height `4.5`, the facial anchor height. Locate the same bracketing samples that `radiusAt` uses. Compute `t=(4.5-a.y)/(b.y-a.y)` and then `r=a.x+t(b.x-a.x)`. Do not substitute the radius of a neighboring control point: the sampled interpolant is the implemented surface authority.

Next choose a test horizontal coordinate `.66`, matching one calm eye center. Derive its shell depth as `sqrt(r²-.66²)` and add the feature's chosen offset. Repeat with negative `.66`. The two depths should match because the shell is rotationally symmetric. Their X coordinates differ; their Y and Z values should not. This checks both symmetry and the sign convention without needing a rendered screenshot.

Finally compare the derived point with the factory's `front` result and check that all coordinates are finite. A useful observable recipe is to create the factory in its normal browser environment, inspect the named black eyes, and compare their neutral local positions. The texture helper needs a document canvas, so do not claim a bare Node factory run works without a canvas adapter. The helper's actual environment requirement is visible at canvas creation (src/handheld.js:68).

MD19-R01 Widen the belly without breaking dependents

Proposed extension. Change the `2.85` control radius to `3.1` in an isolated experiment, retaining its height and the sampling counts. This retry changes geometry, not only material color. Rebuild the shell and derive radii at limb attachment heights and the face height. A local belly change can influence adjacent spline intervals; do not assume the effect ends exactly at the edited control point.

Diagnose three consequences separately. First inspect the silhouette for a new shoulder kink. Second compare projected facial depths using the new shared profile. Third settle the ropes against the new radius function. The rope constraint pushes points outward using body radius plus cord or tip radius, as implemented in body contact (src/pear-rope.js:140). A rope that intersects the widened shell is not repaired by increasing render order.

Also re-evaluate body movement bounds. `PearMotion` has a deliberately simplified box, not a runtime extraction of every new profile vertex. Widening beyond its half-width can invalidate collision expectations even when the visible mesh is excellent.

Movement and clearance are different contracts

The motion controller bounds Pear by `(-3,0,-3)` to `(3,8.1,3)`, with root workspace from `(-22,0,-11.5)` to `(24,25,7.5)`. Those values govern upright body motion; they are not a promise to enclose every hanging foot. See PearMotion configuration (src/pear-motion.js:150).

The factory additionally computes an object box and expands its horizontal reach to at least `±4.5`. Scene clearance starts with that envelope and unions current children except the contact plane. That arrangement protects broader limb reach while avoiding a fake shadow becoming solid furniture. Scene envelope assembly (src/scene.js:797) and the separate placement solver (src/desk-clearance.js:3) explain why a new body shape needs integration checks, not merely a neutral-front capture.

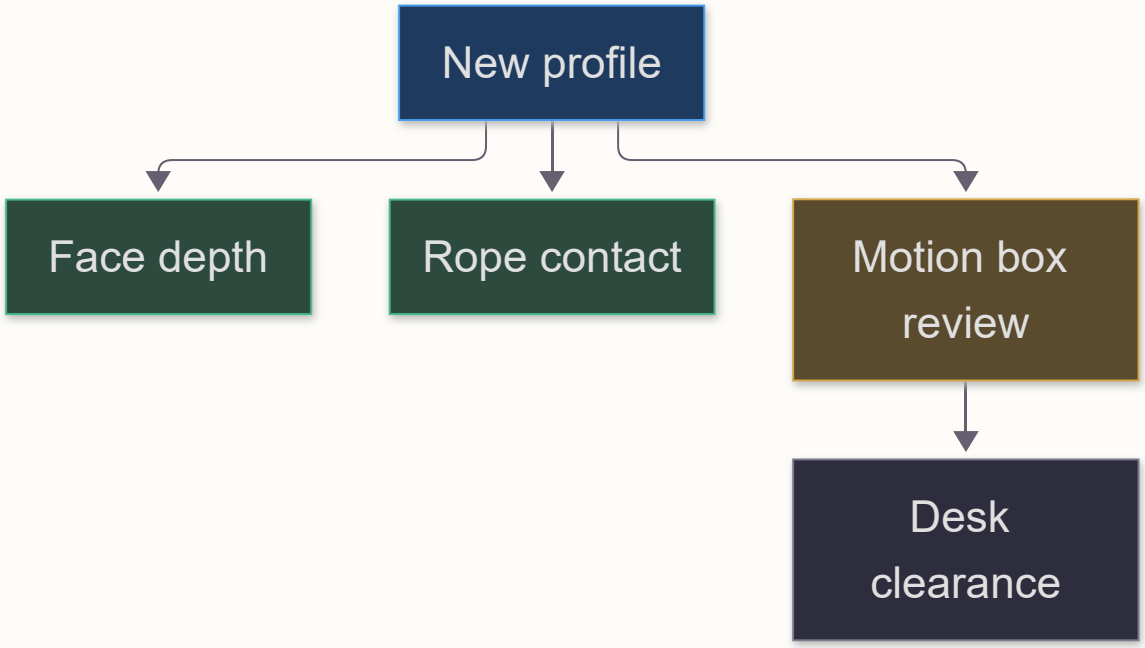


Figure MD19.3. A silhouette edit propagates into three geometric contracts before the model can be considered safely placeable.

Preserve useful invariants during reconstruction

A practical reconstruction should record three independent measurements before experimenting: the shell's local vertical span, sampled radius at the face anchor, and full assembly bounds after settling. The first describes body geometry, the second supports attachment math, and the third describes scene participation. Combining them into one “size” number throws away the distinction that makes later failures diagnosable. The factory exposes both `body` and `bounds`, while motion supplies another explicitly configured envelope (src/pear-boy.js:375; src/pear-motion.js:150).

For a proposed lower-resolution version, reduce only one sampling direction at a time in an isolated copy. Compare front silhouette, top circularity, and projected face clearance under the same coordinates. A lower polygon count is not automatically a valid optimization if broad expression patches start intersecting a coarser shell. Likewise, a higher count is not evidence of a better design if it changes nothing visible at the intended desk scale. The relevant tradeoff is observable quality against geometry cost, with the attachment contract retained.

The existing rope tests offer a useful numerical counterpart to silhouette inspection: finite coordinates, pinned attachment, controlled length error, and correct edge support. Keep the tests' synthetic constant-radius fixture distinct from the actual pear profile. Passing that fixture validates solver mechanics, not every authored radius sample. A profile-specific extension should sample its own critical heights rather than copying the fixture's constant `2.8` as if it were the entire body's radius. The fixture and its assertions are explicit in (tests/unit/pear-rope.test.mjs:6).

MD19-Q01 Choose the right failure detector

Explained answer

A rebuilt Pear looks correct from the front, but its hand enters the shell after a lateral pickup. Should the first repair change shell roughness, add more radial lathe segments, or inspect the shared radius/contact contract? Explain what additional observation would distinguish a bad collision radius from an incorrectly transformed rope.

MD19-A01 Separate shape, contact, and coordinate failures

Back to question

Inspect the radius/contact contract first. Roughness changes shading, and radial segments change the polygonal approximation; neither necessarily changes the constraint's requested clearance. Compare each rope point in world coordinates against the body-local radius evaluated at `point.y-origin.y`. Then check that painting subtracts the root position exactly once. A consistent radial shortfall suggests wrong radius or tip clearance; a displacement that follows root translation suggests a coordinate conversion error.

The failure boundary is important: the rope only collides with the pear and desk, not arbitrary scene props. Existing rope unit cases exercise body contact and stale-momentum removal, but do not prove a future profile is safe at every possible point. Use them as regression contracts, then add focused negative controls for the changed shape. This handbook is a reproducible reconstruction aid, not a recovered internal design memory.

MD20

Projecting a face onto a curved shell

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Projecting a face onto a curved shell

Why a flat face fails on a rounded body

Implemented behavior. The Pear face is real mesh geometry attached to a curved shell. It is not a texture decal and does not use a camera-facing billboard. Its placement function derives Z from the same sampled radius profile that builds the body. The important dependency is therefore geometric: a facial feature must remain outside a shell whose depth changes with both height and horizontal coordinate. See profile, front projection, and patches (src/pear-boy.js:66).

Design interpretation. Keeping the markings close to the surface can make them read as part of a soft lamp rather than floating accessories. That reading does not establish a physical printing process. The procedural problem is simply to balance surface clearance against conspicuous separation, including when the face moves.

Derive the front projection and its domain

At a chosen height, the ideal section satisfies `x²+z²=r²`. Selecting the front half gives `z=sqrt(r²-x²)`. The code adds a positive offset, named `inset`, although numerically it pushes the point outward along positive Z. Its default is `.03`. A literal inward normal displacement would have a different meaning; use the actual equation rather than interpreting the name as a physical direction.

The function clamps the square-root argument to zero. This prevents a negative radicand from generating NaN when `|x|>r`, but does not make such a point a correct surface attachment. It would sit beyond the valid circular section with depth equal to the offset. Nor does `front` include the explicit height guard present in `radiusAt`; an out-of-range height can fail while selecting its interpolation pair. Valid feature authoring must respect both domains.

For a broad patch, it is not enough to project only its corners. A triangle connects projected vertices with planar edges. Across a convex shell that flat chord can cut inward between samples, burying its interior even when the vertices are individually clear. This is the reason to examine tessellation before increasing offset.

Form expressions in two dimensions first

`facePatch` constructs a `ShapeGeometry` with curve-segment argument `16`, converts it to non-indexed triangles, copies its positions into vectors, and disposes both intermediate geometries. It then performs two triangle subdivision passes. Each triangle becomes four by introducing edge midpoints. After two passes an original triangle contributes sixteen triangles. Only then are all vertices projected using `front`.

This ordering is the central lesson: shape placement happens in its XY description before projection. Translating an already projected patch vertically does not re-evaluate the shell radius at the new height. A mouth can consequently enter a region of greater shell depth and disappear behind the body; movement toward a narrower region can instead leave it floating. The implemented anger mark follows the safer pattern too: it adds `.42` to the authored Y coordinate before calling `front`, rather than moving the finished tube afterward. See anger mark construction (src/pear-boy.js:215).

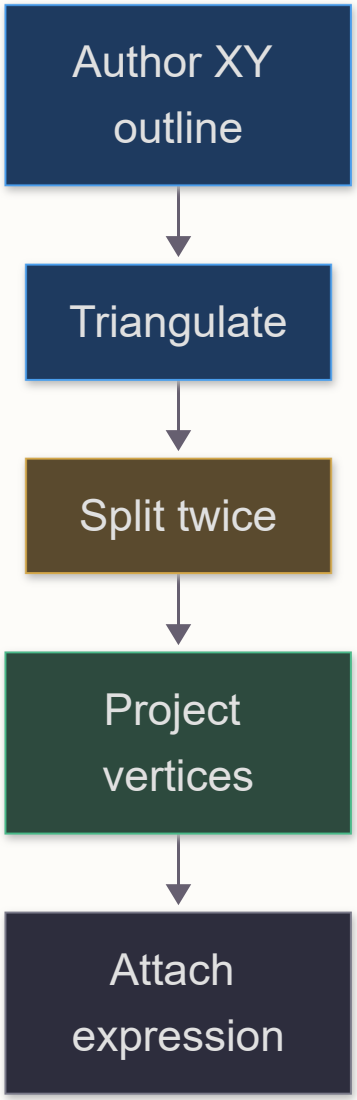


Figure MD20.1. Subdivision precedes projection so the filled interior follows the rounded shell rather than remaining a large cutting plane.

Tubes, patches, and small solid features

The calm eyes are scaled spheres at `x=±.66`, `y=4.5`, with scale `(.18,.22,.09)`. The smile samples seventeen points from X `-.5` to `.5`, using a sine depression of `.32` from height `3.96`, then forms a tube of radius `.055`. These solids have their own thickness; they are not equivalent to a zero-

thickness patch with the same center coordinate.

Expression eyes, cheeks, mouth fills, and tongue use different offsets. Typical patch clearance is `.06`; the white angry-eye interior and tongue use `.085`; rosy cheeks use `.025`. These layers should be understood relative to their projected surface, not as absolute world depths. The mesh helper initially adds meshes to the model root, after which adding them to a face group reparents them. Their coordinates were already authored in model space, and those face groups start with identity transforms.

Researched guidance. Current BufferGeometry documentation states that absent indices, successive triples form triangles. That supports the patch's `setFromPoints` construction. The returned patches do not explicitly compute normals or retain UV coordinates; their `MeshBasicMaterial` colors do not require the same lighting response as the standard-material shell.

Move around the body axis, not the face center

`face.root` is a group at the body origin, containing both expression groups and permanent freckles. Horizontal tracking rotates this group about Y. Because its geometry is already located near the shell's front, that operation moves features around the body axis. Rotating a group newly centered at the face anchor would instead turn markings around their own center, an entirely different trajectory.

Vertical tracking translates the group and scales X and Z by `radiusAt(4.5+dy)/radiusAt(4.5)`. This is a shared center-height approximation, not a per-vertex reprojection at every feature height. It preserves a coherent face while accommodating the narrowing shell. The fixed anchor is a separate object attached directly to the model root, so pointer normalization does not chase the moving face. See anchor and transform update and gaze transform (src/pear-boy.js:248).

MD20-W01 Derive lateral motion of a marking

Take a neutral point `(x,y,z)` on the face. Let the horizontal normalized gaze be one, so the implemented yaw is `.55` radians. Under the conventional Y rotation, its new horizontal coordinate is `x cos(.55)+z sin(.55)`. This includes a contribution from shell depth. An eye can therefore move substantially farther than a simple `.55`-unit lateral translation would suggest. The angle is approximately 31.5 degrees; it is not a screen-pixel distance.

Now set upward gaze to one. The implemented Y offset is `.75` model units, not `.75` radians. Compute the radius ratio at heights `5.25` and `4.5`. Apply that ratio to local X and Z before yaw. The exact numerical result must come from the sampled profile, not from assuming a spherical head.

For an observable reproduction, compare world positions of both named black eyes at neutral and extreme gaze after `updateWorldMatrix(true,true)`. Keep the root position unchanged and separately project the fixed anchor. The browser regression requires upward eye movement over `.65` world units and five screen pixels, with an unchanged anchor. It also checks smile and freckle movement. See pointer assertions.

MD20-R01 Move a broad mouth toward the shoulder

Proposed extension. Raise an expression mouth's XY control points by `.3` before constructing its shape. Rebuild its triangulation and projection; do not translate the finished mesh. The changed condition is increased shell curvature and a smaller radius at the new height. Inspect the patch center as well as its perimeter, because corner-only tests miss chord penetration.

If the interior disappears, first verify subdivision and the resulting vertex depths. Increasing offset may restore visibility but can leave a floating outline in three-quarter views. Increasing tessellation better approximates curvature, at a geometry cost. Replacing the fill with a texture is another tradeoff, but it would introduce UV and texture-resolution contracts absent from the implemented expression pipeline.

Do not assume `DoubleSide` fixes buried geometry. It can expose back faces, but the shell can still occlude the patch. Current Raycaster documentation also distinguishes back-face behavior from closest-hit ordering. Diagnose the actual surface intersection instead of using material settings as a substitute for correct placement.

Visibility means foremost appearance

The freckle test casts toward each dot and selects the nearest intersection whose entire ancestor chain is visible. A dot's own `visible=true` is not sufficient if its parent is hidden. Nor does a visible flag prove the body or another eye is not in front of it. The test examines all nine gaze combinations for four expressions, requiring all six foremost names to be the freckle name. See visibility filtering.

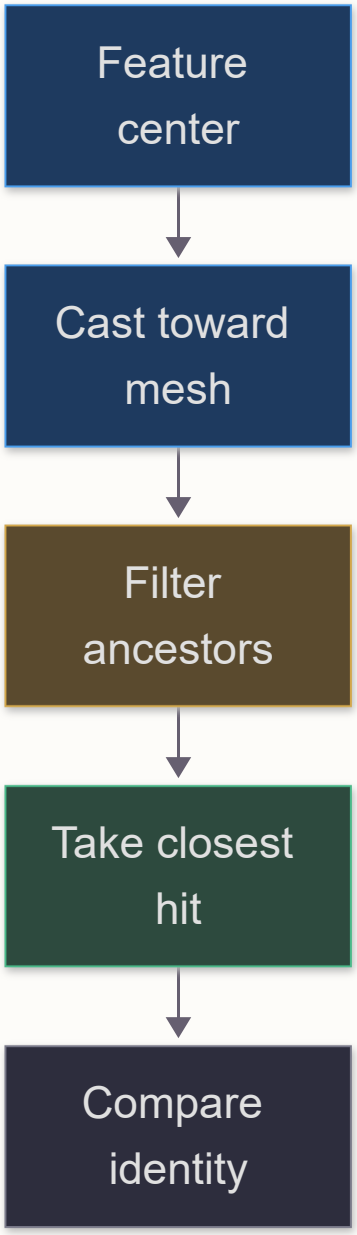


Figure MD20.2. A geometric visibility check rejects hidden ancestors and intervening surfaces before accepting a facial marking.

Integration constraints beyond the face

The same model root also carries moving limbs and lights. Rope painting converts simulated world positions back to root-local coordinates; it must not inherit face-group yaw. Body motion is separate again, using `DeskMotion` integration rather than facial interpolation. These ownership boundaries are explicit in limb painting, rope origins (src/pear-rope.js:48), and body integration (src/pear-motion.js:59).

Scene pointer normalization projects the immutable anchor and clamps gaze to `[-1,1]`, independent of the current markings (src/scene.js:1326). Texture color-space setup is centralized in the canvas helper (src/handheld.js:68), whereas face patches use direct basic colors. Those distinctions explain why changing emission or the canvas gradient is not a repair for a buried mouth.

Depth offsets are a layered budget

The patch and solid-feature offsets form a small but important depth budget. An angry eye's white interior at `.085` sits farther forward than the black outline's default `.06`; the tongue uses the same larger offset against its mouth fill. Blush at `.025` is intentionally closer to the shell than many ink features. These are source values, not universal safe distances. Their relative arrangement is visible in the expression constructors (src/pear-boy.js:136; src/pear-boy.js:202).

When adapting a feature, first identify which layer should be foremost and whether it is a tube, flattened sphere, or triangulated fill. A tube radius contributes additional geometric depth around its centerline, whereas a patch has no comparable thickness. Equal center offsets therefore do not imply equal front surfaces. This explains why moving every feature to the same Z offset can introduce new occlusion even if it removes shell penetration.

A useful proposed diagnostic is to cast separate rays toward the intended foreground layer, its surrounding outline, and a point just beyond the feature. Record object identity, not only hit distance, because two parts can be nearly coplanar while belonging to different expressions. Repeat after upward and horizontal gaze, then compare the same local layer ordering. This extends the existing freckle-center method without pretending it already tests every mouth pixel. If a repair requires disabling depth testing, stop and re-examine geometry: the current implementation's lesson is to construct surface-following features, not to draw facial colors over unrelated objects.

MD20-Q01 Diagnose a face that vanishes only upward

Explained answer

A new cheek fill is visible at neutral gaze but becomes partly hidden looking up. Its object and parent are visible, and its corner vertices are outside the shell. What should be measured next, and why is rotating the whole lamp an invalid repair?

MD20-A01 Inspect interior sampling and transform ownership

Back to question

Measure interior triangle clearance at the changed height, then inspect the group's center-height scale approximation. Large planar triangles may penetrate between clear vertices; a shape extending far above the anchor may also exceed the approximation's useful range. Rebuild with more appropriate sampling or change its authored XY extent before projection.

Rotating the whole lamp changes body, ropes, lights, and desk collision relationships. It conceals the local error by changing the viewing condition. The failure boundary is that the current face transform is designed for the authored range, not arbitrary expressions spanning the entire pear. History records wider gaze and later upward travel, but those commits do not prove every proposed larger face is safe. Extend the geometric test grid and inspect actual appearance rather than converting historical qualification into universal approval.

PART 3 / CHARACTERS, SOFTNESS AND MOVEMENT

MD21

Expressions, gaze and six visible freckles

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Expressions, gaze and six visible freckles

Separate where the face looks from what it expresses

Implemented behavior. `createPearBoy` builds four expression groups beneath one movable face root: calm, scared, angry, and sad. The public expression labels are `curious`, `scared`, `angry`, and `sad`; the final sad appearance is a squeezed-eye pout, not the earlier glossy worried face. Gaze moves the shared face root, while visibility selects the expression. This separation prevents each expression from implementing slightly different tracking. Face ownership (src/pear-boy.js:43) is the authoritative hierarchy.

Design interpretation. The compact eyes, short brows, blush, and puckered mouth can communicate a gentler reaction than large wedges. “Cuter” is a design interpretation and historical task vocabulary, not a measurable shader property. This chapter instead makes the mechanics reproducible: which geometry appears, what triggers it, how long it remains eligible, and which markings never belong to a temporary expression.

Permanent freckles are structural, not duplicated decoration

Each side creates three dots at positive-coordinate pairs `(1.05,4.25)`, `(1.16,4.03)`, and `(.92,4.04)`, mirrored in X by the side sign. Each dot is a sphere of radius `.055`, with ten width and eight height segments, flattened in Z by `.4`. The material is orange `MeshBasicMaterial` with tone mapping disabled. All six are attached directly to `face.root`, not to calm or scared. See freckle creation (src/pear-boy.js:160).

This ownership makes six a persistent inventory invariant. Duplicating the dots into four groups would multiply maintenance points and could accidentally leave two sets visible. Attaching them outside the moving root would preserve their count but break whole-face tracking. Neither error is caught by simply counting named objects in the entire model; both hierarchy and actual foremost visibility must be checked.

Gaze math and a fixed reference

The scene projects an immutable anchor at the original face center, converts cursor displacement into fractions of twenty percent of viewport width and height, and clamps each axis to `[-1,1]`. Upward screen movement reverses the usual screen-Y direction. The anchor is separate from the animated face group, avoiding a feedback loop in which the target moves because the face already moved. See scene normalization (src/scene.js:1326).

The gaze interpolation blend is `1-exp(-12*delta)`, or one for reduced motion. For a constant target, the remaining error after elapsed time `t` behaves like `exp(-12t)` until the snap threshold is reached. The characteristic time is about `.0833` seconds. A squared error below `1e-7` copies the target exactly, preventing perpetual microscopic changes.

Horizontal gaze produces Y-axis rotation up to `.55` radians. Positive vertical gaze produces up to `.75` model units of rise; negative gaze produces down to `-.18`. These are deliberately different dimensions. A radius ratio scales X and Z to follow the changing shell width. It is not a `.75`-radian pitch.

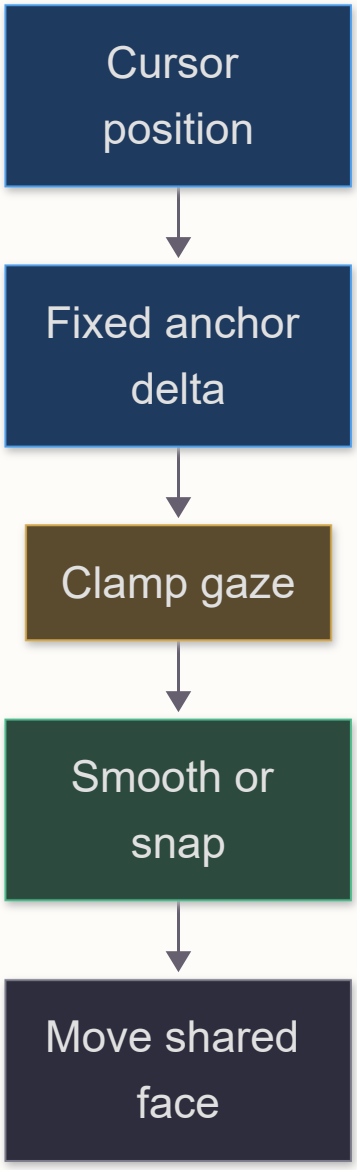


Figure MD21.1. Gaze is derived from a stationary reference and applied once to every expression and permanent dot.

Expression state and paint acknowledgment

`showExpression` switches group visibility, restores the wobbly scared mouth, hides the open scream mouth, resets elapsed time, and clears `presented`. `setFaceLifted` selects scared or curious unless an impact reaction currently owns the face. A reset or new lift clears that reaction. `reactToImpact` ignores speeds below ten and ignores another impact while already reacting. State setters (src/pear-boy.js:224) establish these rules.

While reacting, angry lasts at least `.9` accumulated seconds and sad at least `1.25`, but transitions additionally require `presented=true`. The scene sets that flag after submitting the main room render. Consequently a stalled renderer cannot silently consume the entire reaction sequence before any expression was submitted. This is a visibility contract layered over elapsed time, not a promise of exact wall-clock dwell on every device.

When lifted without a reaction, the two scream mouths alternate after `.9` seconds only if the current mouth was presented. Reduced motion suppresses that alternation and forces the wobbly mouth if the open one was visible. Recovery can still progress through static expressions. See face update.

MD21-W01 Reproduce the mouth dwell without a stopwatch

Use the factory in its normal browser environment and explicitly call `setFaceLifted(true)`. Acknowledge one presentation, then update with `.6` seconds and zero gaze. The wobbly mouth remains selected. Add `.31` seconds: the accumulated `.91` crosses `.9`, so the open mouth becomes visible and `presented` resets.

Without another acknowledgment, advancing time does not authorize another switch. After acknowledgment, repeat the `.6` and `.31` pair. The deterministic browser test records `[false,true,true,false,false]`, where the final false comes from a reduced-motion update forcing the closed variant. This recipe distinguishes state timing from automation latency and does not require loosening any screenshot deadline.

Then test a hard impact separately: set lifted false with reset, call `reactToImpact(20)`, and inspect the expression label. Advance beyond `.9` without acknowledging paint; angry should remain. Acknowledge, update, and observe sad. Repeat beyond `1.25` to return to curious if no longer lifted. The actual browser timing and visibility fixture provides an existing executable model for this separation.

MD21-R01 Retry under sparse rendering and reduced motion

Change conditions from regular paints to a long update with no paint acknowledgment. If all expressions disappear immediately into curious, inspect whether a caller set `presented` before rendering or bypassed the state setter. Do not conclude that large deltas inherently require skipping intermediate states. The implementation explicitly preserves them until presentation.

Next enable reduced motion. Gaze should snap rather than ease; mouth alternation should stop; impact recovery remains eligible after its timing and presentation conditions. Rope motion has a different accommodation: it settles constraints immediately instead of swinging. See reduced-motion limbs and the settling procedure (src/pear-rope.js:34). Reduced motion is therefore not a global instruction to freeze every state variable or teleport a dropped body home.

Researched guidance. MDN's requestAnimationFrame reference, consulted 2026-11-07, warns that callback frequency varies and background callbacks can pause. That supports elapsed-time reasoning, but does not itself guarantee GPU completion or prove that a particular expression reached the screen.

Prove freckles are not merely enabled

The browser fixture iterates X and Y values `-1`, `0`, and `1` under each expression, casting rays from a point in front of the Pear toward each dot. It filters intersections whose object or any ancestor is invisible, then requires the closest remaining object to be the freckle. Thirty-six combinations times six dots provide 216 center-ray checks.

This is stronger than `visible=true`, but narrower than a complete perceptual proof. It checks centers, not every silhouette pixel or every possible camera angle. Antialiasing, tiny projected size, and color contrast still affect appearance. Current Raycaster guidance supports nearest-first intersections and explains mesh face orientation; the repository's explicit ancestor filter supplies its visibility policy.

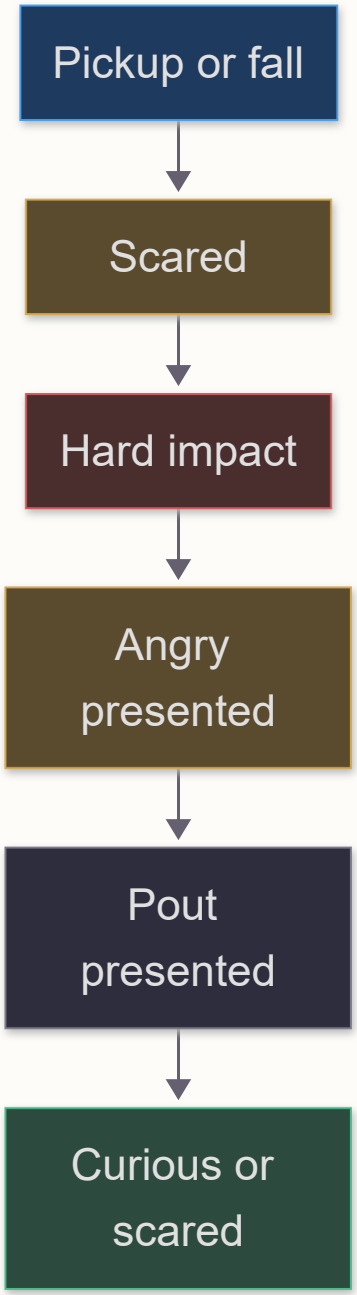


Figure MD21.2. Hard-impact recovery requires time and submitted presentation; permanent freckles remain outside this expression selection.

History explains changes, not hidden reasoning

Documented history. Commit `1510777ff05b82414e8e1b5958e8c76e446a0eaa` replaced large angry-eye shapes and glossy recovery eyes, and changed scream dwell from `.24` to `.9`. Commit `44b5dc2213bcf9ee464357cfa68ee44b8c1d9bbe` widened yaw from `.22` to `.55`. Commit `dd269dea6f5cc6ada0038a2245b0ec3a420768c5` increased only positive vertical travel from `.18` to `.75`. Their scoped source diffs establish those changes; the task records describe observed failures and qualification. They do not recover private deliberation or imply aesthetic approval. The numerical

changes are recorded at (TODO/118_PEAR_BOY_FACE_TRAVEL.todo:23) and (TODO/121_PEAR_BOY_UPWARD_FACE_TRAVEL.todo:22); the approval boundary is explicit at (TODO/113_PEAR_BOY_CUTER_RECOVERY.todo:22).

Physics supplies an impact speed rather than directly selecting facial meshes. The motion controller resets that report per update and records downward support contacts; horizontal wall contact is not the same event (src/pear-motion.js:59). The scene consumes it before updating the face (src/scene.js:1322). Desk placement remains independently responsible for collision-free paint, rather than expression code moving the lamp aside (src/desk-clearance.js:3). These boundaries prevent a visual reaction from becoming an accidental collision repair.

Geometry makes each reaction readable

The scared face uses squeezed tube eyes, projected pink cheeks, and either a wobbly line mouth or a filled open scream. The wobbly mouth samples 33 points across width `1.2`, with a cosine oscillation of amplitude `.075` and six half-turns across the normalized parameter. Its geometry is fixed; the visible tremble impression comes from its contour and alternation, not continuous random vertex motion. The open mouth is a distinct curved patch (src/pear-boy.js:186).

Angry uses compact ellipse-based black outlines and smaller white interiors, short curved brows, lower blush strokes, a filled shouting mouth, a pink tongue, and an anger mark built higher on the forehead. Sad uses squeezed eyes, small frustrated brows, rosy cheeks, and a narrow alternating-X pout. These are different geometric vocabularies, not merely recolored calm eyes. When reconstructing them, preserve the XY coordinates before projection; raising a complete already-projected patch can bury it in the tapering shell.

A proposed alternative based on a single animated texture could reduce mesh inventory, but would introduce atlas coordinates, texture filtering, and expression-frame lifecycle concerns. The pinned assembly instead makes individual features inspectable by name and allows ray-based occlusion checks. Neither approach is categorically superior; the reusable lesson is to state which representation owns expression identity and how its visibility is proved.

Keep acceptance layers separate

The historical task records distinguish technical qualification, public deployment, and aesthetic approval. Repeating a recorded “passes” statement does not execute a test in a new checkout. This authoring pass reads the fixtures and history; the parent owns rendering and publication. Another agent should retain that boundary rather than treating prose as an invisible test run.

For a new expression, preserve permanent-dot count and hierarchy, test both reduced-motion and ordinary timing, then instrument actual room submissions for state order. Only after those mechanics pass should visual review decide whether the reaction reads as intended. A six-dot center-ray result still does not establish that a tiny dot is perceptually readable on every display. The failure boundary includes projected size, contrast, camera angle, and unreviewed combinations beyond the existing nine-point gaze grid.

MD21-Q01 Explain a motionless angry expression

Explained answer

A hard drop reports speed twenty, angry is selected, and two seconds elapse, but no room render occurs. Is remaining angry a bug? What observation would demonstrate a genuine sequencing failure after rendering resumes?

MD21-A01 Require the right acknowledgment

Back to question

Remaining angry is expected while `presented` is false. The purpose is to avoid consuming an expression entirely between submitted paints. After the main room render calls `acknowledgeFacePaint`, a subsequent eligible update can advance to sad, whose own presentation must also be acknowledged before recovery completes.

A genuine failure would be a submitted angry frame followed directly by curious without the sad state being presented, or an acknowledgment from an unrelated reflection render that incorrectly unlocks progression. The boundary is main-room submission, not proof of physical monitor scanout. Preserve that distinction when instrumenting tests. The painted-drop browser case samples the actual renderer call and expects `scared, angry, sad, curious` in order, rather than using animation callbacks alone as evidence (tests/browser/pear-boy.spec.mjs:496).

PART 3 / CHARACTERS, SOFTNESS AND MOVEMENT

MD22

Limbs, rope and soft motion

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD22 / LIMBS, ROPE AND SOFT MOTION

Contents

Limbs, rope and soft motion

What softness actually means here

Implemented behavior. Pear's shell does not squash. Soft motion comes from four separately simulated ropes, each ending in a rounded hand or foot. `PearRope` owns simulated points, prior points, segment lengths, collision queries, and sleep state. The factory owns their tube meshes and terminal geometry. Calling this a skinned character or finite-element silicone solver would invent an authoring pipeline absent from the source. Rope state (src/pear-rope.js:1) and factory limb creation (src/pear-boy.js:110) define that division.

Design interpretation. Inertial lag, rounded tips, and resting feet can suggest a small hanging toy. The measurable implementation contracts are stronger and less subjective: attachment stays pinned, segment lengths remain bounded, body and desk contact are respected, and the final tube follows finite simulated points without allocating a new mesh every frame.

Establish rest geometry and terminal dimensions

Each authored limb starts from a three-point Catmull–Rom curve. The factory uses `getSpacedPoints(6)`, producing seven particles and six rest lengths. Arms start around `(±2.48,3.45,.35)` and terminate initially around `(±3.05,1.6,.8)`. Legs begin at `(±1.2,.65,2.35)` and extend toward `(±2.05,.36,3.85)`. These are authoring seeds, not guaranteed final settled particle positions.

Cord radius is `.14` in the factory, overriding the solver's `.085` default. Hands use contact tip radius `.32`. Feet use radial contact extent `.55` but floor height `.36`, matching their visibly wider-than-tall scale. Distinguishing `tipRadius` and `tipFloor` is essential at the rounded desk edge: a spherical collision assumption would lift a flattened foot too high.

The factory runs up to 600 warm-up updates, stores the settled points as rest, resets, then later settles again at the actual model origin. This makes the visible default depend on desk placement, not merely the original curve. Hand and foot seeds are the reproducible starting values.

Derive Verlet-style movement

For each unpinned point, the solver computes an inertial displacement `d=.985*(p-pPrevious)`, copies the current point into previous, adds `d`, and subtracts `45*h²` from Y, where `h=1/120`. The gravity displacement per step is `.003125` model units. This is a position-based integrator; there is no independently stored velocity vector for each particle.

Root movement updates the pinned first point immediately. If a pointer teleport moves the root farther than twice total rope length, both current and previous free points receive the excess translation. Moving both avoids inventing velocity from the correction. Even when less than one fixed step has accumulated, a moved root invokes constraint solving, preventing a rapid pointer event from leaving the rope stretched until the next tick. Update and integration (src/pear-rope.js:48) implement these safeguards.

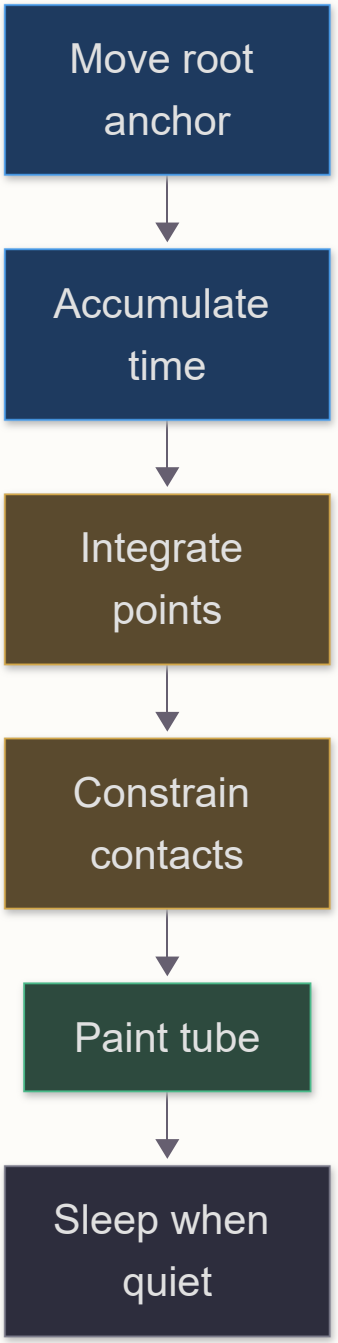


Figure MD22.1. Solver time and visible tube updates are separate stages; root movement can constrain attachment even before a full physics step.

Length constraints and convergence limits

For neighboring points A and B, let current separation be `d`, length `l`, and rest length `L`. The correction vector is `d*(l-L)/(l*(wA+wB))`. A moves by `wA` times that vector; B moves oppositely by `wB` times it. The attachment has weight zero, ordinary particles weight one, and the last particle weight `.3`. Thus the terminal particle receives less positional correction than an ordinary point; it behaves as a relatively heavy tip in this constraint distribution.

The solver performs at most 120 iterations, interleaving segment corrections and collision corrections. It exits early when every segment differs from its rest length by less than `.002`. A near-zero segment length is skipped to avoid division by zero. This is bounded iterative convergence, not an exact algebraic guarantee for every impossible configuration.

Sleep requires every free point's squared movement to remain below `2e-6` for 24 consecutive integration steps. On sleep, previous positions are copied to current and the accumulator clears. Those operations eliminate residual momentum instead of merely suppressing drawing.

Finite tabletop contact and the hanging foot

The rope's desk model has front Z `10`, bottom Y `-2.4`, and a separate lower safety floor near Y `-35`. Points over the tabletop use their cord or tip floor. Beyond the front edge, the solver permits hanging below the table instead of applying an infinite horizontal plane.

At the upper front corner, contact uses an ellipse: `((z-10)/radius)²+(y/floor)²=1`. Points inside that rounded corner are projected outward. Below the top, the front fascia can push a particle to `10+radius`. Supported points receive tangential damping by changing previous X and Z. This is why a foot partly over the edge can slide down instead of resting on an invisible square-expanded ledge. Contact branches show the actual finite geometry.

Body collision is radial around the translated root, using `radiusAt` at the particle's local height plus cord or tip radius. Arbitrary props, rotating supports, and other ropes are not included. Do not extend the desk result into a claim of general scene collision.

MD22-W01 Reproduce edge-to-table settling

Start with the exact leg seed and options from the unit test, including `.14` cord radius, `.55` tip radius, and `.36` tip floor. Settle at origin `(18,0,7.3)`. The terminal point should hang below `-.5` and report `tipSupported=false`. For each point, check either top support, front clearance, or the elliptical corner condition. Also check the first particle equals rest attachment plus origin.

Move the origin back to `(18,0,1)` and update for sufficient fixed steps. The terminal point should return to top support, rather than preserve its previous hanging offset. Then return to the edge and allow settling again. This tests the same rope under changed support topology without swapping meshes or resetting its identity.

The executable finite-edge tests also include a foot centered only partly beyond the edge. That negative control is valuable because an over-expanded rectangular tabletop can pass a simple “never below floor” test while producing the wrong hanging pose.

Paint a stable tube without rebuilding topology

Each rope mesh starts with 24 longitudinal and six radial divisions. Painting samples the simulated curve at 25 longitudinal positions, producing seven radial vertices per ring including the seam: 175 positions. The first normal is derived from tangent crossed with either up or a fallback Z vector when the tangent is nearly vertical. Subsequent rings remove the normal's tangent component and renormalize; binormal is tangent cross normal.

For radial angle `a`, the outward direction is `-cos(a)*normal+sin(a)*binormal`. It supplies the normal attribute; multiplying by cord radius and adding the curve point supplies position. Both attributes are marked dirty, and bounding box and sphere are recomputed. The tip follows the last local curve point. Supported tips use identity orientation; hanging tips align their up axis toward the preceding segment. Dynamic tube painting is explicit about these ownership conversions.

MD22-R01 Retry with a multi-second callback

Replace regular `1/120` updates with a three-second callback for the same total elapsed time. The solver accepts at most five seconds per update and integrates fixed steps; it does not turn gravity into one giant unstable Euler jump. The unit trajectory test compares sub-Hz, 2 Hz, 30 Hz, and 120 Hz results. Then try a `.001`-second root movement to exercise the separate inter-step constraint path.

If attachment is correct numerically but the rendered cord detaches, inspect world-to-local conversion in painting. Simulated points are world coordinates; curve points subtract `root.position`. Subtracting twice or also rotating through the animated face root introduces a visible error unrelated to constraint convergence.

Researched guidance. Current BufferGeometry documentation explains why changed vertex data needs updated bounds for culling and ray queries. Omitting that maintenance can make a correct rope disappear at a camera boundary. It is not evidence that the physics slept.

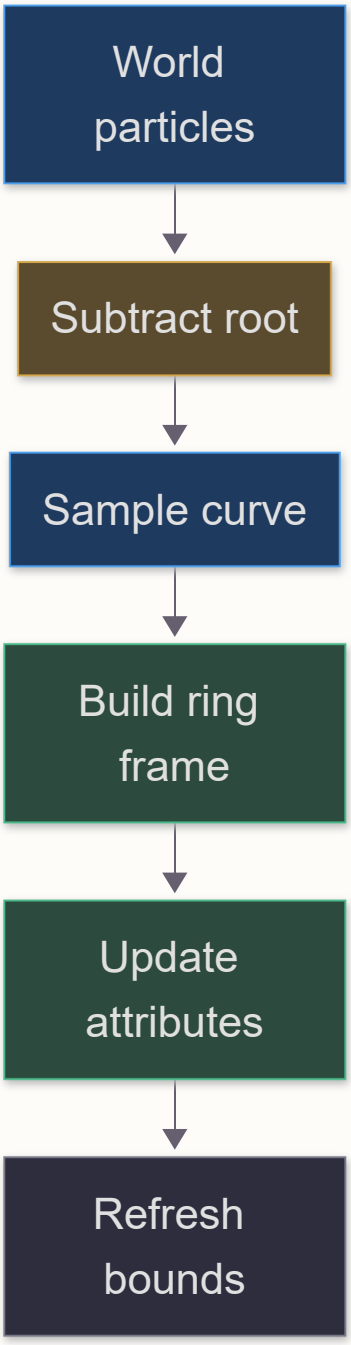


Figure MD22.2. The visible rope is reconstructed from world simulation into root-local rings, with explicit normals and refreshed spatial bounds.

Integration and limits

Reduced motion settles each changed rope rather than preserving oscillation. Body movement itself belongs to `DeskMotion` (src/pear-motion.js:59); it is not driven by rope tip forces. Scene rendering invalidates its clearance signature when limbs move (src/scene.js:1367), because the

envelope can change without root translation. The horizontal placement solver then evaluates geometry-derived obstacles (src/desk-clearance.js:3). These three supporting files establish why an idle root does not necessarily mean a settled layout.

The braided cord appearance is a repeating canvas texture with crossing diagonal strokes, not modeled individual fibers. The shared canvas helper uses sRGB color space (src/handheld.js:68). This texture supplies tactile suggestion cheaply; the six-sided tube still determines silhouette and collision radius.

Distinguish solver particles from visible curvature

Seven particles do not mean the viewer sees six straight cord segments. The factory interpolates a Catmull–Rom curve through those points and samples that curve for the dynamic tube. Conversely, collision checks act on the particles, not every interpolated point of the visible curve. A highly curved new limb could therefore look closer to a surface between particles than the particle contacts suggest. This is a meaningful limit of the implemented representation, not evidence that the present authored limbs fail.

Proposed extension. If longer or more tightly bent cords are needed, compare increasing particle count against adding collision samples along the interpolant. More particles increase constraint work and change the dynamic system; more paint rings improve visible smoothness without necessarily improving contact. State which problem is being solved before increasing either resolution. The source separates `getSpacedPoints(6)` from the 24-division tube precisely enough to make this comparison reproducible (src/pear-boy.js:112).

Another useful negative control is a root teleport exceeding twice total rope length. Check that the solver shifts both current and previous free points by the excess and that no enormous throw appears afterward. Then check an ordinary short move still produces lag. Suppressing all lag would make the teleport test pass trivially while breaking the intended soft motion. The existing attachment-and-inertia unit case provides the positive counterpart. Together those tests explain why the teleport correction is bounded rather than a blanket reset on every pointer event.

MD22-Q01 Identify the unsupported extension

Explained answer

A new swinging hand passes through a cartridge while still clearing the pear and tabletop. Does that contradict the existing solver contract? Propose the minimum additional responsibility required for a real repair.

MD22-A01 Add collision authority, not a cosmetic offset

Back to question

It does not contradict the current contract: arbitrary props are absent from `PearRope.constrain`. A genuine extension must provide obstacle geometry in a consistent coordinate frame and reconcile those contacts with pinned length constraints. Merely moving the hand mesh leaves the simulated

endpoint and painted tube behind.

The failure boundary is convergence under competing contacts. Add a negative test where the cartridge blocks the old trajectory, verify finite coordinates and bounded length errors, and define what happens if no feasible rope shape exists. Without that policy, increasing iteration count can hide an impossible configuration rather than solve it.

PART 3 / CHARACTERS, SOFTNESS AND MOVEMENT

MD23

Axolotl: the silicone body

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD23 / AXOLOTL: THE SILICONE BODY

Contents

Axolotl: the silicone body

Reconstruct the actual primitive assembly

Implemented behavior. `createAxolotlLamp` creates a compact cream-and-pink companion from shared spheres, extruded outlines, a smile tube, and an interior point light. There is no imported organic mesh, sculpting session, skeletal rig, or silicone deformation simulation in this factory. Its returned handles are `root`, `head`, `silicone`, `pink`, and `light`. The smaller feature count and lack of face animation distinguish it from Pear Boy, despite their shared movement controller. Complete body setup (src/axolotl-lamp.js:3) is the reconstruction starting point.

Design interpretation. A large rounded head and low elongated body can read as a small friendly night-light. “Silicone” is the implementation's material name and intended visual category, not proof of subsurface scattering, material measurement, or a physical fabrication process. Build the actual geometry and then evaluate the visual interpretation separately.

Establish axes and the two principal ellipsoids

A single unit `SphereGeometry(1,32,24)` is shared among the oval helper's meshes. The body center is `(0,1.12,-1.35)` and its scale is `(1.22,.97,1.98)`. The head center is `(0,1.45,.2)` with scale `(1.65,1.3,1.32)`. Thus the body extends mainly toward negative Z, while the face is authored on the positive-Z hemisphere of the head.

Derive approximate principal extents before adding details. The body spans X `±1.22`, Y `.15` to `2.09`, and Z `-3.33` to `.63`. The head spans X `±1.65`, Y `.15` to `2.75`, and Z `-1.12` to `1.52`. These are ellipsoid extrema from scale and translation, not the final whole-model box: gills, feet, toes, and tail extend it.

The two meshes overlap. There is no Boolean union or watertight remeshing step. Their common material and rounded overlap create continuity in appearance, but a future export or physical manufacturing workflow would need an explicit solid-union policy. Do not infer one from a smooth-looking render.

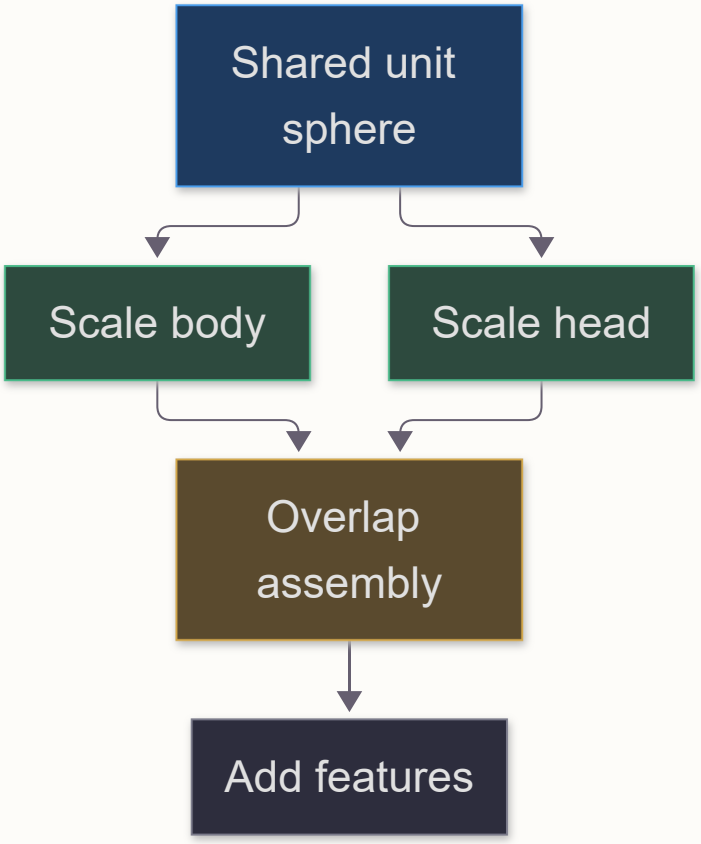


Figure MD23.1. Two transformed copies of one sphere geometry form the visible body assembly without a Boolean union.

Geometry sharing and transform ownership

The `oval` helper adds a mesh under the root and changes object scale, not the shared sphere's positions. That distinction makes sharing safe: changing one eye's object scale does not resize the head. Editing the shared geometry's vertices, however, would affect every oval using it. A proposed shape-specific deformation must clone the geometry or introduce a separate primitive.

Each mesh is named with the `axolotl` prefix, tagged `userData.axolotlLamp=true`, and marked to receive shadows. The entire root owns placement and scene yaw. Eyes and cheeks receive their own quaternions for surface orientation, while the head remains an independently returned mesh. The factory does not animate these transforms after creation.

Researched guidance. Current BufferGeometry documentation describes geometry attributes, normals, and disposal. That reference explains the general distinction between modifying a geometry and transforming an object; the actual sharing policy here is established by the one sphere allocation and the helper calls in pinned source.

Project features onto the head

For head-relative coordinates, the ellipsoid equation is `(x/1.65)²+((y-1.45)/1.3)²+((z-.2)/1.32)²=1`. Solving the positive-Z branch gives `z=.2+1.32*sqrt(1-(x/1.65)²-((y-1.45)/1.3)²)+offset`. The default offset is `.035`. Unlike Pear's projection, this function does not clamp the radicand. A proposed feature outside the ellipse produces nonfinite coordinates and must be rejected or handled explicitly.

Eyes are placed at `x=±.79`, `y=1.55`, while cheeks use `x=±1.05`, `y=1.24`. Their small depth scales make them surface-attached ovals rather than large protruding spheres. The factory aligns their positive-Z axes to an ellipsoid-gradient direction. That supports the sideward tilt needed near the cheeks without requiring a separate facial parent rig. Projection and orientation (src/axolotl-lamp.js:29) supplies both operations.

Material identity is not optical simulation

Cream uses color `#fff0dd`, roughness `.72`, emissive `#ffdcab`, and factory intensity `45`. Pink also uses roughness `.72`, with its own base and emissive colors and intensity `24`. The factory's black ink and blush use basic materials with tone mapping disabled. No transmission, thickness, subsurface scattering, or custom skin shader is configured here.

Current MeshStandardMaterial guidance, consulted 2026-11-07, explains roughness and emissive modulation. It does not justify describing a high-emission surface as physically measured silicone. In particular, visible emission and light arriving on the desk are separate: the factory also creates a point emitter. Scene controls later adjust both.

Pear supplies a useful implemented contrast: its shell is lathed and its emission has a canvas gradient, whereas this lamp uses constant standard materials and shared spheres (src/pear-boy.js:8). The difference is an intentional procedural assembly distinction, not evidence that either model went through Blender.

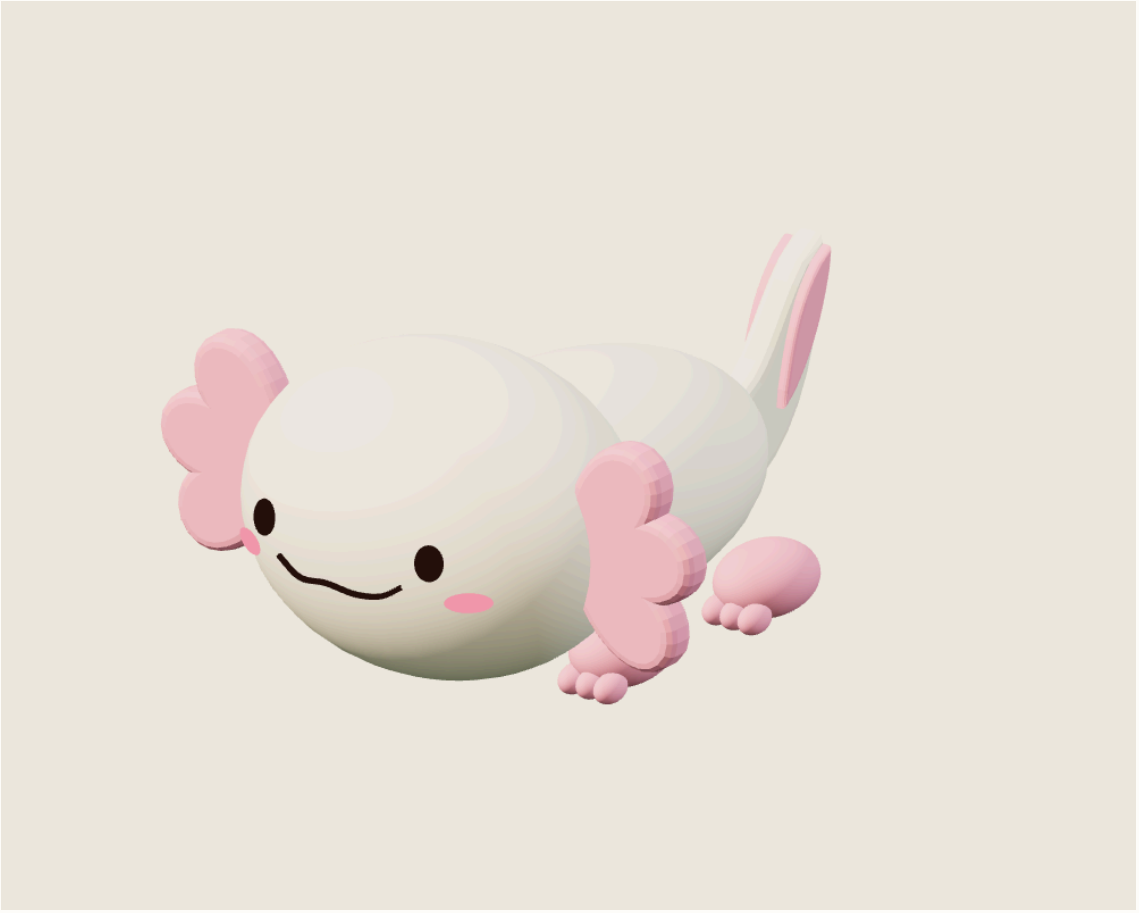


Figure MD23.2. Actual original-factory neutral-studio diagnostic showing head/body overlap; it does not establish native-room brightness or falling motion.

MD23-W01 Calculate and orient one eye

Choose the positive eye center `x=.79`, `y=1.55`. First compute `u=.79/1.65≈.4788` and `v=.1/1.3≈.0769`. The radicand is about `.7648`, safely positive. Its square root is about `.8745`; multiplying by `1.32` and adding `.2+.035` yields a front depth near `1.389`. These rounded calculations are explanatory; use actual double precision values for a reproduction comparison.

Next form the gradient-like vector `(x/1.65²,(y-1.45)/1.3²,(z-.2)/1.32²)` and normalize it. The code uses the already offset feature Z, so this is not exactly the gradient evaluated on the unoffset ellipsoid. Reproduce that detail rather than quietly replacing it with a mathematically cleaner variant. Set a quaternion from local positive Z to the normalized direction.

Finally apply the eye scale `(.115,.145,.07)`. In a browser inspection, read the named eye's position, scale, and transformed local positive-Z vector. Compare the mirrored eye: X position and outward X direction should change sign while the Y and Z magnitudes remain symmetric. This is observable without claiming a new screenshot or a full rendering qualification.

MD23-R01 Retry with a wider head

Proposed extension. Increase head X scale from `1.65` to `1.9` while retaining its center and Y/Z radii. If the projection formula and gradient denominators remain `1.65`, the features no longer follow the visible shell. The changed condition is not repaired by scaling the eye meshes alone. Update the authoring parameters consistently, then recompute front positions and surface orientation.

Measure the new whole-model box after all features are assembled and after the scene's `.3`-radian Y rotation. The scene captures axolotl motion bounds from the actually rotated root, translates that world AABB back by its root position, and derives root-workspace limits from its offsets. See scene construction and bounds (src/scene.js:130).

If the shape widens but its stored motion bounds do not, side collisions can accept visible overlap. Recreate or recompute those bounds deliberately in the experiment. A dynamic shape editor would need an invalidation policy; the pinned factory builds a fixed shape.

Movement uses the assembly, not anatomical joints

The axolotl's root moves through `DeskMotion`. Its gills, head, body, toes, tail, and light follow as children. The controller does not independently articulate feet or change the smile during impact. It stores one velocity and keeps orientation fixed during dragging and falling (src/pear-motion.js:5).

Scene clearance can temporarily displace the lamp during console focus and restore its remembered table XZ coordinates afterward. It treats the model as a spatial participant, not as a special-case painted decoration. The nearest-position helper expands obstacle rectangles by local envelope offsets and a default `.25` gap (src/desk-clearance.js:3). Thus topology choices can affect layout even when the body center stays unchanged.

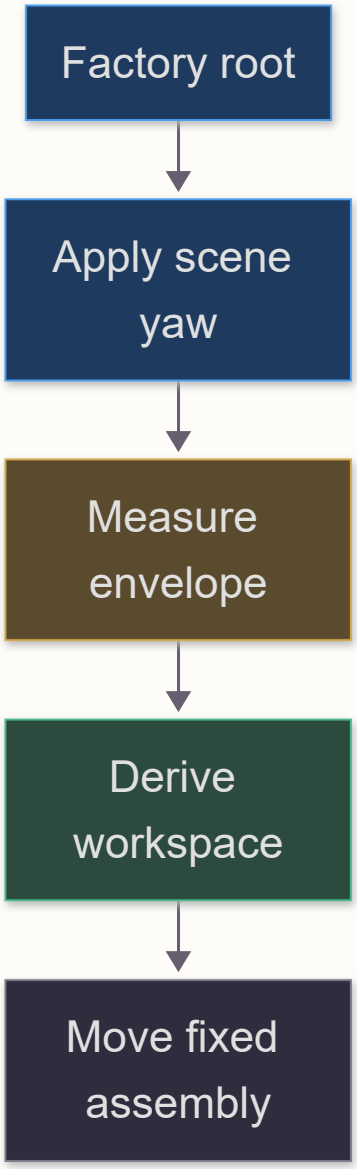


Figure MD23.3. Scene orientation is applied before the movement envelope is measured, preserving the assembled model's actual horizontal reach.

Historical constraint versus current behavior

Documented history. `TODO/110_SMALL_AXOLOTL_LAMP.todo` records the initial fixed-lamp scope, including the internal-emitter correction and independent brightness levels. `TODO/111_MOVABLE_AXOLOTL.todo` explicitly supersedes its fixed-position constraint with shared companion movement. The scoped `f10fab1752c4d15167baeaaace4623820f0c2bee` source history contains the assembled lamp. These records explain chronology, not recovered internal reasoning or fresh user approval. The supersession is recorded at (TODO/111_MOVABLE_AXOLOTL.todo:40), while the earlier optical correction remains historical evidence (TODO/110_SMALL_AXOLOTL_LAMP.todo:52).

A reconstruction agent must therefore implement the pinned movable behavior, not revive the older fixed-position rule because it appears in an acceptance record. Conversely, movement does not authorize inventing face animation, rope gills, or persistent saved lamp locations. Those are absent from this factory and would be separately labeled extensions.

Build an inspection checklist from the representation

Begin a new reconstruction with root identity transforms and verify the head/body centers before applying scene placement. This isolates an assembly error from a camera or layout error. Then check the single shared sphere geometry is used through object transforms, and that the two principal materials remain distinct handles. Their later brightness updates depend on shared material identity: accidentally cloning a cream material for one part can leave that part glowing after the scene switches the original `silicone` handle off.

That observation follows directly from the factory returning material handles and the scene mutating them, rather than traversing every mesh to find colors (src/axolotl-lamp.js:90; src/scene.js:586). A proposed material override needs an explicit control policy. Either include the new material in the power mapping or document it as deliberately nonemissive. Visual similarity at startup is insufficient if later state changes diverge.

Next inspect front, back, and three-quarter geometry under a neutral diagnostic setup, without treating those views as native-room evidence. A front view checks bilateral arrangement; a back view reveals the elongated body and tail overlap; a three-quarter view exposes attachment depth. Their value is complementary rather than redundant. A beautiful front image cannot prove that a tail layer is centered through its thickness.

Finally compare the complete rotated envelope with the scene workspace and test one rejected occupied placement. This connects the local assembly to its interactive role. The current model is smaller than Pear, but “smaller” alone cannot guarantee clearance when gills spread laterally or a tail extends behind the root.

MD23-Q01 Explain an apparently seamless body

Explained answer

The head/body junction looks smooth under warm emission. Can another agent claim the model is one watertight silicone mesh? What evidence would be required before exporting it for physical manufacture?

MD23-A01 Distinguish appearance from solid topology

Back to question

No. The implemented model contains overlapping sphere-based meshes with shared material, not a Boolean-unified shell. A manufacturing export would require explicit union, internal-face cleanup, manifold validation, wall-thickness decisions, and scale units. None follows from the visible overlap.

The useful failure boundary is that scene rendering tolerates intersecting parts inside a deliberately assembled character, while inter-object desk clearance forbids overlap between prioritized models. These are different contracts. Do not “repair” the intentional head/body overlap with the desk placement solver or describe the latter as an anatomical solid-model validator.

PART 3 / CHARACTERS, SOFTNESS AND MOVEMENT

MD24

Gills, tail and cute features

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Gills, tail and cute features

Detail as silhouette, not unverified anatomy

Implemented behavior. The axolotl factory adds six pink gill petals, four feet, twelve toes, two eyes, two cheeks, one smile tube, and layered tail geometry around the head/body assembly. These are procedural shapes, not scanned anatomy or imported reference-image meshes. Their names describe a stylized lamp. Biological realism is outside the demonstrated contract. Feature construction (src/axolotl-lamp.js:33) makes the inventory auditable.

Design interpretation. Large side petals, short feet, small bead eyes, and an upturned tail can make the silhouette readable at desk scale. Calling that arrangement cute is interpretation, not a test result. The engineering question is whether each detail survives projection, lighting, mirroring, and collision-envelope measurement without becoming visually or numerically unstable.

Author one gill petal, then mirror instances

The petal starts at `(0,0)` and uses two cubic Bézier curves. The first reaches `(1.02,.19)` through `(.28,.34)` and `(.86,.53)`; the second returns to origin through `(1.14,-.19)` and `(.45,-.34)`. The geometry is extruded `.12`, with bevel size and thickness `.09`, three bevel segments, and one depth step.

The factory shares that geometry among all gills. For each side, three meshes are placed at `(side*1.43,1.48,.24)` and rotated around Z using angles `side*[-.8,0,.8]`. Their X scale is `side`, so the negative side reflects the local petal. Negative scale is not a request to reverse the authored control points manually; doing both would double-reflect the shape.

The attachment point is the petal's local origin. Rotation therefore fans the tip around its base, not around the center of the whole extruded shape. This pivot ownership is what allows one shared shape to form three distinct silhouette directions. Centering the geometry automatically would change the fan even if all rotations remained identical.

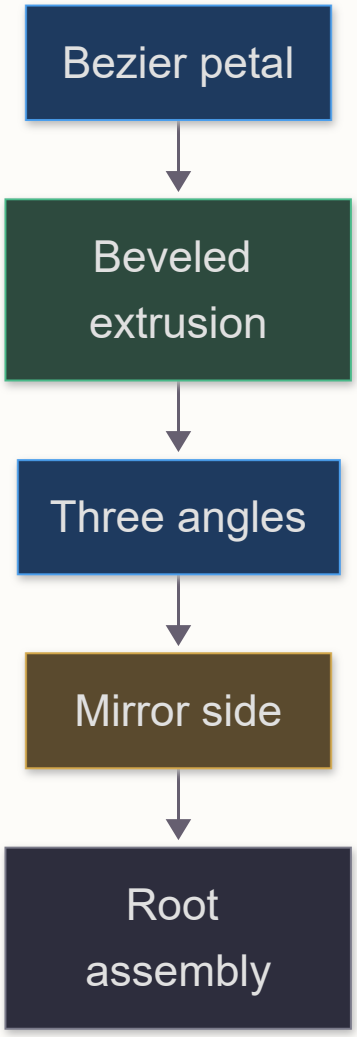


Figure MD24.1. Shared petal topology becomes a bilateral fan through base-pivot rotations and one X reflection per side.

Derive a petal's transformed reach

For a local planar point `(u,v)`, X reflection produces `(s*u,v)`, where `s` is the side sign. A Z rotation by `s*a` then mixes the horizontal and vertical coordinates. Add the mesh translation only after applying those local operations. This derivation explains why a petal's maximum world X is not simply `1.43+1.14`: its rotation changes which point is extremal, and bevel geometry adds reach beyond the original planar outline.

The actual Three.js object matrix also includes the scene root's `.3`-radian yaw. A desk-facing screenshot cannot by itself recover the unrotated gill dimensions. For reproduction, inspect local geometry and mesh transforms separately, then compute the assembled world box. Avoid reporting a single hand-estimated width as a verified final envelope.

Researched guidance. Current BufferGeometry documentation explains that geometry bounds must reflect changed positions. Here geometry is fixed after construction, but a proposed deforming petal would need both normal and bounds maintenance. Merely changing the object's rotation uses a different transform responsibility.

Bead eyes and cheeks follow a curved normal

Eyes use the shared unit sphere at `x=±.79`, `y=1.55` with scale `(.115,.145,.07)`. Cheeks use `x=±1.05`, `y=1.24` and scale `(.19,.085,.045)`. Their centers come from the positive-Z ellipsoid function, and their quaternions align local positive Z to a normalized ellipsoid-gradient vector. See facial feature attachment (src/axolotl-lamp.js:46).

The interpretation is a thin colored oval riding the head, not an eye embedded in a flat plane. Near the side, orientation matters as much as depth: an unrotated flat cheek would increasingly cut across the head curvature. The code's formula uses the offset center when computing its gradient-like direction; reproduce that detail before proposing a more exact normal-offset construction.

Pear offers a contrasting strategy: broad expression patches are subdivided and projected vertex by vertex, whereas these axolotl features remain transformed solids (src/pear-boy.js:90). The distinction matters when adapting a large flat marking to this smaller head. Enlarging an oval without revisiting curvature can reveal intersections absent at the original scale.

Feet and toes communicate support

For each side, feet are placed at Z `-.65` and `-2.45`, giving four feet. Their centers are `(side*1.08,.3,z)` with scale `(.34,.3,.46)`, and their Y rotation is `side*-.2`. Each foot receives three separate toe meshes at X offsets `-.17`, `0`, `.17`, center Y `.12`, and forward offset Z `.37`. Each toe scale is `(.105,.12,.17)`.

The foot's Y minimum is zero, and so is the toe's, from center minus vertical radius. Those authored support heights help explain why a root Y of zero places the lamp on the tabletop. They do not prove all extruded tail and gill vertices remain above it; inspect the complete assembled bounds for that.

Toes are direct root children, not foot children. Rotating a foot therefore does not rotate its toes. This is an important ownership fact for a proposed walking animation: the pinned model is static, and anatomical-looking naming does not create a joint hierarchy automatically.

MD24-W01 Construct the raised tail in the correct plane

Begin with the cream fin outline. It starts at the origin, curves toward `(1.55,1.65)`, and returns through a second contour to close. Extrude it `.12`, with `.09` bevel size and thickness and three bevel segments. Place the mesh at `(0,.55,-2.72)` and rotate around Y by `pi/2`. The original XY outline now reads as a vertical tail extending along the body's depth direction rather than as a side-facing sign.

Next build the separate pink tip outline using its own Bézier coordinates. Its extrusion depth is `.4`, bevel size and thickness `.035`, again with three bevel segments. Apply the same position and Y rotation, then call `translateZ(-.14)`. That last operation is local-axis translation after rotation, not a subtraction from world Z. It aligns the thicker pink layer across the cream fin's thickness direction.

For an observable reproduction, compare the two tail meshes' world matrices, not only their `.position.z` values. Transform the local unit Z direction through each quaternion to see where the offset acts. The exact sequence is visible in tail construction (src/axolotl-lamp.js:69).

The smile is a sampled tube

The smile uses six XY control points, projected to the head with offset `.045`. Their heights undulate from `1.31` through `1.2`, `1.23`, `1.2`, `1.23`, and `1.34`. A Catmull–Rom curve connects them, and `TubeGeometry` uses 32 longitudinal divisions, radius `.024`, six radial divisions, and an open path. It is neither a filled mouth patch nor a texture.

The small radius keeps the line modest relative to the head, but it creates a sampling tradeoff at distant camera scales: a physically present line can project to less than a pixel. Increasing emission will not necessarily restore black-line readability. A proposed thicker smile should be evaluated for both projected size and shell clearance, not just assigned a larger radius. Current Raycaster documentation helps distinguish a geometrically hittable tube from perceptually legible pixels; neither property automatically proves the other.

MD24-R01 Retry with longer gills and a closer camera

Proposed extension. Scale each gill's local X magnitude by `1.2` while preserving side sign and base pivot. The changed condition increases silhouette reach and may crowd nearby consoles. Recompute the full rotated model envelope; do not retain motion bounds measured before the edit.

Then move to a closer, three-quarter diagnostic camera. Inspect both front and back faces, because the extruded thickness and mirrored transforms can look acceptable from the front while exposing an awkward seam behind. The parent-produced atlas views are appropriate for this diagnostic comparison, but they are neutral-studio views, not proof of room light or animation. No new screenshots are implied by this recipe.

If enlarged gills collide with another desk model during drag, compare the scene's stored `axolotlMotion.bounds` with the newly measured assembly. The controller integrates one rigid upright root (src/pear-motion.js:5); it does not fold the gills away. The placement helper uses the provided envelope and cannot discover omitted geometry by itself (src/desk-clearance.js:3).

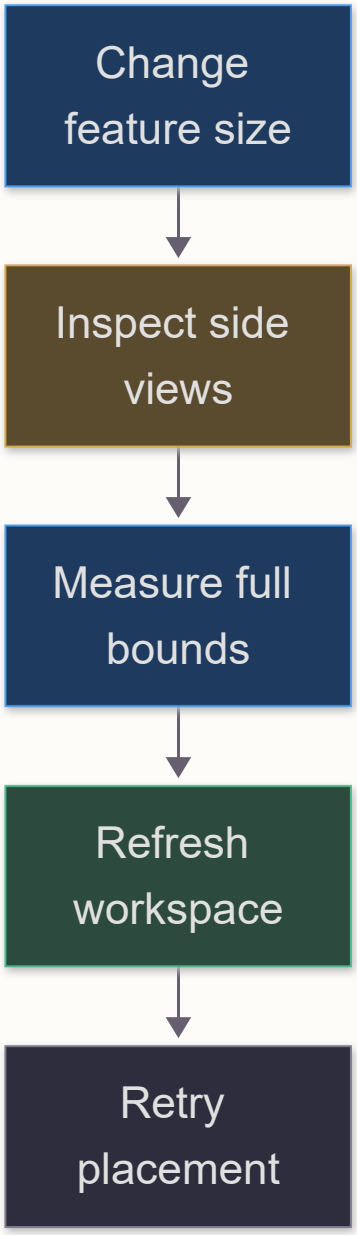


Figure MD24.2. A decorative silhouette edit becomes a movement-envelope edit once the feature extends beyond the old assembly.

Integration, history, and material tradeoffs

The scene tags and picks the actual companion meshes, applies yaw before measuring bounds, and keeps the emitter under the same root (src/scene.js:130). Thus details are not merely display ornaments: their geometry contributes to the measured footprint. The cream and pink materials share roughness but not emission color or strength. Current MeshStandardMaterial guidance describes that distinction; the chosen values remain pinned source facts.

Documented history. The axolotl task record describes a contact-sheet review that found overly dark wine-colored gills and a correction to pink material/emission. That is evidence of a recorded visual iteration, not permission to claim all future lighting conditions look correct. The same record explicitly separates local technical qualification from owner acceptance. The later movable-lamp record supersedes fixed placement without adding gill animation or a facial state machine. See the recorded color correction (TODO/110_SMALL_AXOLOTL_LAMP.todo:54) and movement supersession (TODO/111_MOVABLE_AXOLOTL.todo:40).

Count details without confusing count and quality

The browser geometry fixture checks six gill petals, four feet, and an axolotl-to-Pear height ratio below `.4`, alongside initial nonoverlap and a real mesh pick. These are useful assembly assertions: a missing mirrored loop or an accidentally doubled root scale becomes detectable without relying on subjective description. They do not prove twelve toes have ideal spacing or that the smile remains readable at every camera distance (tests/browser/dock.spec.mjs:662).

For a proposed feature revision, retain those inventory checks and add a specific observation for the changed condition. Lengthening gills calls for an envelope check; thickening a smile calls for surface clearance and visible line-width comparison; reparenting toes calls for transform tests. A generic “model exists” assertion is too weak to distinguish these failure modes.

The contrast with Pear is instructive. Pear's six freckles are checked as foremost ray hits across expression and gaze states, because animated facial neighbors can occlude them. Axolotl has no corresponding expression state machine, so copying that entire test grid would not automatically add value. Instead, choose observations that match its actual static geometry and shared-root movement. This avoids documenting imaginary animation simply to make two companions appear symmetrical in the handbook.

MD24-Q01 Find a misleading pivot assumption

Explained answer

A developer rotates the front foot and expects its three toes to rotate with it. Nothing happens to the toes, although the foot mesh visibly turns. Should the repair change toe geometry, the light intensity, or transform ownership?

MD24-A01 Build the hierarchy the animation actually needs

Back to question

Change ownership if articulation is the goal. The implemented toes are root children, so foot rotation cannot affect them. A proposed animated foot would need a group with an explicit pivot, reparented foot and toe meshes, and coordinates converted to that group's local space without changing the neutral appearance.

The failure boundary is not only the joint transform. Reparenting changes how bounds and picking traverse the assembly, and animated toes can change the silhouette without root motion. Add envelope invalidation and tests for the new hierarchy. Until that work exists, document the pinned model honestly as a static stylized assembly moved as one companion.

MD25

Companion light emission and material tuning

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD25 / COMPANION LIGHT EMISSION AND MATERIAL TUNING

Contents

Companion light emission and material tuning

Two different meanings of luminous

Implemented behavior. Both companions combine emissive mesh materials with point lights. Emission makes their surfaces appear luminous; point emitters illuminate surrounding geometry. The factories do not configure a physically transmitting silicone volume. Treating the emissive material as if it alone casts the warm desk pool would misdescribe the actual rendering architecture. Pear's setup appears in material creation (src/pear-boy.js:8), and axolotl's in two material families (src/axolotl-lamp.js:6).

Researched guidance. Current MeshStandardMaterial documentation, consulted on 2026-11-07, describes emissive color, intensity, and maps as separate factors. It also distinguishes roughness from emission. These general definitions help reason about the model, but pinned source remains the authority for its actual values and controls.

Reconstruct Pear's surface emission

Pear's diffusion map is a `16×256` procedural canvas. A vertical gradient uses `#b8b8b8` at zero, white at `.65`, and `#d6d6d6` at one. The standard material uses cream `#fff3d4`, roughness `.65`, emissive `#ffe4a0`, the gradient map, and factory intensity `77`. That gradient modulates a generated lathe surface; it is not a photographed silicone texture.

The shared `canvasTexture` helper creates the canvas, throws if no 2D context exists, paints it, sets `SRGBColorSpace`, and assigns anisotropy four. Texture helper (src/handheld.js:68) therefore belongs to the material evidence chain. An offscreen numerical factory recreation that bypasses this helper may reconstruct shape but not the same color pipeline.

Four point lights form a rectangular distribution at X `±1.1`, Y `4.4`, and Z `±1`. Each factory light has warm color `#ffd9a8`, intensity `560`, distance `70`, and decay `2`. The shell does not cast shadows, explicitly avoiding self-blocking by the opaque shell representation (src/pear-boy.js:325). This is an appearance approximation, not simulated light transport through silicone.

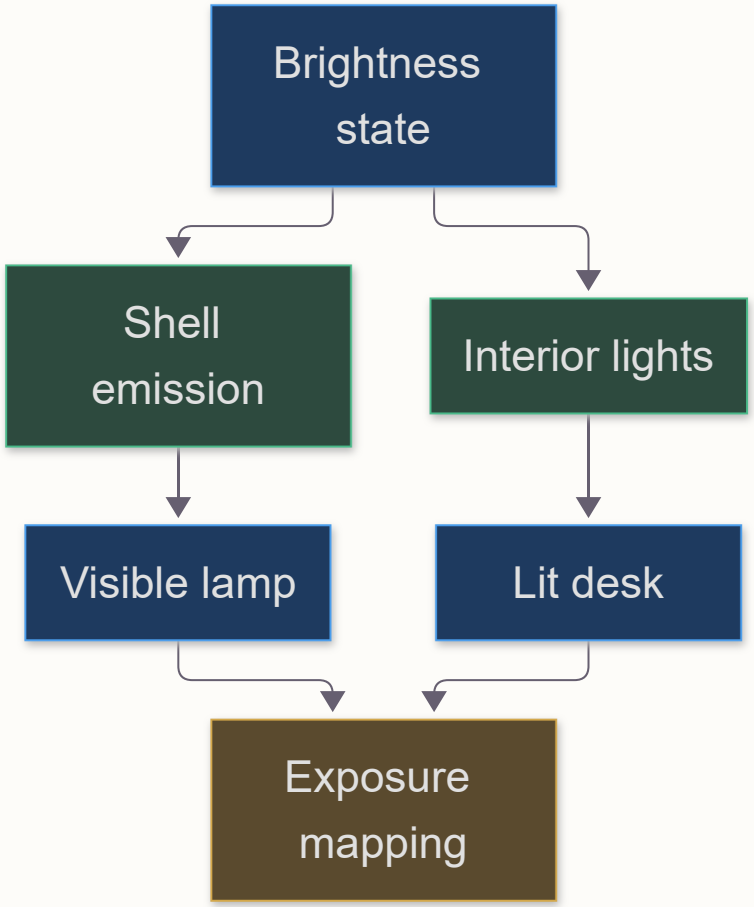


Figure MD25.1. Surface glow and received illumination are independently generated, then jointly affected by scene exposure.

Factory values versus scene-controlled values

The scene starts Pear on at brightness `.7`. `updatePearLight` assigns each emitter `800*b` and shell emission `110*b` when on. At `.7` these reproduce the factory values `560` and `77`. Power off hides the point lights and sets shell emission to zero. Brightness accepts finite values from `.1` through one and does not inherently enable power.

Axolotl starts on at `.5`. The scene assigns point intensity `260*b`, cream emission `90*b`, and pink emission `48*b`. At `.5` these become `130`, `45`, and `24`, again matching factory defaults. Its brightness setter accepts only `.5` or one, not an arbitrary continuous slider. Power off sets all three intensities to zero; its light need not be hidden to stop contributing. Control mapping (src/scene.js:563) is more precise than saying both lamps share identical brightness behavior.

Axolotl's internal emitter and readable face

The axolotl emitter is at `(0,.65,-.5)` beneath the head region inside the assembled body, with color `#ffd29e`, factory intensity `130`, distance `24`, and decay `2`. Its cream and pink materials have roughness `.72` but different base and emissive colors. Black eyes and smile remain basic-material markings rather than luminous parts. Emitter placement is a critical geometric parameter, not merely a lighting metadata detail.

Documented history. `TODO/110_SMALL_AXOLOTL_LAMP.todo` records that an initial exposed emitter near the face saturated both brightness levels. Moving it inside the body allowed the unchanged stronger rendered-pixel assertion to distinguish them. The record also describes correcting overly dark gill colors. Those are reported iteration outcomes, not recovered internal reasoning or evidence of fresh aesthetic approval. The source record preserves both the failed probe and the stronger unchanged pixel assertion (TODO/110_SMALL_AXOLOTL_LAMP.todo:50).

The lesson is not to memorize one magical intensity. A point light very close to a visible surface can dominate the local result. Moving it can improve readability while retaining an actual desk pool. Merely lowering emission would change a different contribution.

MD25-W01 Derive the two axolotl brightness states

At Sleep `.5`, compute emitter `260*.5=130`, cream `90*.5=45`, and pink `48*.5=24`. At Bright one, these become `260`, `90`, and `48`. All three double, but the final displayed RGB values need not double: lighting, material response, exposure, and tone mapping intervene. This distinction explains why a property-value assertion is insufficient for visible brightness.

For an observable reproduction, follow the browser fixture: turn off tripod, room, and Pear; select each axolotl level through controls; wait for its state, settled exposure near `.018`, and completed pending paint work. Sample the same `12×12` patch around the projected head target and compare mean RGB. The existing assertion requires Bright to exceed Sleep by more than five mean channel units. See actual shell photometry.

Keep the sample camera and location fixed. A changed sample that lands on a black eye or background is not a valid brightness comparison. Read the bitmap into the canvas before closing it, and normalize by retained canvas dimensions. The historical record notes that premature dimension handling first produced an invalid probe before the real saturation problem was found.

Exposure makes all-off a separate condition

The scene targets exposure `.018` if any tripod, room, Pear, or axolotl light is on; otherwise it targets `.45`. Damping uses rate four and current delta, with an explicit convergence threshold. See exposure adaptation (src/scene.js:1222).

Consequently toggling the last companion off changes more than its own emission. A before/after image taken during exposure adaptation can make the dark state unexpectedly bright elsewhere. Tests must wait for the intended condition rather than treating an immediate screenshot as a settled optical measurement.

Pear's light test samples near and farther desk locations, requires a red increase greater than blue increase, and checks meaningful illumination beyond the lamp's feet. That is evidence for warm received light, distinct from the axolotl shell-brightness test (tests/browser/pear-boy.spec.mjs:65). Neither test establishes a calibrated lux value or physical bulb efficiency.

MD25-R01 Retry after moving the lamp

Change conditions by relocating Pear while keeping its brightness constant. The lights are children of its root, so their world positions change with it. A desk sample that was bright can become dim even though all intensity properties are unchanged. First record emitter world positions and confirm the sample still hits the intended desk surface.

Then isolate two questions. To test production movement, let body and lights move together. To reproduce a historical fixed-light material comparison, explicitly hold the historical emitter positions while keeping legal body clearance. Label the latter as a controlled diagnostic, not native behavior. The verification ledger records exactly this distinction when clearance moved Pear and altered a DS deck color measurement (docs/VERIFICATION.md:462).

Do not “fix” that comparison by weakening RGB thresholds or pretending the lamp's local coordinates determine received light independently of layout. Root motion comes from the motion controller (src/pear-motion.js:36), and layout can also move it before painting (src/desk-clearance.js:3). These geometric sources belong in a lighting diagnosis because they change emitter-to-surface relationships.

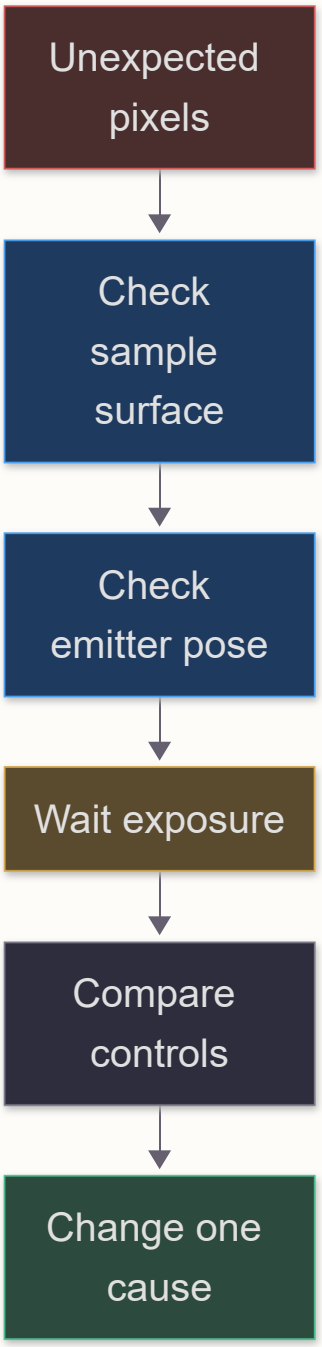


Figure MD25.2. Diagnose geometry, exposure, and state before modifying materials; a lighting mismatch can originate in layout rather than color.

Material and performance tradeoffs

Proposed extension. A transmissive material might produce a different silicone impression, but it would introduce thickness, environment, sorting, and performance questions not answered by the present setup. It must not be documented as an already implemented quality upgrade. Similarly, replacing four Pear point lights with one would alter the spatial distribution even if total nominal intensity stayed similar.

Current MeshStandardMaterial documentation clarifies that a displacement map or edited vertices affect shape differently from normal or roughness adjustments. Here no displacement-driven silicone surface is authored. The useful diagnostic sequence is silhouette first, material response second, actual received light third, and exposure last as an explicitly controlled presentation variable.

Design interpretation. High roughness and broad warm output may suggest soft diffusion. That remains an appearance interpretation. It does not justify using “physically accurate silicone” as a factual claim, especially when self-shadow suppression and multiple interior emitters are deliberate approximations.

Design a bounded optical comparison

Use paired measurements rather than a single impressive image. For Pear, hold camera, desk sample, exposure condition, and root pose fixed while switching its power. Compare the red-channel change with the blue-channel change to test warm contribution, and retain a farther sample to avoid qualifying only a tiny bright hotspot. The existing near/far test explicitly requires both effects rather than reading a light's color property (tests/browser/pear-boy.spec.mjs:65).

For axolotl, compare the same head patch at Sleep and Bright with other lights disabled. Then switch power off and check all three numeric contributions become zero. Selecting Sleep while off must leave power off. Finally wait for all-off exposure to approach `.45`. These observations test different layers: control semantics, surface response, and adaptation. Combining them into one broad screenshot assertion would make a failure harder to diagnose.

The scene marks reflections dirty when companion lighting changes. That does not mean a reflection capture happens immediately during every held movement. The render path waits for suitable settled conditions before updating DS reflections (src/scene.js:1375). A stale reflection during a drag is therefore a different question from whether the moved lamp's direct light follows its root. Keep those claims separate in both tests and captions.

Avoid misleading brightness units

The point-light constructor includes intensity, distance, and decay parameters, but this chapter does not infer a real bulb wattage from them. The displayed result includes exposure values far from a generic default and intentionally strong mesh emission. Without a calibrated scene, physical camera model, and measurement convention, reporting a consumer-lamp lumen equivalence would be invented precision.

A proposed tuning session should record model pose, camera pose, enabled lights, brightness state, exposure convergence, and the sampled surface. Those are reproducibility inputs, not private conversation memory. Another agent can repeat the comparison from those conditions even if it does not share the original author's aesthetic judgment. The interpretation “soft warm silicone” should remain clearly labeled as a visual aim, while exact colors, offsets, roughness, and intensity mappings remain source-backed facts.

When an edit improves the face but weakens the desk pool, do not report unqualified improvement. State which observation improved and which changed adversely. Emission, internal emitter location, and brightness mapping are independent controls with coupled appearance; a responsible tradeoff preserves that causal structure instead of collapsing it into one saturation slider.

MD25-Q01 Explain indistinguishable brightness screenshots

Explained answer

Both brightness buttons correctly double the three axolotl intensities, yet the sampled face is nearly identical. What should be checked before declaring the control broken? Why is a metadata-only test inadequate?

MD25-A01 Test the displayed response and its causes

Back to question

Check saturation, emitter proximity to the face, sample location, other enabled lights, and exposure convergence. If the face is already clipped, doubling linear contributions can leave displayed pixels almost unchanged. Metadata proves that a setter executed, not that the intended appearance survived the render pipeline.

The existing stronger boundary is an actual shell-pixel difference with other lights isolated and settled exposure. Preserve that assertion while changing one causal parameter at a time. Conversely, if only a neutral-studio atlas is available, report its geometry diagnostic value and leave native-room photometry unverified. No atlas caption should imply the movement or brightness sequence was captured.

PART 3 / CHARACTERS, SOFTNESS AND MOVEMENT

MD26

Dragging, gravity and dropping

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Dragging, gravity and dropping

One root controller, two companion assemblies

Implemented behavior. Pear uses `PearMotion`, a configured subclass of `DeskMotion`; axolotl uses `DeskMotion` with bounds measured from its rotated assembly. Both share companion gesture handling in the scene. The controller stores a reference to the root position, velocity, previous position, fixed-step accumulator, held/sleep state, and optional presentation state. It does not simulate angular momentum or soft-body deformation. Controller state (src/pear-motion.js:5) is the authoritative model.

This separation is reusable: model factories build geometry and local parts; the scene interprets input; the controller advances upright translation. Pear's rope and face systems respond to body movement afterward. Axolotl's static feet, gills, and emitter simply follow its root (src/axolotl-lamp.js:3). Similar interaction does not imply identical anatomy or animation.

Construct a camera-plane drag

`beginCompanionMove` switches toward table mode, saves the pickup origin, and creates a plane whose normal is the camera's world direction. The plane passes through the actual hit point, or a bounds-center fallback. Intersecting the pointer ray with that plane gives a 3D drag point. The saved offset from that point to the root prevents the object center from snapping under the pointer.

Pointer movement must exceed four screen pixels before the gesture counts as moved. A short click can therefore remain a power toggle; a drag releases into physics without toggling. Pointer capture keeps ownership when the pointer leaves the canvas. Cancellation, lost capture, blur, and related reset paths restore the saved pickup origin rather than inventing a new home. Gesture setup (src/scene.js:611) and completion show these responsibilities.

Wheel input adds camera-direction depth using clamped `deltaY*.015`, bounded to `[-2,2]` model units per event. The drag plane is translated by the accepted root displacement, preserving subsequent pointer continuity. Keyboard arrows and Page Up/Down use `.65`-unit world-axis steps, and clear velocity after each deliberate placement.

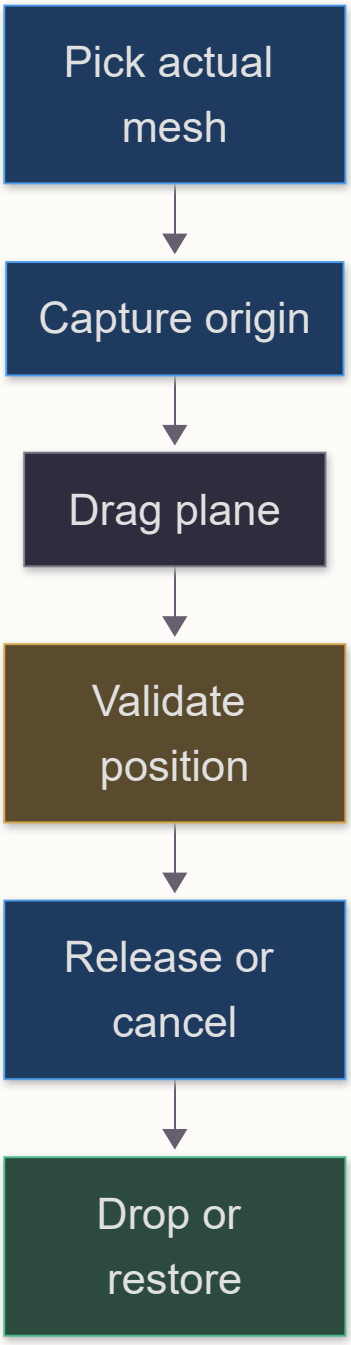


Figure MD26.1. Input owns a bounded translation transaction; release and cancellation intentionally take different state paths.

Derive release velocity without unbounded throws

`moveTo` estimates seconds from the previous movement timestamp, clamped between `.008` and `.1`. After the first sample, velocity is displacement divided by that duration, then clamped to total length 22. The clamp limits three-dimensional speed, not each component independently. The first movement has no previous finite timestamp and does not create an arbitrary throw.

`drop` clears held state, impact report, and accumulator. If more than 100 milliseconds passed since the last movement, velocity becomes zero. This makes holding still before release produce a fall rather than preserve a stale throw. `pickUp` and `reset` also remove old velocity and pending presentation. These details explain why keyboard placement explicitly zeroes velocity rather than depending on the user's key-repeat timing. Move and drop are the reproducible rules.

Fixed-step gravity and contact response

Each physics step is `1/120` second. Velocity Y decreases by `45*h`; horizontal velocities multiply by `.998`; position advances by `velocity*h`. The method accepts at most five seconds from one callback and processes at most 600 steps in one update. It does not integrate an arbitrarily large unbounded suspension.

At the workspace floor, root Y clamps to its minimum. Incoming downward speed is reported as impact speed. If its magnitude exceeds `1.2`, vertical velocity rebounds at `.23` of that magnitude; otherwise it becomes zero. Supported horizontal motion is additionally damped by `.85`. Total speed squared below `.0025` allows sleep and clears accumulated time.

Obstacle collision expands each box horizontally by the moving model's offset bounds. Previous vertical position distinguishes landing from hitting an underside. If neither applies, the controller chooses the nearest horizontal side. These are box-based contacts, not continuous triangle collision or a general guarantee against tunneling at arbitrary speeds and obstacle thicknesses. The implemented bounded speed and step size define the useful regime.

MD26-W01 Derive the first drop and rebound

Start Pear at height ten with zero release velocity and no obstacles. After one step, `vy=-45/120=-.375`, and the height decreases by `.375/120=.003125`. This immediate change is small but physical; a snap straight to zero would violate the implementation.

Ignoring discrete-step error and damping of horizontal components, the continuous reference gives impact speed `sqrt(2*45*10)=30`. The `.23` rebound suggests initial upward speed near `6.9` and a free rebound height near `6.9²/(2*45)=.529`. These are explanatory estimates, not exact expected floating-point values: collision is detected at fixed steps and the controller may clamp to supports.

For observable reproduction, use the unit test's raised position, call `pickUp`, move to height ten, and drop after the stale-velocity threshold. Assert that an early update remains between nine and ten, later updates contain positive vertical velocity, height never goes negative, and the final velocity is zero. See fall and contact tests. This establishes trajectory, not yet visible presentation.

Physical integration is not a painted fall

Documented history. Commit `120d9bd5a3e785d24a9bdc5e96d6e9b332613510` added paint-aware presentation. The verification ledger records an earlier failure where 48 physics steps covered fall and rebound without a main-room render. Correct final position and animation-callback samples did not prove the user saw the fall (docs/VERIFICATION.md:443).

With `requirePaint=true`, the controller first holds an actual descending pose at or below halfway between release height and relevant support. It preserves the remaining accumulator and refuses further integration while presentation is pending. Main-room rendering acknowledges that pose, allowing the controller to continue toward contact and a first upward pose.

During rebound, it chooses a rising threshold `contactHeight+vy²/(4*45)`, halfway toward the ideal ballistic apex height. The next pending pose is still an integrated rising state, not a fabricated animation key. A second acknowledgment releases the remaining catch-up. Presentation logic explains why queued time and visible state must be managed together.

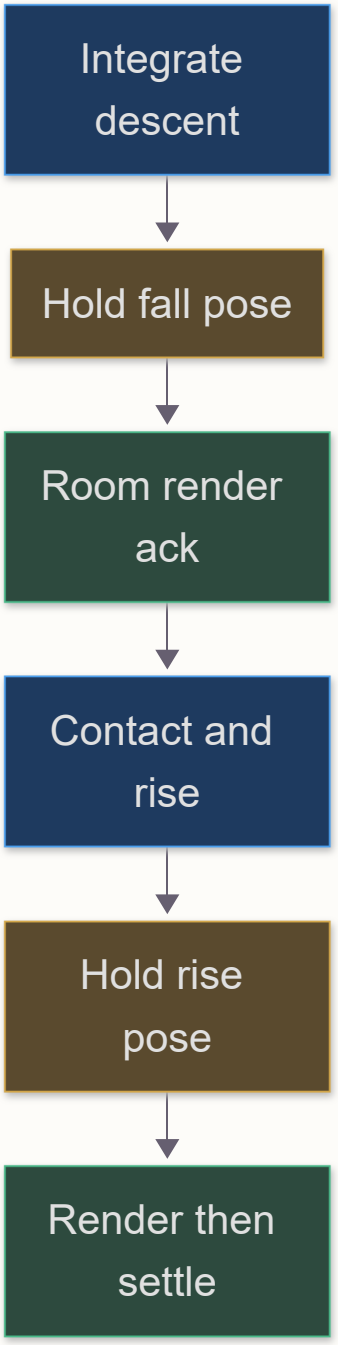


Figure MD26.2. Sparse updates retain actual descending and ascending poses until main-room submission acknowledges them.

MD26-R01 Retry with a three-second callback

Change the callback interval from `1/120` to three seconds. First run without paint requirements: the unit trajectory test expects equivalent integrated contacts over the same bounded elapsed time. Then enable `requirePaint`. The first long update should stop mid-fall; a further update should leave position unchanged while adding queued time. Acknowledge paint, update with zero new time, and observe a rising pose. Acknowledge again and allow sleep.

If the body remains frozen, inspect whether the main room render actually occurred. Scene code acknowledges Pear, axolotl, and console motion only after `renderer.render(scene,camera)` (src/scene.js:1378). A reflection capture or dirty flag is not an equivalent acknowledgment. If it snaps directly to rest, inspect whether release requested paint-aware behavior or bypassed the normal finish path.

Researched guidance. MDN's requestAnimationFrame reference documents variable refresh rates and paused background callbacks. It supports the need for elapsed-time handling, but an animation callback is not evidence of a completed GPU frame. Keep physics, render submission, and observed pixels as separate evidence layers.

Couple reactions and ropes without mixing authorities

Pear receives `frightened=true` while held or not sleeping. Downward support impact speed can trigger anger at ten or above, followed by pout recovery. Rope updates then follow the changed root. The shell's root remains upright; there is no impact-driven body pitch. The actual hierarchy and face state are in `createPearBoy` (src/pear-boy.js:235), and rope origin changes are handled independently (src/pear-rope.js:48).

Scene clearance can still adjust horizontal placement before a paint. That is why a sleeping controller is not enough to assert final saved console coordinates. The browser console test explicitly waits for a settled table paint before recording the moved position (tests/browser/console-movement.spec.mjs:159). Geometry reconciliation belongs to the placement solver (src/desk-clearance.js:3), not to a hidden movement animation.

Actual axolotl drop assertions wrap the room renderer, requiring descending and rising positive-height samples and no negative heights (tests/browser/dock.spec.mjs:791). Pear's corresponding fixture additionally checks the painted expression sequence. These are test definitions read for this chapter, not fresh execution claims.

Raised supports and offset origins

A root's floor is not necessarily the mesh's visible bottom. The generic controller uses local bounds offsets to land on an obstacle: support root height is `obstacle.max.y-bounds.min.y`. The unit fixture with local minimum Y `-.5` lands at `8.5` on an eight-unit support, not at eight. The workspace minimum is separately configured to `.5` for the table in that fixture. This is why reusing a controller with a new model requires measuring origin offsets rather than copying Pear's zero floor (tests/unit/pear-motion.test.mjs:102).

For an underside hit, the correction uses obstacle minimum minus the moving model's maximum bound. Side contact uses horizontally expanded intervals and reflects the affected velocity component. A proposed nonupright companion would need current-orientation or conservative rotational bounds; the existing translational controller does not update a spinning collision shape for you (src/pear-motion.js:91).

The paint-aware fall threshold also considers raised supports below release height. Otherwise a halfway-to-table hold could lie inside a tall obstacle. The raised-support unit case checks that the held descending pose remains above the support, then verifies catching clears presentation and ordinary integration can settle correctly. This is a concrete changed-condition test, not a blanket claim that all stacked furniture is supported.

Select instrumentation that cannot miss the event

Arm a release observer before dispatching pointerup. Starting a recorder only after a later automation evaluation can miss a genuine short fall and falsely blame physics. Conversely, sampling every animation callback can record positions that were never submitted to the main room renderer. The Pear fixture therefore records both progression and actual room-render samples, and the axolotl fixture wraps the renderer for its descending/rising assertions (tests/browser/pear-boy.spec.mjs:448; tests/browser/dock.spec.mjs:791).

These are complementary controls. An event-time recorder avoids an observation race, while the render wrapper proves presentation was attempted. Neither alone establishes physical monitor scanout, and neither authorizes relaxing the existing no-penetration or expression-order assertions. If a test fails under sparse software rendering, retain the failure evidence and inspect whether time was discarded, presentation was bypassed, or the observer started late.

The historical paint-aware correction changed the controller's presentation contract while preserving gravity and rebound values. That is a more precise lesson than “slow down the animation.” Another agent can reconstruct the mechanism from fixed steps, pending poses, acknowledgments, and tests, without claiming literal continuity with any prior agent's memory.

MD26-Q01 Distinguish cancellation from resetting home

Explained answer

A companion is moved from a nondefault table location, lifted, then canceled. Should it return to the original factory home, the last accepted held point, or the position captured at pickup? What would reveal stale momentum?

MD26-A01 Restore the transaction's origin

Back to question

Cancellation restores the pickup origin. Factory reset is a separate operation with occupancy checks; normal release drops from the last accepted point. After cancellation, velocity and pending presentation must be cleared. A subsequent update moving the canceled lamp would expose stale controller state rather than a valid continuing throw.

The failure boundary is occupied or externally changed scene geometry. Restoring an origin is not permission to hide overlapping objects; normal pre-paint clearance still applies. Tests should separate cancellation identity, collision reconciliation, and actual painted result instead of conflating all three with a single “position changed” assertion.

MD27

Clearance, collision envelopes and home poses

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Clearance, collision envelopes and home poses

Why sleeping physics cannot resolve every overlap

Implemented behavior. `resolveDeskClearance` runs in the main paint path after motion and geometry updates. It reconciles objects according to current interaction mode before rendering. A stationary Pear can otherwise be intersected by an incoming rotated console even though Pear's own physics is asleep. Moving the objects apart is the actual repair; visibility tricks, render-order changes, and disabled depth tests are not substitutes. Pre-paint integration (src/scene.js:1367) establishes this sequencing.

The system has several kinds of remembered position: authored seed coordinates, current root positions, saved console table poses, first legal console homes, temporary focus parking, and companion table snapshots. Their names are not interchangeable. This chapter treats layout as an explicit transaction with geometric evidence rather than a collection of convenient coordinate constants.

Choose envelopes for their actual responsibility

Pear's motion controller uses a simplified body box from `(-3,0,-3)` to `(3,8.1,3)`. The factory also computes broader bounds and expands horizontal reach to at least `±4.5`. For clearance, the scene starts with that broader envelope and unions current child bounds, excluding the contact shadow. The envelope therefore covers more than the instantaneous shell while still accounting for current limbs.

The contact shadow is a `6.6×6.6` plane with transparent material and a clipping plane. It is not a solid support object. Including it in an envelope would make a presentation aid displace real models. The source explicitly excludes it during Pear envelope assembly here (src/scene.js:806).

Axolotl bounds are measured after its scene Y rotation `.3`, so the offset AABB reflects the actual assembled orientation. Focused consoles can rotate and hinge; current world bounds are therefore needed instead of a historical unrotated width. A current AABB is still not an all-future rotational envelope. Re-evaluation and invalidation make it useful at each eligible paint.

Derive forbidden root rectangles

`nearestDeskPosition` receives local offset bounds, desired root position, root workspace, obstacle boxes, and a default gap `.25`. It first rejects desired heights outside workspace limits. Only obstacles with strict vertical overlap contribute forbidden horizontal rectangles.

For obstacle X interval `[oMin,oMax]` and local moving bounds `[bMin,bMax]`, the forbidden root interval is `(oMin-bMax-gap,oMax-bMin+gap)`. The same expansion applies in Z. This is a Minkowski-style reduction: instead of translating an entire box through every candidate, test one root point against expanded rectangles. The actual implementation is the complete solver (src/desk-clearance.js:3).

Candidate X values are the clamped desired X and every clamped rectangle edge; candidate Z values follow the same rule. Their Cartesian product includes mixed corners formed by different obstacles. Testing only “left, right, front, back” of one obstacle would miss the nearest legal corner in a crowded layout.

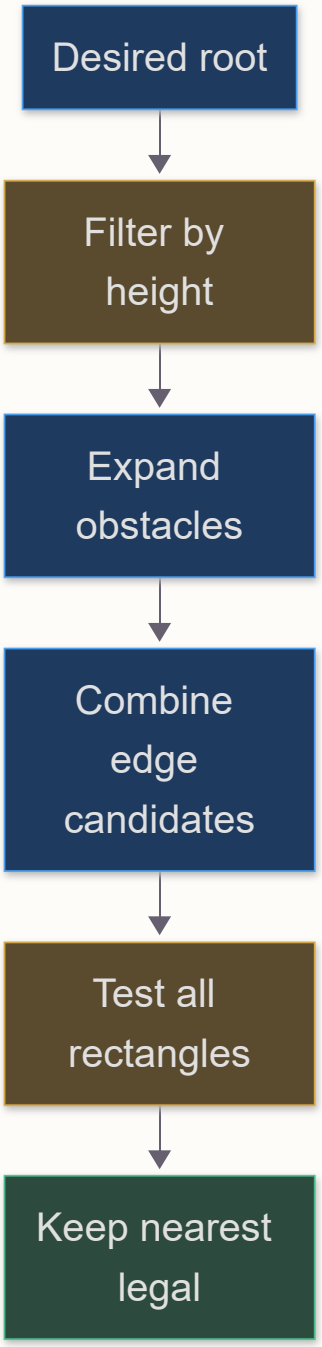


Figure MD27.1. Expanded forbidden rectangles reduce box placement to a finite search over desired coordinates and obstacle edges.

MD27-W01 Derive a two-obstacle corner

Use local bounds `[-.5,0,-.5]` to `[.5,1,.5]`, desired root `(0,7,0)`, and workspace X/Z `[-2,2]`. The first obstacle extends X `-2.25` to `.25` and Z `-4.25` to `4.25`; its right expanded edge is `.25-(-.5)+.25=1`. The second obstacle extends X `-4.25` to `4.25` and Z `-2.25` to `.75`; its forward expanded edge is `.75-(-.5)+.25=1.5`.

The nearest legal combination is therefore `(1,7,1.5)`, with horizontal distance `sqrt(1²+1.5²)=sqrt(3.25)`. Neither isolated single-axis correction is sufficient because the other obstacle still blocks it. The unit test also reverses obstacle order for this fixture and expects the same answer. See corner regression.

For reproduction, clone inputs before calling the solver and compare them afterward. The result is a new vector; desired position, workspace, bounds, and obstacle boxes must remain unchanged. Mutation would make later retry results depend on whether a previous diagnostic had already run.

Stable ranking and two different tolerances

Distance ranking uses squared horizontal distance. A candidate is rejected when `squared>=bestDistance-1e-9`; thus numerically equivalent candidates retain the earlier accepted choice. This `1e-9` is a ranking tolerance, not a collision gap and not the vertical overlap tolerance.

Obstacle-interior classification uses `8*Number.EPSILON*max(1,coordinate magnitudes)`. It accepts legal edges that move a few representable values during world/local bounds reconstruction. It does not replace the `.25` geometric gap with a broad collision allowance. The negative test verifies that genuine `.01` penetration still moves clear.

Documented history. Commit `6b5d2c552e6b3206bfb3ae36d845896773147a1c` added the scale-relative interior allowance. The ledger records an inactive DS return changing X by one ULP, from `11.314174109565958` to `11.31417410956596` (docs/VERIFICATION.md:410). A strict edge classifier rejected the formerly legal point after reconstruction. That evidence supports the bounded numerical fix, not loosening exact-pose tests or claiming all software latency was solved.

Mode priority and transactional placement

In focus, the active console is inserted as an immovable obstacle. The inactive console is placed next, then Pear, then axolotl, then loose props. The active camera-facing pose and hinge are not moved to accommodate lower-priority objects. On the table, companion positions become obstacles first, loose props attempt their anchors, and consoles yield afterward.

The resolver collects placements without immediately mutating roots. A failed placement reports the blocked model and returns false. For equal-priority table consoles, it first tries GBA then DS; if that ordering fails, it restores the candidate arrays and tries DS then GBA. This is a bounded order retry, not a global optimization over every permutation of every object. Priority and retry (src/scene.js:820) provides the exact ordering.

On commit, moved roots update motion's previous position and clear horizontal velocity. Elevated unheld objects can wake to fall. Pear limbs are reset against the new origin. This prevents a layout correction from leaving stale inertial state or detached rope geometry.

Capture legal homes, not overlapping seeds

The scene initially copies console seed positions into home slots, but after the first successful nonfocused clearance it replaces those homes with the collision-free console table poses. This is guarded by `initialDeskLayoutStored`. Resetting consoles later therefore targets a legal initial layout rather than the old seeds that predated companion clearance. Home capture (src/scene.js:868) is the important implementation point.

The wording “settled home” needs care: the guard records first successful table reconciliation; it does not explicitly wait for a new independent all-physics-sleep predicate. Preserve the exact contract instead of implying a stronger settling algorithm. Companion reset homes remain authored constants, with occupancy checks and explicit errors if another lamp blocks them. They are not automatically replaced by this console-home mechanism.

MD27-R01 Retry after focus parking and explicit movement

First choose a nondefault Pear table position and resolve table clearance. Enter Play or Inspect so consoles take priority. Save the original table snapshot once on entering focus; temporary parking must not overwrite it. Return to table and verify the remembered XZ values are restored while lower-priority models yield.

Now change conditions: reset Pear or explicitly move it while a table snapshot exists. Those operations update the remembered table position so an older focus snapshot cannot undo the user's newer choice. Canceling a move restores its pickup origin and updates the remembered snapshot accordingly. The browser focus fixture exercises both reset and explicit movement overrides (tests/browser/desk-clearance.spec.mjs:163).

If the lamp jumps back after a valid move, inspect snapshot ownership before changing geometry. If it oscillates between sides on successive paints, inspect tie ranking and reconstructed bounds before rounding all positions. Rounding would destroy exact pose identity and can introduce genuine overlap.

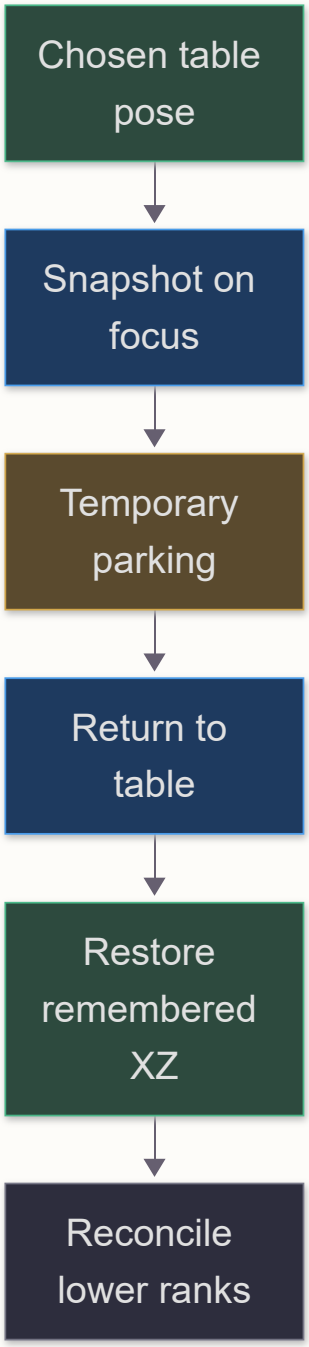


Figure MD27.2. Focus parking is temporary geometry reconciliation, not a new authored table pose.

Invalidate a layout when its evidence changes

The clearance signature includes focus state, DS lid rotation, cartridge motion presence, and relevant roots' positions and quaternions. Moving limbs or insertion activity explicitly clears the signature, because geometry can change without root movement. A future animated axolotl tail would require similar invalidation; its current static construction does not supply that automatically.

Rope simulation updates world points and tube bounds (src/pear-rope.js:48; src/pear-boy.js:330). Axolotl geometry contributes its actual extruded feature reach (src/axolotl-lamp.js:33). Body physics separately uses support and side contact boxes (src/pear-motion.js:91). These supporting sources explain why neither a current AABB alone nor a broad static envelope alone is a complete future-motion guarantee.

Researched guidance. Current BufferGeometry documentation notes that edited vertices may need recomputed bounding boxes and spheres. The repository does this for moving ropes. That general advice supports invalidation discipline, but does not prove a new model's envelope is sufficient without measured geometry and tests.

Order dependence has a bounded meaning

The two-obstacle worked example is insensitive to reversing its obstacle array because its nearest corner is unique. Do not generalize that result into universal permutation invariance. Candidate coordinates are stored in sets with insertion order, and equal-distance ranking deliberately retains the first accepted candidate within tolerance. A truly symmetric arrangement can therefore expose deterministic ordering choices. The source's guarantee is stable evaluation under its ordered inputs, not an abstract canonical answer independent of every permutation.

The console-order retry addresses a different problem. Two equal-priority models have different widths, so placing the narrower one first can leave no legal region for the wider one. Retrying the reverse order is a bounded search improvement. It does not remove all layout path dependence, and it does not promote a lower-priority model above the active focus console. Keep rank policy, candidate tie policy, and retry order as separate concepts when extending the solver.

Verify geometry at the paint boundary

The browser clearance fixture deliberately creates overlap without calling the resolver, then requests an actual paint. It records nonintersection and unchanged active pose from the renderer wrapper. This checks integration, not merely the helper in isolation. It additionally verifies that sleeping Pear yields in focus, returning to table restores remembered placement, and console rotations and tripod pose remain unchanged (tests/browser/desk-clearance.spec.mjs:75).

That test does not prove arbitrary triangle-level separation for every future model. It demonstrates the current envelope-based contract at the point the room is submitted. A proposed articulated companion must supply both an adequate envelope and invalidation when articulation changes. Without those responsibilities, a cached signature can certify stale geometry even though the helper's rectangle math remains perfectly correct.

MD27-Q01 Distinguish no solution from unstable choice

Explained answer

One crowded scene returns null; another alternates equally distant sides after bounds reconstruction. Should both be repaired by reducing the gap? What evidence distinguishes them?

MD27-A01 Preserve geometry and diagnose the actual failure

Back to question

No. Null can mean no legal root candidate exists in the fixed-height workspace. The correct response is an explicit blocked result or a changed layout, not quiet penetration. Alternating sides can instead indicate unstable tie ranking or edge classification. Compare exact desired coordinates, candidate distances, and before/after reconstructed bounds.

The solver's failure boundary is fixed-height horizontal placement with provided envelopes and priorities. It does not search arbitrary heights, rotate models, prove mesh-level clearance, or guarantee a globally optimal layout. The existing occupied-workspace, exact-edge, negative-overlap, and paint-path tests establish useful bounded contracts. Retain their assertions when extending the scene; a visually plausible screenshot cannot replace those geometric and state-identity checks.

PART 4

A complete, verifiable 3D world

1. The desk, room and tripod lamp
2. Light units, exposure and color
3. Shadows, reflections and shaders
4. Screen materials and pixel preservation
5. Picking, occlusion and input ownership
6. Render loops, GPU budgets and lifecycle
7. Algorithms and data structures
8. Design principles and failure boundaries
9. Validation and bounded recovery
10. The causal modeling history
11. Agent reconstruction and extension

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD28

The desk, room and tripod lamp

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD28 / THE DESK, ROOM AND TRIPOD LAMP

Contents

The desk, room and tripod lamp

A room is a coordinate contract

The shared desk is not an imported interior. It is a small, deliberately constructed Three.js world whose furniture establishes where other models can rest, move and receive light. Begin reconstruction with its support surfaces, not with decorative detail. Otherwise every later contact, camera and lighting decision inherits an uncertain origin. The application uses Three.js **0.186.1**, exactly, rather than a Blender scene, CAD assembly, model importer or general physics engine (package.json:7–9).

In the constructor of `HandheldScene`, the tabletop is a box measuring 1000 by 2.4 by 26 world units. Its center is `(0,-1.2,-3)`, so its upper face is exactly world Y zero, its front edge is Z ten, and its rear edge is Z minus sixteen. Its enormous width is a framing expedient: it prevents a side edge from entering ordinary views. It is not evidence that the depicted physical desk is seventeen meters wide. Keep authored scene dimensions separate from claims about real furniture (src/scene.js:73–97).

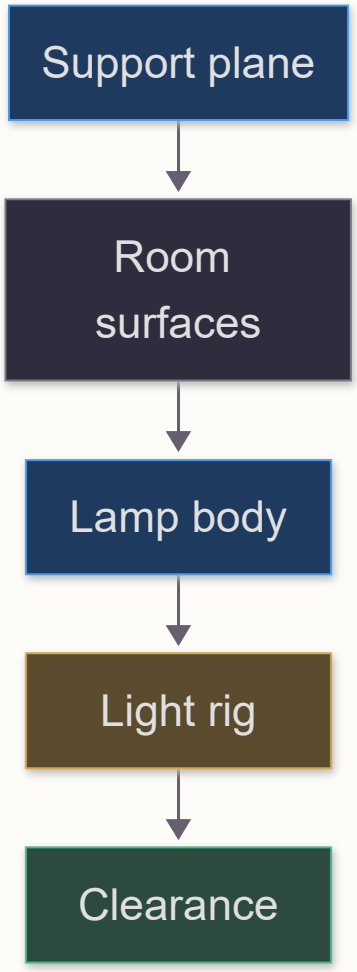


Figure MD28.1. Construct support and illumination before arranging movable objects; clearance is a consequence of real geometry, not a cosmetic final pass.

Constructing the support surfaces

Create two independently repeated wood textures. The tabletop repeats `1000/16` horizontally and `26/11.5` along the other texture coordinate; the fascia repeats `1000/16` and `2.4/11.5`. Separate repetition prevents the narrow edge from inheriting the top's stretched grain. The material array assigns the lighter, roughness `.83` material to the positive-Y box face; the remaining faces use the darker `.88` edge material. Do not assign a single UV scale to the whole box merely because it has one geometry.

The shared `createWood` helper makes a 1024-square canvas with a deterministic integer generator seeded at 41. It paints eight 128-pixel planks, each with a slight wash, 110 sinusoidally meandering grain strokes and paired seam lines. Repeat wrapping is enabled before the scene overrides its default repetition. Consequently, rebuilding the texture does not randomly change the grain on every reload. This determinism is useful for visual comparison: a material adjustment should not also move every grain line. The lamp has its own smaller ash-grain generator; do not replace both with one texture just because both are wood (src/handheld.js:657–685).

The wall is a 1000-square plane at `(0,464.5,-16)`, with warm plaster color and roughness one. Its lower edge is Y minus 35.5, close to the floor at minus 35. The floor is a 180-square plane rotated minus ninety degrees about X. Both receive shadows. These broad planes provide a continuous room background without introducing a window, ceiling mesh, chair, skirting board or extra prop that does not exist in this construction. The scene background itself starts black and changes separately with the room-light switch.

Build the physical world and the interaction workspace as different objects. The floor is not the drop destination for tabletop movement. `DeskMotion` uses a root-coordinate workspace whose minimum Y determines support. The visible table is much wider than that workspace. This intentional restriction keeps interactions near the camera and other models rather than turning the whole oversized support mesh into a draggable territory (src/pear-motion.js:59–89).

Building the tripod in metric-local space

`createDeskLamp` creates a root and a body group. The body is positioned at `(-13.5,0,2)` in world units and scaled by `1/.017`. Its children are authored in meter-like local dimensions. This is the essential ownership rule: parent translation remains a world-space placement, while child lengths are converted by parent scale. Do not divide the parent position by `.017` a second time (src/lighting.js:5–49).

Draw one leg as a closed four-point shape, beginning at `(.076,0)`, then `(.094,0)`, `(.027,.174)` and `(.009,.174)`. Extrude `.013` without bevels, then translate the extrusion by minus `.0065` along Z. Centering its thickness before rotating avoids a small systematic twist in the footprint. Reuse this geometry three times. Each leg rotates around Y by `.35 + index * 2π/3`. Thus the tripod is a radial construction of flat sloping members, not three cylinders leaning toward a guessed center.

The wood material uses a 128-by-512 generated grain canvas and roughness `.72`. Its strokes vary mildly with X and meander with Y. A short cylinder at Y `.162`, radius `.022`, height `.018`, supplies the shade support. There is no separately authored tall central pole in this factory: record the short support cylinder as the pole/support component rather than inventing a hidden stem. The cream trim is a standard material with roughness `.8`.

The shade is an open cylinder, radius `.09`, height `.15`, with 96 radial segments, centered at Y `.225`. Its open ends are important: a closed cylinder would introduce opaque caps absent from the model. The double-sided fabric material shows the interior and exterior of this thin surface. Two torus rims sit at Y `.15` and `.30`, each rotated around X to lie horizontally. A sphere at Y `.215`, radius `.018`, acts as a warm diffuser. The source explicitly disables shadow casting on shade, rims and bulb; copying the mesh helper's default casting flag without those overrides would change the result (src/lighting.js:51–81).



Figure MD28.2. The supplied neutral-studio front render is for checking leg silhouette, open shade and support proportions, not proof of actual room illumination.

Constructing a lamp that also illuminates

The visible glowing fabric does not replace scene lights. The factory adds a downward spotlight, transmitted-shade area light, open-bottom area light and separate tabletop-bounce area light. The first three belong to the scaled body; bounce belongs directly to the root. This hierarchy makes the finite bounce patch a world-sized approximation rather than a meter-local rectangle accidentally enlarged fifty-nine times (src/lighting.js:82–110).

There is no modeled toggle lever or cord switch. The mesh helper marks the lamp surfaces with `userData.lampToggle`, and scene picking invokes the lamp callback. This is an interaction model, not an omitted anatomical detail to fabricate. Switching sets light intensities to zero or restores their remembered values and changes shade/bulb emission. It keeps the light objects present to avoid changing shader light counts (src/scene.js:1053–1075).

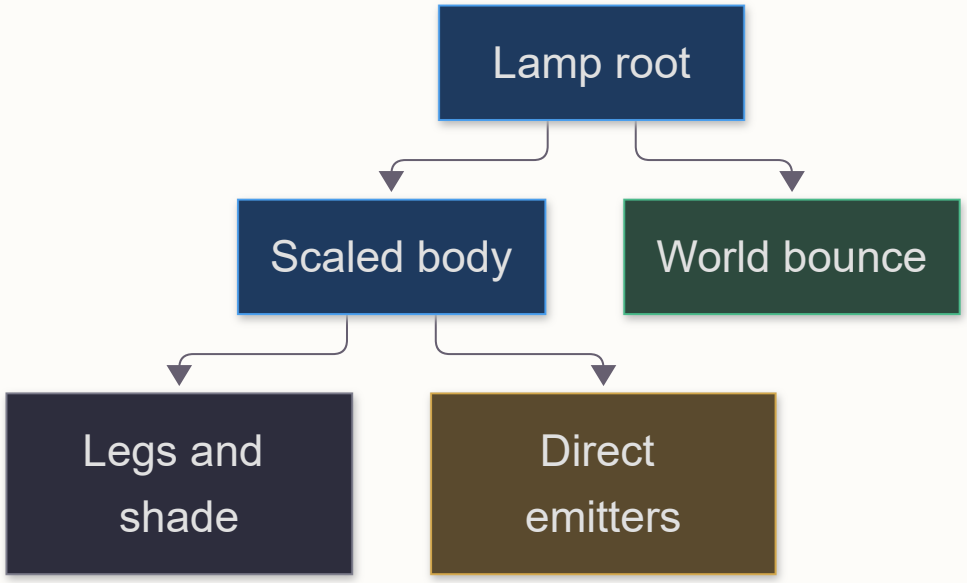


Figure MD28.3. The bounce patch has a different space owner from the lamp's metric-local parts; preserving that distinction prevents silent size errors.

Reconstructing the weave rather than painting a flat white drum

The fabric canvas is 1024 by 256. At each pixel, the factory combines two period-four cosine signals for warp and weft, their product, and a smaller sinusoidal fiber variation. The base value is 237; both main signals contribute six levels and their product contributes two. The same generated texture feeds ordinary color and emission, which keeps the weave visible when the shade glows. This is actual procedural texture synthesis, not a downloaded fabric photograph or a normal map (src/lighting.js:53–72).

For a material diagnostic, retain geometry, light and exposure while temporarily comparing the generated map with a constant near-white map in an isolated harness. The comparison asks whether texture contrast survives emission, not whether the whole room looks brighter. Restore the original map afterward and dispose only resources the harness created.

MD28-W01 Convert the shade and tabletop by hand

Work through three dimensions before writing geometry. With one world unit representing `.017` meters for this factory, shade diameter is `.18/.017 = 10.5882` world units; shade height is `.15/.017 = 8.8235`; top height is `.30/.017 = 17.6471`. The shade center becomes world `(-13.5,13.2353,2)`. The support cylinder center becomes Y `9.5294`. These are derived construction values, not measurements taken from a rendered frame.

The tabletop's upper face is `-1.2 + 2.4/2 = 0`, which matches the legs' local starting Y. That is why the factory can sit directly on the shared support plane without a compensating root lift. Check this equation before adjusting any contact shadow.

This original browser-module recipe assumes the project's import map, decoded brand assets and served source are already available. It constructs only the lamp; it does not start a renderer or modify the application. Expected output is three leg names and the derived top height.

```
import { createDeskLamp } from './src/lighting.js';
const lamp = createDeskLamp();
const body = lamp.root.children[0];
const legs = body.children.filter(
  item => item.name.startsWith('Ash tripod leg')
);
console.assert(legs.length === 3);
console.log(legs.map(item => item.name));
console.log(.30 * body.scale.y); // 17.647...
```

MD28-R01 Retry after changing the shade diameter

Suppose a proposed extension changes radius from `.09` to `.11` while retaining height. This is not a request to scale the whole body. Rebuild the shade and both rims with corresponding radii, preserve the support height, and reconsider whether the transmitted area emitter still lies outside the fabric. The current `.11` emitter X offset was chosen to avoid an intersecting plane and a hard lighting seam. A wider shade can invalidate that relationship even though every mesh remains legal.

Next recompute the lamp's actual world bounds and rerun desk clearance. The lamp is a fixed obstacle, not a lower-priority object the resolver may quietly move aside. A radius increase expands the occupied territory and can displace companions, cartridges and consoles. A visual improvement therefore has an interaction cost. The clearance browser test explicitly checks that the lamp's pose remains unchanged (tests/browser/desk-clearance.spec.mjs:59–72).



Figure MD28.4. The supplied diagnostic three-quarter view helps distinguish the three radial legs; it cannot establish the spotlight target or bounce strength.

MD28-Q01 Exercise: add a floor without moving the desk

Explained answer

A reconstruction sets the floor at Y zero because that is the usual ground convention in its previous project. Explain which contract breaks, how to diagnose the apparent intersection, and why moving every handheld upward is the wrong repair. Also explain why the wall's huge height is not a furniture dimension to copy into a product specification.

MD28-A01 Explained answer

Back to question

Here Y zero already belongs to the tabletop. A second horizontal room surface at the same height overlaps the support and can alter depth, lighting and visible grain. Restore the authored floor at minus 35 and leave table and object homes unchanged. Raising every model would require changing workspace floors, saved poses, light targets and camera fits to compensate for one mistaken surface. The wall is oversized scenery with its lower edge near the floor; its dimensions ensure coverage, not architectural fidelity.

Finally separate geometry from acceptance. A leg count assertion verifies factory structure; a clearance assertion verifies placement; a neutral studio image helps judge silhouette. None says the owner likes the lamp in the integrated room. That distinction is explicit project practice (docs/VERIFICATION.md:1–8).

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD29

Light units, exposure and color

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD29 / LIGHT UNITS, EXPOSURE AND COLOR

Contents

Light units, exposure and color

Begin with the source of each brightness

Three distinct mechanisms make this room appear bright: surface emission, actual light objects and renderer exposure. They are not interchangeable. A bright shade material makes the shade visible; a spotlight illuminates surfaces; exposure maps scene radiance into display values. Replacing a misplaced light with global brightness can make a screenshot legible while destroying the all-lights-off behavior. The renderer explicitly uses sRGB output, ACES filmic tone mapping and initial exposure `.018` (src/scene.js:39–47).

The checked current Three.js color-management guidance explains why lighting should be computed in Linear-sRGB and output converted for an sRGB display. This is contemporary guidance consulted on 2026-11-07, not proof of which library revision the application runs. That proof is the exact dependency in package.json:7–9. Never substitute a current documentation default for an explicit source assignment.

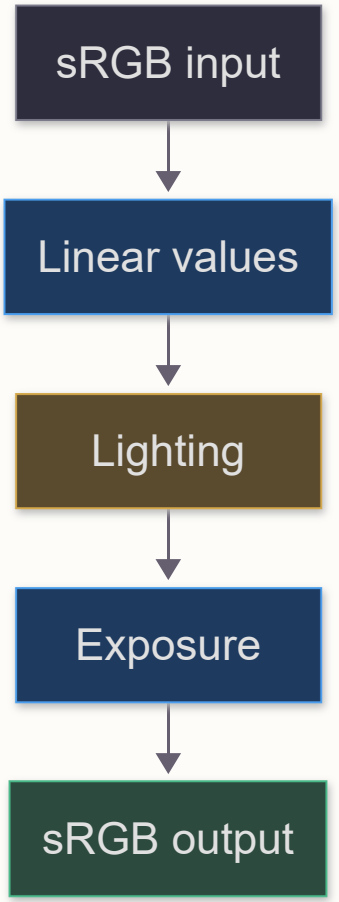


Figure MD29.1. Input encoding, physical-light arithmetic and display encoding are separate stages; changing light intensity cannot repair a missing conversion.

The authored hierarchy and its units

The tripod factory uses `.017` meters per world unit. Its spotlight intensity is `12/.017²`, approximately `41522.49`. This scaling preserves an inverse-square relationship when meter-authored geometry becomes world-sized geometry: at corresponding distances, intensity and squared distance acquire the same factor. It does not certify the modeled lamp against a photometer or establish a real bulb wattage.

The spotlight has decay two, unlimited distance, cone angle `π/3` and penumbra `.65`. Its local target includes the inverse placement needed to aim toward the desk. The transmitted-shade rectangle has intensity 300 and explicit dimensions `.18/.017` by `.15/.017`; the open-bottom rectangle uses 90 and `.16/.017` square. Dimensions are explicit because the source warns that rectangle-light dimensions do not inherit parent scale. Its bounce rectangle is six by six with intensity 50, rooted outside the metric-scaled body (src/lighting.js:82–110).

Room lighting is a separate switchable group: a 14-by-12 ceiling area light at `(0,10,5)`, intensity 132, looks toward the origin; a hemisphere light with intensity 22 represents warm reflected room illumination. The ceiling is a light, not a visible ceiling mesh. The hemisphere and tabletop patch

are authored approximations, not a traced global- illumination solution (src/lighting.js:113–123).

Companion lamps are part of the same exposure problem. At defaults, Pear brightness `.7` produces intensity `800*.7 = 560` on each of its lights and body emission `110*.7 = 77`. Axolotl brightness `.5` produces point-light intensity 130, silicone emission 45 and pink emission 24. These numbers describe source coefficients, not equivalent radiometric units across material and light properties. Turning off only the tripod and room therefore does not establish an LCD-only room (src/scene.js:563–603).

Sampling screen light in linear space

`ScreenIllumination` samples a 24-by-16 canvas, so each sample visits 384 pixels. A 256-entry floating-point lookup table converts byte channels from sRGB to linear values. The code sums converted values, divides by 384, and interpolates its current color toward that target. Averaging encoded bytes first would not conserve average light energy: half black plus half white averages to linear `.5`, not the linear equivalent of encoded middle gray (src/lighting.js:5–7, src/lighting.js:163–187).

Sampling a live source happens at most once per 125 milliseconds unless the source changes. Interpolation can still advance on eligible room paints. Its coefficient is `1-exp(-18*delta)`, making the response depend on elapsed seconds instead of a fixed fraction per callback. A maximum channel error below `.00001` snaps to the target, allowing eventual idle rendering rather than an asymptote that stays dirty forever.

The maximum current channel, called peak, determines area-light intensity as `peak*12`. Normalizing color by peak separates hue from overall strength. Experimental control spill uses `peak^.25*.1`; it makes dim content contribute a small amount near coplanar controls without inventing global ambient light. At peak zero, intensity is zero. The code avoids dividing by zero, and `clear()` explicitly zeros both emitters when a source goes away.

Exposure is stateful, not a one-off correction

During rendering, exposure targets `.018` if any tripod, room, Pear or axolotl lamp is on, and `.45` only when all are off. It damps toward the target with rate four. That is a factor of 25 between endpoints, not a sudden multiplication applied to LCD pixels. The GBA indicator's emission compensates by multiplying `80*.018/currentExposure`, preventing dark adaptation from bleaching its green appearance (src/scene.js:1222–1236).

The DS accent fill also follows this branch. Its emission is copied from material color only in the brighter-room target state and cleared for the dark-room target. It is an explicit stylization, not evidence that metallic paint generates light. Document this approximation rather than hiding it behind the word realistic. Active LCD materials avoid tone mapping entirely, so their picture and the room's adaptation have different ownership.

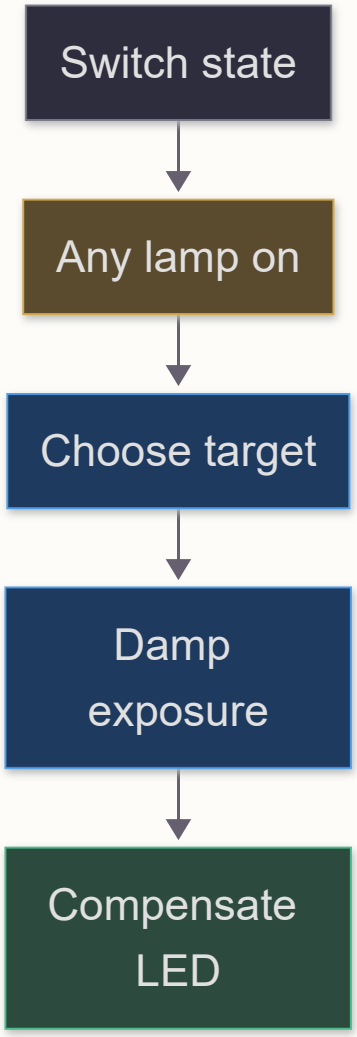


Figure MD29.2. All four switch families participate in adaptation; forgetting one changes both illumination and exposure.

Diagnose color by changing one stage

Use an ordered diagnostic rather than tuning several coefficients. First establish which object a sample actually hits. A ray landing on the glossy screen instead of blue paint invalidates a shell-color conclusion. Next identify material state: active basic LCD, inactive physical glass, ordinary painted shell or emissive accent. Then record every lamp switch and exposure target. Only after those conditions are explicit should a light coefficient be changed.

A color texture and a data texture also have different meanings. The checked current color-management guide recommends sRGB annotation for color imagery, while roughness, normal and other noncolor data must not receive a display-color conversion. The lamp's generated fabric is color imagery; calling every generated canvas “data” would therefore be an incorrect shortcut. Conversely, a proposed numeric roughness texture should not inherit the color helper's annotation without review. This is researched extension guidance, not a claim that a new roughness canvas exists in the lamp.

Finally compare settled states. A frame taken during exposure damping mixes old and new adaptation even if the light switch changed instantly. Record current exposure or deliberately wait for convergence. Do not demand an arbitrary fixed wait universally: eligible GPU paints and interpolation time determine progress. For exact LCD invariance, material type plus unchanged samples is stronger than a loosely described screenshot. For shell appearance, acknowledge that chosen probes cover only their local surfaces and views. A passing blue-channel threshold does not establish calibrated color across all monitors, camera angles or tone-mapping alternatives.

MD29-W01 Work a half-white screen through the sampler

Assume the downsampled canvas contains 192 white pixels and 192 black pixels, with equal channels and no resampling ambiguity. Converted linear white is one and black zero, so target RGB is `(.5,.5,.5)`. Starting from black, after `.1` seconds the interpolation coefficient is `1-exp(-1.8)`, approximately `.83470`. Current RGB becomes approximately `(.41735,.41735,.41735)`. Peak is `.41735`, the primary area intensity becomes about `5.0082`, and normalized light color is white. Control spill is approximately `.0804`. These are calculated expectations under stated assumptions, not recorded browser samples.

The following standalone Node recipe uses no browser or model assets. It reproduces that arithmetic and should log a primary intensity close to five. It intentionally does not approximate the renderer's ACES curve, which would add a different stage to the experiment.

```
const target = .5;
const delta = .1;
const current = target * (1 - Math.exp(-18 * delta));
const primary = current * 12;
const spill = current ** .25 * .1;
console.assert(primary > 5 && primary < 5.02);
console.log({ current, primary, spill });
```

For comparison, taking an encoded average `.5` and then linearizing it yields about `.214`, less than half the desired average energy. This explains why a lookup table is a correctness mechanism rather than merely a micro-optimization.

MD29-R01 Retry a dark upright DS without global ambient

Change the condition from a console lying on the table to the straight, camera-facing DS play pose. The historical repair moved its target from `(5,5.1,8)` to `(-2,5.1,-8)`. It did not raise exposure, recolor paint or add a camera light. The causal diff explains that the face must remain behind the existing emitter instead of being backlit (commit 9be851e).

Reproduce the diagnosis with the checked desktop and phone fixture: disable both companions, use real static status LCDs, verify the root is straight, ray-check eight shell probes, and compare tripod-only, all-off and room-only pixels. The test requires blue above 75 under the tripod and

maximum channel below three in darkness; it also requires the LCD RGB values to remain identical across states. These are assertions read in source, not a new successful run (tests/browser/ds.spec.mjs:1412–1486).

MD29-Q01 Exercise: why is “all off” still bright?

Explained answer

An agent switches off the tripod and ceiling controls, sees bright table pixels, and reduces LCD spill until they darken. Identify two uncontrolled inputs and explain why the apparent repair would damage a valid dark-room experiment.

MD29-A01 Explained answer

Back to question

Pear and axolotl remain independent illumination sources and also keep the low-exposure target selected. Disable both using their scene methods, then wait for adaptation and a real eligible paint. Otherwise the experiment mixes changed light energy with changed exposure. Reducing LCD spill would compensate for an incorrect fixture and remove legitimate screen-driven lighting from the actual all-lamps-off state.

The public verification record explicitly separates fixture isolation from production changes and reports parked lighting coverage as debt, not fixed behavior (docs/VERIFICATION.md:21–37). Use that same discipline when deciding whether a color mismatch belongs to encoding, material, geometry, exposure or evidence.

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD30

Shadows, reflections and shaders

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Shadows, reflections and shaders

Three effects that must not share a diagnosis

A dark patch beneath an object, a bright moving patch on lacquer and a discolored LCD may all look like lighting defects, yet they arise from different pipelines. Shadows estimate blocked direct light. Environment reflection samples surrounding radiance. Screen shaders or canvas effects transform image content. Start by naming the pipeline before changing a parameter. This scene does not contain a universal reflection or global-illumination engine: it combines a spotlight shadow map, authored fill, selected physical materials and a cached room cubemap (src/scene.js:39–47, src/lighting.js:82–110).

The current official shadow guide, checked 2026-11-07, explains the basic cost: shadow-casting lights render casters from their own viewpoint before the main view. Its examples are recommendations, not measurements of this application. In particular, adding shadow-casting point lights to every glowing companion would introduce extra scene views rather than simply making their existing glow more accurate.

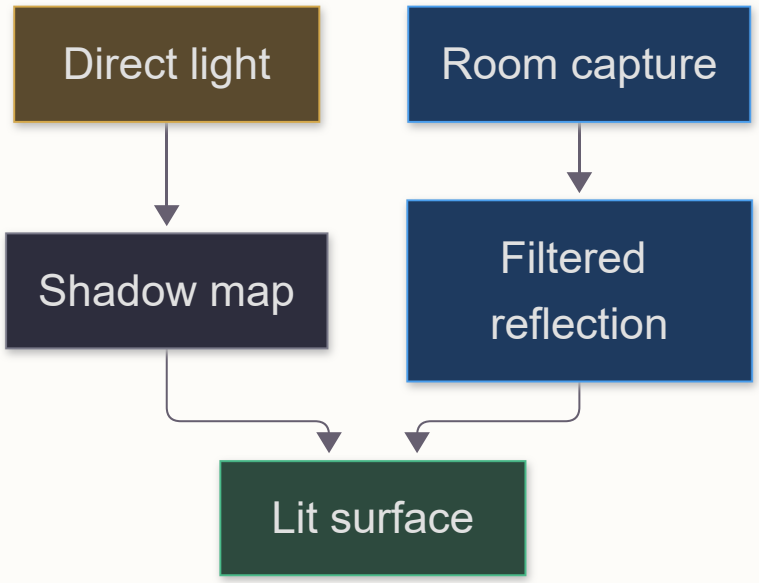


Figure MD30.1. A surface combines different evidence paths; a cubemap highlight is not a shadow and neither is a screen filter.

Constructing the shadow path

The renderer enables shadows, selects `PCFShadowMap` and disables automatic shadow updates. The tripod spotlight casts the mapped shadow. Its map is 1024 square, near plane `.7`, far plane 120, bias `-.00012`, normal bias `.008` and radius 16. These are authored settings, not guarantees of a particular penumbra width on every GPU. Preserve them as a baseline before testing an alternative shadow technique (src/lighting.js:82–92).

The lamp mesh helper initially makes objects both casters and receivers, but shade, rims and diffuser override casting to false. The tabletop casts and receives; wall and floor receive. This asymmetry is purposeful. An opaque shadow-casting thin drum could block the very light used to suggest transmitted fabric. It would also make an illumination approximation look like a modeling hole. Shadow flags therefore belong to the material-and-light contract, not merely to a mesh-construction convenience.

The main render path marks shadows dirty when scene change, active pose change or button motion requires it. Reflection capture also requests shadow updates before and after its temporary renderer state change. Disabling automatic updates is safe only while these invalidation paths remain complete. A new animated prop that never sets `changed` can leave a stale shadow even if its own transform is correct (src/scene.js:1367–1386).

Pear additionally has a stylized contact plane. The scene keeps it at world Y `.012` by subtracting body height from its local Y, fades opacity no lower than `.12`, and enlarges its scale with height. Picking explicitly ignores it. Treat this as a visual attachment, not a physical obstacle or an accurate soft-shadow solution. The room's mapped shadow and the character's contact cue serve different jobs (src/scene.js:547–553, src/scene.js:894–899).

Building the reflected environment

The constructor allocates a 128-square half-float cube render target, a cube camera with near `.1` and far 1500, and a `PMREMGenerator`. It traverses the DS and stores its physical materials in a Set. That Set avoids duplicate assignment when meshes share a material; it does not mean every glossy object in the room receives this environment map (src/scene.js:161–167).

`updateDsReflections` records console visibility and renderer tone mapping, hides both consoles, moves the capture camera to the DS root, and captures with `NoToneMapping`. It then filters the cube texture, assigns the filtered texture to collected physical materials, disposes the previous filtered target, and clears the dirty flag. A `finally` block restores visibility and tone mapping even if capture or filtering throws (src/scene.js:1077–1100).

Hiding both consoles prevents self-capture and the other console from appearing in this approximation. It also limits fidelity: this is not a full mutually reflective scene. A single capture point cannot reproduce parallax-correct nearby reflections across the whole clamshell. Describe

the result as an actual-room sampled environment with explicit omissions, not as perfect mirror physics. Physical material roughness can blur that environment but cannot supply missing spatial information.

The current official MeshPhysicalMaterial reference supports understanding clearcoat and roughness as distinct surface layers and parameters. It now contains properties and deprecations newer than the pinned runtime. Consult it for concepts only; do not silently add a current property to a reconstruction of revision `1e4ef518`.

Cache policy is part of appearance

Room capture waits until there is no transition, inspection drag, pinch, companion move or console move, both companions are sleeping, and consoles are not moving. Light switching paints direct illumination first, then permits reflection refresh on a later settled paint. This is a latency tradeoff: a temporarily stale reflection is preferable to making every direct-light switch wait for six extra views and filtering.

Switching tripod lights changes intensity instead of removing light objects. Keeping shader light counts stable reduces compilation churn. This is a concrete implementation choice, not a blanket promise that material changes never compile shaders. The browser fixture compares program counts and requires a direct-light paint while reflections are still pending; it is explicitly CI-parked in the pinned source (tests/browser/scene.spec.mjs:150–199).

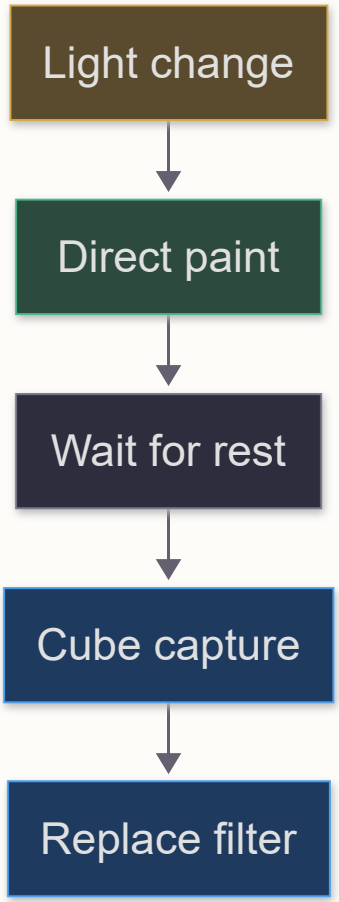


Figure MD30.2. Reflection quality is scheduled behind responsiveness; the first switched frame intentionally uses the previous environment.

Shader selection is not the canvas CRT pipeline

The word shader is particularly easy to overextend here. Renderer materials compile GPU programs; preferences expose validated native CRT choices; generated status pictures receive a canvas overlay. `drawLcdCrt` uses ordinary 2D composite operations and a cached pattern. It is not a custom imported GLSL file secretly responsible for the DS clearcoat or room shadow map (src/lcd-crt.js:1–39).

When diagnosing a dark edge, first reproduce it with a static status image and fixed camera. If it remains painted into that image independent of room state, inspect the vignette and picture generation path. If it moves with viewing direction on inactive glass, inspect physical material and environment. If another object changes it by blocking the lamp, inspect the shadow path. These experiments isolate causes without rewriting three pipelines at once.

For a proposed higher-quality reflection mode, state the budget before implementation. Doubling cube-face side length quadruples texel count per face, and there are six faces before filtering. A 128-square cube contains 98,304 base face texels; 256-square contains 393,216. Those counts exclude mip levels, attachment formats, filtering intermediates and implementation overhead. They are arithmetic, not measured VRAM consumption. Any claim of actual memory use or

timing needs profiling on the target renderer. Preserve current responsiveness and settled-refresh behavior while evaluating the new quality mode, and make any changed resolution contract explicit rather than silently altering the baseline.

MD30-W01 Trace a light switch through two eligible paints

Assume the scene has settled with no outstanding fence and the tripod is on. The user switches it off. First, `setLamp(false)` stores/restores known intensities, sets all four to zero, disables shade and bulb emission, and sets `changed`, `lightingChanged` and `dsReflectionsDirty`.

On the next eligible render, `lightingChanged` allows recovery bypass but not outstanding-fence bypass. Reflection capture is excluded because `lightingChanged` is true. The main view paints the switched direct lighting, resets that flag and inserts a GPU fence. Once the fence and any severe-load recovery permit another frame, a settled scene may capture and filter the new room environment. No numeric duration follows from these steps: eligibility depends on actual GPU completion.

This explicit pseudocode is a teaching reduction, not replacement application code. It assumes capture succeeds and retains the ordering needed for a responsive switch.

```
on switch:
  directDirty = true
  environmentDirty = true
on eligible paint:
  if environmentDirty and not directDirty and settled:
    capture and replace environment
  render main view
  directDirty = false
  insert completion fence
```

Expected outcome is a direct-light frame before environment replacement. Record frame order, flags and actual sampled pixels; a checkbox changing alone does not demonstrate the effect.

MD30-R01 Retry after a moving companion changes reflections

Change the condition: the axolotl is dragged while its light is on. Do not capture a fresh environment on every pointer event. Let movement dirty the main view and retain deferred reflection refresh until the object sleeps. If the DS seems to reflect the old position during movement, that can be the intentional cache policy rather than failed light propagation.

After landing, verify that dirty state eventually clears and that the previous filtered target is disposed. If it never clears, inspect held/moving flags and pending presentation acknowledgments before increasing capture frequency. If memory grows across repeated replacements, inspect

target ownership rather than deleting arbitrary scene textures. The checked official cleanup guide reinforces that render resources require explicit release; material disappearance from a scene is not disposal.

MD30-Q01 Exercise: can clearcoat fix missing shadows?

Explained answer

A proposed change increases clearcoat until the DS has a brighter edge, claiming it restored lamp shadow detail. Explain the category error and propose independent observations for reflection, direct occlusion and cached-state correctness.

MD30-A01 Explained answer

Back to question

Clearcoat changes the reflective response of a material; it cannot create a missing caster or update an old shadow map. Check caster, receiver, light and renderer flags for the shadow path. Then move a known light or object and inspect whether shadow invalidation occurs. For reflection, inspect the assigned environment texture, capture origin and settled refresh conditions separately. For cached state, compare dirty flags before and after an eligible paint, not after an arbitrary animation callback.

Keep the evidence limitation visible. CI parking retains the original switch assertions but does not prove they currently pass; the verification record calls such exclusions coverage debt (docs/VERIFICATION.md:93–107). An appealing reflection and a passed structural assertion still require separate human aesthetic review.

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD31

Screen materials and pixel preservation

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Screen materials and pixel preservation

A screen has several independent representations

The physical LCD is geometry, its picture is a texture, its powered-off appearance is a reflective material, and its contribution to room light is a separate emitter. Combining those into one vaguely emissive glass material would make room exposure alter game colors and would confuse touch mapping with a decorative overlay. Reconstruction begins by preserving these responsibilities rather than choosing a shader that looks screen-like (src/scene.js:1106–1142).

The library is exactly Three.js 0.186.1 (package.json:7–9). The checked current color-management manual supports the distinction between color textures, linear lighting and sRGB output, but does not establish historical runtime defaults. Keep the source's explicit color-space annotations even if a newer example appears to work without them.

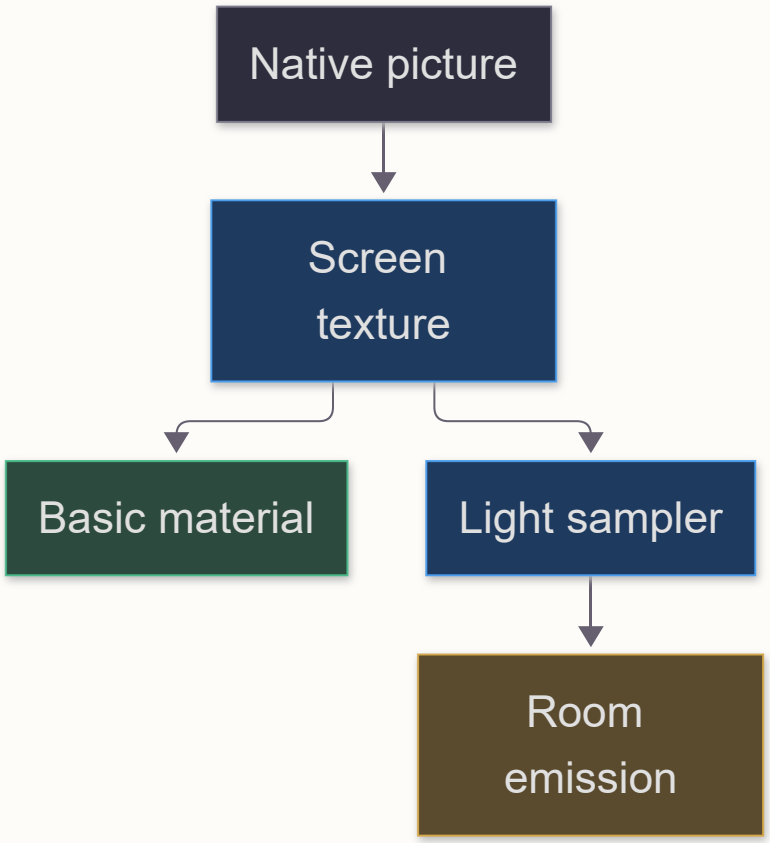


Figure MD31.1. Picture display and room illumination share an image source but not a material or exposure path.

Active display versus inactive glass

When the GBA powers off, the scene clears its source and screen illumination, then installs `MeshPhysicalMaterial` with color `#101713`, roughness `.24`, clearcoat one and clearcoat roughness `.22`. That is deliberately reflective dark glass. When a live canvas is attached, `setScreen` creates an sRGB `CanvasTexture`, sets both filters to `LinearFilter`, disables mipmaps and uses `MeshBasicMaterial` with `toneMapped:false`. These assignments mean the active picture does not acquire the room's tone mapping or direct-light response.

The DS follows the same active/inactive distinction but not the same live-texture filtering. Its powered-off path uses reflective physical material. Status screens use basic non-tone-mapped materials. When the retained native framebuffer starts, the controller creates two 256-by-192 canvas textures with both filters set to **NearestFilter**, no mipmaps, and non-tone-mapped basic materials (src/ds-controller.js:266–294, src/ds-controller.js:415–435).

This difference matters. “All active LCDs use nearest” would be false for the pinned GBA path. Nearest is the exact DS native pixel-preservation contract; linear filtering is an explicit GBA choice for a supplied canvas that can already contain filtered content. Neither implies a one-to-one mapping between source pixels and final device pixels at every camera angle. Perspective projection, canvas density and screen footprint remain involved.

The controller verifies the native canvas layout and requires a retained drawing buffer before accepting it. `paintScreens` copies each region into its output canvas with `imageSmoothingEnabled=false`, then marks the corresponding texture dirty. These copies occur even when the room-render budget defers another 3D paint (src/ds-controller.js:662–675). Do not describe texture dirtiness as proof that the main view has already presented the new picture.

Curved outlines without a second framebuffer warp

`createLcdGeometries` gives the GBA viewport width 4.05 and height `width*2/3 = 2.7`. The curved option builds a shape outline; the flat option builds a plane. For curved geometry, UVs are explicitly set from XY position divided by viewport dimensions plus one half. This preserves a direct image mapping inside a curved boundary instead of warping an already-filtered framebuffer again (src/handheld.js:49–65).

The recess is regenerated with the screen, and old geometries are disposed during a curve change. Geometry owns silhouette; the emulator or built-in drawing path owns the picture. Moving the outline vertices and then adding another barrel distortion to UVs would create two unrelated distortions. That might be a proposed artistic experiment, but it is not the implemented reconstruction.

Pixel preservation also requires preserving the original source. Changing world exposure to rescue dark shell paint should leave active LCD RGB unchanged. A fixture that samples powered-off glass cannot assert that rule because the powered-off material is intentionally light-responsive. The

upright DS lighting test therefore installs real status LCDs before comparing light states, and checks material type as well as RGB (tests/browser/ds.spec.mjs:1421–1484).

Built-in CRT is a canvas effect, not an imported model

`drawLcdCrt` uses a WeakMap keyed by the destination canvas. It caches a three-column mask tile, a repeated pattern and a radial vignette. Cache identity includes canvas width, height and row count. It saves context state, resets the transform, multiplies the mask over the picture, overlays the vignette and restores state. The effect belongs to generated startup/status pictures, not to a new modeled CRT tube or a universal postprocessor (src/lcd-crt.js:1–39).

Each tile column has a slightly different near-white tint. The last row gets a translucent black scanline. The radial gradient stays transparent until its outer region and darkens the edge. These are restrained overlays; no random noise or frame-time-dependent mask is generated. A WeakMap avoids keeping an otherwise unreachable canvas alive solely because its cached pattern exists, although GPU textures still require their separate disposal.

Preferences list supported native CRT presets and allow `disabled`; validation rejects unrecognized values. That list does not mean all presets run through `drawLcdCrt`. Distinguish native preset selection from the built-in canvas treatment (src/preferences.js:1–13, src/preferences.js:67–90).

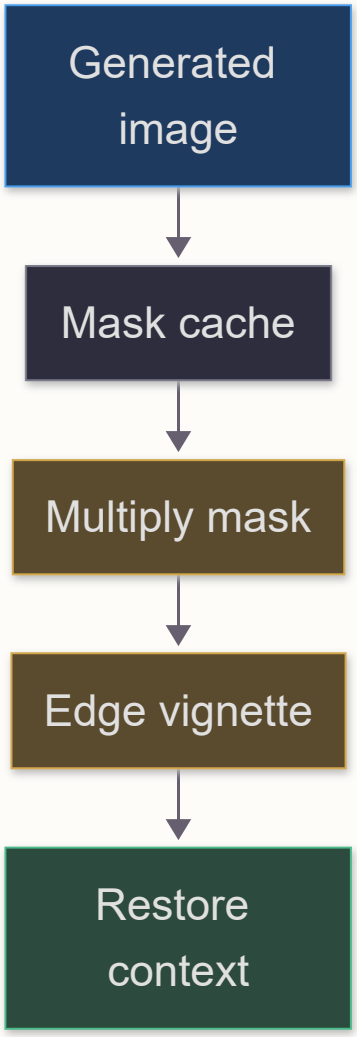


Figure MD31.2. The built-in treatment overlays a generated canvas and returns its drawing state to the caller.

A practical pixel-preservation audit

Follow the image through its boundaries. At the native boundary, record canvas dimensions, retained-buffer eligibility and region layout. At the 2D boundary, record source rectangle, destination rectangle and smoothing state. At the texture boundary, inspect color space, filters and mipmap generation. At the material boundary, inspect basic versus physical and tone-mapping state. At the final view boundary, record camera, projected viewport size and renderer pixel ratio. Each stage can be correct while another introduces blur or color change.

Use a chart with alternating single-source-pixel columns to expose resampling. At exact integer enlargement, nearest should retain hard repeated blocks; linear should interpolate transitions. At arbitrary perspective tilt, the chart may alias under nearest or blur under linear. That is a sampling tradeoff, not evidence that the native framebuffer was corrupted. Keep a direct native or canvas sample beside the projected-view observation when isolating the cause.

Do not assume disabling mipmaps is universally best for every scene texture. It is the explicit LCD choice here. The wood and fabric maps have different visual frequencies, screen footprints and lifecycle. Copying an LCD texture policy to all room materials could worsen distant texture aliasing. Likewise, anisotropy on the general canvas helper does not supersede the explicitly assigned DS nearest filters.

For memory ownership, each newly attached screen map replaces an older one through `setDisplay`, which disposes the prior map and material. A debugging chart that is injected outside that path needs its own cleanup. Removing its DOM canvas does not release a corresponding GPU texture automatically. Keep that distinction in a reconstruction checklist rather than treating debug-only resources as cost-free.

A useful negative control is a solid black active picture. Its basic material should remain black rather than acquiring a reflected lamp highlight, while the separate sampler should converge toward zero emission. Powered-off glass can still show the room. Comparing these two superficially similar dark screens exposes accidental material reuse. A second control, a constant white chart, separates exposure invariance from source animation. Neither requires private gameplay content. Use public generated charts first, then qualify native behavior separately where execution is authorized. This keeps a failed picture pipeline from being misdiagnosed through a moving, uncontrolled emulator scene.

MD31-W01 Work out the cached scanline tile

Use the GBA status size 720 by 480 and default 160 rows. `round(480/160)` is three, so the mask tile is three by three. At DS status size 512 by 384 with 192 rows, the tile height is two. Its three columns repeat across width regardless of whether the width divides evenly by three; the pattern is clipped by the destination fill rectangle.

The following original browser-console recipe assumes same-origin served modules. It draws a generated chart, not emulator content, and deliberately applies the effect only once. Expected outcome is a chart with a fine repeating mask and darker outer corners. The assertion checks caller state, not aesthetic quality.

```
const { drawLcdCrt } = await import('./src/lcd-crt.js');
const canvas = document.createElement('canvas');
canvas.width = 720;
canvas.height = 480;
const ctx = canvas.getContext('2d');
ctx.fillStyle = '#808080';
ctx.fillRect(0, 0, 720, 480);
ctx.globalAlpha = .7;
drawLcdCrt(ctx, 160);
console.assert(ctx.globalAlpha === .7);
document.body.append(canvas);
```

Do not call this repeatedly on the same unchanged image without redrawing its base. Multiplication accumulates darkness. The cache saves pattern construction; it does not make the overlay idempotent. That is an important distinction between caching an effect's resources and caching an effect's final output.

MD31-R01 Retry after doubling the destination height

Change the chart height to 960 while keeping 160 rows. The cached tile must be rebuilt because its height becomes six and the vignette radius changes. If an extension caches only by canvas identity, it will retain the old scale and produce denser-looking lines after resize. Preserve the width/height/rows key, not just the WeakMap container.

If DS native pixels look softened, inspect both stages before raising texture resolution: the 2D copy must disable smoothing, and texture minification/magnification must remain nearest. Then inspect actual projected footprint. A subpixel-wide source feature cannot be universally preserved after arbitrary perspective minification. No renderer flag establishes that impossible guarantee.

MD31-Q01 Exercise: fix a false LCD-light regression

Explained answer

A test powers off the DS, toggles room light, and fails because LCD center RGB changes. It proposes disabling tone mapping on every material in the scene. Explain the wrong premise and design two narrower tests.

MD31-A01 Explained answer

Back to question

Powered-off glass is physical and reflective, so its appearance should respond to surroundings. Test that state for reflective material properties and sensible room dependence. Separately install an active status or native display and require a basic non-tone-mapped material plus unchanged picture samples across lighting states. Disable both companions when claiming LCD-only illumination; otherwise their light and exposure branch remain uncontrolled.

Keep any native-input test independent from this static color fixture. A generated status chart can prove material isolation but cannot prove that the emulator advanced or received touch. The public verification document explicitly distinguishes technical evidence from human acceptance and from deployment (docs/VERIFICATION.md:1–8).

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD32

Picking, occlusion and input ownership

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Picking, occlusion and input ownership

Visibility does not automatically grant input

An interactive procedural model needs more than triangles under the cursor. It needs a rule for which surface owns a gesture, when that ownership ends, and how an action reaches its intended subsystem. The same canvas supports table movement, inspection, lamp switching and DS stylus interactions. Those actions cannot all respond to the same pointerdown. `HandheldScene` therefore dispatches by mode, transition state, hit metadata and current movement ownership (src/scene.js:180–231).

Begin with a state table rather than adding event handlers until the desired click works. In table mode a console click can become movement or play selection; in inspection it rotates the active body or edits a hinge; in DS play a lower-screen touch can become native touch. A modal dialog, transition or held stylus changes what is available. These are behavioral ownership boundaries, not incidental UI details.

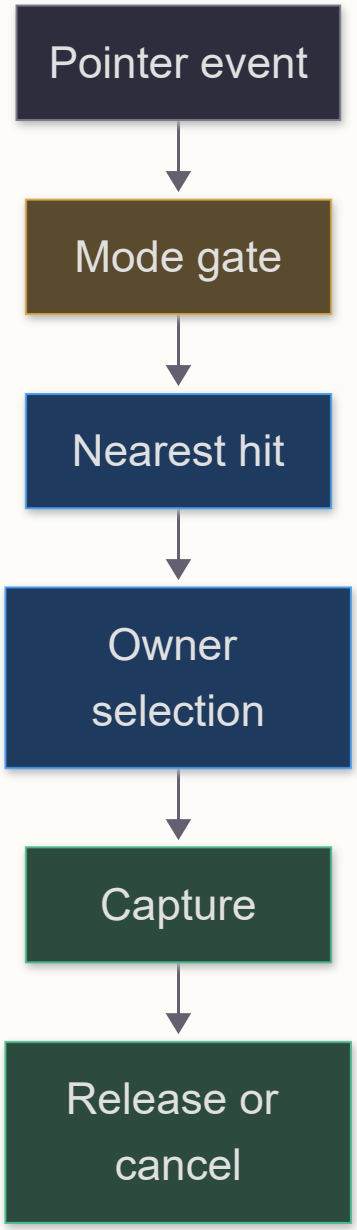


Figure MD32.1. A successful pick is only the middle of an interaction; gates and cleanup make ownership reliable.

Converting browser coordinates into a world ray

`pick` gets the scene canvas's CSS bounding rectangle. It converts client coordinates into normalized device coordinates by subtracting the rectangle origin, dividing by its CSS size, mapping X into minus one through one, and flipping Y. `Raycaster.setFromCamera` then produces the world ray. It intersects GBA, DS, tripod, Pear and axolotl roots recursively and uses the first permitted hit (src/scene.js:547–561).

Do not divide client coordinates by the drawing buffer dimensions. A device-pixel ratio of two doubles the buffer but does not double CSS pointer coordinates. Conversely, framebuffer reads need buffer coordinates. Many apparently small picking offsets come from mixing these spaces, especially after moving a canvas away from the viewport origin.

Two exceptions refine nearest-hit selection. Pear's contact plane is ignored because it is a visual cue rather than a selectable object. A docked stylus marked `dsStylus` is ignored while the stylus is held, preventing its inactive original geometry from stealing input. These exclusions should remain specific. A general rule that ignores all transparent meshes could break legitimate screen or decorative-surface interactions.

DS native touch has a stricter ray test. It intersects the complete DS model and accepts only when the first permitted hit is the lower screen itself. It does not raycast directly against the LCD and accept a point through an intervening lid. The resulting UV is converted by `dsTouchFromUv`, then sent with a pressed or cancelled state to the retained iframe (src/ds-controller.js:490–508).

Dragging owns a plane, offset and pointer identity

A console movement record stores system, original position, pointer ID, initial CSS coordinates, a camera-facing plane and grab offset. The plane passes through the picked point, so dragging follows a constant-depth surface initially. Subtracting the ray-plane point from root position preserves where the user grabbed the object; without it, picking a corner snaps the root under the cursor (src/scene.js:635–654).

Movement becomes a drag only after more than four CSS pixels of pointer travel. Wheel depth is separately treated as movement. Its displacement follows camera forward, clamped to two world units per event. When a move is accepted, the interaction plane is translated by the actual accepted displacement, not the requested displacement. This matters at workspace edges: using the unclamped request would leave the next pointer event intersecting a plane the object never reached (src/scene.js:233–246, src/scene.js:295–323).

Pointer capture keeps the original owner receiving subsequent events outside the element. Release, cancellation and lost capture have different meanings. A normal release after motion drops with bounded inertia; cancellation restores the original position; a stationary console release activates play. Blur, resize, view change and console selection also cancel owned movements. A robust extension must cover these exits before claiming a new drag interaction is complete.

Keyboard movement uses `.65`-unit steps along X, Y and Z. It clears velocity after each accepted step because deliberate keyboard placement should not imitate a fast pointer throw. Enter drops and Escape cancels. Inspection has different arrow-key semantics, so movement ownership is checked first. The project architecture documents that positions are session state, not reload-persisted placements (docs/ARCHITECTURE.md:43–68).

Projected controls need their own occlusion contract

DS control DOM elements project from physical anchors. The controller first rejects wrong mode, transitions, back-facing normals and offscreen positions. It then raycasts the lid and hides a control if the lid is nearer than the anchor distance minus `.08`. Only visible targets receive projected CSS position and scaled size, with a minimum 44-pixel power target and 14-pixel ordinary target (src/ds-controller.js:677–700).

The stylus button uses a separate obstruction check and margin `.16`. Do not copy one tolerance blindly into the other: their anchors and target geometry differ. Likewise, these DOM checks are not a universal scene-wide occlusion solver. They implement the specific DS lid and stylus ownership requirements of the current application.

Projection cannot wait for the next eligible GPU paint. `updateViewMatrices` updates camera and active-model matrices and projects controls immediately after pose/camera changes. Otherwise a deferred room frame could leave current DOM targets paired with stale touch rays (src/scene.js:484–500).

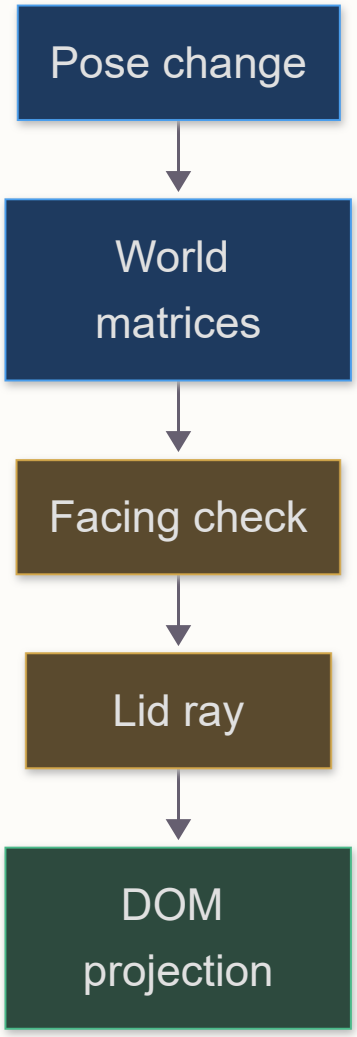


Figure MD32.2. Input geometry becomes current before GPU scheduling decides whether another room image can be submitted.

Native messaging has session ownership too

`touch` requires both a frame and a running controller. Its message includes the touch payload and current session and is posted to `location.origin`. A visually correct stylus therefore does not prove a native touch was accepted: the controller must still have a valid retained target. Keep session identity in any diagnostic log, without publishing unrelated private user data (src/ds-controller.js:498–508).

A complete regression observes at least three stages. First, the ray produces a legal lower-LCD UV under current geometry. Second, the controller sends press and release for the current session. Third, a native response demonstrates effect where the fixture supports it. A moving physical button cap proves neither the second nor the third stage. Conversely, an iframe accepting input does not establish that its hit target matches the visible handheld.

For accessibility, preserve the distinction between target size and physical size. The power control's minimum 44 CSS pixels is an input affordance; it does not enlarge the modeled button cap or change native geometry. A projected target can be larger than its visible cap while still belonging to the same anchor and occlusion rule. Test pointer hit area and physical travel independently. If the target remains active through a closed lid, a generous size is not an excuse: visibility gating must hide it before sizing. This is a practical reason to keep projection checks in their current order.

This distinction also guides cancellation tests. Hold a pointer, change view or open tools, and inspect whether all owned sources release. Repeat with keyboard and touch ownership separately; one synthetic release cannot stand in for every source. Then resume the retained session and confirm no old hold remains. These are proposed diagnostic steps derived from the controller's cleanup contract, not claims of newly executed browser results. They make the boundary reusable for a future model without assuming that another console shares the DS's exact message schema or stylus availability rules.

MD32-W01 Convert a pointer and preserve its grab

Assume the canvas CSS rectangle is left 100, top 50, width 800, height 600. A pointer at `(500,200)` produces X `(400/800)*2-1 = 0` and Y `-(150/600)*2+1 = .5`. Those coordinates stay the same at pixel ratio one or two. If the ray intersects the movement plane at world `(3,4,2)` while the object's root is `(5,4,1)`, the stored offset is `(2,0,-1)`. A later intersection `(4,5,2)` requests root `(6,5,1)`, preserving the original grab.

This runnable Node recipe checks the coordinate arithmetic only; ray generation still requires camera matrices and Three.js. It is intentionally small enough to print beside a diagram.

```
const rect = { left: 100, top: 50, width: 800, height: 600 };
const event = { clientX: 500, clientY: 200 };
const x = (event.clientX - rect.left) / rect.width * 2 - 1;
const y = -(event.clientY - rect.top) / rect.height * 2 + 1;
console.assert(x === 0 && y === .5);
const offset = [2, 0, -1];
const nextHit = [4, 5, 2];
console.log(nextHit.map((v, i) => v + offset[i]));
// [6, 5, 1]
```

Expected behavior after a rejected placement is different: the model stays in its accepted position, and no fabricated successful displacement is recorded. Keep intent, acceptance and presentation as separate observations.

MD32-R01 Retry with a closed lid and deferred room paint

Change the condition from a flat, unobstructed DS to a lid covering the lower LCD while GPU recovery is active. Update pose matrices immediately. The lower-screen-only raycast shortcut would still return UVs, but the complete-model nearest-hit rule should reject touch through the lid. DOM controls obscured by the lid should likewise disappear.

If the visible image is one frame old, that does not justify sending input through current occluding geometry. Test current projection and ray ownership independently, then verify the next eligible painted pose. Do not remove recovery or fake future timestamps simply to make the test click its old target. The DS browser coverage explicitly includes inaccessible controls and lower-screen rejection in an occluded pose (tests/browser/ds.spec.mjs:250–268).

MD32-Q01 Exercise: explain a stuck native press

Explained answer

A new tool dialog opens during a held DS pointer action. The dialog looks correct, but gameplay continues receiving the old hold. Why is visual correctness insufficient, and which ownership cleanup should the tool reuse?

MD32-A01 Explained answer

Back to question

Opening a dialog changes who can receive the eventual release. The existing controller's `openTools` first calls `releaseInputs`: keyboard release, cancelled touch, stylus return, release of recorded pointer sources, and map clearing. A tool that only changes visibility bypasses that contract (src/ds-controller.js:265–266, src/ds-controller.js:509–514).

Reuse the owner-level cleanup rather than synthesizing one arbitrary keyup. Preferences separately validate unique bindings and reserve browser-sensitive keys; valid bindings do not remove the need for release semantics (src/preferences.js:63–90). A complete interaction test checks press, ownership transfer, release/cancellation and absence of stale state, not just whether a dialog appeared.

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD33

Render loops, GPU budgets and lifecycle

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Render loops, GPU budgets and lifecycle

A callback is not a completed picture

The browser can call JavaScript again before previously submitted graphics work finishes. A modeling application that treats every animation callback as a completed picture can queue expensive room renders faster than the GPU drains them. It can also integrate an entire fall between visible frames. This project separates callback time, render submission, completion fences and presentation-aware motion (src/scene.js:1167–1217).

The main renderer uses antialiasing and caps device-pixel ratio at two. It does not dynamically lower sharpness after a slow frame. Root dependencies pin Three.js 0.186.1 (package.json:7–9). Budgeting here primarily changes when another room image may be submitted, not how many source pixels are discarded.

The checked MDN WebGL best-practices page recommends avoiding blocking calls in production, flushing when awaiting fence results, and budgeting memory conservatively. Those are current recommendations consulted 2026-11-07. They explain the design, but do not constitute a benchmark of this room or proof of universal browser behavior.

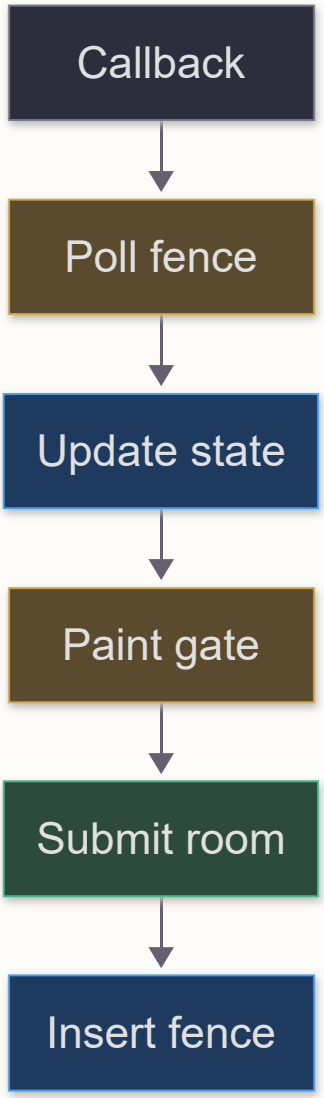


Figure MD33.1. The render loop updates more often than it necessarily submits room work; a fence protects the queue rather than creating another animation clock.

Constructing the scheduling state

The constructor checks whether `fenceSync`, `clientWaitSync`, `deleteSync` and `flush` are available. Following a main room render, `fenceRoomPaint` inserts a completion fence and flushes. Later callbacks poll it with zero timeout. A timeout retains the outstanding fence and withholds another room submission; failure reports an explicit synchronization error. A signaled fence allows deletion and scheduling of the next eligible paint.

This proves completion of submitted GPU commands before the fence, not that a human saw a frame at a particular display scanout. It is nevertheless stronger evidence than “one rAF elapsed.” Startup uses this distinction: `waitForFirstPaint` waits for scene changes and reflections to settle, then fences the completed first frame when supported (src/scene.js:413–449).

Recovery depends on observed completion delay. For a fence observed no later than 100 milliseconds after submission, no extra idle interval is added. Beyond that threshold, delay is twice observed elapsed time, capped at 1000 milliseconds for settled playing and 200 otherwise. “Otherwise” includes transitions and cartridge insertion. The no-fence fallback has its own earlier-callback heuristic and is not identical to the fence path.

`paintDue` also requires no failed synchronization, no native busy operation and no open dialog. A lighting change bypasses the recovery timestamp, but not an outstanding fence or those other gates. This precise exception lets direct switching feel responsive without submitting work on top of an unfinished room frame.

Different loops have different owners

The scene still updates body motion, gaze, limbs and control state before returning at the room gate. DS `paintStylus` and `paintScreens` also run before that return. The latter copies retained native framebuffer regions into output canvases; it is not the expensive full 3D room render. Moving the copy behind the gate would couple native LCD freshness to room recovery again (src/scene.js:1303–1342, src/ds-controller.js:662–675).

At an eligible paint, screen illumination samples use elapsed time since the last paint. Clearance resolves before actual rendering. An idle guard then checks sources, dirty flags, active pose, button movement, adaptation and illumination settling. If nothing requires a new picture, it returns without waking a settled room. A new component must dirty the scene when its appearance changes and eventually stop dirtying it when settled.

After rendering, body motion acknowledges its required presentation pose; transition and cartridge elapsed values are committed to that painted step. GPU completion remains tracked by the fence. This is a pipeline with separate submission and completion contracts, not a claim that every physics acknowledgment waited for monitor presentation (src/scene.js:1372–1412).

Fullscreen screen focus suspends the room animation loop. On return it resets `lastRender`, recovery, rendered-frame state and dirtiness before restarting (src/scene.js:537–545). Context loss reports an error without claiming saves were deleted; restoration clears synchronization failure and requests a fresh paint. Failure of LCD light sampling similarly clears and parks that sampler, reports the error, and retains the game instead of retrying it every frame.

Resource lifecycle is a model contract

`setDisplay` releases the old screen map and material. `setScreenCurve` releases replaced geometries. Reflection replacement disposes the previous filtered target. Final scene disposal stops animation, removes registered global handlers, disconnects resize observation, deletes the pending fence, releases reflection resources, traverses geometry/material resources, and disposes the renderer (src/scene.js:1114–1130, src/scene.js:1430–1450).

Do not exaggerate this into a universal resource manager. The traversal handles current material maps and explicitly handled DS bump maps; it does not enumerate every possible texture property a future material might acquire. Shared leg geometry and materials can also be encountered more than once. A proposed extension should track unique owned resources with a Set and define external ownership rather than blindly disposing every referenced texture.

The current official cleanup guide explains explicit geometry, texture and material disposal, including resource tracking. Apply that principle when extending this implementation, but label a new tracker as a proposal until it is actually installed and tested.

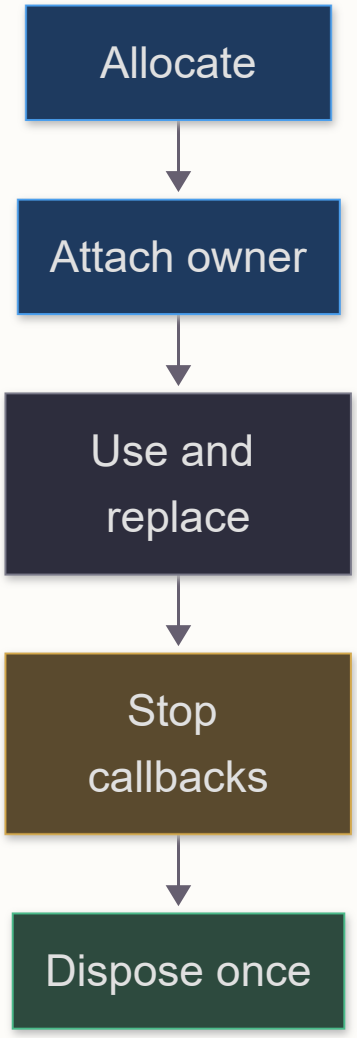


Figure MD33.2. Releasing GPU resources starts with ownership and callback shutdown; removing a mesh alone is not cleanup.

Quantify a budget without inventing performance results

Pixel count is a useful first estimate, not a benchmark. For a CSS canvas 800 by 600, pixel ratio one gives 480,000 drawing-buffer pixels; ratio two gives 1,920,000. That fourfold count helps explain why density matters without asserting that total GPU time must also quadruple.

Geometry, shaders, shadows, captures, copies and driver behavior contribute different fixed and variable costs.

Similarly, the screen sampler visits 384 pixels per sample regardless of the native picture's source size. Its readback remains a synchronization-sensitive operation, so reducing sample dimensions does not prove zero stall. The source catches readback failures and quarantines the failed emitter. A proposed optimization should measure actual sample cost and retain error visibility rather than merely removing the catch because the canvas is small.

For a trustworthy experiment, record CSS size, buffer size, device ratio, browser/GPU environment, active lights, moving objects and whether an environment capture occurred. Separate CPU callback duration from observed fence delay. The latter includes queued completion observed by a later callback; it is not automatically an isolated GPU timer query for one mesh. Also distinguish initial shader compilation from settled switching, since their costs and remedies differ.

The checked current MDN guidance suggests batching, smaller buffers when appropriate and avoiding needless synchronous queries. Here, smaller buffers would alter an explicit product constraint. Investigate redundant submission, repeated environment refresh or perpetual dirty state first. If a new quality mode later changes density, name it as a new authorized feature with its own visual acceptance and regression criteria, not as a transparent repair of the existing full-resolution path.

For lifecycle testing, repeat an owned operation rather than reading one final memory number. Replace status screens several times, toggle curve geometry, refresh reflections and dispose the scene. Track which resources should be created and released at each step. A stable memory reading alone can hide a retained object if driver allocation is pooled; a rising reading alone can include intentional caches. Pair ownership counts with renderer diagnostics and actual disposal paths. This is a recommended future harness, not a claim that a repeated-resource profile was run here. Keep the current cleanup limitations visible until such coverage exists.

MD33-W01 Work through three fence observations

Assume settled play, no transition, no insertion and a room frame submitted at time 1000 milliseconds. If its fence is observed complete at 1100, elapsed is exactly 100: the strict `>100` check is false, so `nextPaint=1100`. If observed at 1101, elapsed 101 gives recovery 202 and `nextPaint=1303`. If observed at 2000, twice elapsed would be 2000, but the settled-play cap gives 1000, so `nextPaint=3000`.

Now change only mode to a moving transition. At observation 2000, recovery caps at 200 and next paint is 2200. These are source-derived scheduling examples, not measured frame rates. The following runnable Node calculation reproduces the threshold and cap.

```
function nextPaint(submitted, observed, moving) {
  const elapsed = Math.max(0, observed - submitted);
  const limit = moving ? 200 : 1000;
  const idle = elapsed > 100
    ? Math.min(limit, elapsed * 2) : 0;
  return observed + idle;
}
console.assert(nextPaint(1000, 1100, false) === 1100);
console.assert(nextPaint(1000, 1101, false) === 1303);
console.assert(nextPaint(1000, 2000, true) === 2200);
```

MD33-R01 Retry after a long real GPU wait

Change the test condition: callbacks have been manually paused while real time advances during GPU recovery. Starting the next rest sample from stale `lastRender` can submit a synthetic timestamp earlier than the real operation's creation time. The sparse-paint regression therefore retires the prior fence, waits recovery, resets the old render baseline and starts from `performance.now()`, not from a guessed old timestamp (tests/browser/scene.spec.mjs:338–363).

The reusable fixture waits real recovery separately and rejects a caller timestamp earlier than `nextPaint`. It does not fabricate later time merely because waiting took longer. Preserve both clocks in a retry; otherwise a passing fixture can conceal a scheduling defect (tests/browser/pixel-lab.mjs:8–41).

MD33-Q01 Exercise: make a slow scene faster responsibly

Explained answer

An extension proposes lowering pixel ratio after every 100-millisecond frame and calling `gl.finish()` before each readback. Explain why neither matches the existing budget contract, and identify a safer first experiment.

MD33-A01 Explained answer

Back to question

Automatic ratio reduction violates the retained full- resolution sharpness contract. A blocking completion call can stall the main thread and native work rather than solve queue pressure. First measure which work is being submitted, whether reflections are recaptured unnecessarily, whether idle invalidation ever settles and whether real fences are respected. Change one scheduling cause while retaining resolution and assertion bounds.

The original GPU recovery browser scenario is explicitly CI-parked, so its retained assertions are a specification, not a fresh success claim (tests/browser/scene.spec.mjs:437–445). Record both performance observations and missing coverage when handing off the experiment.

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD34

Algorithms and data structures

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD34 / ALGORITHMS AND DATA STRUCTURES

Contents

Algorithms and data structures

Simplification is useful only when its domain is explicit

This desk uses custom clearance and motion algorithms, not a general collision engine. Their strength comes from a limited problem: upright or orientation-locked objects, axis-aligned bounds, a bounded workspace and a small set of obstacles. Understanding those assumptions is more reusable than copying a function name. A nearest legal translation is not the same problem as continuous collision detection during arbitrary rotation (src/desk-clearance.js:3–35, docs/ARCHITECTURE.md:43–101).

The important data structures are modest. Relative `Box3` bounds describe an object around its root. A `Vector3` stores a proposed root position. Arrays hold obstacles and staged placements. Sets deduplicate candidate coordinates and reflection materials. Small state objects preserve pointer ownership, motion velocity and pending presentation. There is no spatial tree to imply a large-world scalability claim.

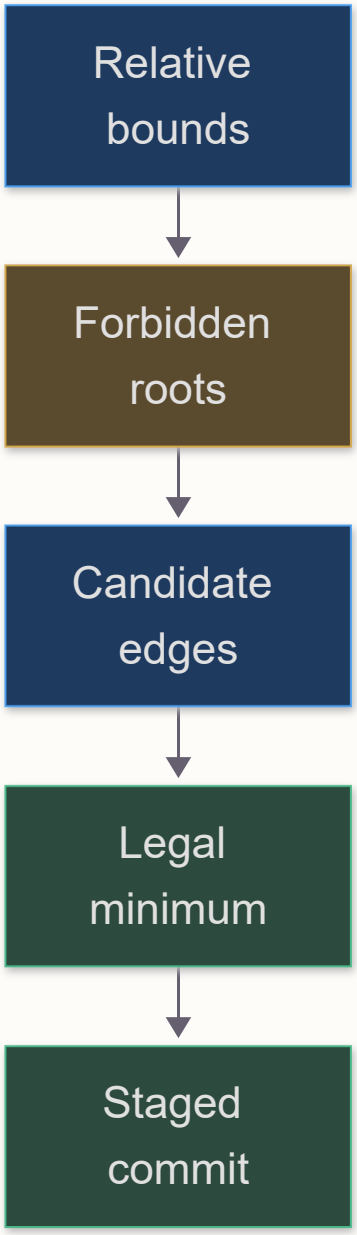


Figure MD34.1. Clearance converts geometry bounds into a finite root-position search before mutating any scene object.

Deriving forbidden root rectangles

Let a moving object's local X bounds be `bmin` and `bmax`. At root X `p`, its interval is `[p+bmin,p+bmax]`. For obstacle interval `[omin,omax]` and gap `g`, horizontal overlap occurs when `p > omin-bmax-g` and `p < omax-bmin+g`. These two limits form the forbidden root interval. Repeat for Z, after filtering out obstacles that do not overlap at the proposed fixed Y.

The asymmetric formula is essential. A console root can be off-center after rotation, so replacing its bounds with one radius would either miss a corner or unnecessarily move it. The function refuses proposed Y outside workspace limits; it does not search for an alternative height. Its

returned vector preserves Y and is independent of the input vector.

For each axis, the algorithm collects the clamped desired coordinate and both clamped edges of every forbidden rectangle in a Set. It tests their Cartesian product, rejects points in a strict forbidden interior, and minimizes squared distance to the original desired root. For this fixed-height rectangle problem, a nearest free point occurs at a desired coordinate or a boundary coordinate. Squared distance avoids a square root without changing ordering.

With `n` vertically relevant obstacles there are at most `2n+1` coordinates per axis, thus at most `(2n+1)²` candidate pairs. Each may test `n` rectangles: worst-case time is cubic, coordinate and rectangle storage linear. This bound excludes the scene traversal that first computes boxes. It is acceptable for a small authored desk, not a reason to apply the same search to thousands of actors.

Numerical stability and deterministic choice

The strict-interior test allows boundary contact, with a tolerance of eight machine epsilons multiplied by coordinate magnitude. This repairs tiny world/local round-trip errors without forgiving a visible overlap. The unit test separately accepts a one-ULP boundary reconstruction and rejects a real `.01` overlap. These are complementary obligations (tests/unit/desk-clearance.test.mjs:90–123).

A different tolerance, `1e-9` in squared-distance comparison, preserves the first candidate when distances are effectively tied. Candidate enumeration follows Set insertion order. Do not advertise complete order independence for every conceivable symmetric layout. The test proves the stated multi-blocker example is invariant under reversed obstacles, while stable ties prevent one-ULP noise from switching sides in repeated scene reconstruction.

The scene's multi-object resolver stages placement arrays before committing transforms. In table mode, companions retain priority, then loose props, then equal-ranked consoles. It tries GBA followed by DS; if that fails, it restores the staged arrays and tries DS followed by GBA. This is two bounded orderings, not all permutations and not global optimal packing (src/scene.js:796–890).

Orientation bounds are a separate algorithm

Current-orientation AABBs work for locked translation but cannot guarantee clearance across the DS transition from table pose to upright play. `dsPoseBounds` evaluates each mesh bounding box under hypothetical root and lid transforms and unions the results without changing the live pose. That supports camera framing and candidate-space reasoning for an intended endpoint (src/scene.js:502–519).

Even endpoint hypothetical boxes do not prove continuous rotational clearance. An extension promising an unobstructed whole rotation needs swept bounds or sampled intermediate poses with a justified margin. Label that as proposed work. Do not infer it from a successful upright endpoint or from the current desk translation solver.

Fixed-step motion and visible state

`DeskMotion` integrates at `1/120` second. Update elapsed is capped at five seconds, so at most 600 substeps are processed. Gravity is 45; horizontal damping per step is `.998`; support restitution is `.23` for sufficiently large incoming speed and zero for small contacts. Motion remains upright with no angular momentum, impulse exchange or articulated rigid-body solver (src/pear-motion.js:1–58, src/pear-motion.js:59–89, src/pear-motion.js:91–157).

Pointer velocity is based on a bounded `.008`–`.1` second sample and capped in magnitude at 22. Releasing after more than 100 milliseconds of stale input clears inertia. These limits prevent timing noise or a stationary hold from becoming a huge throw.

Sparse painting needs additional state. A drop can require an intermediate falling pose, an impact and a rising pose. While a required pose awaits paint acknowledgment, motion queues elapsed time instead of racing through the entire event unseen. Unit coverage checks sparse `.5`- and three-second updates, support speed, orientation locking and presentation transitions (tests/unit/pear-motion.test.mjs:1–180).

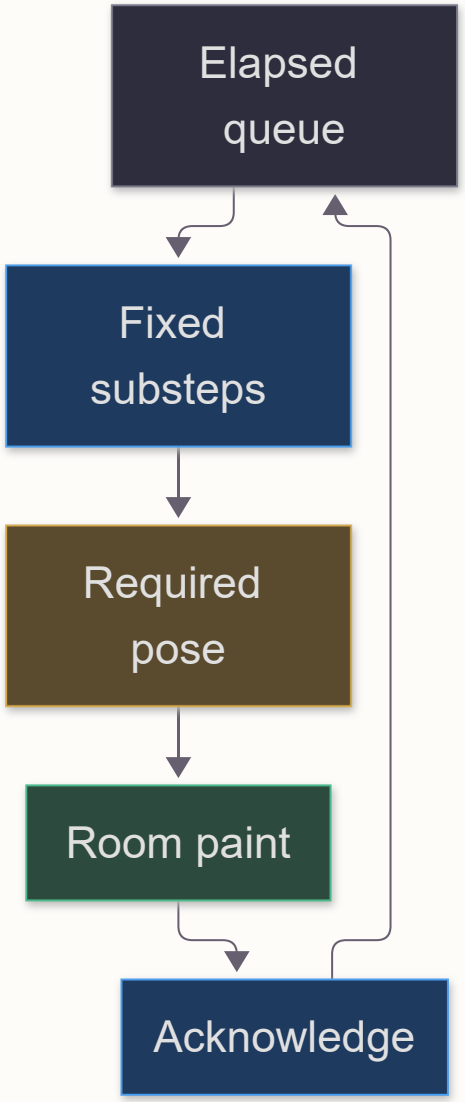


Figure MD34.2. Presentation state constrains when simulation may consume queued time; fixed steps alone cannot guarantee a visible fall.

Separate caches from authoritative state

The scene holds authoritative transforms in object roots and motion state. Its clearance signature is only a shortcut for detecting unchanged inputs, not an alternative source of truth. It is saved only after successful placement. If an extension updates the signature before attempting a layout, a failed attempt could look clean on the next callback and suppress the required retry. This is a general transaction lesson derived from the resolver's actual commit boundary.

The array of staged positions is likewise temporary. Successful commit may zero lateral velocity of displaced objects while preserving vertical velocity and waking an elevated sleeper. That detail prevents clearance repair from cancelling a real fall or leaving an unsupported object asleep. A data structure that stores only positions is therefore insufficient to describe the whole commit's behavior.

Three other small structures solve different problems. The screen sampler's lookup array trades a fixed 256-entry allocation for repeated color conversion. The CRT WeakMap allows per-canvas caches without a permanent global strong reference. The reflection material Set deduplicates shared physical materials. Choosing among Array, Set and WeakMap follows access and lifetime requirements; it is not a generic preference for one fashionable container.

When extending the scene, write down expected cardinality. For ten relevant blockers the candidate upper bound is 441 pairs, each checking at most ten rectangles. For one hundred it is 40,401 pairs and potentially over four million interior checks before geometry traversal. These are worst-case arithmetic bounds, not measured runtimes. They show where a spatial index or different packing strategy might become worth researching. Do not introduce that complexity into the small desk without evidence, and do not promise large-world scalability from the current limited test fixtures.

MD34-W01 Solve one asymmetric placement

Assume bounds X `[-1,2]`, Z `[-.5,.5]`, obstacle X `[4,6]`, Z `[0,2]`, overlapping height, and gap `.25`. Forbidden root X is `[1.75,7.25]`; forbidden root Z is `[-.75,2.75]`. Desired root `(3,1)` in XZ lies inside. Moving left to `(1.75,1)` costs squared distance `1.5625`. Moving to the nearer Z boundary `(3,-.75)` costs `3.0625`. Therefore the left edge wins if the workspace admits it and no second obstacle blocks it.

The following runnable Node recipe imports the actual solver and makes these assumptions explicit. Expected root X is 1.75, Y stays one and inputs remain unchanged. Save it as `clearance-example.mjs` at the repository root and run `node clearance-example.mjs` in an authorized scratch checkout with dependencies installed.

```
import { Box3, Vector3 } from 'three';
import { nearestDeskPosition } from './src/desk-clearance.js';
const box = (a, b) => new Box3(new Vector3(...a),
  new Vector3(...b));
const bounds = box([-1, 0, -.5], [2, 2, .5]);
const workspace = box([-10, 0, -10], [10, 10, 10]);
const obstacle = box([4, 0, 0], [6, 4, 2]);
const wanted = new Vector3(3, 1, 1);
const answer = nearestDeskPosition(
  bounds, wanted, workspace, [obstacle]);
console.assert(answer.x === 1.75 && answer.y === 1);
console.assert(wanted.x === 3);
```

MD34-R01 Retry with a second blocker and swapped order

Add a blocker covering the previous left-edge candidate. Rebuild forbidden rectangles and search all candidate pairs; do not merely push right by a fixed distance. The closest free point may combine one rectangle's X edge with another's Z edge. That is why the Cartesian product exists.

For multiple objects, a different failure occurs when the first console consumes the only legal position for the second. The scene retries the opposite console ordering before declaring failure. Keep the lamp fixed, preserve companion priority, and roll back the temporary placement arrays. Mutating the first root before success would make the retry operate on a different starting problem and could visibly jump even when both orders fail.

MD34-Q01 Exercise: add free console rotation

Explained answer

An agent keeps the current relative box and adds arbitrary rotation during dragging, claiming clearance remains exact. Identify which assumptions cease to hold and what evidence would be needed before making that claim.

MD34-A01 Explained answer

Back to question

Relative bounds now change with orientation, so a box computed before rotation no longer describes the object. Recompute transformed geometry bounds for each candidate orientation, then consider swept motion between old and new poses. A fixed endpoint AABB can be conservative there yet miss an intermediate sweep. Full free rotation also changes support, pivot and inertia assumptions in `DeskMotion`.

The existing transition helper freezes elapsed when paint is unavailable and preserves an intermediate DS desk step; it is not a continuous collision detector (src/transition-motion.js:1–8). Build explicit rotation and swept-clearance tests before expanding the solver's advertised domain.

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD35

Design principles and failure boundaries

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD35 / DESIGN PRINCIPLES AND FAILURE BOUNDARIES

Contents

Design principles and failure boundaries

Principles must explain a concrete choice

A useful modeling principle predicts what to do when a condition changes. “Keep it modular” is too weak unless it identifies ownership, dependencies and failure behavior. This application provides stronger examples: a model factory owns procedural parts; scene integration owns world placement; native controllers own emulator input; clearance stages movement before committing; and publication tools copy an explicit allowlist. These are different boundaries with different reasons to exist (src/lighting.js:9–16, src/scene.js:796–890, scripts/site-files.mjs:4–25).

Do not infer an elaborate design-pattern framework from ordinary functions. The code is direct JavaScript with Three.js objects and explicit state. Its reuse value lies in contracts, not in invented class hierarchies. A correct reconstruction can explain where a dimension is authored, who converts its space and who must invalidate a cached result when it changes.

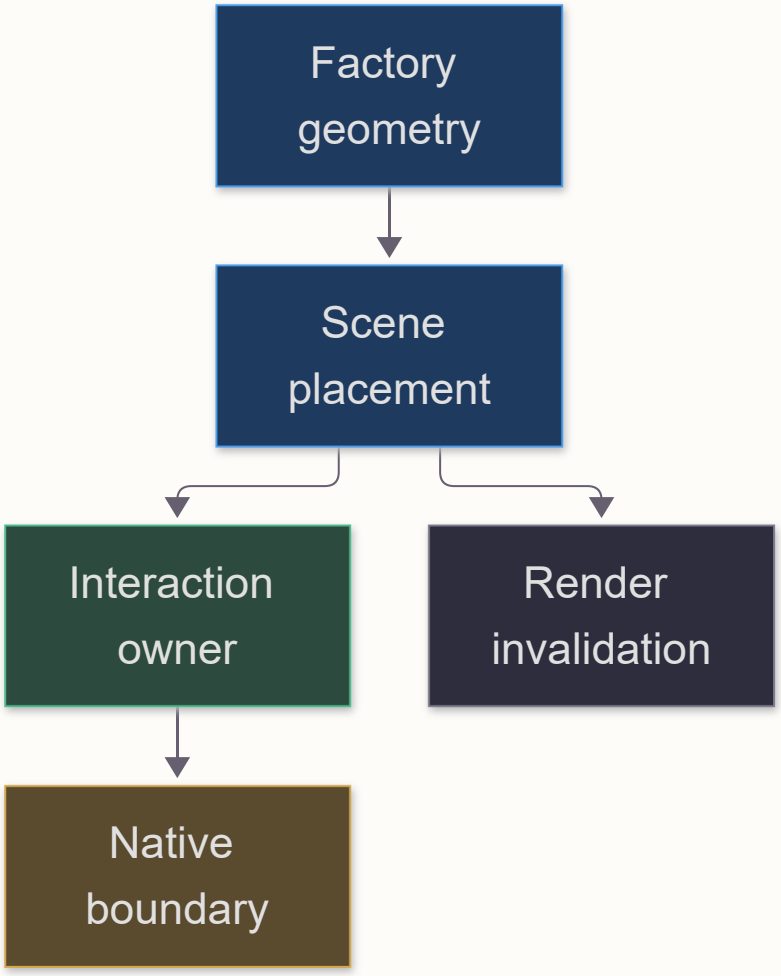


Figure MD35.1. Reusable responsibility boundaries follow actual space, input and rendering ownership, not a decorative architecture taxonomy.

Preserve invariants rather than apparent screenshots

The tabletop's upper face at world Y zero is an invariant. The lamp's body converts metric-local child lengths exactly once. Active LCD pixels use a basic non-tone-mapped material. Locked console translation preserves its orientation. Each statement is stronger than “the screenshot looks similar,” because it explains what must survive a new camera, device size or light state.

Invariants should also say what is deliberately approximate. The tabletop bounce is a finite authored rectangle, not computed global illumination. Screen spill averages a 24-by-16 image, not per-pixel indirect lighting. `DeskMotion` is an upright bounded integrator, not a general rigid-body solver. The reflection cube omits both consoles. These limitations are useful design choices when their costs and consequences remain visible (src/lighting.js:103–110, src/lighting.js:126–188, src/pear-motion.js:91–157).

The current official physical-material reference describes capabilities such as clearcoat and transmission. That does not obligate every modeled surface to use every capability. More physically elaborate materials increase cost and demand better illumination data. The correct question is which additional behavior solves a demonstrated problem, not which property makes a material sound advanced.

Make illegal transitions explicit

Clearance is transactional. The scene computes staged positions under fixed priorities, tries the two permitted console orders, and only then commits roots. If no complete layout exists, it reports an error and withholds the next paint rather than knowingly displaying an overlapping repair. This is a failure boundary: an impossible scene is not silently reclassified as acceptable.

Reset is similarly conditional. A companion may have occupied a saved console home, or another object may occupy a companion home. The application refuses that reset with an explicit reason rather than moving the fixed tripod or teleporting an unrelated object (src/scene.js:698–712, src/scene.js:768–794).

An interaction failure should preserve unrelated state. The context-loss message says the game and saves were retained and offers recovery tools. Screen-light sampling failure clears the failed source and quarantines it instead of retrying the same readback every animation callback. Neither strategy declares success; it narrows the failed subsystem while retaining a usable path forward (src/scene.js:383–391, src/scene.js:1343–1364).

Boundaries for identity and assets

Website identity is not interchangeable with hardware decoration. Architecture documentation distinguishes the site chrome, purple GBA identity and DS styling. Replacing every surface with the website logo would flatten those roles and make the modeled objects less legible (docs/ARCHITECTURE.md:11–31).

`src/brand.js` decodes four approved images and exposes cached assets only after successful decoding. Site file mapping explicitly names vendor modules and eligible top-level source/assets files; it is not a recursive copy of the repository. This matters when adding a procedural model with reference material: a private reference directory does not become public merely because the project has an assets directory (src/brand.js:6–15).

The build script clears distribution, copies that mapping, generates the two original demo artifacts and writes the Pages marker. It is a publishing preparation action, not a harmless manuscript validator. This writing task does not run it (scripts/build.mjs:7–16). Keep permission to document separate from permission to build, synchronize, commit or deploy.

Invalidating caches at the owner boundary

A cache is correct only when all changes to its inputs are represented. `drawLcdCrt` keys by canvas plus dimensions and row count. The scene's clearance signature includes relevant poses, hinge and cartridge state. Reflections have their own dirty flag because direct lighting can paint before environment capture. These caches have different lifetimes and must not be collapsed into one global “needs update” boolean.

When adding an extension, list its inputs beside the cache. A shade radius affects geometry, lamp bounds, possible emitter seams and reflections. A brightness change affects lighting and exposure but not necessarily leg geometry. A texture replacement affects resource disposal as well as appearance. This dependency exercise is more valuable than adding updates everywhere until the visible bug disappears (src/lcd-crt.js:1–27).

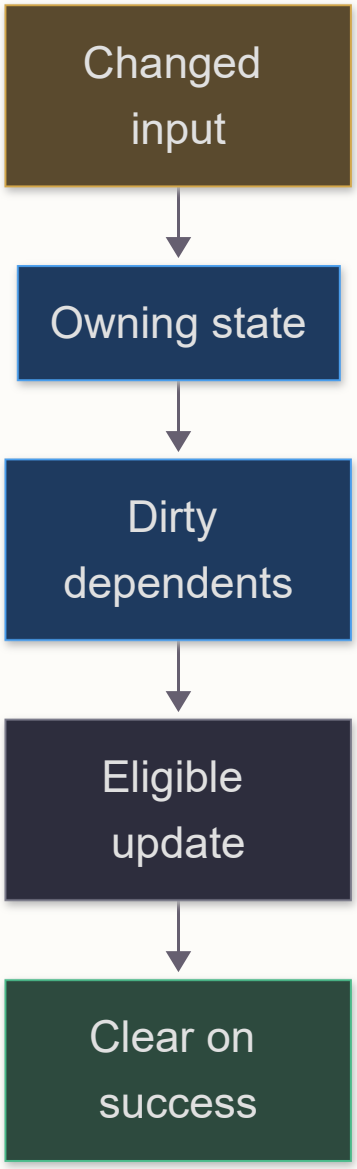


Figure MD35.2. Clear dirty state only after the dependent update succeeds; a failed refresh must remain distinguishable from a valid cache.

Prefer narrow adapters over cross-system shortcuts

The scene asks the DS controller to copy screens and project controls through explicit methods. It does not reach into native gameplay and reinterpret emulator state as room lighting state. Likewise, a screen picture becomes illumination through a sampler rather than by changing every shell material to emissive. These narrow adapters allow one subsystem to fail or run at a different cadence without silently redefining another.

The same principle applies to new models. A proposed desk clock might return a root, a display source and an interaction callback. Its scene adapter would own placement, light sampling and invalidation. It should not directly mutate the active console's camera target or private native

session. This is a hypothetical extension example, not an assertion that a clock exists in the current room.

Prefer explicit state transitions to “helpful” global repair. If a lamp switch needs fresh reflections, mark that dependency. Do not rebuild all model factories. If a new movement cannot fit, report it. Do not move the fixed environment. If a display source is invalid, surface that failure and retain unrelated native state. Such narrow responses make causal debugging possible because one user's action has a bounded set of effects.

Finally design rollback at the same abstraction level as the change. A material experiment restores material properties; a staged layout restores staged arrays before trying another order; a capture restores renderer tone mapping and object visibility in `finally`. Mixing those levels produces partial recovery: for example, restoring only the camera after a failed capture leaves both consoles invisible. The concrete restoration in `updateDsReflections` is a better teaching model than a generic instruction to “handle errors gracefully.”

Use names as diagnostic interfaces, not as magical selectors for broad behavior. The factory gives legs, shade and emitters useful names, while interaction uses explicit metadata or returned references. An extension should preserve both roles: names make ray-probe failures understandable, and references keep application logic independent of display wording. If a test deliberately depends on a name to establish which shell surface was sampled, a rename changes that evidence contract and deserves a coordinated test update. Do not silently drop the surface check because a friendlier label was introduced. Naming is a lightweight but real part of observability.

MD35-W01 Design a reversible lamp-material experiment

Propose changing shade roughness from `.95` to `.8`. First identify the owner: shade and both rims share the same material. One assignment changes all three. Record the old value, perform the proposed assignment, dirty the scene and reflection cache, and compare the same camera and light state. Do not separately mutate each rim with a cloned material unless a different finish is the explicit goal.

Expected effects are surface-response differences, not new emitted power: retain emissive intensity 60, diffuser intensity 80 and all four light coefficients. If the room becomes broadly brighter, inspect accidental light or exposure changes before attributing the result to roughness.

The following pseudocode is a proposed transaction for an isolated harness, not a new application API. It assumes a lamp factory result and a scene invalidator.

```
remember shade.material.roughness
try:
  set shared roughness to 0.8
  invalidate main view and reflected environment
  capture matching diagnostic observations
finally:
  restore remembered roughness
  invalidate the same dependents
```

A reversible harness protects the baseline and makes the changed condition explicit. The comparison still requires human review; no roughness value alone proves that the cloth reads more convincingly.

MD35-R01 Retry when the experiment changes geometry

Now change the shade diameter instead. The previous material-only invalidation list is insufficient. Rebuild the cylinder and rims, dispose replaced owned geometry, recompute lamp bounds and test nearby placement. Inspect the transmitted rectangle's position relative to fabric. Then request reflection refresh.

This retry illustrates why a generic “visual tweak” work item is poorly specified. State precisely which inputs change, which invariants remain fixed and which old tests cease to cover the new domain. Do not weaken the fixed-lamp assertion because a larger lamp makes a layout inconvenient. The test checks a product behavior, not an arbitrary obstacle to editing (tests/browser/desk-clearance.spec.mjs:59–72).

MD35-Q01 Exercise: can automation approve the redesign?

Explained answer

An agent reports valid geometry, no detected overlaps, passing color thresholds and a green deployment. It concludes that the redesigned desk has owner approval. Explain which inference fails and how to word a truthful handoff.

MD35-A01 Explained answer

Back to question

Each observation covers a separate technical layer. Geometry tests do not establish aesthetic preference; color thresholds cover chosen probes; deployment shows a revision became available. None supplies the owner's visual acceptance. Report exactly those observations and mark aesthetic approval pending.

The verification document makes this distinction explicit and treats previously parked scenarios as coverage debt, not silently passing cases (docs/VERIFICATION.md:1–8). Likewise, the GIF-first root README introduces the project while standalone books carry deep detail; new documentation should extend that navigation instead of replacing the public identity with a wall of engineering receipts (README.md:1–30).

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD36

Validation and bounded recovery

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

Contents

Validation and bounded recovery

Evidence is a ladder, not one green checkbox

A procedural model can have valid geometry and still fail when touched, lit, moved or published. Validation should therefore identify the layer being observed. Source inspection establishes construction intent; unit tests establish mathematical contracts; browser tests establish integrated behavior under particular fixtures; screenshots support visual review; deployment records identify a published artifact. Human aesthetic acceptance remains separate. The public verification record explicitly warns against collapsing those layers (docs/VERIFICATION.md:1–8).

This chapter gives reproducible validation instructions, not a claim that tests ran while this handbook was written. The pinned assertions are evidence of what the project checks and where gaps remain. A recipe without an execution receipt is a planned experiment, not a result.

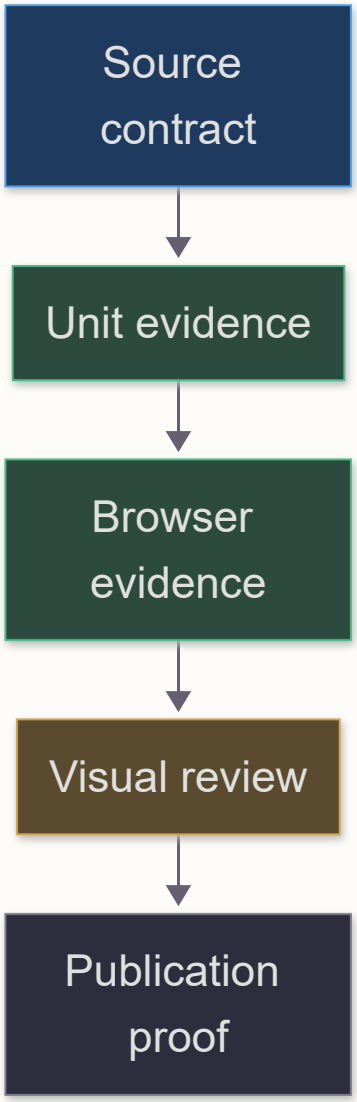


Figure MD36.1. Later evidence adds information rather than replacing earlier contracts; publication still does not imply owner approval.

Validate construction before appearance

Begin with counts, dimensions, transforms and ownership. For the tripod, verify three shared-geometry legs, an open drum, two rims, support cylinder, diffuser and the four actual light objects. Confirm that body scale is `1/.017` and root placement is not scaled twice. Those checks detect a malformed reconstruction before a pleasing light setup can conceal it (src/lighting.js:9–110).

Next validate bounds under the intended orientation. The desk solver's tests cover multi-blocker nearest placement, independent returned vectors, vertical separation, asymmetric extents, numeric gaps and one-ULP boundary stability. They do not prove free rotational collision

behavior (tests/unit/desk-clearance.test.mjs:1–123). For a DS endpoint, use hypothetical geometry bounds; for a proposed full rotation, add explicit swept coverage rather than rebranding endpoint tests.

Motion validation checks trajectories and state transitions separately. The unit test with sparse three-second updates requires falling and rising presentation states to survive until acknowledgment. It also verifies that resetting clears pending presentation and inertia (tests/unit/pear-motion.test.mjs:75–154). A final resting position alone cannot reveal an entire skipped bounce.

Browser fixtures must establish their own conditions

The upright DS lighting fixture is unusually instructive. It uses both desktop and narrow-phone dimensions, turns off Pear and axolotl, establishes a straight root, installs genuine status LCD materials and ray-checks shell probes before sampling pixels. This avoids accidentally measuring a background, reflection or occluding lid instead of paint (tests/browser/ds.spec.mjs:1412–1486).

The fixture's tripod-only blue threshold above 75 and all-off maximum below three express a bounded product expectation. They are not universal color calibration. If another renderer produces different pixels, inspect probe ownership, fixture state, exposure convergence and actual geometry first. Do not lower the threshold until the intended condition disappears.

Room-paint fixtures must honor GPU readiness. `installRoomPaintFixture` retires the real fence, waits bounded recovery, rejects timestamps before `nextPaint` and requires `renderedLastFrame`. It does not equate any callback with a picture (tests/browser/pixel-lab.mjs:8–41). Use it to distinguish state mutation from actual render submission. A later GPU fence gives stronger completion evidence, though not proof of display scanout or human observation.

One bounded repair, then explicit parking

The project has documented repeated headless application failures rather than pretending every scenario is green. Its policy permits one bounded repair, then parks a repeating scenario while retaining its assertions, deadlines and resolution contract. Coverage debt remains linked to an actual issue. Parking is neither a code fix nor permission to call the scenario passed (docs/VERIFICATION.md:93–107).

A bounded repair changes one evidenced cause: for example, disabling both companions when a fixture claims LCD-only lighting, or starting a rest experiment from current `performance.now()` after real GPU waiting. Removing native framebuffer assertions, reducing density, inventing synthetic future time or extending every deadline is not equivalent. Those actions can change the product contract merely to satisfy the harness.

If failure repeats, preserve its original assertion body, report the exact exclusion and open or update the corresponding issue and Project status through the authorized owner. Do not retry indefinitely until environmental luck produces a receipt. The ledger records this distinction and rejects receipt-only deployment loops (TODO/04_DEVELOPMENT_LEDGER.todo:1–106).

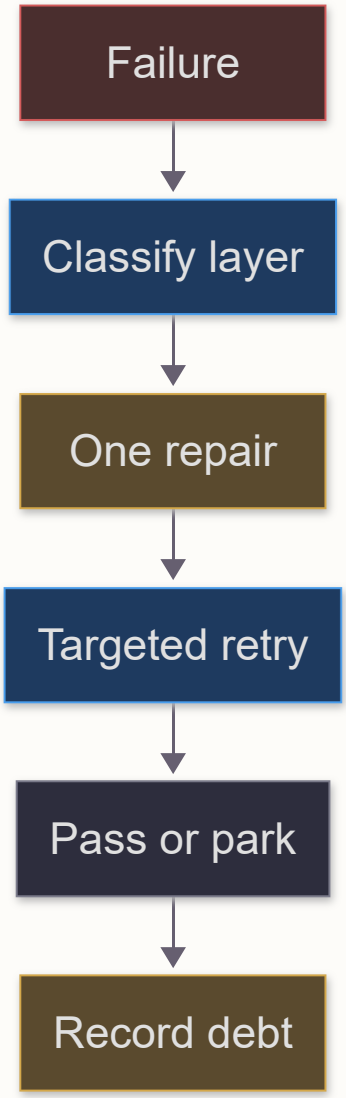


Figure MD36.2. Bounded recovery preserves assertions and records unresolved coverage instead of manufacturing a green result.

The actual workstream record

Numbered TODO workstreams carry eight core fields: Status, Milestone, Dependencies, Purpose, Tasks, Acceptance criteria, Completion evidence and Completion log. `TODO/116_DS_UPRIGHT_LIGHTING.todo` shows this structure and links actual GitHub issue 115. A workstream number is therefore not an issue-number formula (TODO/116_DS_UPRIGHT_LIGHTING.todo:1–38).

Tracking documentation describes local TODO text as the managed source, preview-first synchronization, refusal to overwrite divergent remote changes, persisted backlinks and final readback of issue body, state, Project membership and status. Index and perpetual ledger are not ordinary numbered workstreams to mirror blindly. This is a real workflow, not authorization for every documentation agent to synchronize or publish (docs/GITHUB-TRACKING.md:1–63).

Write completion evidence in past tense only for actions that actually occurred. Planned checks belong in Tasks or Acceptance criteria. A chapter describing a test cannot reuse a historical success as its own execution receipt.

Keep the experiment's stopping rule explicit

Before a browser run, state the expected terminal outcomes: pass with retained assertions, fail with diagnostic evidence, or explicit exclusion after the bounded repair policy. Also state which exact command, revision and shard inventory will be monitored. Without that stopping rule, an agent can accidentally substitute an endless series of partial runs for one complete verification attempt.

A useful failure packet contains the scenario name, fixture setup, actual assertion text, relevant console or application error, timing/fence state and any captured diagnostic image. It also distinguishes a harness timeout from a product assertion failure. Both matter, but they suggest different next actions. A timeout before startup ownership is established does not prove an LCD shader defect. A native source assertion after valid startup should not be dismissed as cosmetic rendering noise.

Preserve negative controls. The upright-light fixture's all-off state protects against adding global ambient as a shortcut. Its LCD equality protects against rescuing shell brightness by changing picture output. Its ray probes protect against sampling the wrong surface. Removing any one can allow a different wrong implementation to satisfy the remaining checks. This is why a good regression contains several orthogonal observations rather than one elaborate screenshot threshold.

For documentation validation, use a different scope. Check chapter names, required markers, balanced fences, Mermaid parsing, source-path existence, verified line bounds and prose depth. Those checks can establish manuscript integrity without building the application or exercising native gameplay. Report them as documentation checks only. A successful Markdown validator must not be added to the runtime test-pass total, just as a runtime deployment must not be presented as an aesthetic review. This separation keeps future handoffs honest even when several agents produce different evidence at the same time.

Also record what a diagnostic image cannot establish. A still image can show a shell silhouette and local brightness, but not a press/release sequence or a deferred native update. A short animation can show motion, but may omit the exact moment that a held source should cancel. A GPU fence can establish command completion, but not who reviewed the picture. Choose evidence to match each claim and preserve the missing dimensions explicitly. This avoids requesting ever more screenshots for a state-ownership defect that needs event traces, or replacing visual review with a numeric bound that covers only one small region.

MD36-W01 Reconstruct a truthful result summary

The public record identifies historical exact-source run `387537b602f44a5bc85d10ee0196d0ccb099e968`. Its declared browser inventory is 119: 98 passed, 21 explicitly skipped, zero failed and zero unrun. The arithmetic `98+21=119` reconciles declared coverage, but “119 passed” would be false. The record

also identifies all nineteen shards and build/deploy evidence. Those receipts belong to that source revision and run (docs/VERIFICATION.md:9–37).

This runnable Node recipe checks only reconciliation. Expected output keeps passed and excluded separate.

```
const result = {
  declared: 119, passed: 98, skipped: 21,
  failed: 0, unrun: 0
};
const accounted = result.passed + result.skipped
  + result.failed + result.unrun;
console.assert(accounted === result.declared);
console.log(`${result.passed} passed; ` +
  `${result.skipped} excluded; ${result.unrun} unrun`);
```

The same record says 48 public runtime files matched, while a stricter 49-entry checker still exited one because the Pages marker returned 404. Report both facts. Do not convert an explained checker failure into a forty-ninth successful response, and do not imply this chapter repeated the published-byte checks.

MD36-R01 Retry when the fixture, not the model, is wrong

Suppose the first upright LCD test used powered-off glass and observed changed pixels after switching lights. The changed condition is reflective material, not corrupted active LCD output. The bounded repair is to install status LCDs, verify their basic non-tone-mapped material and retain the original RGB equality requirement.

Rerun the affected desktop and phone scenarios under the exact same geometric and lighting assertions. If application failures repeat, park them explicitly and preserve coverage debt. Do not replace an active-LCD assertion with a weaker “image exists” assertion. The historical lighting TODO records this correction and its limited reason; it does not justify generalized fixture weakening.

MD36-Q01 Exercise: identify the unsupported conclusion

Explained answer

A report says “Pages deployed successfully, so the shader, native input and owner-approved appearance are verified.” Which additional receipts would make each narrower claim defensible?

MD36-A01 Explained answer

Back to question

Deployment proves availability of a particular artifact only after revision identity is checked. Shader behavior needs material/state observations and appropriate pixel or visual evidence. Native input needs a real transition or native framebuffer response, not merely button animation. Owner approval needs the owner's explicit review.

For each claim, record revision, command or fixture, environment, exact outcome and remaining exclusions. Monitor the exact run to a terminal state and reconcile all shards rather than substituting a later unrelated green run. This separation turns a report into reusable engineering evidence rather than persuasive but untraceable optimism.

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD37

The causal modeling history

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD37 / THE CAUSAL MODELING HISTORY

Contents

The causal modeling history

History teaches causality, not just chronology

A useful modeling history explains why the present implementation has its shape. Listing commits is insufficient: each change should connect a symptom, a tested mechanism, a bounded repair and an evidence limit. This chapter distinguishes checked diffs from documented historical observations. It does not recreate private conversation, claim perfect memory or treat an old screenshot as proof that today's source was tested.

The current manuscript's baseline is `1e4ef518d6943ab1af39cef3b4d558f209a17891`. When an earlier diff has a different constant, teach the earlier value only as history. The current runtime's exposure, GPU thresholds and placements come from the pinned source, not a commit message (src/scene.js:14–25, src/scene.js:1182–1236).

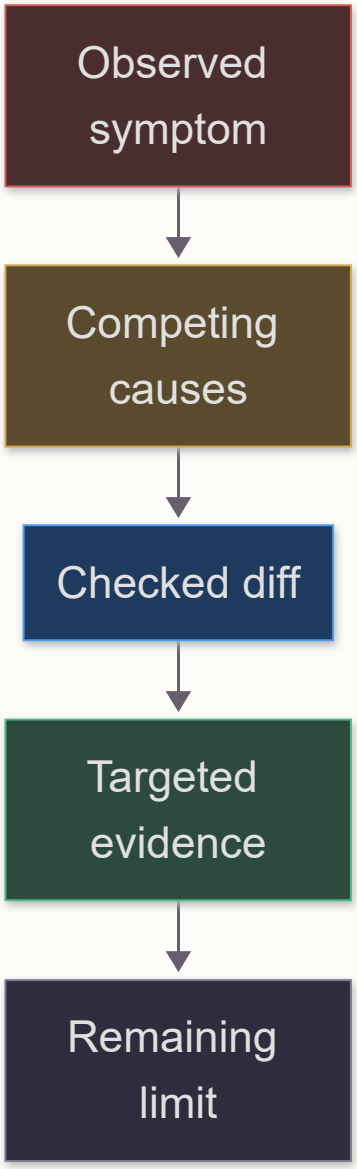


Figure MD37.1. Causal history preserves rejected explanations and evidence limits instead of converting every commit into an unqualified success story.

The upright DS: a spatial repair, not brighter paint

Commit `9be851e96098e0dfd4beedffd911185cf33a11a6` changes the DS play position from `(5,5.1,8)` to `(-2,5.1,-8)`. The checked application diff does not alter shell materials, light intensities or exposure. That is the causal point: a straight camera-facing shell had been behind the wrong side of the existing lamp, so the repair moved the shell into the authored illumination.

The workstream records old blue-channel values around nine or ten, and final documented shell blue ranges 108–113 on the lid and 117–157 on the deck. These are historical observations reported in the TODO, not values newly measured for this handbook. It also records an

intermediate candidate `(-2,5.1,-2)` that improved lighting but had a rotation-clearance gap of `-1.047`. Brightness alone therefore could not select the final placement (TODO/116_DS_UPRIGHT_LIGHTING.todo:1–56).

The final regression is geometrically specific: eight probe rays must hit named deck/lid shell surfaces inside the frame; all lamps off must be nearly black; tripod and room conditions must produce the expected blue shell. Status LCD RGB must not change. Those controls rule out a fake repair that simply increases global ambient or recolors the paint (tests/browser/ds.spec.mjs:1412–1486).

GPU fences changed the meaning of readiness

Commit `34484498de6be19a48c5191c4d717035bf84a320` introduced real room-paint fences and separated DS LCD copying from room-paint-only controller work. This addressed two coupled problems: JavaScript could outpace graphics completion, and budgeting the room could accidentally freeze native LCD copying.

The earlier diff used a 50-millisecond slow-frame threshold. The pinned implementation uses 100 milliseconds. Do not copy the historical threshold into a current recipe simply because the commit explains the mechanism well. The key lasting design is zero-timeout polling, explicit flush, bounded recovery and independent native picture updates.

Later commit `836fc093bc4e72b06898e983cdf524c8fe772047` retained severe-load recovery while avoiding an added wait for moderate completed frames. Its checked tests distinguish the exact 100-millisecond boundary from 101 milliseconds. This is not a claim that all devices now reach a particular frame rate. It is an algorithmic policy improvement under stated observations.

Another subtle repair concerns the test clock. After a real GPU wait, starting a rest sample from stale `lastRender` can place synthetic time before actual work creation. The current test resets the baseline and starts at real `performance.now()` (tests/browser/scene.spec.mjs:338–363). History therefore teaches a testing lesson as well as a renderer lesson: clocks are part of a fixture's causal model.

Why fixed steps needed presentation state

Commit `120d9bd5a3e785d24a9bdc5e96d6e9b332613510` added fall/bounce/rise presentation behavior. Ordinary fixed stepping can compute a correct final rest position after a long gap while never showing the fall. The repair preserves meaningful intermediate poses until the room acknowledges them, while bounding integration to 600 substeps.

This is deliberately not a replacement for accurate rigid-body physics. Motion remains upright, support and side contact are simplified, and queued elapsed time serves the visible interaction. The unit tests prove the intended sparse-update sequence and eventual behavior, not angular momentum or arbitrary contact manifolds (src/pear-motion.js:91–157, tests/unit/pear-motion.test.mjs:75–154).

Tiny numeric errors, large visible layout changes

Commit `6b5d2c552e6b3206bfb3ae36d845896773147a1c` adds a magnitude-scaled eight-epsilon tolerance to forbidden-rectangle interior checks. A legal edge can move a few ULPs when world bounds are translated to local bounds and reconstructed. Without tolerance, the candidate can be misclassified as inside and jump to a different side of the desk.

The repair does not replace the `.25` physical clearance gap with a broad collision fudge. The separate squared-distance tie tolerance remains, and the tests retain rejection of an actual `.01` overlap. Historical context explains why both numerical and product-scale tests are necessary (src/desk-clearance.js:18–31, tests/unit/desk-clearance.test.mjs:90–123).

Built-in CRT became explicit reusable drawing

Commit `06e9c8240b6f3a8c5e2c91a408db0be61ccdfef2` introduced the cached built-in CRT treatment. The present helper caches mask and vignette by canvas dimensions and row count, then restores drawing state. This history supports a bounded claim about generated status images, not a claim that all native CRT presets use this canvas implementation (src/lcd-crt.js:1–39, src/preferences.js:1–13).

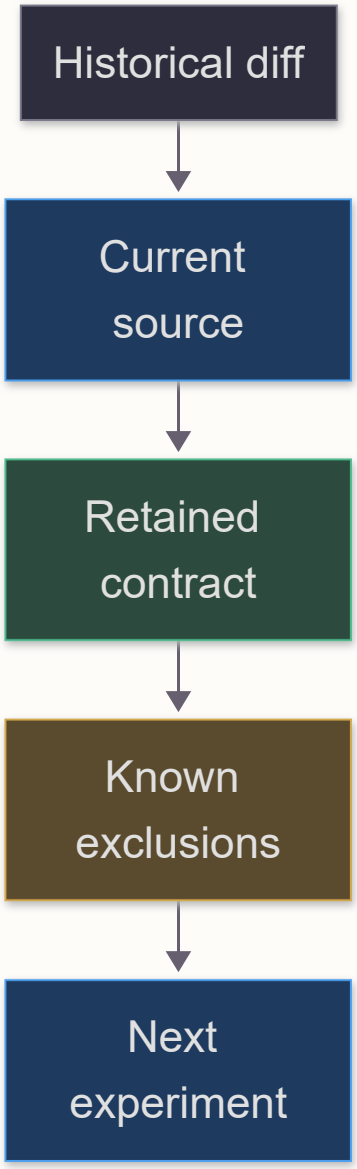


Figure MD37.2. A historical mechanism becomes a current teaching claim only after comparison with the pinned implementation and its exclusions.

Read a history entry in both directions

Forward reading asks what changed after a symptom. Backward reading asks which present invariant would break if the change were removed. Removing the ULP tolerance can destabilize a legal edge. Removing presentation acknowledgment can hide a fall under sparse painting. Putting DS copies behind the room gate can stale native pictures. These counterfactuals make the history useful for future regression design.

Counterfactual reasoning is not a new experiment. Unless the removal was actually tested, label it as an explanation derived from source and tests. The distinction is especially important when historical entries include numerical observations: those values depend on their fixture and environment, and cannot be reproduced merely by repeating them in prose.

Look for unchanged files as well as edited ones. The upright repair's lack of material or light edits rules out several claimed mechanisms. Conversely, a commit that touches renderer, tests and verification prose does not automatically prove all three are correct. Read its actual changes and compare to the final baseline, where later edits may alter thresholds, exclusions or fixture preparation.

The development ledger is a causal index, not the complete evidence itself. Its newest entries distinguish exact-source deployment, book artifacts outside the runtime allowlist and preservation of nine unrelated dirty historical images (TODO/04_DEVELOPMENT_LEDGER.todo:1–36). Those distinctions explain why documentation work must not casually rebuild or commit the whole worktree. A model change, a book change and a published runtime receipt can coexist while having different owners. Record which one a history claim concerns, and avoid promoting a navigation or receipt update into an application-behavior change.

For each historical measurement, preserve its subject as carefully as its number. The lighting TODO's final ranges refer to lid and deck samples, not two universal device classes. Its rendered desktop inspection is explicitly not owner approval. Its focused browser run is not an all-suite result. These qualifiers may look small beside the headline repair, but losing them changes the claim. A causal history should make later engineering decisions safer, not accumulate impressive numbers stripped of their conditions. When a receipt lacks a needed condition, mark the gap rather than manufacturing a more convenient interpretation.

MD37-W01 Build a causal receipt for the DS repair

Write four rows: symptom, changed source, controlled variables and evidence limit. Symptom is an upright blue shell reading too dark under the tripod. Changed source is the play-position constant. Controlled variables include material, light intensity, exposure, straight root and both disabled companions. Evidence includes eight verified shell rays and separate LCD invariance; the limit is that documented historical samples are not a new execution.

Then compute the displacement: new minus old is `(-7,0,-16)`, whose horizontal length is `sqrt(49+256)`, about 17.464 world units. That large relocation explains why framing and rotational clearance also needed attention. It does not explain brightness by distance alone: surface orientation relative to the lamp and cone also matters.

This runnable Node recipe checks displacement only. Expected output is the stated vector and length, not a predicted pixel value.

```
const oldPose = [5, 5.1, 8];
const newPose = [-2, 5.1, -8];
const delta = newPose.map((v, i) => v - oldPose[i]);
const length = Math.hypot(delta[0], delta[2]);
console.assert(delta[1] === 0);
console.log({ delta, length });
```

MD37-R01 Retry when a historical test becomes unreliable

Change the condition from a successful local fixture to repeated headless application failure. Commit `19f499643c9d02a0be219713ae0059d9fd531309` records loader-aware restoration and explicit parking. The public account retains the failed broad SwiftShader attempt—52 passed, 55 failed, two unrun—and says that broad change was reverted. It does not prove SwiftShader was the sole cause (docs/VERIFICATION.md:79–107).

Apply one bounded evidenced repair, then park repeating scenarios with assertions intact and coverage debt visible. History is most useful when it preserves this uncomfortable outcome rather than smoothing it into “CI fixed.”

MD37-Q01 Exercise: which constant should a reconstruction use?

Explained answer

An agent reads the fence-introduction commit, copies its 50-millisecond threshold and claims it reproduced the pinned application exactly. What should its correction include beyond changing one number?

MD37-A01 Explained answer

Back to question

Use the pinned 100-millisecond threshold and its strict boundary, recovery caps and lighting exception. Explain that 50 belongs to an earlier iteration. Reconcile tests at exactly 100 and 101, inspect the real-time fixture baseline, and report whether the retained browser scenario is excluded in CI. Correcting a constant alone does not establish the whole scheduling contract.

Keep the ledger's causal account and exact-source receipts separate from this documentation's validation. Old successful runs remain valuable history, but they cannot be silently promoted into tests of newly written chapters or a future implementation.

PART 4 / A COMPLETE, VERIFIABLE 3D WORLD

MD38

Agent reconstruction and extension

> *Inspect the shape. Reconstruct the mechanism. Verify the visible result.*

MD38 / AGENT RECONSTRUCTION AND EXTENSION

Contents

Agent reconstruction and extension

Reconstruct behavior, not an imagined memory

A useful agent handoff contains enough explicit contracts for another engineer to continue without pretending to remember an earlier conversation. It names the baseline, files, space conventions, ownership, experiments and unresolved gaps. It does not claim a perfect mental replica or universal accuracy. This chapter provides such a contract for the integrated room, renderer and interaction layer; detailed console and companion anatomy remains the responsibility of their dedicated factory chapters.

The canonical evidence baseline is `1e4ef518d6943ab1af39cef3b4d558f209a17891` in `Zanark/BowserBoy`. Runtime dependency evidence is the explicit Three.js 0.186.1 declaration (package.json:7–9). A later repository checkout or current online manual can differ. Always distinguish “observed at the baseline” from “recommended for a new extension.”

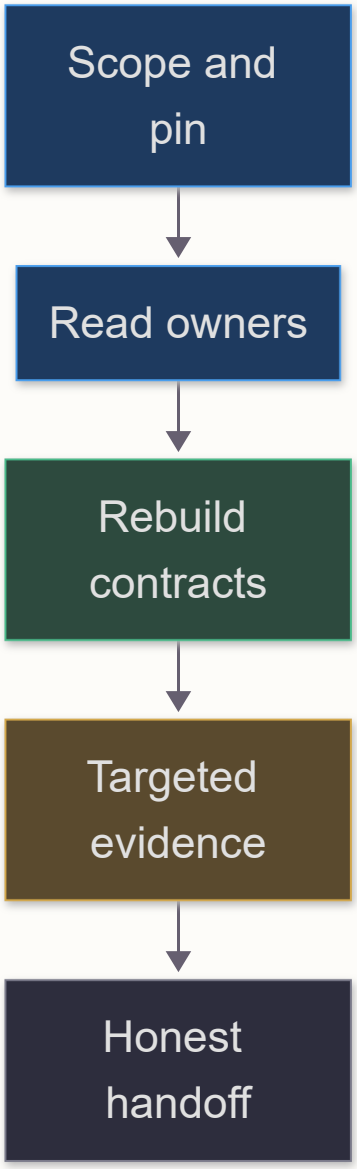


Figure MD38.1. A reconstruction is a chain of explicit receipts, not an appeal to hidden conversation memory.

The reusable assignment

The following instructions are intended to be copied as an assignment, with writable paths and execution permissions filled in by the requesting owner. They do not grant publication, tracking synchronization or application-edit permission by themselves.

Establish scope and read before editing

Record the requested output, exact baseline and allowed write paths. Inspect the worktree without resetting unrelated changes. Preserve historical images, existing books and concurrent work. Do not inspect private ROMs, conversations, credentials or unrelated personal context. Report missing public evidence rather than searching private stores to fill a documentation gap.

Read complete `src/lighting.js`, `src/scene.js` and `src/lcd-crt.js`. Read relevant controller, preferences, motion, clearance and asset sections before describing their behavior. Trace actual callers and state transitions where source evidence permits; do not invent a code graph for a repository without one. Read tests as specifications and mark any skipped coverage. Public architecture is a useful orientation, not a substitute for the implementation (docs/ARCHITECTURE.md:43–146).

Rebuild the world in dependency order

First establish Y zero as tabletop support. Construct the tabletop, rear wall and lower floor with the source dimensions and separate top/edge material ownership. Add the tripod body at `(-13.5,0,2)` with child scale `1/.017`. Draw one extruded leg and share it three times around Y; add support, open drum, rims and diffuser. Preserve double-sided fabric and the explicit non-casting overrides.

Add the spotlight, two local area emitters and world-owned tabletop bounce. Do not invent a tall central pole, mechanical switch or extra room props. The actual switch model is surface metadata plus scene/UI dispatch (src/lighting.js:9–110). Attach existing console and companion factories through their returned roots and named references; do not reconstruct their detailed anatomy in the integration module.

Preserve image and light ownership

Use sRGB output and the authored tone mapping. Keep active displays basic and non-tone-mapped, but preserve the actual distinction between DS nearest-filter native canvases and GBA linear- filter canvases. Keep powered-off glass reflective. Do not treat `drawLcdCrt` as a native emulator shader implementation (src/scene.js:1106–1160, src/ds-controller.js:415–435).

Sample screen emission in linear space using the 24-by-16 average, bounded sampling cadence and elapsed-time smoothing. Include both companions in exposure decisions and obstacle inventory. Never call a fixture LCD-only unless tripod, room, Pear and axolotl conditions are explicitly controlled.

Preserve interaction and scheduling

Convert pointer coordinates through the canvas CSS rectangle. Respect dispatch priority, pointer capture, grab offsets, cancellation and release. For lower DS touch, raycast the whole model and accept only a nearest lower- LCD hit. Update camera/model matrices and DOM control projection before waiting for a GPU paint (src/ds-controller.js:490–514, src/scene.js:484–500).

Separate callback updates, native screen copies, room submission and fence completion. Retain resolution, threshold boundaries and bounded recovery; start new real-time experiments from current `performance.now()`, not stale render timestamps. Keep required fall/bounce/rise poses until presentation acknowledgment. Do not label this as full rigid-body physics.

Research and evidence instructions

Consult three to five official references for concepts that extend beyond source. For this part the checked references are Three.js color management, physical materials, shadows and cleanup, plus MDN WebGL best practices. Record URL, consultation date, supported claim and scope. Current documentation is guidance, not pinned-runtime proof. Never send local code to public search.

Maintain four evidence labels: actual implementation, documented historical observation, researched recommendation and proposed extension. For history, inspect scoped diffs and say which mechanism changed. An old test receipt does not validate a new source revision. For a new experiment, record assumptions, changed condition, fixed variables, expected outcome and the observation that would falsify the explanation.

The scene's cleanup is a useful baseline, not a universal resource tracker. If adding texture slots or shared resources, define their owner and disposal path explicitly (src/scene.js:1430–1450). The checked current cleanup guide supports unique-resource tracking, while MDN best practices supports avoiding unnecessary blocking and releasing GPU resources eagerly.

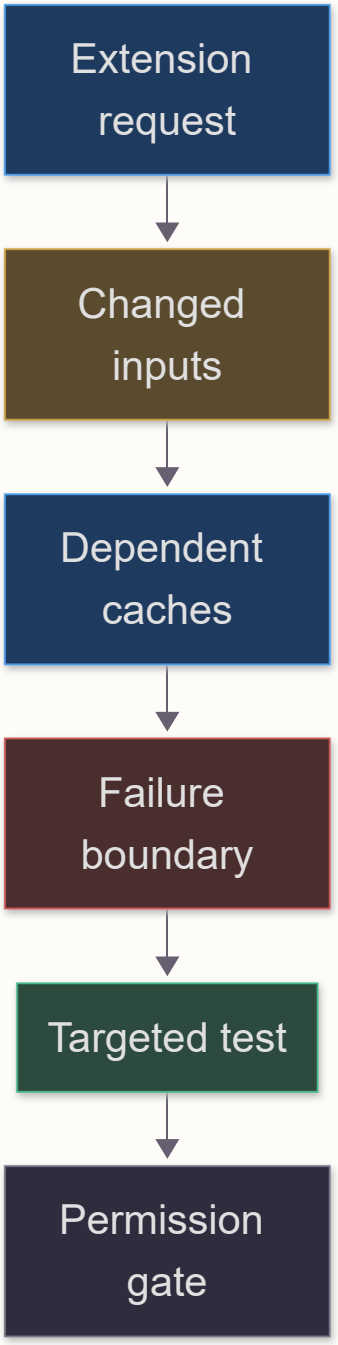


Figure MD38.2. Extension work ends at its authorized boundary; successful technical checks do not automatically authorize synchronization or publication.

Deliver a self-contained reconstruction packet

The final packet should let another agent find every necessary public source without guessing. Include a model inventory mapping components to source symbols, chapters and relevant tests. Record absent components explicitly: there is no modeled mechanical lamp switch or separate tall pole to hunt for. Include research receipts with supported concepts and historical receipts with checked changes. List what remains unverified, especially render execution, parked browser coverage and owner acceptance.

For a proposed extension, add a dependency table with changed input, owner, invalidated cache and retained invariant. A wider shade changes geometry and bounds; its material brightness need not change. A new light changes exposure-state logic and reflections; its decorative mesh need not be an obstacle unless the model actually occupies space. Such a table prevents a successor from mistaking all visible properties for one undifferentiated “appearance” setting.

Include runnable arithmetic or clearly labeled pseudocode with assumptions and expected outcomes, not fragments that reference imaginary application APIs. State whether a recipe was executed. When commands are authorized, retain exit status and exact run identity. When they are prohibited, stop at manuscript validation and say so. A complete handoff is not one that claims everything is done; it is one that makes the next permitted action and remaining evidence gap unambiguous.

MD38-W01 Specify a larger shade extension completely

Suppose the owner requests a ten-percent wider shade while preserving height and leg footprint. Compute radius `.09*1.1=.099` and diameter `.198/.017=11.6471` world units. The original diameter was 10.5882, so the increase is 1.0588 world units across the drum. Keep center Y `.225`, height `.15` and rim Y values `.15` and `.30`.

The rim's original radius `.0895` is `.0005` inside the drum radius. Preserving that radial inset gives new radius `.0985`, not `.0895`. Retain the tube radius unless a thicker seam is separately requested. Check the transmitted rectangle at X `.11` against the wider fabric: the source's anti-seam offset deserves renewed inspection, not automatic rescaling.

This runnable Node recipe calculates proposed dimensions only. It does not modify any model or claim the enlarged shade is accepted.

```
const metersPerUnit = .017;
const radius = .09 * 1.1;
const rim = radius - .0005;
const diameter = 2 * radius / metersPerUnit;
console.assert(Math.abs(rim - .0985) < 1e-12);
console.log({ radius, rim, diameter });
```

The implementation plan must include new geometry creation, old owned geometry disposal, lamp-bound recomputation, clearance with all companions, main-view invalidation and reflection refresh. Acceptance should include fixed leg footprint, unchanged light switches, no invented pole, no seam and unchanged active LCD colors. Human review decides whether the wider shade is preferable.

Keep calculated values unrounded in code and round only their displayed explanation. The diameter increase is exactly `.018/.017` world units under these assumptions. Independently rounding old and new diameters before subtraction can produce a different last printed digit. This

small example mirrors the clearance algorithm's larger lesson: numerical presentation is not authoritative simulation state.

MD38-R01 Retry when the new bounds block the desk

Change the condition: the widened shade's world bounds leave no legal table layout under current priorities. First verify that the bounds include actual geometry and were not enlarged twice by metric conversion. If the bound is correct, do not move the fixed tripod silently.

The resolver must stage both equal-ranked console orders with stable tie behavior, then report failure if neither works (src/scene.js:849–890). Offer an explicitly proposed alternative, such as a smaller diameter or a separately authorized world placement change. Preserve saved poses and rollback semantics. If the application regression repeats after one bounded repair, retain the assertions and park coverage debt rather than shrinking gaps until it passes.

MD38-Q01 Exercise: write the final handoff

Explained answer

The agent has written the extension and run only unit tests. Browser startup failed twice, images have not been reviewed, and publication was not authorized. What should the final message contain, and what must it not imply?

MD38-A01 Explained answer

Back to question

Report exact changed files, baseline, computed dimensions, commands actually executed, unit outcomes and browser failure receipts. State the bounded repair attempted, retained assertions, parked scenario and issue/Project update needed from the authorized owner. Mark visual and aesthetic review pending. Say explicitly that no deployment occurred.

Use the real eight-field numbered workstream and causal ledger practice when authorized. Preview synchronization, read the actual issue backlink rather than guessing its number, and verify remote state after any approved apply (docs/GITHUB-TRACKING.md:1–63). That process still grants no implicit publishing permission.

Finally preserve the public presentation: the root README remains GIF-first, and standalone books provide detailed teaching (README.md:1–30). Do not replace an inviting entry point with exhaustive validation logs. A truthful handoff makes the next action clear without claiming tests, approval, memory continuity or publication that did not occur.

The standalone agent handoff

This appendix is an execution contract for an agent that did not participate in BowserBOY's original development. Read it with the relevant model chapters and the source register. It is designed to preserve reasoning boundaries and reproducible techniques, not to impersonate an earlier assistant or assert access to its hidden reasoning.

Establish authority and the baseline

Begin by stating the requested visible outcome in ordinary language. Examples include making a facial feature travel farther without entering the body, making a hinged lid preserve its relationship to the lower deck, or making a moved lamp illuminate the scene from its new position. Keep the outcome separate from a proposed implementation. A user asking for greater movement has not necessarily asked for a new geometry representation.

Record the repository revision, worktree state and relevant model factory. Preserve unrelated edits, supplied artwork and previously published editions. The implementation documented here is pinned to `1e4ef518d6943ab1af39cef3b4d558f209a17891`; a later working tree is a new baseline, not a reason to silently reinterpret the book. Before using a chapter's numeric value, check whether the active code still agrees.

Define allowed writes and output boundaries. A diagnostic capture does not authorize editing the application model to make the capture easier. A book request does not authorize publishing private ROMs, raw recordings or conversation history. A local verification does not imply permission to push or deploy. Follow the actual user's authority for those operations.

Inspect the existing numbered workstreams and contributor instructions. For this project's established workflow, retain the TODO index, numbered workstream, development ledger and all required status, purpose, task, acceptance, evidence and completion fields. Keep technical completion separate from aesthetic approval. Where remote tracking is authorized, verify the real issue and Project status rather than merely writing an issue URL into a local file.

Construct a model contract

Write a compact contract before changing geometry. It should identify the root group, local axes, scale convention, primary silhouette, geometry helpers, material roles, texture sources, movable subgroups, returned handles and disposal owner. Add the important interaction surfaces and collision envelopes. If any of those are unknown, make them a question to resolve in source or a small experiment; do not fill them with a plausible guess.

For a console, include the front/back convention, shell layers, seam, bezel opening, active display, inactive glass, buttons, grips, ports, labels and cartridge relation. For the DS, add the lower-deck origin, upper-lid pivot, hinge geometry, separate screens, stylus and touch coordinates. For a companion, include body shape, face projection, expression groups, limbs, light contribution, drag/rest states and full bounds.

Separate appearance from mechanism. An object can look rounded because of an actual bevel, a smooth normal field or a material response. Those are different constructions with different costs. A character can look soft because of the silhouette, curvature and diffuse material without a deformable physical simulation. A screen can look bright because it is unlit, emissive or externally illuminated; those mechanisms affect color and exposure differently.

Name dimensions by what they control. A parameter in radians is not a distance in model units. A local offset is not a world-space displacement after parent rotation. A physical lamp dimension in meters is not necessarily the scene unit used by the console. Carry units through the worked calculations and into diagnostic output.

Rebuild the hierarchy in a small scene

Instantiate the original factory in an isolated scene before reconstructing it manually. Load required image resources first. In particular, await `loadBrand()` before calling the GBA factory, because its canvas textures draw the shared brand image during construction. Do not treat an import as resource readiness.

Capture a front, rear and three-quarter view with a declared projection. Compute bounds from the complete hierarchy after updating world matrices. Fit the camera to all corners rather than a guessed center and radius that can crop a protruding limb or stylus. Make any pose override explicit. In this edition's atlas, the DS returned lid group receives a fifteen-degree local-X rotation; this is a diagnostic pose, not a public `setHinge` method or a claim about the application's play angle.

Use neutral lighting to understand base surfaces, then test the real application lighting separately. When disabling material emission or model lights, record that policy in the capture receipt and caption. Do not present a studio render as proof that the table lamp produces the same result in the application.

Inspect large masses before details. Compare silhouette width, height, recess placement and front/back thickness. Then inspect seams, bevels, controls and small markings. Rebuilding detail before establishing scale tends to produce local perfection attached to the wrong global proportions.

If a primitive is transformed, inspect its local axes before applying the parent transform. A cylinder's construction axis, an extruded shape's depth and a plane's normal can differ. The relevant chapter explains each usage; do not assume one helper's orientation is a repository-wide convention.

Preserve the visible surface

For layered hard-surface models, draw a cross-section on paper or in a small diagram. Identify which surface forms the recess floor, which forms the surrounding rim, which is an active display and which is glass. Check that the layers do not intersect, produce z-fighting or overwrite one another's intended color.

For facial patches on a curved body, apply the intended expression shape before projecting onto the shell. Translating already projected geometry can bury one end of the feature or detach it from the surface. The correct operation depends on the body's actual surface parameterization and the intended transform owner.

Test every movement extreme with every supported expression, not only a centered smile. Check the visibility of the foremost geometry from the real camera. A local `visible` property or a finite vertex coordinate does not establish that the eye or freckle is outside the body and visible to the user. Ancestor visibility and intervening geometry also matter.

Preserve the complete feature set when changing the gaze range. Tiny freckles and facial accents are easy to omit from an expression-only test while still being visibly wrong. Compare the projected screen-space displacement to the user's intended viewer-left, viewer-right, up or down direction; a world-axis sign alone does not establish that result after rotation.

Do not hide a surface problem with brighter emission or a different camera. Isolate geometry, normal orientation, material settings and illumination as separate variables. Restore the actual scene conditions before accepting the change.

Treat materials as separate contracts

Record which surfaces use lit physical/standard materials and which deliberately do not. Preserve the active LCD's color path separately from inactive reflective glass and the surrounding plastic. Nearest sampling and mipmap choices belong to the display contract, not to every texture on the model.

Color values, output color space, tone mapping and exposure have to agree. A material that is pleasant in a studio can become unreadably dark or white in the application. A companion's emissive surface can overwhelm its facial detail even when its point light is correctly positioned. Raising global exposure can fix one object while breaking every other object.

When tuning, hold most variables fixed. Start with the visible body and screen under the real camera, then isolate the lamp, companion emitters and ambient contribution. Compare the same pose before and after the change. Record what the experiment actually excludes; turning off one companion is not the same as turning off both.

Small labels need an explicit resolution and filtering policy. Check the canvas crop, alpha processing, physical size and distance to the camera. A high-resolution texture is not automatically readable if the mark is physically too small, tilted, occluded or washed out. Respect supplied artwork and distinguish it from newly generated labels.

Integrate movement without losing ownership

Separate authoring pose, interaction pose and settled desk pose. A factory's initial coordinates do not automatically become the safe home position after the scene places several objects together. A focus or inspection mode may temporarily move a console without authorizing the overwrite of its saved table pose.

Use the actual animation timestamp appropriate to the mechanism. A stale render timestamp, an arbitrary number of updates and a wall-clock delay can disagree with painted motion. For a fall or cartridge sequence, inspect the intermediate visible pose, not only the final state variable.

After changing shape or pose, re-evaluate the complete component envelope. A limb can extend beyond the body's current axis-aligned box; a rotated object can sweep through a larger envelope than its final orientation. Collision-free contact at one angle does not establish clearance throughout manipulation.

When choosing a home position, inspect deterministic ordering and tie behavior. An algorithm can be correct for one object order and fail when equally ranked objects are considered in the other order. Preserve existing tolerances and cache invalidation semantics unless the requirement specifically changes them.

Do not equate smooth animation with a physics simulation. The project includes explicit motion and settling mechanisms; explain the actual equations and state transitions used. If introducing a real physics engine, that is a substantial new dependency and behavior change requiring its own requirements, tests and resource budget.

Preserve input ownership

Find the complete path from pointer location to selected geometry and from that geometry to the interaction owner. Occlusion, invisible ancestors, controls, stylus, lamp clicks, console dragging and inspection can compete. A model change can affect picking even when it does not change the visual design.

Test representative rays through the new shape and through neighboring controls. A broad invisible hit target may improve one interaction while stealing another. A decorative mesh may need to be ignored for a particular action without becoming globally invisible. Keep the existing ownership rules explicit.

For DS touch, preserve the relationship among the screen geometry, its world transform and touch coordinates. A visual change to a curved screen or frame should not silently change the emulated touch domain. Inspect real coordinate conversion and the active surface, not an unrelated screenshot rectangle.

Use a layered validation plan

Start with the smallest deterministic checks that can reject the intended bug. For geometry, inspect component presence, dimensions, local relationships and finite values. For surface visibility, use actual foremost-hit or rendered evidence at the movement extremes. For interaction, exercise the correct pointer owner. For motion, inspect the painted intermediate and settled poses.

Add focused coverage only for the behavior introduced or fixed. Avoid tests that merely repeat constants or assert implementation spelling. Prefer a case that would fail on the old behavior and succeeds on the new visible outcome. Keep assertions meaningful when diagnosing environmental

instability.

Use the established bounded-recovery rule for application tests: one justified repair attempt, then explicitly park a repeatedly failing case with its assertions retained and coverage debt recorded. Do not add retries until a flaky test happens to pass, weaken an unrelated requirement or report a skip as a fixed behavior.

Broaden validation when shared construction helpers, transform ownership, render scheduling or input routing are affected. A small visual patch can have a wide dependency boundary. Conversely, do not rerun an unrelated full suite repeatedly after the relevant checks have passed and nothing has changed.

Record the exact revision and environment of a hosted run. A local browser result is not a hosted deployment result; a successful build is not gameplay verification; a screenshot is not an input test. Count declared, passed, skipped, failed and unrun cases separately, and inspect all required shards and the actual deployment job.

Reproduce the diagnostic atlas

Run the existing generator from the repository root in an environment with the locked application and production dependencies and an installed supported Chromium browser:

```
node docs\modeling-book\tools\capture-model-atlas.mjs
python docs\modeling-book\tools\make-atlas-plates.py --review $env:BOOK_REVIEW_DIR
```

Set `BOOK_REVIEW_DIR` to an explicitly chosen private review directory before the second command. It is an output location, not a source of model images. A missing value must be corrected, not replaced by a success-shaped empty image report.

The capture generator verifies the pinned first-party sources, records raw and committed hashes, blocks external requests and writes eighteen PNGs plus its manifest. The plate tool composes those captures into an original six-model cover and a contact sheet. It does not redraw geometry or synthesize a replacement model image.

Read the receipt and inspect the actual contact sheet. Check for cropped limbs, missing controls, wrong front/back labeling, invisible faces and incorrect lid pose. The expected mesh count establishes inventory continuity, not visual correctness by itself. A future intended geometry change requires a new source cutoff and an explicitly new atlas, not editing an old receipt.

Rebuild this book without touching the old editions

The modeling edition uses its own assembler and route:

```
node --test docs\book\production\production.test.mjs
python -B -m unittest discover -s docs\book\production -p native_rules_test.py
node docs\book\production\build-models.mjs
node docs\book\production\render.mjs --models
```

The default route belongs to the GBA book; `--nds` belongs to the NDS-only book. Do not use either to update this handbook. The three routes have separate candidate and output directories. Existing qualified PDFs and the NDS reader are preserved by exact hash checks.

The modeling assembler verifies source spans against cached immutable Git blobs, checks the four model-coverage receipts, enforces substantive per-chapter and total prose floors, and embeds approved metadata-clean illustrations. It rejects a repeated long paragraph rather than counting a copied passage as new substance. Code blocks, image markup and URLs do not satisfy the prose requirement.

The renderer lays out the actual manuscript, renders Mermaid diagrams and computes chapter page ranges. Both browser and native checks require at least two hundred real pages and two hundred unique chapter-content pages. Cover, contents, part/chapter openers and reference appendices do not supply those chapter-content pages. Typography is inherited from the readable shared print style; padding and oversized type are not a depth strategy.

The native reviewer checks searchable content coverage, chapter markers, A4 bounds, links, bookmarks, embedded fonts and private-path leakage. Its privacy expression distinguishes a real Windows drive from an ordinary HTTPS URL. The offline reader is tested without scripts or external requests. Those are specific mechanical qualifications, not proof that all wording is correct or every page has been visually reviewed.

Inspect the generated contact sheets and selected native pages. Record the exact inspected scope and PDF hash in the edition review. A rebuild invalidates an earlier visual-review note. Do not describe a thumbnail-level review as full-resolution inspection of every diagram.

Package and publish only what was authorized

Keep published byte-preservation attributes with the artifact checksums. On Windows, automatic line-ending conversion can change generated HTML or JSON between a qualified working file and its staged Git object. Verify the staged object bytes, not only the working tree, before declaring that the sealed edition will be delivered unchanged.

Stage an explicit allowlist. Exclude private inputs, raw captures not intended for publication, transient candidates, local review sheets, unrelated artwork and context stores. Keep the front README short and GIF-first; its role is to point to this book, not to absorb the book's construction explanations.

After an authorized push, verify the exact remote revision and actual downloaded artifact bytes. Monitor the exact deployment associated with that revision, including required browser shards, build and deployment jobs. A previous successful run cannot qualify a later commit.

Create or update only the owned MeshBoard persona when publishing the requested local reference thread. Use a professional-English question root, register the open thread, and add substantive replies with correct ownership and reply references. Verify the generated index and both thread views. A local board post is not an external social-network publication and does not wake another agent.

Leave a better handoff

End with the visible outcome, the meaningful mechanism changed, the exact source/artifact identity and any real limitation. Update public development evidence and private continuity for their separate audiences. Preserve prior history instead of rewriting a failed experiment into an uninterrupted success story.

For a future extension, leave a small reconstruction recipe: required inputs, coordinate assumptions, geometry sequence, material roles, hierarchy, interaction owner, relevant views, regression checks and disposal policy. If the technique came from researched guidance rather than the implementation, label it. If a statement remains unverified, say so plainly.

The new agent's advantage should be better evidence and a clearer method, not borrowed confidence. That is how this handbook can improve over time without pretending that one document contains every possible modeling decision.

Sources, model coverage and research limits

Immutable implementation, later manuscript

The public implementation cutoff for this handbook is `1e4ef518d6943ab1af39cef3b4d558f209a17891` in `Zanark/BowserBoy`. Chapter source links use that revision. The assembler checks each cited file and line range against the actual committed blob and caches repeated reads. It does not treat a successful HTTP response, a search-result preview or a matching filename as source verification.

The source manifest records the exact manuscript, production, atlas and coverage inputs used for the edition. Those later-created files are not retroactively placed at the implementation cutoff. Relative references to new supplementary files under this book may point to the living `main` branch; those links are recorded separately and are not immutable implementation evidence.

The relevant public entry points are the application dependency declaration, the handheld factory and geometry helpers, the DS factory, Pear Boy, the axolotl, the lamp and lighting systems, and scene integration. The chapters provide narrower line-level evidence and supporting motion, input and regression references.

Coverage is an explicit inventory

Part one covers the GBA's shell, seams, front/rear layers, recess, LCD, controls, grips, generated details, cartridges and texture/artwork boundaries. It explicitly distinguishes the curved LCD outline from a thick optical lens; no separate transmissive GBA glass slab exists at this cutoff. Its foundation chapters explain the library, reference and geometry contracts needed to reconstruct those objects.

Part two covers the DS lower deck, upper lid, hinge-related construction, hardware, controls, markings, separate planar displays, reflective powered-off glass material, stylus, touch domain, cartridge choreography, inspection and model regression evidence. Curved DS cover geometry is a labeled extension proposal, not an implemented model. The stylus and cartridge are not omitted merely because they are subordinate objects rather than independent top-level factories.

Part three covers Pear Boy's body and face, projection, expressions, gaze, freckles, limbs and rope, plus the axolotl's body, gills, tail and facial features. It then connects the companions to light emission, dragging, gravity, settling and complete clearance envelopes.

Part four covers the desk, room, tripod lamp and shared systems that make every model behave coherently: exposure, color, shadows, reflections, screen materials, picking, render lifecycle, algorithms, design principles, bounded recovery and causal history.

The four `coverage/part-*.json` files retain per-model source, symbol, component, chapter, test and limitation records. They also list actual source reads, consulted research and historical evidence. The assembler checks their structure and basic family coverage. That mechanical inventory is not

a semantic proof that every code line or visual nuance has a correct explanation; the chapter review and explicit limitations remain necessary.

An empty mapped-test list explicitly means no mapped test, never a pass. Installed dependency metadata and this later manuscript are recorded separately from files that exist at the implementation cutoff.

Libraries and asset provenance

The application models are constructed procedurally with Three.js; browser canvas code creates many textures and markings. Playwright is a browser-automation dependency, not the mesh-authoring library. The book pipeline separately uses Markdown-it, Mermaid, Paged.js, Playwright Core, PyMuPDF and Pillow. Do not merge those roles into a claim that a PDF tool built the handheld geometry.

Version declarations, lockfiles, installed-package identities and actual served resource hashes answer different questions. Chapter one explains the application toolchain; the edition manifest identifies production inputs; the atlas receipt records the actual browser and Three package used for the captures. A lockfile alone does not establish that an arbitrary installed directory is intact.

The factories use existing supplied brand artwork as well as generated labels. The atlas and cover are original diagnostic compositions of the project's actual procedural geometry, not photographs, historical design drawings or AI-generated replacements. The capture manifest describes lighting, pose, projection, source identity and limitations. The project's third-party notices remain part of the delivered edition.

No commercial ROM, firmware, private game recording or raw conversation is required to reconstruct the illustrated models. The atlas deliberately leaves factory displays in their non-gameplay state. Existing authorized application demonstrations, when referenced in the living documentation, have their own media provenance and do not become proof of every mechanism discussed here.

External research is bounded

Each part's coverage receipt records consulted external references, consultation date, supported mechanism and scope. Chapter citations explain the specific use. Official library or browser documentation is used to clarify geometry, transforms, materials, rendering and platform behavior, not to invent an original modeling history.

Current documentation may have changed since the pinned implementation or may describe a different release. Verify version-specific behavior against the installed package and the relevant code before applying a recommendation. If a chapter proposes an alternative, it is not evidence that the alternative is already deployed.

Short explanatory code and equations are teaching tools unless explicitly identified as exact source. Preserve actual source attribution, and do not confuse a reduced worked example with a complete drop-in replacement for a factory.

Verification and non-claims

The production manifest records actual chapter prose counts, diagrams, learning markers, illustration derivatives, implementation source hashes, atlas policy and preservation checks. The browser layout report and native PDF review identify actual pages, chapter ranges, text coverage, links, fonts and bounds. The checksum file identifies delivered bytes.

The edition's review note states the actual visual inspection performed for that exact PDF. Generated review images are candidates for inspection, not proof that inspection occurred. Mechanical success does not imply human acceptance, independent rights clearance, PDF/UA accessibility, printer prepress, a fresh-machine rebuild or byte-for-byte reproduction across browsers.

The modeling edition guards the earlier GBA PDF, NDS PDF and NDS offline reader without rebuilding them. Their preservation is not new validation of the old application or repair of previously parked browser cases. Any application CI accounting must name the actual revision and run and keep passed, skipped, failed and unrun counts separate.

This book aims to make modeling decisions reconstructable, but it is not a literal replica of an assistant and cannot guarantee complete transfer of undocumented knowledge. Improvements should add source-backed mechanisms, experiments and corrected limitations rather than confident filler. When evidence ends, the claim ends with it.